

Матвеев М. Д.

Справочник *Линуксоида*

Все • что нужно • под рукой

- Командная строка, сеть, файловая система
- Управление процессами и загрузкой ОС
- Ядро, маршрутизация и брандмауэр



Справочник *Линуксоида*

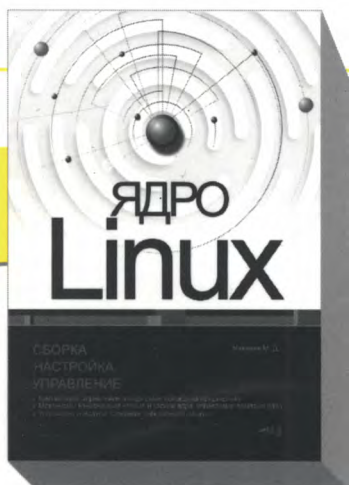
Все ● что нужно ● под рукой

Диапазон материалов справочника достаточно широк: от самых основ (командная строка, локальная сеть, файловая система, пользователи и группы) до более продвинутых тем (управление загрузкой ос, работа с хранилищем, управление процессами Linux). Отдельное внимание уделено таким важным темам, как: ядро Linux, маршрутизация и настройка брандмауэра, командный интерпретатор bash.

издательство

НАУКА и ТЕХНИКА

рекомендует



ISBN 978-5-907592-17-9



9 785907 592179 >

«Издательство Наука и Техника» г. Санкт-Петербург

Для заказа книг: т. (812) 412-70-26

E-mail: nitmail@nit.com.ru

Сайт: nit.com.ru



МАТВЕЕВ М. Д.

СПРАВОЧНИК ЛИНУКСОИДА

ВСЕ, ЧТО НУЖНО, ПОД РУКОЙ



"Издательство Наука и Техника"

г. Санкт-Петербург

УДК 004.42 ББК 32.973

ISBN 978-5-907592-17-9

Матвеев М. Д.

Справочник линуксоида. Все, что нужно, под рукой. — СПб.: Издательство Наука и Техника, 2023. — 352 с., ил.

Серия "Справочник"

В этом справочнике вы найдете ключевую информацию об операционной системе **Linux** в удобной и наглядной форме. Структура справочника позволяет линуксоидам быстро находить нужную информацию, получать примеры использования тех или иных элементов Linux.

Диапазон материалов справочника достаточно широк: от самых основ (ж-мандная строка, локальная сеть, файловая система, пользователи и группы) до более продвинутых тем (управление загрузкой ос, работа с хранилищем, управление процессами Linux).

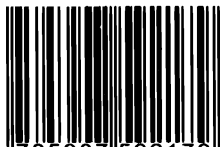
Отдельное внимание уделено внимание таким важным темам, как: ядро Linux, маршрутизация и настройка брандмауэра, командный интерпретатор bash.

Книга будет полезна как начинающим линуксоидам, так и более опытным пользователям, которым в процессе работы в Linux нужна информация под рукой.

Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Издательство не несет ответственности за возможный ущерб, причиненный в ходе использования материалов данной книги, а также за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими. Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги.

ISBN 978-5-907592-17-9



9 785907 592179 >

Контактные телефоны издательства:

(812) 412 70 26

Официальный сайт: www.nit.com.ru

© Матвеев М. Д.

© Издательство Наука и Техника (оригинал-макет)

СОДЕРЖАНИЕ

ГЛАВА 1. ОСНОВЫ КОМАНДНОЙ СТРОКИ.....11

1.1. ВВОД КОМАНД.....	12
1.2. АВТОДОПОЛНЕНИЕ КОМАНДНОЙ СТРОКИ	13
1.3. ПЕРЕНАПРАВЛЕНИЕ ВВОДА/ВЫВОДА	14
1.4. СПРАВОЧНАЯ СИСТЕМА <i>MAN</i>	16
1.5. КОМАНДЫ ДЛЯ РАБОТЫ С ФАЙЛАМИ И КАТАЛОГАМИ.....	16
1.5.1. Команды для работы с файлами.....	16
1.5.2. Команды для работы с каталогами	20
1.6. КОМАНДЫ СИСТЕМНОГО АДМИНИСТРАТОРА.....	23
1.6.1. Команды для работы с устройствами и драйверами.....	23
1.6.2. Команды настройки сетевых интерфейсов	25
1.6.3. Программы тестирования и настройки жесткого диска	26
1.7. КОМАНДЫ ОБРАБОТКИ ТЕКСТА.....	27
1.7.1. Редактор <i>sed</i>	27
1.7.2. Подсчет количества слов/символов	29
1.7.3. Сравнение файлов	29
1.7.4. Разбивка текста на колонки	30

1.7.5. Команды <i>diff</i> и <i>diff3</i>	31
1.7.6. Команда <i>grep</i>	34
1.7.7. Замена символов табуляции пробелами	34
1.7.8. Форматирование текста	34
1.7.9. Команды постраничного вывода <i>more</i> и <i>less</i>	35
1.7.10. Команды <i>head</i> и <i>tail</i> : вывод первых и последних строк файла	35
1.7.11. Команда <i>split</i>	36
1.7.12. Команда <i>unexpand</i>	37

ГЛАВА 2. ЛОКАЛЬНАЯ СЕТЬ.....39

2.1. ФИЗИЧЕСКАЯ НАСТРОЙКА СЕТИ ETHERNET	40
2.2. НАСТРОЙКА СЕТИ С ПОМОЩЬЮ ГРАФИЧЕСКОГО КОНФИГУРАТОРА	43
2.3. КОМАНДА <i>IFCONFIG</i>	48
2.4. ИМЕНА СЕТЕВЫХ ИНТЕРФЕЙСОВ В LINUX	52
2.5. ОБЩИЕ КОНФИГУРАЦИОННЫЕ ФАЙЛЫ	56
Файл /etc/hosts	56
Файлы /etc/hosts.allow и /etc/hosts.deny	56
Файл /etc/host.conf	57
Файл /etc/hostname	57
Файл /etc/motd	57
Файл /etc/resolv.conf	58
Файл /etc/services	59
Файл /etc/protocols	59
Файл /etc/network/interfaces: конфигурация сети в Astra Linux	59
Каталог /etc/NetworkManager/system-connections: конфигурация сети в Ubuntu	60

ГЛАВА 3. УСТАНОВКА ПРОГРАММ В LINUX.....63

3.1. СПОСОБЫ УСТАНОВКИ ПРОГРАММ.....	64
3.2. ТИПЫ ПАКЕТОВ И ИХ СОДЕРЖИМОЕ	66
3.3. ИСТОЧНИКИ ПАКЕТОВ.....	67
3.4. МЕНЕДЖЕРЫ ПАКЕТОВ	68
3.5. ГРАФИЧЕСКИЕ СРЕДСТВА УСТАНОВКИ ПРОГРАММ	75
3.6. СНАПЫ	79
3.7. ОШИБКА ПРИ ВЫПОЛНЕНИИ ART: <i>UNABLE TO ACQUIRE THE DPKG LOCK /VAR/LIB/DPKG/LOCK</i>	83
3.8. НЕВОЗМОЖНО НАЙТИ ОПРЕДЕЛЕННЫЙ ПАКЕТ	86

ГЛАВА 4. ПОПУЛЯРНЫЕ ПРОГРАММЫ.....89

4.1. ОФИСНЫЕ ПАКЕТЫ	90
4.2. ГРАФИЧЕСКИЕ ТЕКСТОВЫЕ РЕДАКТОРЫ.....	91
4.3. КОНСОЛЬНЫЕ ТЕКСТОВЫЕ РЕДАКТОРЫ.....	93
4.4. ПРОГРАММЫ ДЛЯ РАБОТЫ С ИНТЕРНЕТОМ.....	98
4.5. LINUX-АНАЛОГИ WINDOWS-ПРОГРАММ	99

ГЛАВА 5. ФАЙЛОВАЯ СИСТЕМА.....113

5.1. КАКИЕ ФАЙЛОВЫЕ СИСТЕМЫ ПОДДЕРЖИВАЕТ LINUX	114
5.2. КАКУЮ ФАЙЛОВУЮ СИСТЕМУ ВЫБРАТЬ?.....	117
5.3. ЧТО НУЖНО ЗНАТЬ О ФАЙЛОВОЙ СИСТЕМЕ LINUX.....	118

5.3.1. Имена файлов и каталогов.....	118
5.3.2. Файлы устройств.....	119
5.3.3. Корневая файловая система и основные подкаталоги первого уровня	120
5.4. ССЫЛКИ	122
5.5. ПРАВА ДОСТУПА	123
5.5.1. Общие положения	123
5.5.2. Смена владельца файла.....	124
5.5.3. Определение прав доступа	125
5.5.4. Специальные права доступа.....	128
5.6. АТРИБУТЫ ФАЙЛА	128
5.7. ПОИСК ФАЙЛОВ.....	130
5.8. МОНТИРОВАНИЕ ФАЙЛОВЫХ СИСТЕМ.....	133
5.8.1. Монтируем файловые системы вручную.....	133
5.8.2. Имена устройств.....	136
5.8.3. Монтируем файловые системы при загрузке.....	139
5.8.4. Автоматическое монтирование файловых систем	141
5.9. РАБОТА С ЖУРНАЛОМ	142
5.10. ПРЕИМУЩЕСТВА ФАЙЛОВОЙ СИСТЕМЫ <i>EXT4</i>	143
5.11. СПЕЦИАЛЬНЫЕ ОПЕРАЦИИ С ФАЙЛОВОЙ СИСТЕМОЙ	144
5.11.1 Монтирование NTFS-разделов.....	144
5.11.2. Создание файла подкачки	145
5.11.3. Файлы с файловой системой	146
5.11.4. Создание и монтирование ISO-образов.....	147
5.12. ФАЙЛЫ КОНФИГУРАЦИИ LINUX	148

5.12.1. Содержимое каталога <code>/etc</code>	148
5.12.2. Конфигурационные файлы	149
5.12.3. Подкаталоги с конфигурационными файлами	159
5.13. ПСЕВДОФАЙЛОВЫЕ СИСТЕМЫ	169
5.13.1. Псевдофайловая система <code>sysfs</code>	170
5.13.2. Псевдофайловая система <code>proc</code>	172
ГЛАВА 6. УПРАВЛЕНИЕ ХРАНИЛИЩЕМ.....	177
6.1. ПОДКЛЮЧЕНИЕ НОВОГО ЖЕСТКОГО ДИСКА И ЕГО РАЗМЕТКА ..	178
6.2. МЕНЕДЖЕР ЛОГИЧЕСКИХ ТОМОВ	186
6.2.1. Введение в LVM	186
6.2.2. Уровни абстракции LVM	188
6.2.3. Немного практики	188
6.3. РАСШИРЕНИЕ LVM-ПРОСТРАНСТВА.....	192
ГЛАВА 7. УПРАВЛЕНИЕ ЗАГРУЗКОЙ ОС.....	197
7.1. ЗАГРУЗЧИКИ LINUX.....	198
7.2. ЗАГРУЗЧИК GRUB2.....	199
7.2.1. Конфигурационные файлы	199
7.2.2. Выбор метки по умолчанию	207
7.2.3. Загрузка Windows	208
7.2.4. Пароль загрузчика GRUB2	209
7.2.5. Установка загрузчика	211
7.3. СИСТЕМА ИНИЦИАЛИЗАЦИИ	212
7.3.1. Принцип работы	213

7.3.2. Конфигурационные файлы <i>systemd</i>	216
7.3.3. Цели	219

7.4. УПРАВЛЕНИЕ СЕРВИСАМИ ПРИ ИСПОЛЬЗОВАНИИ *SYSTEMD*..221

ГЛАВА 8. УПРАВЛЕНИЕ ПРОЦЕССАМИ.....223

8.1. КОМАНДЫ <i>PS</i> , <i>NICE</i> И <i>KILL</i>	224
8.1.1. Получение информации о процессе	224
8.1.2. Изменение приоритета процесса	230
8.1.3. Аварийное завершение процесса.....	230
8.2. КОМАНДА <i>TOP</i>	233
8.3. ИНФОРМАЦИЯ ОБ ИСПОЛЬЗОВАНИИ ПАМЯТИ И ДИСКОВОГО ПРОСТРАНСТВА	235
8.4. КОМАНДА <i>FUSER</i>	237
8.5. ПЛАНИРОВЩИКИ ЗАДАНИЙ	238
8.5.1. Планировщик <i>cron</i>	238
8.5.2. Планировщик <i>anacron</i>	241

ГЛАВА 9. ПОЛЬЗОВАТЕЛИ И ГРУППЫ.....243

9.1. ВВЕДЕНИЕ В УЧЕТНЫЕ ЗАПИСИ <i>LINUX</i>	244
9.2. ПОЛУЧЕНИЕ ПОЛНОМОЧИЙ <i>ROOT</i>	248
9.3. УПРАВЛЕНИЕ УЧЕТНЫМИ ЗАПИСЯМИ ПОЛЬЗОВАТЕЛЕЙ.....	255
9.3.1. Создание учетной записи пользователя	255
9.3.2. Файлы <i>/etc/passwd</i> и <i>/etc/shadow</i>	257
9.3.3. Изменение и удаление учетных записей	261
9.3.4. Группы пользователей.....	266

9.4. ГРАФИЧЕСКИЕ КОНФИГУРАТОРЫ.....	266
9.5. МОДУЛИ РАМ	268
9.5.1. Ограничиваем доступ к системе по IP-адресу.....	272
9.5.2. Ограничиваем время входа в систему	274
9.5.3. Ограничение системных ресурсов с помощью РАМ	275

ГЛАВА 10. ЯДРО LINUX.....279

10.1. ЧТО ТАКОЕ ЯДРО	280
10.2. ПАРАМЕТРЫ ЯДРА	284
10.3. ОБНОВЛЕНИЕ ЯДРА ДО ВЕРСИИ 5.7	288

ГЛАВА 11. МАРШРУТИЗАЦИЯ И НАСТРОЙКА БРАНДМАУЭРА.....291

11.1. ПРОСМОТР ТАБЛИЦЫ МАРШРУТИЗАЦИИ	292
11.2. ИЗМЕНЕНИЕ И СОХРАНЕНИЕ ТАБЛИЦЫ МАРШРУТИЗАЦИИ..	295
11.3. НАСТРОЙКА БРАНДМАУЭРА <i>IPTABLES</i>	302
11.3.1. Преобразование сетевого адреса.....	302
11.3.2. Цепочки и правила	303
11.3.3. Команда <i>iptables</i>	305
11.3.4. Практический пример	309
11.4. НАСТРОЙКА БРАНДМАУЭРА <i>UFW</i>.....	316
11.4.1. Проверяем состояние брандмауэра.....	316
11.4.2. Базовая настройка.....	317
11.4.3. Создаем правила для других приложений	319
11.4.4. Разрешаем IP-адреса	320

11.4.5. Запрещаем IP-адреса и службы.....	321
11.4.6. Удаление/сброс правил	321
11.4.7. Отключение файрвола.....	321

ПРИЛОЖЕНИЕ 1. КОМАНДНЫЙ ИНТЕРПРЕТАТОР BASH.....323

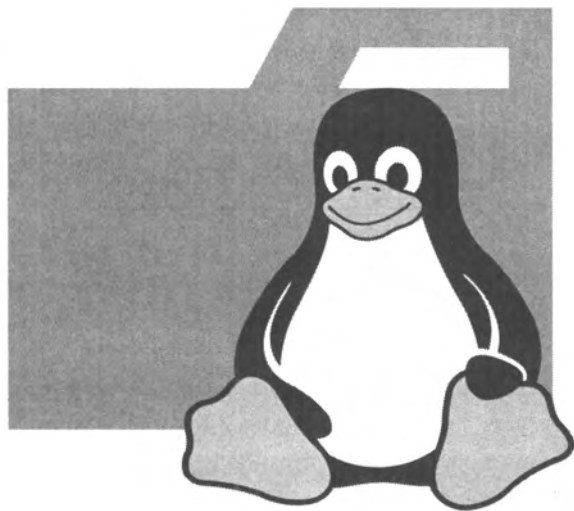
П1.1. НАСТРОЙКА BASH.....	324
П1.2. ЗАЧЕМ НУЖНЫ СЦЕНАРИИ <i>BASH</i>	329
П1.3. СЦЕНАРИЙ "ПРИВЕТ, МИР!"	330
П1.4. ПЕРЕМЕННЫЕ В СЦЕНАРИЯХ.....	331
П1.5. ПЕРЕДАЕМ ПАРАМЕТРЫ СЦЕНАРИЮ.....	333
П1.6. ОБРАБОТКА МАССИВОВ	335
П1.7. ЦИКЛЫ <i>FOR</i> И <i>WHILE</i>	335
П1.8. УСЛОВНЫЕ ОПЕРАТОРЫ	337
П1.9. ФУНКЦИИ В <i>BASH</i>	339
П1.10. ПРАКТИЧЕСКИЕ ПРИМЕРЫ СЦЕНАРИЕВ.....	340
Проверка прав пользователя.....	340
Проверка свободного дискового пространства с уведомлением по e-mail...	341

ПРИЛОЖЕНИЕ 2. СЕТЕВАЯ ФАЙЛОВАЯ СИСТЕМА NFS...343

П2.1. ВКРАТЦЕ О NFS И УСТАНОВКЕ НЕОБХОДИМЫХ ПАКЕТОВ	344
П2.2. ФАЙЛ /ETC/EXPORTS	345
П2.3. ИСПОЛЬЗОВАНИЕ NFS-КЛИЕНТА	348
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ ИНФОРМАЦИИ	350

ГЛАВА 1.

ОСНОВЫ КОМАНДНОЙ СТРОКИ





Командная строка — неотъемлемая часть Linux.

Посредством командной строки осуществляется выполнение команд Linux. Эффективно сможет работать с Linux только тот, кто освоил принципы работы в командной строке. Именно поэтому наше справочное руководство хотелось бы начать с этой главы.

1.1. Ввод команд

Ввод команд осуществляется в приложении **Терминал**. Это эмулятор консоли Linux, позволяющий вводить команды. Можно переключиться из графического режима в консоль (нажав Ctrl + Alt + F1) и вводить команды непосредственно в консоли Linux. Но для большинства пользователей будет удобнее работа с эмулятором терминала в графическом режиме. Приложение *Терминал Fly* изображено на рис. 1.1.

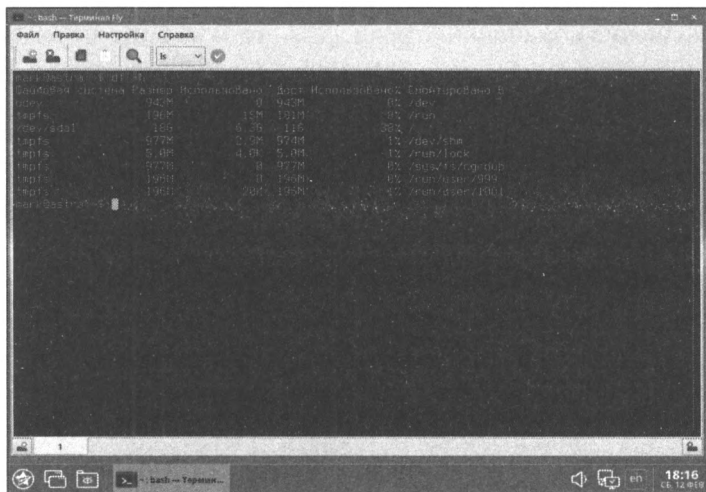


Рис. 1.1. Приложение Терминал Fly

Для ввода команды нужно ввести ее в приглашение командной строки и нажать **Enter**. После чего вы увидите результат выполнения команды. Обычно приглашение командной строки имеет вид:

```
пользователь@компьютер:текущий_каталог<$|#>
```

Символ \$ обозначает, что вводимая команда будет выполняться с привилегиями обычного пользователя. Если команда выполняется с привилегиями суперпользователя *root*, то символ приглашения командной строки меняется на #.

1.2. Автодополнение командной строки

Linux содержит множество команд. Если вы забыли точное название команды или просто хотите ускорить ее ввод, вы можете использовать



автодополнение командной строки. Для этого введите первые буквы названия команды и нажмите **Tab**. Далее система или автоматически дополнит команду, или выведет список доступных вариантов, если доступно несколько вариантов по введенной команде.

Аналогично, автодополнение может использоваться для имен файлов и каталогов. Например, вы хотите перейти в каталог `applications/xnfbdh73/public_html`. Вместо того чтобы вводить этот длинный путь (и помнить его!), вводите команду так:

```
cd a <нажмите Tab> / x <нажмите Tab> / p <нажмите Tab>
```

В конечном итоге вместо ввода 33 символов пути вам нужно будет ввести 5!

1.3. Перенаправление ввода/вывода

Рассмотрим следующую команду:

```
cat very_long_file.txt | less
```

Здесь мы пытаемся просмотреть очень длинный файл `very_long_file.txt`. Поскольку он очень длинный и не помещается на одном экране, мы перенаправили вывод первой части команды (`cat very_long_file.txt`) на стандартный вывод команды `less`, которая обеспечивает удобный просмотр длинных файлов.

Синтаксис следующий:

```
команда_1 | команда_2
```



Особых ограничений не существует, и вы можете передать вывод второй команды на ввод третьей и т.д.:

```
команда_1 | команда_2 | команда_3
```

С помощью такого перенаправления можно автоматизировать некоторые команды, что важно в сценариях **bash**, например, вот как можно утвердительно ответить на запрос об удалении файла:

```
echo y | rm file.old
```

Очень часто перенаправление ввода/вывода используется в таком контексте, в котором использовали его мы: большой вывод перенаправляется на программу просмотра (**less**) или на программу-фильтр, такую как **grep**.

Кроме перенаправления вывода программы на ввод другой программы, его можно перенаправить в файл, например:

```
ps -A > processes.txt
```

Если файл **processes.txt** не существует, он будет создан. Если существует — перезаписан. Если нужно дописать вывод программы в конец файла, не удаляя существующий файл, то используйте два знака больше:

```
ps -A >> processes.txt
```

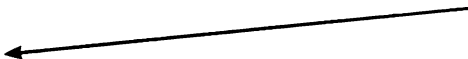
В этом случае, если файл не существует, то он будет создан, а если существует, то информация будет дописана в конец файла.



1.4. Справочная система *man*

В Linux справочная система всегда под рукой. Например, вы забыли параметры команды *df*, просто введите команду:

```
man df
```



Откроется страница руководства (в большинстве случаев — на русском языке), в котором будут описаны все возможные параметры по интересующей вас команде и даны рекомендации по их применению. Для работы справочной системы не требуется соединение с Интернетом, поскольку все страницы справочного руководства уже загружены на ваш компьютер.

Далее будут рассмотрены некоторые полезные команды, знание которых просто обязательно для каждого пользователя Linux.

1.5. Команды для работы с файлами и каталогами

1.5.1. Команды для работы с файлами

В каждой операционной системе есть команды для работы с файлами и каталогами. Linux — не исключение. Рассмотрим стандартные команды Linux для работы с файлами (см. табл. 1.1).

**Таблица 1.1. Стандартные команды Linux для работы с файлами**

Команда	Описание
<code>cat файл</code>	Выводит текстовый файл. Файлы могут быть довольно длинными, поэтому лучше использовать ее в паре с командой <code>less</code> , например, <code>cat /etc/services less</code>
<code>tac файл</code>	Подобна команде <code>cat</code> , но выводит файл в обратном порядке. Данная команда удобна для чтения журналов, в которых самые свежие сообщения заносятся в конец файла, например, <code>tac /var/log/messages less</code>
<code>cp файл1 файл2</code>	Копирует файл1 в файл2. Если второй файл существует, программа спросит вас, нужно ли его перезаписать
<code>mv файл1 файл2</code>	Используется для перемещения файла1 в файл2. Можно использовать для переименования файлов
<code>rm файл</code>	Удаляет файл
<code>touch файл</code>	Используется для создания нового пустого файла
<code>locate файл</code>	Быстрый поиск файла. Позже мы рассмотрим процесс поиска подробнее



<code>which</code> исполнимый_файл	Производит быстрый поиск программы (исполнимого файла). Если программа находится в пути PATH, то <i>which</i> выведет каталог, в котором находится программа
------------------------------------	--

В таблице 1.1 представлены основные команды, которые используются для создания (*touch*), копирования (*cp*), перемещения (*mv*) и удаления (*rm*) файлов, а также несколько дополнительных команд.

Рассмотрим несколько примеров:

```
$ dmesg > kernel.messages
$ cat kernel.messages | less
$ cp kernel.messages krn.msg
$ rm kernel.messages
```

Первая команда выводит загрузочные сообщения ядра в файл *kernel.messages*. Вторая выводит содержимое этого файла на экран, а команда *less* организует удобный постраничный просмотр этого файла. Далее команда *cp* копирует файл *kernel.messages* в файл *krn.msg*, а последняя команда удаляет наш исходный файл *kernel.messages*. В принципе, вместо последних двух команд можно было использовать одну команду *mv*:

```
mv kernel.messages krn.msg
```

При указании имени файла вы можете использовать маски *** и *?*. Как обычно, символ *** заменяет любую последовательность символов, а *?* — один символ.

Например:



```
rm *.tmp  
rm /tmp/*  
cp *.txt /media/ext-usb  
cp ????.txt /media/ext-usb
```

Первая команда удаляет все файлы, заканчивающиеся на «.tmp», в текущем каталоге.

Вторая — удаляет все файлы из каталога /tmp.

Третья копирует все файлы с «расширением» .txt из текущего каталога в каталог /media/ext-usb.

Четвертая команда копирует все файлы, имя которых состоит из трех любых символов и заканчивается на «.txt», например, abc.txt, в каталог /media/ext-usb.

ПРИМЕЧАНИЕ. Как вы уже догадались, к каталогу /media/ext-usb можно подмонтировать внешний USB-диск, и тогда копируемые файлы физически окажутся на внешнем жестком диске. Подробнее о монтировании мы поговорим в других главах справочника.

ПРИМЕЧАНИЕ. Обратите внимание, что в таблице 1.1 команды представлены без параметров. Хотя дополнительные параметры есть у каждой команды. Вы не обязаны помнить все параметры, для этого есть справочная система *man*. Вам нужно помнить только названия команд, а параметры вы всегда сможете «подсмотреть» в *man*.

Мы не рассмотрели команды редактирования текстовых файлов. Они не являются стандартными (кроме программы *vi*, которой пользоваться очень неудобно), но в вашей системе по умолчанию могут быть установлены следующие текстовые редакторы:



- **nano** — удобный текстовый редактор;
- **joe** — небольшой и удобный текстовый редактор;
- **pico** — текстовый редактор, устанавливаемый вместе с почтовым клиентом **pine**;
- **mcedit** — текстовый редактор, устанавливаемый вместе с файловым менеджером **mc**.

Если у вас нет этих редакторов, вы можете установить их. Например, установите **mc** — вы получите и файловый менеджер, и текстовый редактор сразу:

```
sudo apt install mc
```

1.5.2. Команды для работы с каталогами

Аналогично командам для работы с файлами, команды для работы с каталогами представлены в таблице 1.2.

Таблица 1.2. Стандартные команды Linux для работы с каталогами

Команда	Описание
<code>cd каталог</code>	Изменение каталога
<code>ls каталог</code>	Выводит содержимое каталога
<code>rmdir каталог</code>	Удаляет пустой каталог
<code>rm -r каталог</code>	Рекурсивное удаление непустого каталога
<code>mkdir каталог</code>	Создает каталог



<code>cp каталог1 каталог2</code>	Команду <i>cp</i> можно использовать и для копирования каталогов. В данном случае <i>cp</i> копирует <i>каталог1</i> в <i>каталог2</i>
<code>mv каталог1 каталог2</code>	Команду <i>mv</i> можно использовать и для перемещения каталогов. В данном случае <i>mv</i> перемещает <i>каталог1</i> в <i>каталог2</i>

Обратите внимание, что команда *rmdir* не может удалить непустой каталог, поэтому если не хотите удалять сначала файлы и подкаталоги из удаляемого каталога, то лучше использовать команду:

```
rm -r каталог
```



Например:

```
$ mkdir /home/bagira/test
$ touch /home/bagira/test/test-file
$ rm -r /home/bagira/test
```

Как и в случае с командой *rm*, вы можете задать параметр *-r* для команд *cp* и *mv*. В этом случае операция копирования или перемещения будет выполняться рекурсивно.

Очень важной операцией является просмотр содержимого каталога, для которого используется команда *ls*.

Поэтому сейчас сделаем исключение для этой команды и рассмотрим ее параметры (табл. 1.3). А общий формат вызова этой команды таков:

```
ls [параметры] [каталог]
```

Таблица 1.3. Параметры команды *ls*

Параметр	Описание
-C	Выводит список файлов в колонках с вертикальной сортировкой
-F	Для каждого каталога добавлять суффикс '/', а для каждого исполняемого файла — '*', а для каждого FIFO-канала — ' '
-R	Рекурсивный вывод, то есть команда <i>ls</i> будет выводить не только содержимое каталога, но и подкаталогов
-a	Показывать скрытые файлы
-i	Показывать иноды для каждого файла (будет показан серийный номер файла)
-l	«Длинный» формат вывода, в котором отображается тип файла, права доступа, количество ссылок на файл, имя владельца, имя группы, размер файла, метка времени создания файла и имя файла. В колонке типа файла могут быть следующие значения: d (каталог), b (блочное устройство), c (символьное устройство), l (символическая ссылка), r (FIFO-канал), s (сокет)
-r	Сортировка в обратном порядке



В таблице 1.3 приведены не все параметры команды *ls*, а только самые основные.

При задании имени каталога можно использовать следующие специальные имена:

- `.` — обозначает текущий каталог.
- `..` — обозначает родительский каталог.
- `~` — домашний каталог пользователя, например, если вы вошли под пользователем *bagira*, то путь `~/file.txt` равноценен `/home/bagira/file.txt`.

1.6. Команды системного администратора

Существуют команды, которые нужно знать не только обычному линуксоиду, но и, желательно, каждому системному администратору. В этой главе рассматривается необходимый минимум таких команд. Нужно отметить, что команд системного администратора гораздо больше в Linux, по сути, в каждой главе мы рассматриваем те или иные команды администратора. В этой главе мы рассмотрим некоторые базовые команды. Возможно, они не пригодятся вам прямо сейчас, но вы еще не раз вернетесь к этой главе в будущем.

1.6.1. Команды для работы с устройствами и драйверами

В таблице 1.4 приведены некоторые команды, которые помогут вам обнаружить аппаратную проблему — проблему с устройством либо с его драйвером.



Таблица 1.4. Команды, предоставляющие информацию об устройствах

Команда	Описание
<code>uname -a</code>	Очень важная команда, сообщающая версию ядра. Очень важно, чтобы устанавливаемые модули были откомпилированы под вашу версию ядра
<code>lsdev</code>	Выводит информацию об устройствах. По умолчанию эта команда не установлена, нужно установить пакет <code>procinfo</code>
<code>lshal</code>	Выводит параметры всех устройств
<code>lspci, lsusb, lshw</code>	Выводят соответственно список PCI-устройств, USB-устройств и список оборудования компьютера
<code>lsmod</code>	Выводит список загруженных модулей ядра
<code>dmidecode</code>	Отображает информацию о BIOS компьютера
<code>cat /proc/cupinfo</code>	Выводит информацию о процессоре
<code>cat /proc/meminfo</code>	Отображает информацию о памяти
<code>cat /proc/mounts</code>	Показывает точки монтирования



<code>cat /proc/net/dev</code>	Выводит сетевые интерфейсы и статистику по ним
<code>cat /proc/version</code>	Похожа на <code>uname</code> , выводит версию ядра
<code>cat /proc/interrupts</code>	Отображает информацию по прерываниям
<code>cat /proc/swaps</code>	Выводит информацию о файлах подкачки

1.6.2. Команды настройки сетевых интерфейсов

Подробно настройка сети будет рассматриваться в следующей главе, а пока рассмотрим таблицу 1.5, в которой представлен короткий список команд, которые вам могут пригодиться при настройке сети.

Таблица 1.5. Некоторые команды настройки сети

Команда	Описание
<code>route</code>	Просмотр и изменение таблицы маршрутизации
<code>dmesg less</code>	Просмотр сообщений ядра, которые выводятся ядром при загрузке системы
<code>iwconfig</code>	Выводит информацию обо всех беспроводных интерфейсах



<code>iwlist scan</code>	Поиск беспроводных сетей
<code>dhclient wlan0</code>	Обновляет IP-адрес и другую сетевую информацию беспроводного интерфейса <code>wlan0</code>
<code>iwevent</code>	Просмотреть события беспроводной сети
<code>sudo /etc/init.d/dbus restart</code>	Перезапуск всех сетевых служб (работает не во всех дистрибутивах)
<code>sudo systemctl restart <служба> или service <служба> restart</code>	Перезапуск службы. Например, <code>sudo systemctl restart networking</code> перезапускает сеть

1.6.3. Программы тестирования и настройки жесткого диска

Команды для тестирования и настройки жесткого диска, подобно ранее приведенным командам, также представлены в виде таблицы — табл. 1.6.

Таблица 1.6. Команды для тестирования и настройки жесткого диска

Команда	Описание
<code>badblocks -v <имя_устройства></code>	Осуществляет проверку жесткого диска на наличие «плохих» блоков. Параметр <code>-v</code> включает подробный отчет.



hdparm	Тестирование производительности и настройка жесткого диска. Например, параметр <code>-t</code> может протестировать производительность (<code>hdparm -t /dev/sda</code>), а параметр <code>-E</code> установить скорость привода CD/DVD: <code>hdparm -E 2 /dev/sr0</code>
hddtemp	Отображает температуру жесткого диска
bonnie	Тестирует производительность жесткого диска
cpuburn	Тестирование процессора (стресс-тест процессора)
screentest	Тестирование и настройка монитора
smartmontools	SMART-мониторинг. Нужно, чтобы ваши жесткие диски поддерживали S.M.A.R.T

1.7. Команды обработки текста

1.7.1. Редактор *sed*

Команда *sed* — мощный потоковый редактор, и ему можно было посвятить целую главу, но не вижу такой необходимости, поскольку



ку в современных дистрибутивах имеется документация на русском языке. Главное знать, что такая программа есть. А чтобы вы заинтересовались, давайте рассмотрим несколько примеров использования этой программы:

Заменить строку «string1» на «string2» в файле report.txt, результат вывести на стандартное устройство вывода:

```
sed 's/string1/string2/g' report.txt
```

Вывести пятую строку файла report.txt:

```
sed -n '5p;5q' report.txt
```

Удалить пустые строки из файла:

```
sed '/^$/d' report.txt
```

Удалить строку «string1» из текста, не изменяя всего остального:

```
sed -e 's/string1//g' report.txt
```

Удалить пустые символы в конце каждой строки:

```
sed -e 's/ *$//' report.txt
```

Удалить пустые строки и комментарии из файла:

```
sed '/ *#/d; /^$/d' report.txt
```

Преобразовать символы из нижнего регистра в верхний:



```
echo 'test' | tr '[:lower:]' '[:upper:]'
```

Удалить первую строку из файла:

```
sed -e '1d' report.txt
```

1.7.2. Подсчет количества слов/символов

Команда *wc* используется:

- Для подсчета слов в текстовом файле:

```
wc /var/log/messages
```
- Для подсчета количества строк (если задан параметр *-l*):

```
wc -l /var/log/messages
```
- Для подсчета количества символов (параметр *-c*):

```
wc -c /var/log/messages
```

1.7.3. Сравнение файлов

Команда *cmp* используется для сравнения двух файлов. Если файлы идентичны, то *cmp* вообще никак не реагирует. А вот если файлы отличаются, то *cmp* выводит номер строки и номер символа в строке, откуда начинается различие.

Команда *cmp* более универсальна, поскольку она может использоваться для сравнения как текстовых, так и двоичных файлов. А вот команда *diff* и ее аналоги умеют сравнивать только текстовые файлы.

Формат вызова команды следующий:

```
cmp [параметры] файл1 файл2
```





Параметры команды `str` указаны в табл. 1.7.

Таблица 1.7. Параметры команды `str`

Параметр	Описание
<code>-c</code>	Вывод отличающихся символов
<code>-i n</code>	Игнорировать первые <code>n</code> символов
<code>-l</code>	Вывод позиций всех отличий, а не только первого
<code>-s</code>	Не выводить информацию на экран, при этом код возврата будет следующим: 0 — файлы одинаковые; 1 — файлы отличаются; 2 — ошибка при открытии одного из файлов

1.7.4. Разбивка текста на колонки

Команда `column` используется для разбивки текста на несколько столбцов.

Текст может быть прочитан как из файла, так и со стандартного ввода, если файл не указан.

Формат вызова команды:

```
column [параметры] [файл]
```

Параметры команды `column` приведены в табл. 1.8.

Таблица 1.8. Параметры команды *column*

Параметр	Описание
-c <i>n</i>	Задаёт количество столбцов (число <i>n</i>)
-s СИМВОЛ	Указанный символ будет использоваться в качестве разделителя столбцов
-t	Текст будет форматироваться как таблицы. По умолчанию разделителем полей считается пробел, но с помощью параметра -s можно задать другой разделитель
-x	Сначала будут заполняться столбцы, а потом строки

1.7.5. Команды *diff* и *diff3*

Команды используются для сравнения двух (*diff*) и трех (*diff3*) файлов.

Формат вызова программы *diff*:

```
diff [параметры] файл1 файл2
```

В выводе программы отличающиеся строки помечаются символами **>** и **<**:

- Строка из первого файла помечается символом **<**;
- Строка из второго файла — символом **>**.



Самые полезные параметры программы *diff* приведены в табл. 1.9.

Таблица 1.9. Параметры команды *diff*

Параметр	Описание
-a	Сравнение всех файлов, в том числе бинарных
-b	Программа будет игнорировать пробельные символы в конце строки
-B	Игнорирует пустые строки
-e	Применяется для создания сценария для редактора ed , который будет использоваться для превращения первого файла во второй
-w	Игнорирует пробельные символы
-y	Вывод в два столбца
-r	Используется для сравнения файлов в подкаталогах. Вместо первого файла указывается первый каталог, вместо второго файла — соответственно второй каталог

Команда *diff3* похожа на *diff*, только используется для сравнения трех файлов. Формат вызова команды таков:

```
diff3 [параметры] файл1 файл2 файл3
```

Программа выводит следующую информацию:

- ===== — все три файла разные;



- `===1` — первый файл отличается от второго и третьего;
- `===2` — второй файл отличается от первого и третьего;
- `===3` — третий файл отличается от первого и второго.

Параметры команды `diff3` указаны в таблице 1.10.

Таблица 1.10. Параметры команды `diff3`

Параметр	Описание
<code>-a</code>	Сравнивать файлы как текстовые, даже если они являются бинарными
<code>-A</code>	Создание сценария для редактора ed , который показывает в квадратных скобках все отличия между файлами
<code>-e</code>	Создает сценарий для ed , который помещает все отличия между файлами файл2 и файл3 в файл файл1 (будьте осторожны!)
<code>-i</code>	Добавить команды <i>w</i> (сохранить файл) и <i>q</i> (выйти) в конец сценария <i>ed</i>
<code>-x</code>	Создание сценария редактора ed , который помещает отличия между файлами в файл файл1
<code>-X</code>	То же, что и <code>-x</code> , но отличия выделяются
<code>-3</code>	Создает сценарий <i>ed</i> , который помещает все различия между файлами файл1 и файл3 в файл1



1.7.6. Команда *grep*

Предположим, что у нас есть файл какой-то большой файл, и мы хотим найти в нем все упоминания строки *hello*. Сделать это можно так:

```
cat file.txt | grep hello
```

Команда `cat file.txt` передаст содержимое файла `file.txt` на стандартный ввод команды *grep*, которая, в свою очередь, выделит строки, содержащие строку *hello*.

1.7.7. Замена символов табуляции пробелами

Команда *expand* заменяет в указанных файлах символы табуляции на соответствующее количество пробелов.

Команде можно передать лишь один параметр *-i*, означающий, что замена должна быть только в начале строки.

Формат вызова команды:

```
expand [-i] файлы
```

1.7.8. Форматирование текста

Команда *fmt* форматирует текст, выравнивает его по правой границе и удаляет символы новой строки. Синтаксис вызова команды:

```
fmt [параметры] файлы
```



Параметры команды *fmt* приведены в табл. 1.11.

Таблица 1.11. Параметры команды *fmt*

Параметр	Описание
-c	Не форматировать первые две строки
-p префикс	Форматировать только строки, начинающиеся с указанного префикса
-s	Не объединять строки
-t	Начинать параграф с красной строки
-w n	Задаёт максимальную длину строки в n символов (по умолчанию 72)

1.7.9. Команды постраничного вывода *more* и *less*

Большой текстовый файл намного удобнее просматривать с помощью команд *less* или *more*. Программа *less* удобнее, чем *more*, если она есть в вашей системе:

```
tac /var/log/messages | grep ppp | less
```

1.7.10. Команды *head* и *tail*: вывод первых и последних строк файла

Команда *head* выводит первые десять строк файла, а *tail* — последние десять.



Количество строк может регулироваться с помощью параметра *-n*.

Пример использования:

```
head -n 10 /var/log/messages  
tail -n 15 /var/log/messages
```

1.7.11. Команда *split*

Команда *split* используется для разделения файлов на части.

По умолчанию создаются части размером в 1000 строк. Изменить размер можно, указав количество строк, например:

```
split -200 файл1
```

В данном случае файл будет разбит на части по 200 строк в каждой (кроме, возможно, последней части, где может быть меньше строк).

Команду можно также использовать для разделения файлов на части по размеру информации, а не по количеству строк, например, с помощью параметра *-b* можно указать количество символов в каждой части. Примеры вызова команды:

```
split -b100b файл  
split -b100k файл  
split -b100m файл
```

Первая команда разделит файл на части по 100 байтов каждая, вторая — на части по 100 Кбайт каждая, третья — по 100 Мбайт каждая.



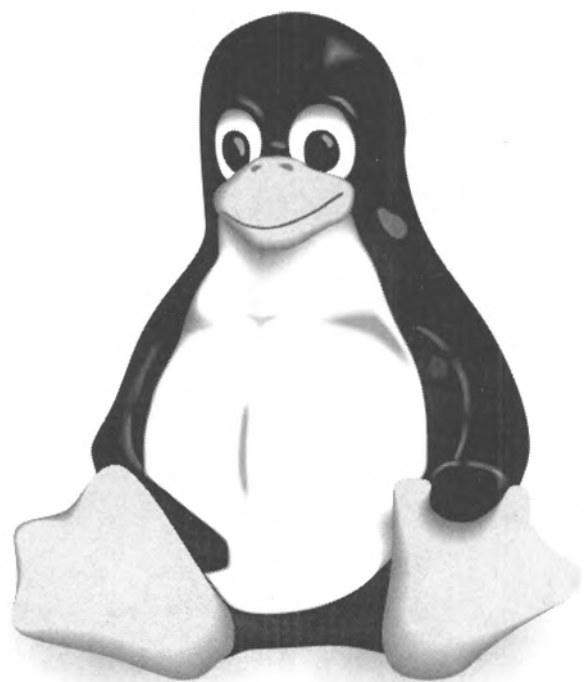
1.7.12. Команда *unexpand*

Команда *unexpand* заменяет последовательные пробелы символами табуляции.

По умолчанию 8 пробелов заменяются одним символом табуляции. Количество пробелов можно задать с помощью параметра *-t n* (где *n* — количество пробелов).

Синтаксис вызова:

`unexpand [параметры] файл`



ГЛАВА 2.

ЛОКАЛЬНАЯ СЕТЬ





2.1. Физическая настройка сети Ethernet

Существует много сетевых технологий, но, так как эта книга имеет справочную направленность, то мы будем рассматривать настройку локальной сети, построенной на технологии Fast Ethernet. Мы рассмотрим ее подробно — от обжатия кабеля до конфигурирования сети в Linux.

Несмотря на наличие уже стандарта Gigabit Ethernet (1000 Мбит/с), стандарт Fast Ethernet (100 Мбит/с) все еще актуален, и его скорости вполне хватает для обеспечения работы локальной сети. А стандарт Gigabit Ethernet пока используется в качестве магистрalli.

Прежде всего вам нужно убедиться, что у вас есть сетевой адаптер, поддерживающий технологию FastEthernet. Большинство современных компьютеров оснащены такими сетевыми адаптерами. Как пра-



вило, в современных компьютерах и ноутбуках сетевые адаптеры являются интегрированными в материнскую плату, и устанавливать их отдельно не нужно.

После этого вам нужно подключить сетевой кабель к вашему сетевому адаптеру. Как правило, кабель обжимается администратором сети.

Для создания Fast Ethernet сети вам нужны следующие устройства:

- **Сетевые адаптеры** — с ними мы уже разобрались;
- **Коммутатор (switch)** — его можно купить в любом компьютерном магазине. Вместо него сойдет *маршрутизатор* (router), который, как правило, обладает 4–8 портами для подключения локальных компьютеров — этого вполне достаточно для построения небольшой SOHO-сети (Small Office Home Office);
- **Витая пара пятой категории** — спрашивайте именно такой тип кабеля¹;
- **Коннекторы RJ-45** — таких коннекторов вам нужно будет в два раза больше, чем число компьютеров, поскольку кабель нужно будет обжать с двух концов;
- **Инструмент для обжимки витой пары** — хороший инструмент стоит относительно дорого (примерно, как коммутатор), а плохой лучше не покупать. Если не хотите выкладываться, возьмите у кого-нибудь на пару дней.

Теперь приступим к самому процессу обжимки. Внутри кабеля будут 4 витые пары, причем у каждого провода будет своя цветовая маркировка. Суть процесса обжимки заключается в том, чтобы подключить каждый из проводов к нужному контакту коннектора. Сначала нужно поместить провода в коннектор (защищать их необязательно — за вас это сделает инструмент), затем коннектор обратной частью



(той, которой он будет вставляться в сетевой адаптер) помещается в инструмент для обжимки и крепко обжимается. Используя приведенную ниже таблицу (табл. 2.1), вы без проблем сможете обжать кабель:

Таблица 2.1. Обжимка витой пары

Контакт	Цвет провода
1	Бело-оранжевый
2	Оранжевый
3	Зелено-белый
4	Синий
5	Сине-белый
6	Зеленый
7	Бело-коричневый
8	Коричневый

Обжимать кабель нужно с двух сторон. Один конец подключается к концентратору (или коммутатору), а второй — к сетевому адаптеру. Если вы неправильно (или слабо) обожмете кабель, то ваша сеть работать не будет или же будет работать только на скорости 10 Мбит/с.

Проверить, правильно ли вы обжали кабель, очень просто: обратите внимание на коммутатор. Возле каждого порта будет два индикатора. Если горят оба, значит, все нормально. Если же горит только один из них, значит, данный порт работает в режиме 10 Мбит/с. А если вообще не горит ни один из индикаторов, значит, вам нужно переобжать кабель.



2.2. Настройка сети с помощью графического конфигуратора

В каждом дистрибутиве Linux есть графические конфигураторы. Конечно, на сервере далеко не всегда устанавливается графический интерфейс, поэтому такие конфигураторы будут недоступны, однако не сказать о них тоже нельзя. Сначала мы рассмотрим графические конфигураторы, а затем рассмотрим настройку сети в консоли — с помощью конфигурационных файлов.

Прежде чем приступить к настройке сети, ради справедливости нужно отметить, что в большинстве случаев настраивать ничего не придется — во всех современных сетях есть DHCP-сервер, который и настраивает все остальные компьютеры. Настройка сети может понадобиться разве что на самом DHCP-сервере, но это только в том случае, когда вы настраиваете сеть с нуля. Опять-таки, если вы подключили свои домашние компьютеры к маршрутизатору, то в качестве DHCP-сервера будет выступать сам маршрутизатор и вам в большинстве случаев вообще не придется ничего настраивать.

В Ubuntu для настройки локальной сети щелкните по значку сети в правом верхнем углу (рис. 2.1). Выберите команду **Параметры соединения**, чтобы открыть конфигуратор сети.

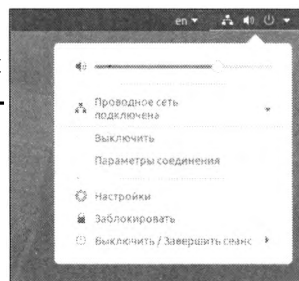


Рис. 2.1. Системное меню

В появившемся окне **Сеть** нажмите кнопку с изображением шестеренки напротив строки **Подключено** — 1000 Мбит/с (рис. 2.2).

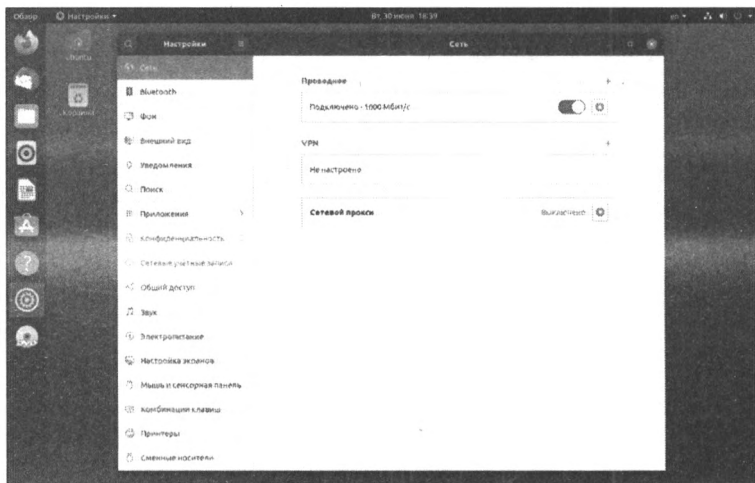


Рис. 2.2. Окно «Сеть»

В появившемся окне на вкладке **Сведения о системе** убедитесь, что включен флажок **Подключаться автоматически**, чтобы соединение устанавливалось автоматически при загрузке системы (или при входе в систему, если выключен флажок **Сделать доступным для других пользователей**).

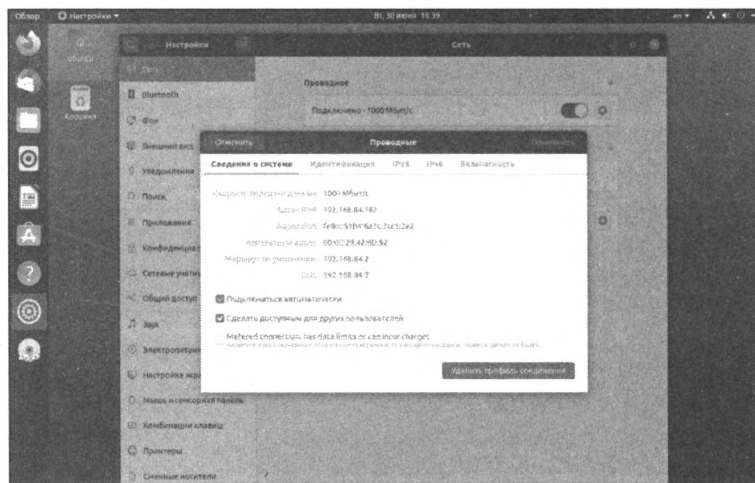


Рис. 2.3. Сведения о системе

Далее перейдите на вкладку IPv4 для изменения параметров протокола IP. По умолчанию сеть настроена на использование протокола DHCP (рис. 2.4). Для ручной настройки выберите **Вручную** и укажите (рис. 2.5):

- IP-адрес, маску сети, IP-адрес шлюза (все эти параметры можно получить у администратора сети);
- IP-адреса DNS-серверов, рекомендуется использовать IP-адреса Google. При указании нескольких IP-адресов разделять их нужно запятыми, как показано на рис. 2.5.

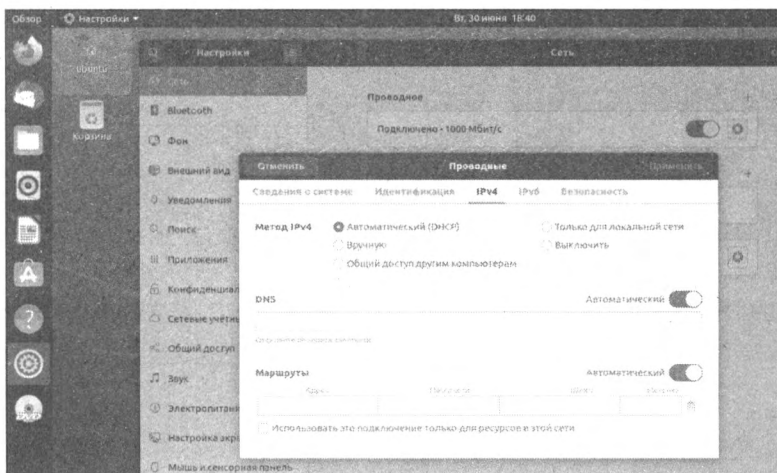


Рис. 2.4. Используем DHCP

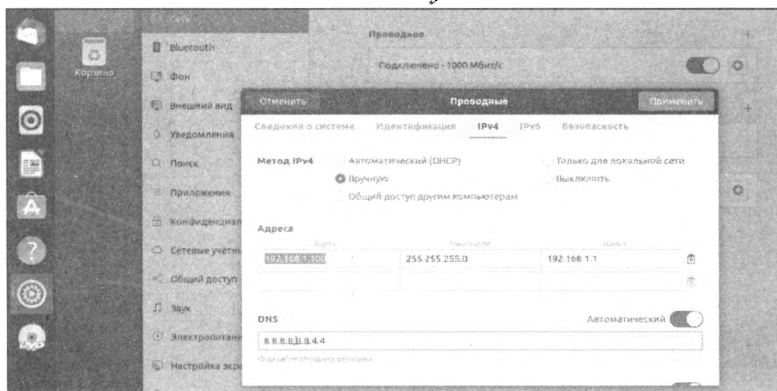


Рис. 2.5. Ручная настройка



Для сохранения настроек нажмите кнопку **Применить**.

В Astra Linux, например, настройка осуществляется аналогично, только интерфейс configurатора немного другой. Щелкните правой кнопкой мыши на значке сетевого соединения и выберите команду **Изменить соединения** (рис. 2.6). В появившемся окне выберите сетевое соединение и нажмите кнопку с изображением шестеренки (рис. 2.7).

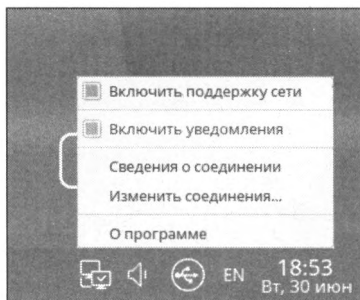


Рис. 2.6. Вызов configurатора сети в Astra Linux

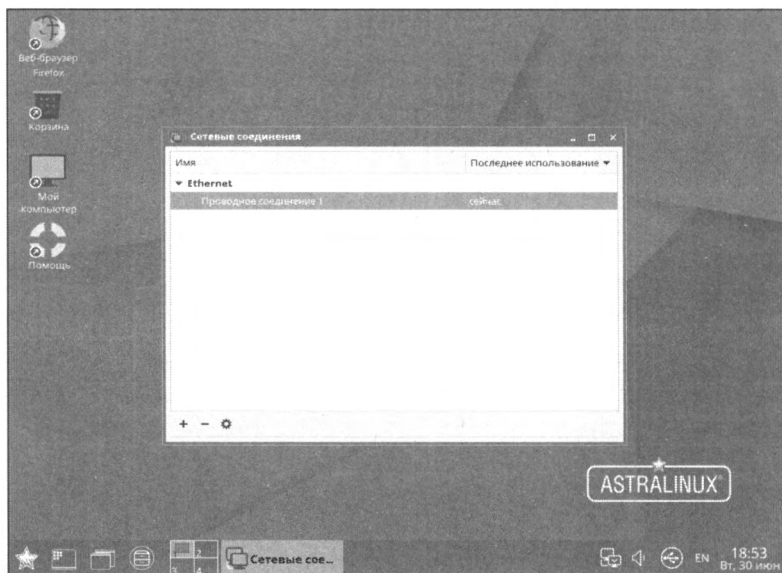


Рис. 2.7. Configurатор сети



Далее нужно перейти на вкладку **Параметры IPv4** и установить параметры сети, как это было сделано ранее в Ubuntu. IP-адреса DNS-серверов также указываются через запятые. Для применения настроек нажмите кнопку **Сохранить**.

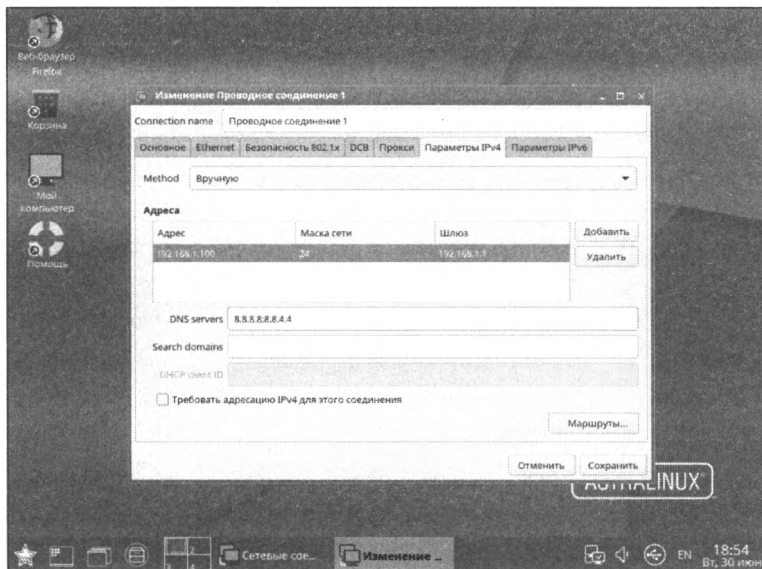


Рис. 2.8. Параметры IPv4

В 99% случаев вам не нужно редактировать параметры Ethernet-соединения, поскольку настройка данного соединения осуществляется по протоколу DHCP автоматически. Конечно, в некоторых случаях, например, когда вам нужно по тем или иным причинам клонировать MAC-адрес устройства или у вас нет DHCP-сервера и нужно определить конфигурацию интерфейса вручную, придется редактировать параметры проводного соединения. Также настройка интерфейса вручную может потребоваться на компьютере, который используется в качестве DHCP-сервера.

На вкладке **Ethernet** можно (рис. 2.9):



- Выбрать физическую сетевую плату, которая будет использоваться для этого соединения (список **Device**);
- Изменить MAC-адрес соединения (**Cloned MAC address**);
- Задать MTU соединения.

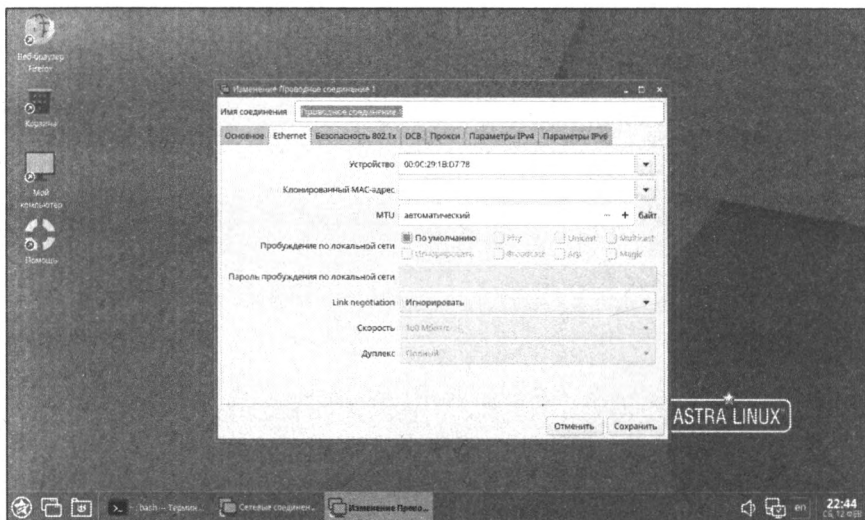


Рис. 2.9. Вкладка *Ethernet* в *Astra Linux*

2.3. Команда *ifconfig*

Команда *ifconfig* используется для настройки и отображения параметров сетевого интерфейса.

Если ввести команду *ifconfig* без параметров, то мы получим список всех активных интерфейсов, например:

```
ens33      Link encap:Ethernet  HWaddr 00:0C:29:C2:0D:D1
          inet addr:192.168.52.154  Bcast:192.168.52.255  Mask:255.255.255.0
```




```

    inet6 addr: fe80::20c:29ff:fec2:ddl/64 Scope:Link
    UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
    RX packets:425 errors:0 dropped:0 overruns:0 frame:0
    TX packets:406 errors:0 dropped:0 overruns:0 carrier:0
    collisions:0 txqueuelen:1000
    RX bytes:41540 (40.5 Kb)  TX bytes:39618 (38.6 Kb)

lo
    Link encap:Local Loopback
    inet addr:127.0.0.1  Mask:255.0.0.0
    inet6 addr: ::1/128 Scope:Host
    UP LOOPBACK RUNNING  MTU:65536  Metric:1
    RX packets:96 errors:0 dropped:0 overruns:0 frame:0
    TX packets:96 errors:0 dropped:0 overruns:0 carrier:0
    collisions:0 txqueuelen:0
    RX bytes:5760 (5.6 Kb)  TX bytes:5760 (5.6 Kb)
```

ПРИМЕЧАНИЕ. Даже если сетевые интерфейсы не настроены, в выводе *ifconfig* должен быть интерфейс *lo*. Если его нет, служба *network* не запущена. Для ее запуска введите команду от пользователя *root*: *service network start*.

Разберемся, что есть что. У нас есть два сетевых интерфейса — *ens33* (Ethernet-адаптер) и *lo* (интерфейс обратной петли). Второй используется для тестирования работы сети и всегда имеет IP-адрес 127.0.0.1. Также он может использоваться для обращения к локальному компьютеру. Например, если на вашем компьютере установлен MySQL-сервер, то в PHP-сценариях можно смело указывать адрес 127.0.0.1 или *localhost*, чтобы указать, что вы обращаетесь к MySQL-серверу, запущенному на этом компьютере.

Рассмотрим поля вывода *ifconfig*:

- ***HWaddr*** — содержит аппаратный MAC-адрес (в нашем случае это 00:0C:29:C2:0D:D1);
- ***inet addr*** — содержит IPv4-адрес интерфейса (192.168.52.154);



- ***inet6 addr*** — содержит IPv6-адрес интерфейса;
- ***Bcast*** — адрес для широковещательной передачи;
- ***Mask*** — маска сети;
- ***MTU*** — значение MTU (Maximum transmission unit);
- ***collisions*** — счетчик коллизий. Если количество коллизий больше 0, с вашей сетью творится что-то неладное. Как правило, в современных коммутируемых сетях коллизии отсутствуют вообще;
- ***RX packets*** — количество принятых пакетов;
- ***TX packets*** — количество переданных пакетов;
- ***RX bytes*** — количество принятых байтов;
- ***TX bytes*** — количество переданных байтов.

Получить список всех интерфейсов, а не только активных, можно с помощью параметра *-a*:

```
sudo ifconfig -a
```



С помощью команды *ifconfig* можно *деактивировать* (down) и *активировать* (up) любой интерфейс:

```
sudo ifconfig ens33 down  
sudo ifconfig ens33 up
```

Иногда такой «перезапуск» интерфейса помогает наладить его работу. Особенно это полезно для «подвисших» беспроводных интерфейсов (wlan*).



С помощью *ifconfig* можно назначить IP-адрес и другие сетевые параметры интерфейса, например:

```
sudo ifconfig ens33 down
sudo ifconfig ens33 192.168.1.1 up
```

Сначала мы деактивируем интерфейс, затем назначаем новый IP-адрес и «поднимаем» (параметр *up*) интерфейс. Можно эту последовательность действий выполнить и за три команды:

```
sudo ifconfig ens33 down
sudo ifconfig ens33 192.168.1.1
sudo ifconfig ens33 up
```

Аналогичным образом можно назначить маску сети и широковещательный адрес:

```
sudo ifconfig eth0 10.0.0.1 netmask 255.255.255.240 broadcast 10.0.0.15
```

После изменений параметров интерфейса не забудьте его активировать с помощью команды *up*:

```
sudo ifconfig eth0 up
```

Изменить MTU интерфейса можно так:

```
sudo ifconfig eth0 down
sudo ifconfig eth0 mtu XXXX up
```

Самой интересной возможностью *ifconfig* является перевод интерфейса в так называемый *promiscuous mode*. В обычном режиме сетевая карта, если принимает пакет, который был адресован не ей,

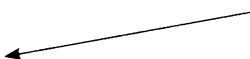


просто его отбрасывает. В *promiscuous mode* карта будет принимать даже те пакеты, которые не были ей адресованы. Такая возможность может понадобиться для перехвата трафика:

```
sudo ifconfig eth0 promisc
```

Перевести карту в обычный режим можно командой:

```
sudo ifconfig eth0 -promisc
```



Помните, что параметры, заданные с помощью *ifconfig*, хранятся только до следующей перезагрузки. Если вы хотите сохранить назначенные параметры, нужно или использовать конфигураторы сети (описаны ранее) или редактировать файлы конфигурации (будут описаны далее).

2.4. Имена сетевых интерфейсов в Linux

В Linux у каждого сетевого интерфейса есть свое имя, чтобы администратор сразу понимал, что это за соединение. Например, у всех PPP-соединений имя `ppp?`, где `?` — номер соединения: первое соединение называется `ppp0`, второе — `ppp1` и т.д. Нумерация соединений начинается с нуля. В таблице 2.2 приведены названия часто используемых сетевых интерфейсов.

*Таблица 2.2. Имена сетевых интерфейсов в Linux*

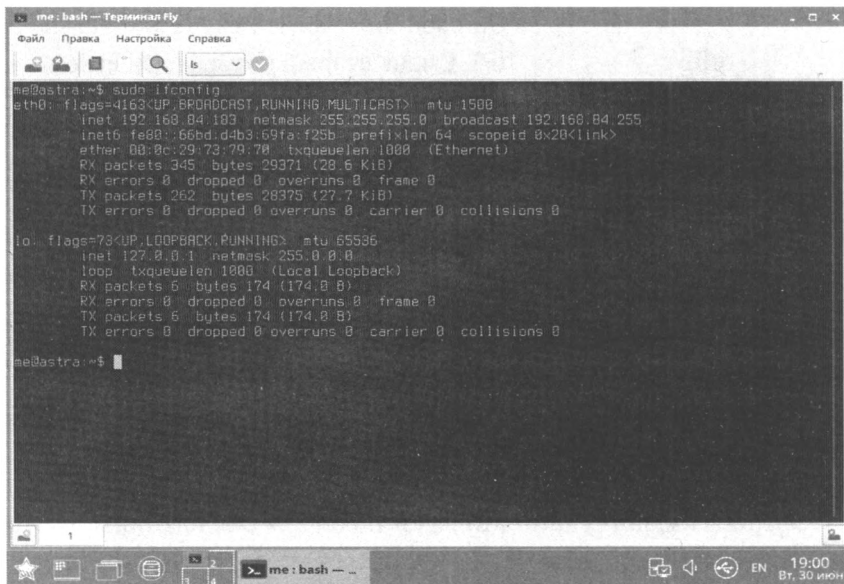
Название интерфейса	Описание
lo	Интерфейс обратной петли
eth , он же ens	Сетевой интерфейс Ethernet
ppp	Соединение PPP (Point-to-Point)
wlan	Беспроводной сетевой интерфейс
tr	Сетевой интерфейс Token Ring
plip	Сетевой интерфейс PLIP (Parallel Line IP). Очень старый интерфейс, вряд ли вы будете с ним иметь дело
sl	Сетевой интерфейс SLIP (Serial Line IP). Тоже «древний» интерфейс
fddi	Интерфейс к карте FDDI
ax	Сетевой интерфейс любительского радио AX.25

Интерфейсы создаются автоматически ядром для каждого обнаруженного сетевого устройства. Исключения составляют разве что интерфейсы **ppp**, которые создаются во время настройки сети. Например, у вас может быть PPPoE-соединение с провайдером и сетевая карта **eth0**. Сетевой интерфейс **eth0** для сетевой карты будет создан автома-



тически ядром. Но ядро не знает, как вы будете использовать эту карту — для подключения к локальной сети или как PPPoE-соединение с провайдером, поэтому оно не может создать интерфейс `ppp0`, прочитав ваши мысли. Поэтому интерфейс `ppp0` будет создан уже во время настройки PPPoE/ADSL-соединения с вашим провайдером.

Все было бы хорошо, если бы по непонятным причинам в последних версиях ядра интерфейсы не переименовали. Так, старый добрый `eth0` теперь почему-то называется `ens33`, в некоторых дистрибутивах даже `enp3s0`. Логике в таких именах не очень много, да и хочется, чтобы имена интерфейсов были такие же, как раньше — как-никак 20-летняя традиция. Ради справедливости нужно отметить, что в Astra Linux, например, интерфейс называется как нужно — `eth0` (рис. 2.10).



```
me@astra:~$ sudo ifconfig
eth0: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.84.189 netmask 255.255.255.0 broadcast 192.168.84.255
    inet6 fe80::66bd:d4b3:69fa:f25b prefixlen 64 scopeid 0x20<link>
    ether 08:0c:29:73:79:78 txqueuelen 1000 (Ethernet)
    RX packets 345 bytes 29371 (28.6 KiB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 262 bytes 28375 (27.7 KiB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    loop txqueuelen 1000 (Local Loopback)
    RX packets 5 bytes 174 (174.0 B)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 5 bytes 174 (174.0 B)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

me@astra:~$
```

Рис. 2.10. Вывод `ifconfig` в Astra Linux



На самом деле, логика в новых названиях есть, но только с точки зрения самого компьютера и разработчиков `udev`. Пользователям и администраторам такая логика не очень понятна. Например, имена, вовлекающие предоставленный BIOS индекс для встроенных в материнскую плату устройств (`ID_NET_NAME_ONBOARD`), выглядят так — `enol`. Имена адаптеров, подключающихся к шине PCI Express, будут содержать номер слота (`ID_NET_NAME_SLOT`), например, `ensl`. Имена, вовлекающие физическое/географическое расположение коннектора (`ID_NET_NAME_PATH`), выглядят еще сложнее — `enp2s0`. Ну и самые «страшные» — это имена с MAC-адресами интерфейсов (`ID_NET_NAME_MAC`). Пример такого имени: `enx78e7d1ea46da`.

Конечно, можно оставить все как есть — все будет прекрасно работать. Но если вам привычнее обычные имена, тогда есть несколько способов вернуть все, как было раньше.

Первый способ заключается в редактировании файла `/etc/udev/rules.d/70-my-network.rules` (или `70-persistent-net.rules`). Если такого файла у вас нет, значит, его нужно создать. Примерное содержимое файла следующее:

```
# PCI device (r8169)
SUBSYSTEM=="net", ACTION=="add", ATTR{address}=="xx:xx:xx:xx:xx:xx",
NAME="eth0"

# USB device (usb)
SUBSYSTEM=="net", ACTION=="add", ATTR{address}=="yy:yy:yy:yy:yy:yy",
NAME="wlan0"
```

Здесь `xx` нужно заменить MAC-адресом вашей сетевой карты, а `yy` — MAC-адресом вашего беспроводного адаптера.

Есть и второй способ — передать ядру параметр `net.ifnames=0`. О том, как это сделать, мы, возможно, поговорим позднее.



2.5. Общие конфигурационные файлы

Наверняка вам интересно, куда конфигураторы сохраняют введенные вами настройки. Об этом мы обязательно поговорим, тем более что файлы конфигурации сети отличаются в зависимости от используемого дистрибутива. Но сначала мы рассмотрим общие файлы конфигурации, имеющие отношение к сети. Эти файлы есть во всех дистрибутивах Linux, и их назначение одинаково.

Файл `/etc/hosts`

Когда еще не было системы разрешения доменных имен (DNS), в этом файле прописывались IP-адреса узлов и соответствующие им имена узлов. Эти файлы тиражировались от компьютера к компьютеру. Если в сеть добавлялся новый компьютер, изменения нужно было внести в файлы `/etc/hosts` на всех компьютерах. Сегодня этот файл практически не используется, и в нем содержится только IP-адрес узла *localhost* — 127.0.0.1 и аналогичные IPv6-адреса. Формат записей этого файла следующий:

IP-адрес	Имя
----------	-----

Например:

127.0.0.1	localhost
-----------	-----------

Файлы `/etc/hosts.allow` и `/etc/hosts.deny`

Содержит IP-адреса узлов, которым разрешен (`hosts.allow`) или запрещен (`hosts.deny`) доступ к сервисам данного узла. Опять-таки — это



устаревшие конфигурационные файлы. Вы можете их использовать, однако в настоящее время ограничения доступа к определенной службе или узлу достигается путем редактирования правил брандмауэра, а не путем редактирования этих файлов. При желании, конечно, вы можете их использовать, но среди своих коллег вы будете выглядеть настоящим динозавром.

Файл `/etc/host.conf`

Содержит параметры разрешения доменных имен, которые указываются директивой `order`. Так, `order hosts, bind` означает, что сначала поиск IP-адреса по доменному имени будет произведен в файле `/etc/hosts`, а затем — с помощью DNS-сервера. Если для вас это существенно, можете изменить порядок на `bind, hosts`, ведь файл `hosts` уже никем давно не используется и искать там нечего.

Директива `multi on` в этом файле означает, что одному доменному имени могут соответствовать несколько IP-адресов.

Файл `/etc/hostname`

Обычно в этом файле содержится имя узла. Оно записано одной строкой, больше никаких директив в этом файле нет.

Файл `/etc/motd`

Содержит сообщение дня, которое может использоваться некоторыми сетевыми службами (например, FTP) и отображаться при входе пользователя в систему. Чтобы ваша сетевая служба использовала именно этот файл, нужно проверить ее файл конфигурации.



Файл /etc/resolv.conf

Обычно в этом файле содержатся параметры резолвера доменных имен.

Директива *search* задает список доменов, в которых будет произведен поиск доменного имени, если оно введено не полностью.

Например, пусть директива **search** определена так:

```
search org org.ru net
```

Вы вводите доменное имя не полностью (например, в браузере) — dkws. Тогда резолвер попытается сначала разрешить доменное имя dkws.org, потом — dkws.org.ru, а уж потом, если ни один из первых двух вариантов не был разрешен в IP-адрес — dkws.net.

Директива *nameserver* позволяет определить IP-адрес сервера DNS.

В одной директиве можно указать только один сервер. Всего в файле конфигурации может быть определено до четырех серверов имен:

```
nameserver 8.8.8.8  
nameserver 8.8.4.4
```

Пример файла конфигурации /etc/resolv.conf приведен в листинге 2.1.

Листинг 2.1. Файл /etc/resolv.conf

```
search dkws.org.ru
```

```
nameserver 8.8.8.8
```

```
nameserver 8.8.4.4
```



Файл /etc/services

Все мы знаем, что за сетевыми службами закрепляются определенные номера портов. Например, службе `pop3` соответствует порт 110, `ftp` — порт 21. Эти соответствия и определяются в файле `/etc/services`. Обычно вам не придется редактировать этот файл. Скорее, вы будете его использовать в качестве информационного, чтобы знать, какой порт соответствует какой службе.

Файл /etc/protocols

Тоже сугубо информационный файл (его можно редактировать, но вряд ли вы будете это делать). В нем описываются имена протоколов, их номера, псевдонимы и комментарии, а также ссылки на RFC-документы. Полезный файл, если вы собрались разобраться по очереди во всех сетевых протоколах.

Файл /etc/network/interfaces: конфигурация сети в Astra Linux

Основной конфигурационный файл сети в Astra Linux называется `/etc/network/interfaces`.

Он содержит параметры сетевых интерфейсов, если не используется Network Manager. Вот пример описания интерфейса `eth1` со статическим IP-адресом:

```
iface eth1 inet static
    address 192.168.1.2          # IP-адрес
```



```
netmask 255.255.255.0
gateway 192.168.1.1
```

```
# Маска сети
# Шлюз
```

Думаю, все ясно. Если интерфейс настраивается по DHCP, его конфигурация будет выглядеть так:

```
auto eth0
iface eth0 inet dhcp
```

Каталог `/etc/NetworkManager/system-connections:` конфигурация сети в Ubuntu

В Ubuntu параметры сетевого интерфейса хранятся в каталоге `/etc/NetworkManager/system-connections`.

В этом каталоге находятся несколько файлов, имена этих файлов соответствуют названиям соединений в NetworkManager. Например, в файле ‘Проводное соединение 1.nmconnection’ находятся параметры первого проводного соединения (лист. 2.2).

Листинг 2.2. Пример настройки интерфейса со статическим IP-адресом

```
[802-3-ethernet]
duplex=full
```

```
[connection]
id=Проводное соединение 1
uuid=00779167-0a51-44ad-a2b3-b5765a65b496
type=802-3-ethernet
timestamp=1419337229
```



```
[ipv6]
method=auto
ip6-privacy=2

[ipv4]
method=manual
dns=8.8.8.8;
addresses1=192.168.1.101;24;192.168.1.1;
```

Как видите, протокол IPv4 настраивается вручную (`method=manual`). Был задан DNS-сервер (8.8.8.8), а также IP-адрес узла (192.168.1.101), маска сети (24 — соответствует 255.255.255.0), а также адрес шлюза — 192.168.1.1. Если нужно добавить еще один IP-адрес, просто добавьте строку *addressesN*:

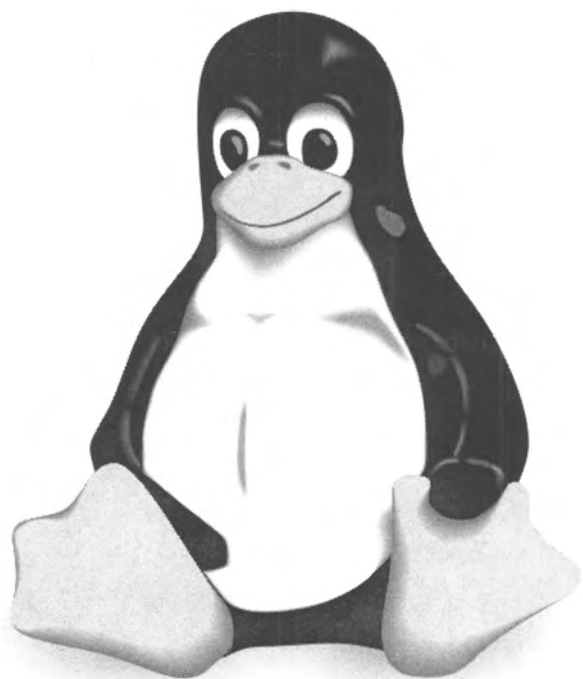
```
addresses2=192.168.1.102;24;192.168.1.1;
```

Для настройки интерфейса по DHCP измените параметр *method*:

```
[ipv4]
method=auto
```

Если вы изменили параметры интерфейса, контролируемого Network Manager, и не через интерфейс NM, а вручную, тогда нужно перезапустить Network Manager:

```
# service network-manager restart
```



ГЛАВА 3.

УСТАНОВКА ПРОГРАММ В LINUX





3.1. Способы установки программ

Существует три способа установки программ: компиляция из исходных кодов, из пакетов, из снапов.

Первый способ устарел и сейчас используется очень редко – когда нет другого способа установить ту или иную программу на свой компьютер. Выглядит все это так: вы сначала устанавливаете программное обеспечение разработчика (заголовочные файлы, компилятор **gcc**, библиотеки, различные утилиты вроде **make** и т.д.), затем загружаете и распаковываете архив с исходными кодами программы. Запускаете компиляцию и ждете, пока программа будет собрана из исходников. Даже если все пройдет гладко и вам не придется адаптировать исходные коды (что требует определенных знаний) под свою систему, процесс компиляции занимает много времени. Удалить установленную таким образом программу можно только вручную.

Сегодня все мыслимое и немыслимое программное обеспечение поставляется в уже откомпилированном виде. Самостоятельная сбор-



ка (компиляция) программного обеспечения имеет смысл только в случаях, когда вам нужно установить программу, а для вашей архитектуры¹/дистрибутива нет пакета или же вы хотите получить самую последнюю версию программы — когда разработчики еще не успели создать пакет для вашего дистрибутива.

Установка *из пакета* гораздо проще. Вы получаете пакет с программой и устанавливаете его. Пакет содержит уже откомпилированную программу и необходимые для работы этой программы файлы, например файлы конфигурации, страницы руководства и др. При удалении пакета вся эта информация будет централизована и удалена, и вам не придется исследовать каждый каталог системы и вручную удалять остатки программы.

Максимум, что может пойти не так при установке пакета, – это нарушение зависимостей. Об этом мы поговорим далее в этой главе. Забегая наперед, обычно для удовлетворения зависимостей нужно удалить или установить какие-то другие пакеты. Это несложно, и в некоторых случаях система сама справляется с поставленной задачей. Установка программы из пакета занимает считанные секунды, в крайнем случае – минуты, большая часть времени уходит на загрузку самого пакета из Интернета.

Получить пакет можно как вручную, собственноручно скачав его из сайта разработчика, так и воспользоваться менеджером установки пакетов, который сам скачает пакет из репозитория и установит его.

Третий способ, *снапы* — относительно недавний. Как уже было отмечено, при установке пакета могут понадобиться дополнительные пакеты. Как правило, это пакеты с какими-то библиотеками. Иногда происходит так, что нужная библиотека отсутствует в вашем дистрибутиве как таковая или конфликтует с имеющейся версией библиотеки. Особенно часто такое происходит, если вы пытаетесь установить

¹ Хотя есть большая вероятность того, что раз для вашей архитектуры нет пакета, то у вас не получится откомпилировать программу.



пакет не из репозитория своего дистрибутива, а скачанный с сайта разработчика. Чтобы избавиться от подобных танцев с бубном, были придуманы снапы. Снап содержит свою программу и все необходимые для ее работы библиотеки. При этом программа, установленная со снапа, будет использовать не системную версию библиотеки, а ту версию, которая шла со снапом. В результате программа будет выполняться корректно, и ее установка никак не отразится на других программах, поскольку процесс установки никак не затрагивает имеющиеся в системе библиотеки. О снапах мы еще поговорим отдельно в этой главе.

3.2. Типы пакетов и их содержимое

Существует два формата пакетов — **DEB** (расширение `.deb`) и **RPM** (`.rpm`). Первые используются в Debian-совместимых дистрибутивах — Ubuntu, Astra Linux, Mint, Denix и др. Второй тип используется в RedHat-совместимых — RHEL, Fedora, CentOS, openSUSE и т.д.

Независимо от формата пакетов в самом пакете, кроме устанавливаемой программы и вспомогательных файлов, содержится различная служебная информация: информация о разработчике, о версии программного продукта, информация о зависимостях и конфликтах, пути для установки (указывает, куда должны быть скопированы файлы, имеющиеся в пакете, в процессе установки пакета).

Отдельного разговора заслуживает информация о зависимостях и конфликтах. Некоторые пакеты для своей работы требуют установки дополнительных пакетов. Например, пакет `ubuntu-desktop` требует для своей работы множества других пакетов — `alsa-base` (файлы звукового драйвера), `alsa-utils`, `bc`, `anacron` и т.д. Говорят, что пакет `ubuntu-desktop` зависит от пакетов `alsa-base`, `alsa-utils` и т.д. В свою очередь эти пакеты могут зависеть от других пакетов. При установке



пакета `ubuntu-desktop` менеджер пакетов выполнит разрешение зависимости, то есть установит все пакеты, от которых зависит наш пакет, и все пакеты, от которого зависят другие устанавливаемые пакеты. Может получиться, что для установки одной небольшой программы будет установлено очень много других пакетов, от которых зависит эта программа. Но ничего не поделаешь — если программа вам нужна, то придется пойти на это. Как вариант — найти аналогичную программу.

Некоторые программы могут находиться в системе только в единственном экземпляре. Например, почтовый агент может быть установлен только один, иначе между ними произойдет конфликт. Поэтому **пакет А** может конфликтовать с **пакетом Б**. Если у вас установлен **А**, а вы пытаетесь установить **Б**, менеджер пакетов сообщит о конфликте и предложит или отказаться от установки, или перед установкой **пакета Б** удалить **пакет А**.

3.3. Источники пакетов

Список источников пакетов в порядке убывания их популярности:

- **Репозитории** — каждый дистрибутив работает с собственным репозитарием (хранилищем) пакетов. Да, для установки программного обеспечения вам понадобится доступ к Интернету, но это самый современный способ установки. Репозитории позволяют эффективно управлять обновлением программного обеспечения. Используя репозитории, вы можете быть уверенными, что у вас будет установлено самое новое программное обеспечение.
- **Установочный диск** — ранее дистрибутивы Linux распространялись на DVD-дисках (некоторые — на одном, некоторые — на нескольких). На DVD, кроме самой системы, была и львиная доля программного обеспечения, которая только может пона-



добиться пользователю. Все программное обеспечение при установке системы не устанавливалась, но была возможность установки с инсталляционного носителя после установки. Когда доступ к Интернету был медленным и дорогим, пользователи предпочитали устанавливать пакеты с DVD-диска. Преимущество этого способа в том, что не нужно загружать пакеты с удаленного сервера, что экономило время и деньги. Сейчас же доступ к Интернету в большинстве случаев высокоскоростной (даже 10 Мбит/с считается высокоскоростным) и безлимитный, поэтому чаще всего на DVD-диске содержится только самое необходимое программное обеспечение, а все остальное загружается из Интернета. К тому же ПО с DVD наверняка устареет к моменту его установки, а из репозитория будет загружена более новая версия — пакеты в репозиториях периодически обновляются.

- **Сайт разработчика программного обеспечения** — иногда программное обеспечение не включено в состав репозитория дистрибутива. Это может произойти по разным причинам. Например, программное обеспечение является *проприетарным* (коммерческим) или же просто узкоспециализированным и его не стали включать в состав дистрибутива. В этом случае вам нужно самостоятельно скачать пакеты и установить их. В некоторых случаях разработчики предоставляют доступ к собственным репозиториям, из которых можно загрузить все необходимые пакеты.

3.4. Менеджеры пакетов

Изначально в RedHat и Debian использовались программы **rpm** и **dpkg**. Сейчас обе эти программы все еще входят в состав дистрибутивов, но использовать их крайне не рекомендуется. Дело в том, что эти программы ничего не «знают» о зависимостях между пакетами. Если для работы **пакета А** нужен **пакет Б**, то программа даже толком не сообщит о том, какой пакет нужен для установки **пакета А**. Она



сообщит о том, что для установки **пакета А** нужна, например, библиотека `lib40`, а в каком DEB-пакете находится эта библиотека и откуда ее нужно заполнить — вам придется догадаться самому.

Также программы **rpm** и **dpkg** ничего не знают о репозиториях, поэтому удел этого менеджера — установка пакета из локального источника. Желательно, чтобы для работы устанавливаемого пакета не требовались другие пакеты — ведь о зависимостях эти программы, как уже было отмечено, ничего не знают. В репозиториях содержится информация обо всем дереве зависимостей. Например, **пакет А** может зависеть от **пакета Б**, **пакет Б** — от пакетов В, Г и Д. Менеджеры пакетов, такие как **apt**, установят все необходимые пакеты в необходимом порядке, чего не дожدهшься от программ **rpm** и **dpkg**.

Именно поэтому для управления пакетами рекомендуется использовать менеджеры пакетов. Вот неполный список возможностей таких программ:

- Поиск пакетов в репозиториях;
- Установка пакетов из репозитариев;
- Установка/удаление пакетов с разрешением зависимостей и конфликтов;
- Обновление пакетов.

Принцип работы менеджера пакетов простой. Представим, что вы устанавливаете **пакет А**. Менеджер производит список вашего пакета по списку всех установленных репозитариев. В одном из репозитариев ваш пакет точно будет, в противном случае менеджер сообщит о том, что пакет не найден. Далее менеджер смотрит на список зависимостей, и если есть пакеты, от которых зависит наш **пакет А**, то они также загружаются и устанавливаются — до установки **пакета А**. Затем производится загрузка и установка самого **пакета А**.



Установка программного обеспечения в Linux требует прав *root*. Поэтому нужно сначала запускать любые команды управления программным обеспечением через команду *sudo*. Например:

```
sudo apt install mc
```

В дистрибутивах Debian, Ubuntu, Astra Linux и других, которые основаны на Debian, используется менеджер пакетов **apt**. В других дистрибутивах могут использоваться другие менеджеры, например, в Fedora используется **dnf**, в CentOS и старых версиях Fedora – **yum**. В openSUSE используется свой менеджер пакетов **zypper**.

Далее будет рассмотрен менеджер пакетов **apt**. При установке пакетов **apt** разрешает зависимости. Вы просто указываете, какой пакет вы хотите установить, а что будет происходить дальше — уже не ваша забота. Вас интересует конечный результат. Лишь бы требуемый пакет был в репозиториях.

ПРИМЕЧАНИЕ. В старых версиях Debian и Ubuntu использовалась команда *apt-get* вместо *apt*. Основные параметры (см. табл. 3.1) у этих двух команд такие же, нет смысла рассматривать устаревшую версию менеджера пакетов. Если вам нужно работать со старой версией, обратитесь к странице руководства (*man apt-get*).

Список репозитариев хранится в файле */etc/apt/sources.list*. Содержимое этого файла показано на рис. 3.1.

Вы вряд ли будете вручную редактировать этот файл. Исключения могут составить лишь ситуации, когда вы добавляете сторонний репозиторий, содержащий какую-то программу, чтобы установить ее впоследствии с помощью **apt**. Что же касается стандартных репозитариев, то ими проще управлять (включать/выключать) посредством приложения **Программы и обновления**, изображенном на рис. 3.2.

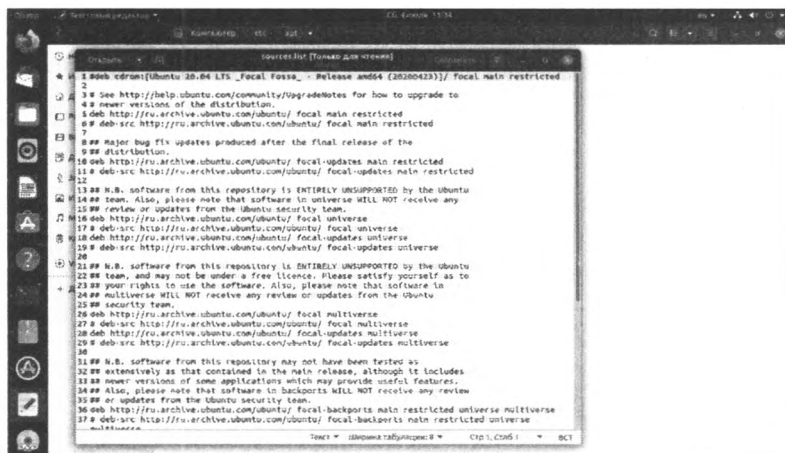


Рис. 3.1. Файл *sources.list*



Рис. 3.2. Программы и обновления

На вкладке **Программное обеспечение Ubuntu** вы можете включить/выключить стандартные репозитории. Также можно включить в качестве источника пакетов инсталляционный DVD-диск Ubuntu. Обратите внимание: вы можете не только включить/выключить репозита-



рии, но и выбрать местоположение сервера, с которого будут пакеты загружаться. На рис. 3.2 показано, что пакеты будут загружаться с сервера, который находится в РФ.

Формат вызова команды **apt** следующий:

```
sudo apt [опции] команды [пакет]
```



В качестве примера приведу команду установки пакета **mc**:

```
sudo apt install synaptic
```

Обратите внимание, что я запускаю команду от имени *root*. Если запустить команду `apt` через команду *sudo* (`sudo apt install synaptic`), нужно, чтобы пользователь, который запускает *sudo*, был внесен в файл `/etc/sudoers`.

По умолчанию пользователь, которого вы создаете при установке Debian, не вносится в этот файл, поэтому нужно или получать права *root* командой *su*, или добавить пользователя в файл `/etc/sudoers`.

Основные команды `apt` приведены в таблице 3.1.

Таблица 3.1. Основные команды менеджера пакетов `apt-get`

Команда	Описание
<code>install <список пакетов></code>	Устанавливает пакеты из списка. Элементы списка разделяются пробелами
<code>remove <список пакетов></code>	Удаляет пакеты из списка. Элементы списка разделяются пробелами



<code>purge <список пакетов></code>	Удаляет не только пакеты, но их конфигурационные файлы. Это означает, что если вы установили какую-нибудь программу, настроили ее, а потом удалили командой <code>apt remove</code> , то конфигурационный файл этой программы останется в системе. Если вы теперь установите эту программу снова, то можно будет использовать предыдущий конфигурационный файл, так как он не был удален
<code>check</code>	Поиск нарушенных зависимостей
<code>clean</code>	Очищает локальное хранилище полученных пакетов. При установке пакеты из репозитория загружаются в каталог <code>var/cache/apt/archive</code> . При интенсивной установке программного обеспечения в этом каталоге накапливается довольно много пакетов, поэтому очистка хранилища помогает сэкономить дисковое пространство
<code>upgrade [список пакетов]</code>	Обновляет указанные пакеты, если пакеты не заданы, обновляет все пакеты, требующие обновления
<code>full-upgrade</code>	Обновляет всю систему
<code>update</code>	Синхронизирует внутреннюю базу данных о пакетах с источниками пакетов, которые описаны в <code>/etc/apt/sources.list</code>



autoremove	Когда вы устанавливаете пакет, то часто устанавливаются дополнительные пакеты, являющиеся его зависимостями. Если теперь вы удалите этот пакет, то зависимости останутся в системе. Команда <i>apt autoremove</i> удаляет эти зависимости, но только те, которые не нужны другим установленным пакетам
list	Выводит список пакетов, соответствующих какому-то критерию. Примеры приведены далее. Вывести список установленных в системе пакетов: <pre>apt list --installed</pre> Вывести список пакетов, которые требуют обновления (у которых вышла новая версия): <pre>apt list --upgradable</pre> Вывести список всех пакетов доступных для вашей системы: <pre>apt list --all-versions</pre>
show <пакет>	Выводит информацию о пакете
search <слово>	Данная команда выполняет поиск указанного слова в названии пакетов и в описании пакетов. Поддерживаются регулярные выражения



<code>edit-sources</code>	Открывает файл <code>/etc/apt/sources.list</code> в текстовом редакторе для редактирования, после сохранения изменений и закрытия редактора выполняет проверку файла на предмет ошибок. В случае наличия ошибок выводит предложение на повторное редактирование файла, чтобы исправить ошибки
---------------------------	---

3.5. Графические средства установки программ

Установка программ из пакетов имеет свои недостатки. Самый главный из них: вам нужно знать имя пакета, в котором находится нужная вам программа. Если вы следуете какому-то руководству, то особых проблем нет — открыл терминал, ввел команду установки нужного пакета и все. Когда вы знаете имя пакета или команду установки, все хорошо. Но если таких познаний у вас нет, тогда вам нужно или найти имя пакета в Интернете, или же воспользоваться графическими средствами для установки программ.

В состав дистрибутива, например, Astra Linux входит, на наш взгляд, один из самых удобных графических менеджеров пакетов — Synaptic. Вы найдете его в программной группе **Системные**. Использовать Synaptic очень просто (рис. 3.3). Слева отображаются группы пакетов и фильтры (под группами пакетов), позволяющие отфильтровать список пакетов по разделам, состоянию, архитектуре и т.д. В правой верхней части находится список пакетов в выделенной группе. Если выделить любой из пакетов, в область ниже будет загружено его описание.

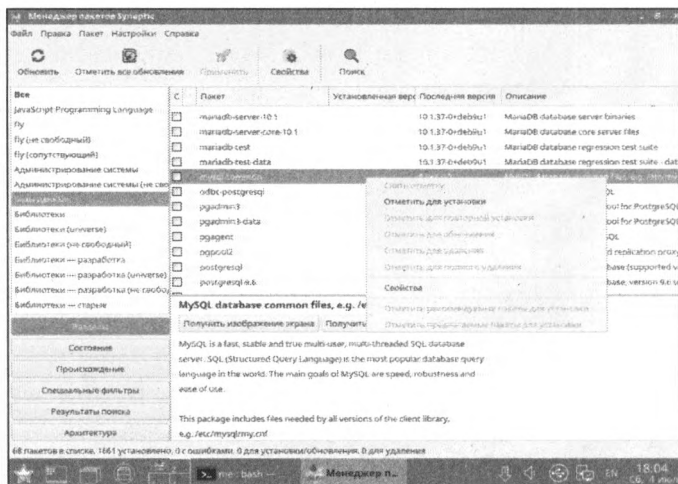


Рис. 3.3. Менеджер пакетов Synaptic

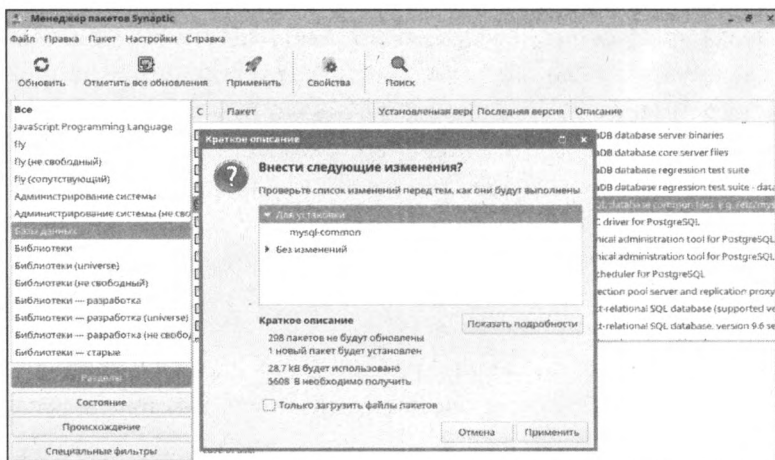


Рис. 3.4. Применение изменений

Установленные пакеты отмечаются зеленым квадратиком. У неустановленных квадратик неокрашенный. Чтобы установить пакет, щелкните на нем правой кнопкой мыши и выберите команду **Отметить**



для установки (рис. 3.3). Значок пакета будет изменен на квадратик с желтой стрелкой. Можете выбрать другие пакеты, которые вы хотите установить. Когда будете готовы, нажмите кнопку **Применить** для установки пакетов (рис. 3.4).

Менеджер пакетов покажет, какой размер будет загружен из Интернета и сколько пакетов будет установлено. Дождитесь установки пакета (рис. 3.5). После этого вы можете использовать установленную программу.

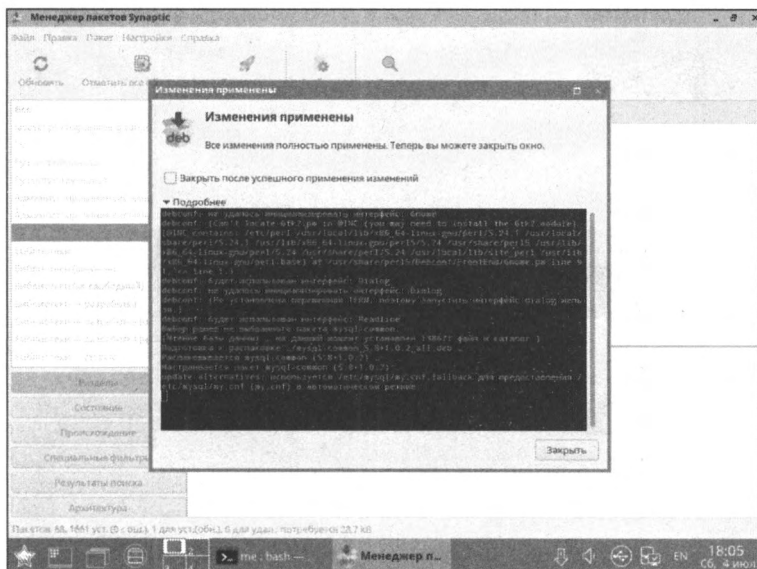


Рис. 3.5. Пакет установлен

В Ubuntu менеджер Synaptic по умолчанию недоступен. Для его установки нужно ввести команду:

`sudo apt install synaptic`



Однако в Ubuntu обычному пользователю больше понравится использовать Ubuntu Software или центр программного обеспечения. Все доступное ПО разбито на группы, вы можете выбрать программу и установить ее.

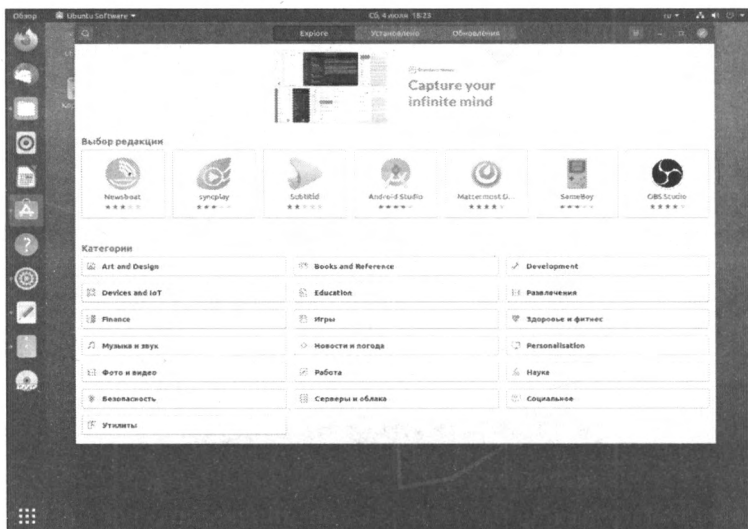


Рис. 3.6. Ubuntu Software

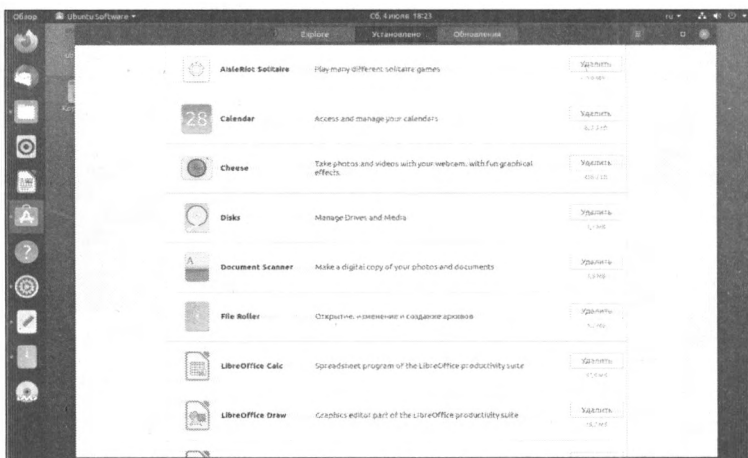


Рис. 3.7. Установленные программы



На вкладке **Установленные** находится список уже установленных программ. Напротив каждой программы есть кнопка **Удалить**, использующаяся для удаления программы.

3.6. Снапы

Ранее уже было сказано, зачем используют **снапы** (snaps). Представим вполне реальную ситуацию. Пользователь устанавливает одну из новых версий Ubuntu, пусть это будет версия 20.04. После установки пользователь не обновляет систему. Его все устраивает, и он спокойно работает некоторое время, скажем, год или полтора. Потом он хочет установить новую версию браузера или какого-то другого приложения и не может этого сделать, поскольку его дистрибутив устарел. Для установки приложения нужны новые версии библиотек, а для их установки нужно обновить уже установленные пакеты. Иногда процесс настолько масштабный, что приходится обновлять дистрибутив. А ведь мы знаем, что пользователь не хочет этого делать по ряду причин, да и это опасно: система может быть разрушена из-за нарушения связей между программами и библиотеками.

Есть и другая ситуация — когда нужно установить приложение, пакет которого конфликтует с уже установленным пакетом. Просто устанавливаемое приложение использует библиотеки, которые не могут «ужиться» с теми, которые уже установлены.

Проблемы настолько частые, то были предложены снапы. Пакет содержит саму программу, а также различные вспомогательные файлы — документацию, ресурсы (картинки, например), файлы локализации, какие-то сценарии и т.д. Но пакет не содержит всего, что нужно для работы этой программы в системе. Например, если программе для работы нужна библиотека **lib**, то просто в пакете «прописывается» зависимость — нужно установить такой-то пакет для работы



этого пакета. При установке программы менеджер пакетов (apt) производит разрешение зависимостей — устанавливает все необходимые для работы этой программы пакеты. С одной стороны, такой подход позволяет экономить место на диске. Ведь одну и ту же библиотеку не нужно устанавливать несколько раз. С другой стороны, это порождает уже описанные ранее проблемы.

Снап — это решение всей головной боли, как пользователя, так и разработчика приложения.

Снап можно считать таким пакетом, в котором содержится не только программа, но и все необходимые для ее работы библиотеки.

Получается, что все, что нужно для работы программы, содержится в снапе. Да, это неэкономно, но, согласитесь, жестким диском размером в 1 Тб сегодня никого не удивить, а головной боли будет меньше. Вы просто установите программу и будете ее использовать, а не два дня пытаться решить проблемы, возникшие при установке ее пакета.

Также не нужно бояться, что система превратится в мусорку. Снапы устанавливаются в отдельной папке в виде защищенного от записи образа. Изменения, которые должен внести снап в файловую систему (например, в каталог /lib), будут внесены в виртуальной файловой системе. Таким образом, установка снапа никак не повлияет на работу основной системы и на другие снапы. Снапы решают проблемы с совместимостью версий разных приложений.

Для начала работы со снапами в старых версиях Ubuntu (например, в 16.04, в более ранних версиях они не поддерживаются вообще) нужно сначала установить пакет **snapt**. В современных версиях данный пакет уже установлен.

Для поиска доступных снапов введите команду:



snap find <название>

Например:

snap find hello-world

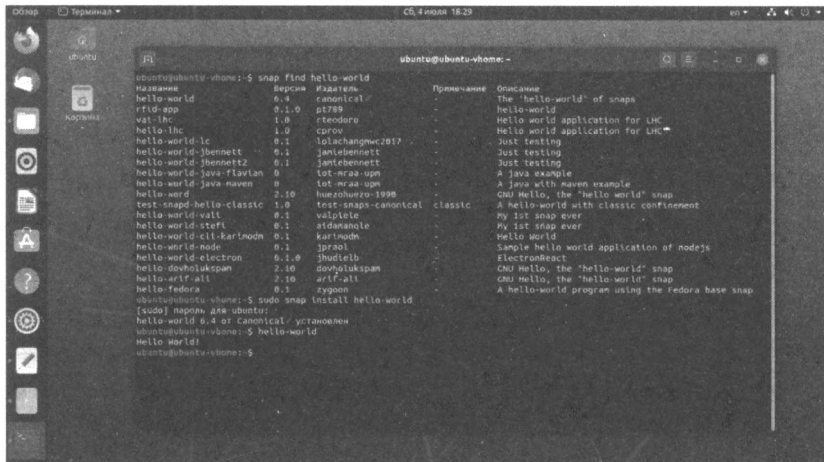


Рис. 3.8. Поиск, установка и запуск снапа

Вывод этой команды изображен на рис. 3.8. В первой колонке приводится название снапа. Пусть мы хотим установить снап *hello-world*:

sudo snap install hello-world

Как видите, команда установки снапа аналогична команде установки пакета, только вместо **apt** используется утилита **snap**.

После установки снапа можно запустить имеющуюся в нем программу:

hello-world
Hello, world!



Просмотреть установленные снапы можно командой *snap list* (рис. 3.9).

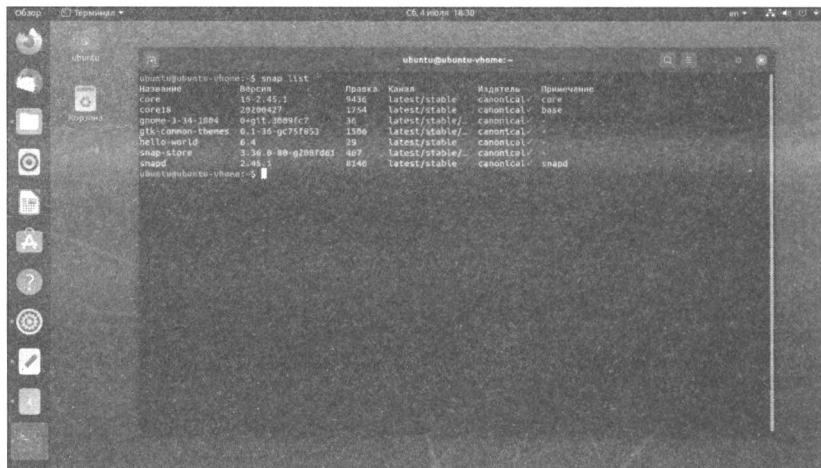


Рис. 3.9. Команда *snap list*

Как и пакеты, снапы можно обновить. Например:

```
sudo snap refresh hello-world
```

Данная команда обновит снап hello-world, если для него доступны обновления. Это очень удобно — вы обновляете не только приложения, но и все необходимые для его работы библиотеки.

Для обновления всех снапов используется другая команда:

```
sudo snap refresh
```



Теперь вы знаете, для чего используются снапы и что это вообще такое. А вот обновлять систему или использовать снапы — каждый решает сам.

3.7. Ошибка при выполнении apt:

Unable to acquire the dpkg lock /var/lib/dpkg/lock

К сожалению, время от времени при работе с Linux возникают различные ошибки. Одна из наиболее часто частых выглядит так:

Не удалось получить доступ к файлу блокировки /var/lib/dpkg/lock-frontent - open (11: Ресурс временно недоступен)

Ошибка возникает при попытке установить пакет в Ubuntu с помощью команды *apt install*.

Сообщение об ошибке может немного отличаться в зависимости от различных условий. Например, могут появляться следующие ошибки:

```
E: Could not get lock /var/lib/dpkg/lock - open (11: Resource temporarily unavailable)
```

```
E: Unable to lock the administration directory (/var/lib/dpkg/), is another process using it?
```

```
E: Could not get lock /var/lib/apt/lists/lock - open (11: Resource temporarily unavailable)
```

```
E: Unable to lock directory /var/lib/apt/lists/
```

```
E: Could not get lock /var/lib/dpkg/lock - open (11: Resource temporarily unavailable)
```

```
E: Unable to lock the administration directory (/var/lib/dpkg/), is another process using it?
```



Данные ошибки появляются, когда программа **apt** не может получить доступ к файлу блокировки `/var/lib/dpkg/lock*`. Данный файл используется, чтобы запретить одновременное выполнение операций, связанных с управлением пакетами в системе, так как при одновременном изменении данных о пакетах будет нарушена целостность «пакетной базы».

Обычно существует две основные причины появления описанных ошибок:

1. В данный момент уже выполняется экземпляр программы **apt**.
2. Предыдущий вызов **apt** завершился некорректно.

Сначала нужно убедиться, что уже не запущен другой экземпляр программы **apt-get** (**apt**). Выполним следующую команду, чтобы проверить, есть ли **apt** в списке запущенных процессов:

```
ps aux | grep -i apt
```

Вывод может быть таким:

```
ubuntu 8425 0.0 0.0 79516 3752 pts/1 S+ 10:31 0:00 sudo aptt install inkscape
ubuntu 8456 0.0 0.0 38892 944 pts/0 S+ 10:32 0:00 grep --color=auto -i apt
```

В первой строке мы видим, что уже есть работающий экземпляр программы **apt**, который имеет PID (идентификатор) 8425. Вторая строка относится к нашей команде **grep**, которую мы запустили с аргументом **apt**, поэтому она вывела саму себя. Итак, нас интересует только первая строка.



Если вы уверены, что не запускали программу **apt** сами или она не запущена в фоновом режиме, например, не выполняется автоматическое обновление системы, то нужно принудительно завершить ее выполнение. Для этого воспользуемся командой *kill -9*. Команде нужно указать числовой идентификатор процесса. В нашем случае это 8425:

```
sudo kill -9 8425
```

После выполнения данной команды процесс с идентификатором 8425 завершится.

Если первый способ вам не помог, то нужно удалить все файлы блокировки. Для этого выполняем команды:

```
sudo rm /var/lib/apt/lists/lock
sudo rm /var/cache/apt/archives/lock
sudo rm /var/lib/dpkg/lock
sudo rm /var/lib/dpkg/lock-frontent
```

Если при выполнении каких-нибудь из этих команд появится сообщение:

rm: невозможно удалить '/var/./lock': Нет такого файла или каталога

это нормально, не обращайтесь на него внимания.

После этого нужно выполнить переконфигурацию пакетов:

```
sudo dpkg --configure -a
```



3.8. Невозможно найти определенный пакет

При работе над нашим справочником мы столкнулись со следующей ситуацией – когда вы вводите команду установки пакета, например:



```
sudo apt install synaptic
```

то менеджер пакетов сообщает вам, что такой пакет не найден. Проблема оказалась в том, что список пакетов почему-то не синхронизировался с сервером `ru.ubuntu.com`.

Для ее решения запустите `update-manager`, нажмите кнопку **Настройки** и настройте менеджер пакетов на использование основного сервера, как показано на рис. 3.10.

Не факт, что такая проблема проявится у вас, но если так произойдет, то вы знаете, что с этим делать.

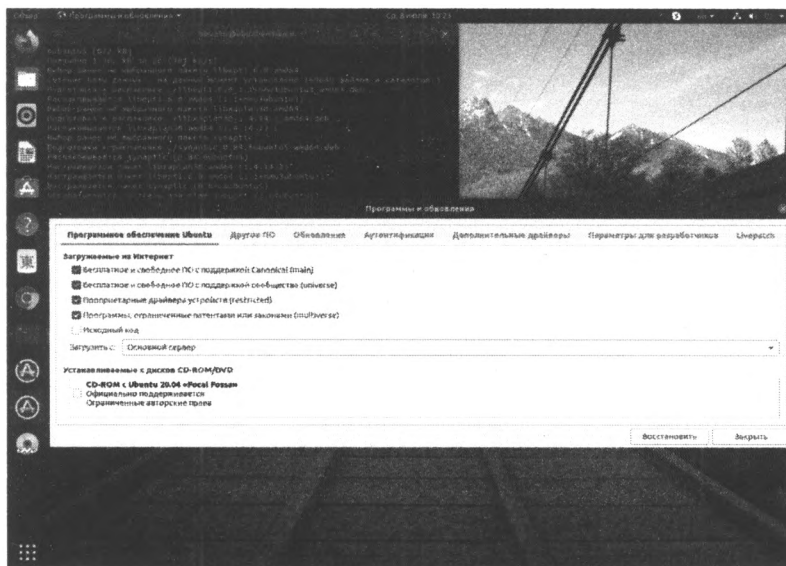
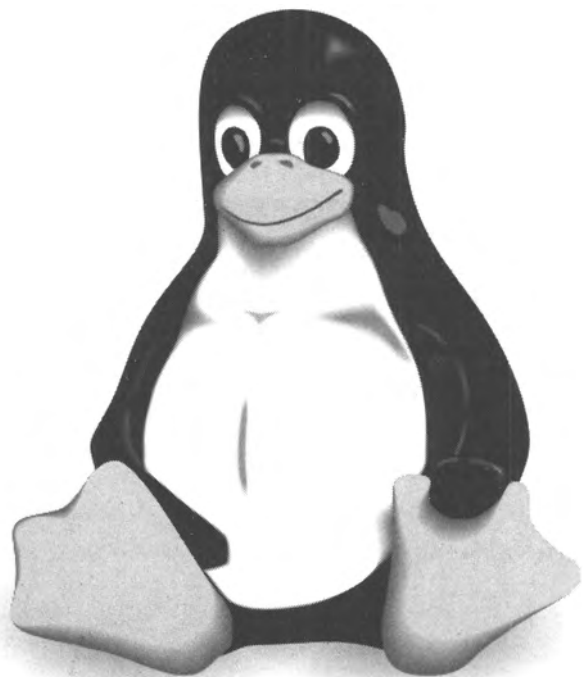


Рис. 3.10. Использование основного сервера пакетов



ГЛАВА 4.

ПОПУЛЯРНЫЕ ПРОГРАММЫ





4.1. Офисные пакеты

Когда-то в Linux было многообразие офисных пакетов. Со временем все эти пакеты были вытеснены одним — LibreOffice, в котором содержатся все необходимые аналоги программ из MS Office:

- Текст LibreOffice (LibreOffice Writer) — текстовый процессор, аналог MS Word.
- Таблица LibreOffice (LibreOffice Calc) — электронная таблица, аналог MS Excel.
- Презентация LibreOffice (LibreOffice Impress) — презентации, аналог MS PowerPoint.
- Рисунок LibreOffice (LibreOffice Draw) — векторный графический редактор для создания блок-схем и диаграмм, аналог MS Visio.
- База данных (LibreOffice Base) — база данных, аналог MS Access.



Данные программы не просто выполняют те же функции, что и программы из пакета MS Office, но и поддерживают форматы MS Office. Да, интерфейс программ несколько специфичен (напоминает старые версии MS Office), но при желании можно разобраться. К тому же есть версия LibreOffice для Windows. Офисный пакет полностью бесплатный, поэтому есть неплохая возможность сэкономить, если установить его еще и на Windows-машины.

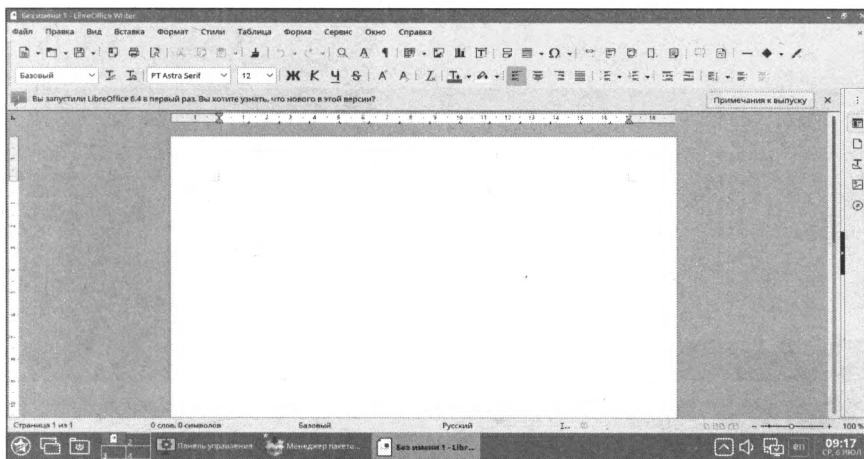


Рис. 4.1. Текстовый процессор LibreOffice Writer

Если же какого-то функционала MS Office будет не хватать, тогда не забывайте об онлайн-версии Office 365, которую можно использовать прямо в браузере.

4.2. Графические текстовые редакторы

Для редактирования обычного текста (без форматирования), например, сценариев командной оболочки или конфигурационных файлов,



можно обойтись стандартным текстовым редактором Kate. Для редактирования файлов конфигурации нужны права *root*, поэтому запускать редактор нужно из терминала так:

```
sudo kate <имя_файла>
```

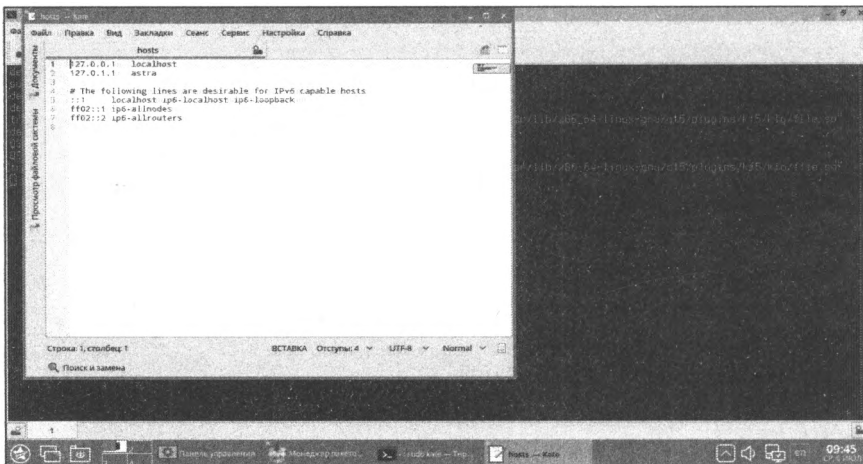
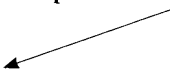


Рис. 4.2. Текстовый редактор kate

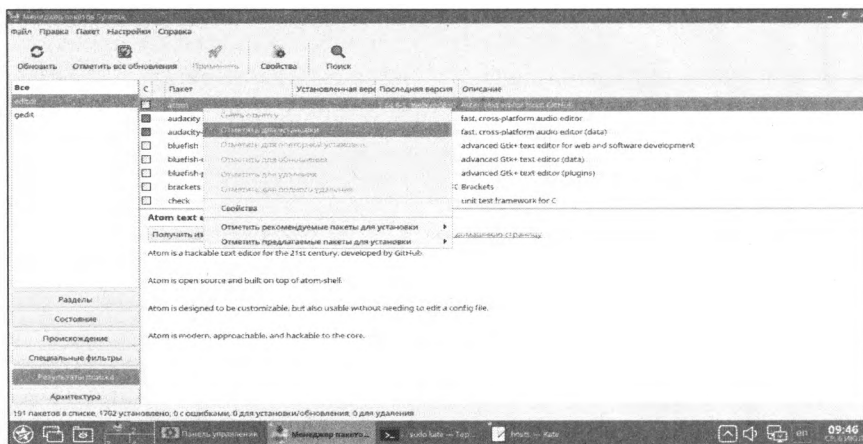


Рис. 4.3. Установка текстового редактора Atom



Штатный текстовый редактор обладает возможностью поиска и замены текста, нумерует строки (**Вид, Границы, Номера строк**), показывает позицию курсора. Для редактирования текстовых файлов конфигурации — то, что нужно.

Если возможностей этого редактора вам окажется мало, используя Synaptic, вы всегда сможете установить редактор Atom (рис. 4.3). Это профессиональный текстовый редактор для программистов, и благодаря системе плагинов его возможности практически безграничны (рис. 4.4).



Рис. 4.4. Текстовый редактор Atom

4.3. Консольные текстовые редакторы

Работая в графическом режиме, вам вряд ли захочется использовать консольные текстовые редакторы. Но при администрировании виртуального сервера выбора у вас не будет.



Виртуальные серверы Linux не оснащены каким-либо графическим интерфейсом, поэтому их администрирование осуществляется через консоль — или посредством Web-консоли, встроенной в панель управления, или же по SSH. Часто у администратора возникает потребность отредактировать какой-то файл конфигурации сервера. Проблем с этим, как правило, никаких нет — запускаешь предпочитаемый текстовый редактор, открываешь файл, редактируешь и сохраняешь. Однако редактирование некоторых файлов конфигурации, в частности `/etc/sudoers`, осуществляется только посредством специальных утилит (в данном случае — **visudo** или **crontab** — при редактировании расписания планировщика), которые запускают текстовый редактор по умолчанию. Таковым редактором является редактор **vi**, перекочевавший в современные дистрибутивы Linux с 1970-х годов, и его нельзя назвать удобным. В этой заметке будут рассмотрены некоторые текстовые редакторы, и будет показано, как по умолчанию установить понравившийся редактор, чтобы он вызывался при редактировании некоторых специальных файлов конфигурации, которые нельзя редактировать вручную.

Самый удобный — редактор **nano** (раньше он назывался **pico** и входил в состав почтового клиента **pine**). Редактор **nano** изображен на рис. 4.5.

Внизу (под текстом) есть подсказка по комбинациям клавиш для управления редактором. Символ `^` означает `<Ctrl>`. То есть для выхода из редактора нужно нажать комбинацию клавиш `<Ctrl>+<X>`, а для сохранения текста — `<Ctrl>+<O>`.

В пакет **mc** (файловый менеджер) входит довольно удобный редактор **mcedit**, который запускается при нажатии клавиши `<F4>` в **mc** (рис. 4.6). При желании вы можете запустить редактор отдельно:

```
mcedit <имя файла>
```

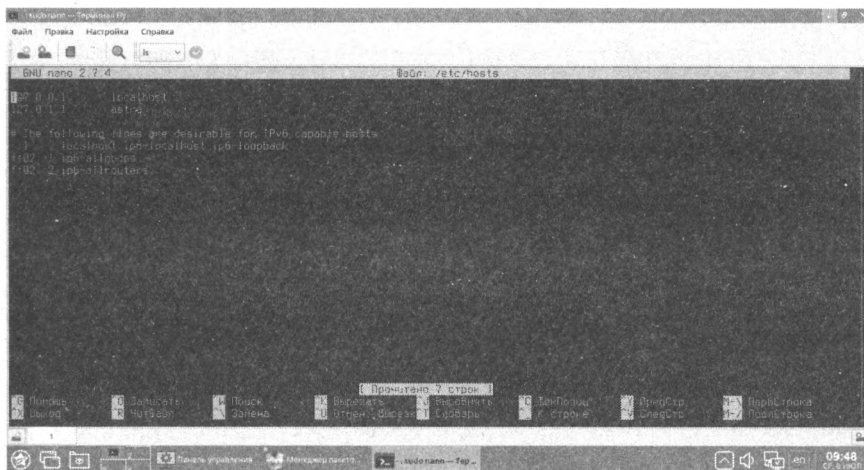


Рис. 4.5. Текстовый редактор nano

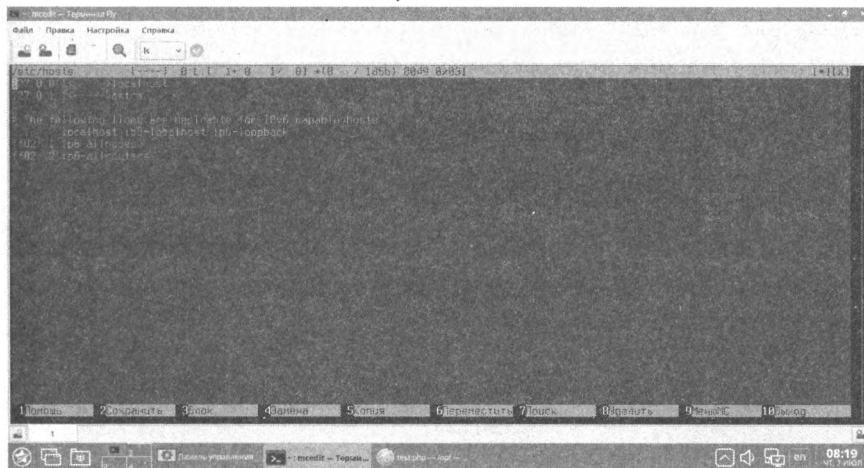


Рис. 4.6. Редактор mcedit

Классический синий фон, подсказки функциональных клавиш внизу и т.д. Редактор не менее удобен, чем **nano**.



Выбор конкретного редактора зависит от ваших личных предпочтений. Но в любом случае, каждый из представленных редакторов будет удобнее, чем стандартный **vi**.

Некоторые утилиты, например, **crontab**, **visudo**, вызывают текстовый редактор по умолчанию для редактирования тех или иных данных. В этом случае будет вызван **vi**, который, как было отмечено, неудобен. Чтобы вызывался нужный вам редактор, его нужно сделать редактором по умолчанию. Для этого нужно установить переменную окружения **EDITOR**:

```
which nano
/bin/nano
export EDITOR=/bin/nano
```

Первая команда (*which nano*) сообщает путь к выбранному редактору. Далее этот путь нам нужно указать в качестве значения переменной **EDITOR**.

Вот только помните, что при следующем входе в систему переменная **EDITOR** будет установлена по умолчанию. Чтобы этого не произошло, нужно отредактировать файл **.bashrc** того пользователя, от имени которого будете редактировать конфигурационные файлы. В случае с **root** это будет файл **/root/.bashrc**:

```
cd ~
nano .bashrc
```

В этот файл нужно добавить команду:

```
export EDITOR=/bin/nano
```



Сохраните файл. Теперь нужно выйти из системы (команда *exit*) и снова войти (по **SSH** или через **Web-консоль**).

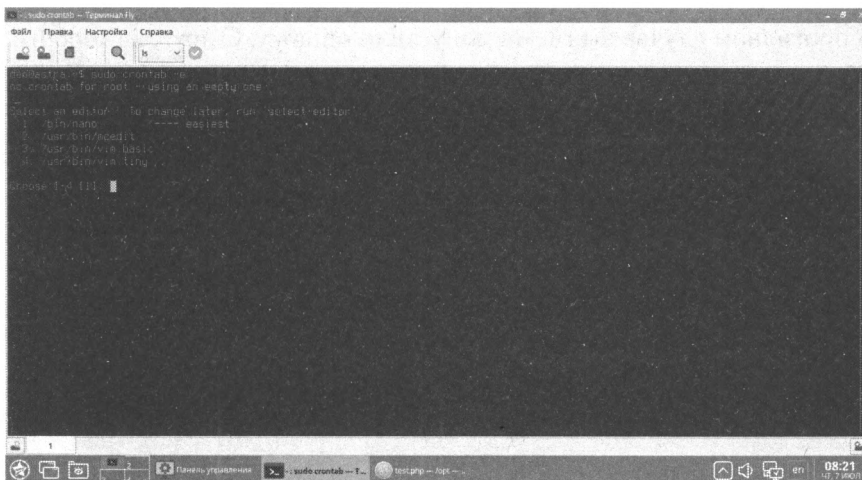


Рис. 4.7. Меню выбора редактора

После этого запустите любую команду, вызывающую стандартный текстовый редактор, например, *sudo crontab -e*. Если вы увидели выбранный вами текстовый редактор, значит, все прошло нормально (рис. 4.8).



Рис. 4.8. При редактировании расписания (команда *crontab -e*) открылся nano. Настройка успешна



В противном случае вы где-то допустили ошибку. С другой стороны, в Linux при первом запуске **crontab** открывается меню выбора редактора, что существенно упрощает процесс настройки (рис. 4.7).

4.4. Программы для работы с Интернетом

Основные программы для работы с Интернетом установлены в Linux по умолчанию.

К ним относятся:

- Браузер Firefox
- Браузер Chromium (аналог Chrome)
- Почтовый клиент Thunderbird
- Мессенджер Psi+ (Jabber)
- Torrent-клиент qBittorrent

По большому счету основной программой сейчас является браузер. Даже почтовый клиент уже не нужен, так как доступ к почтовому интерфейсу осуществляется через веб-интерфейс. Аналогично, посредством веб-интерфейса осуществляется доступ к популярным мессенджерам, например, к таким как:

- Skype — <https://www.skype.com/ru/features/skype-web/>
- WhatsApp — <https://web.whatsapp.com/>



Также есть мессенджер Psi+ — это Jabber, который не очень распространен, и вы вряд ли будете его использовать.

4.5. Linux-аналоги Windows-программ

Новичкам в Linux не всегда просто, поскольку они попросту не знают, какое приложение установить. Не всегда есть Linux-версии известных программ, но в большинстве случаев есть несколько аналогов. Таблица 4.1 поможет вам выбрать Linux-программу на все случаи жизни.

Таблица 4.1. Аналоги Windows-программ

Название Windows-приложения	Linux-аналоги
ACDSee	gThumb F-Spot Shotwel DigiKam PhotoQt GPicView
Acronis True Image	CloneZilla
Adobe Audition	Ardour



Adobe Illustrator	LibreOffice Fraw Inkscape sK1 Xara Xtreme
Adobe Photoshop	GIMP Krita Photivo
Adobe Lightroom	Darktable
Adobe Premiere Pro	Open Movie Editor PiTiVi
Adobe Reader	Xpdf Evince Google Chrome
Alcohol 120	Furius ISO Mount
AnyDesk	Remmina
Apple iCloud for Windows	iCloud for Linux
Ardour	Ardour
Audacity	Ardour



Avira Antivirus	ClamAV Avira AntiVir Personal Avast Antivirus Linux Home Edition
Blender	Blender
CINEMA 4D	Blender
Corel Draw	Inkscape LibreOffice Draw sK1 Xara Xtreme
CPU Z	I-Nex CPU-X CPU-G
CuteFTP	FOFF FileZilla gFTP
CyberLink PowerDVD	Totem



Daemon Tools	Furius ISO Mount Etcher
Darktable	Darktable
DaVinci Resolve	Avidemux
Deluge	Transmission
Discord	Discord
Dr.Web Antivirus	Avira AntiVir Personal Avast Antivirus Linux Home Edition ClamAV
Dropbox	Dropbox
eMule	aMule
ESET NOD32 Antivirus	Avira AntiVir Personal Avast Antivirus Linux Home Edition ClamAV
Etcher	Etcher



Evernote	Boostnote MyNotex Zim CherryTree
FileZilla	FileZilla
FineReader	gImageReader
Foxit Reader	Xpdf Evince Aesop
Freehand	sK1
GeoGebra	GeoGebra
GOG	Steam
Google Chrome	Google Chrome
Google Earth	Google Earth Marble
InterVideo WinDVD	MPlayer Totem



IrfanView	gThumb GPicView Mirage Ristretto
LMMS	Ardour
Maple, Magma, Mathematica, Matlab	Sage
Maya	Blender
Microsoft Access	LibreOffice Base
Microsoft Excel	LibreOffice Calc IBM Lotus Symphony
Microsoft Office Publisher	LibreOffice Draw
Microsoft OneNote	CherryTree
Microsoft Outlook	Evolution Thunderbird Sylpheed



Microsoft Outlook Express	Evolution Thunderbird Sylpheed
Microsoft Paint	Gnome Paint Pinta
Microsoft PowerPoint	IBM Lotus Symphony LibreOffice Impress
Microsoft Visio	LibreOffice Draw
	Code Blocks KDevelop Visual Studio Code NetBeans Eclipse Qt Creator
Microsoft Windows Journal	Xournal



Microsoft Windows Media Player	MPlayer Lollypop Celluloid Totem GMusicBrowser Clementine Rhythmbox Audacious VLC
Microsoft Windows Movie Maker	Open Movie Editor PiTiVi Flowblade Lightworks Avidemux
Mp3tag	EasyTag
MPV	Celluloid



Nero	K3b Brasero GnomeBaker
Norton Commander	emelfM2 Double Commander GNOME Commander Midnight Commander
Notepad++	JuffEd Medit Atom
Opera	Opera Google Chrome Epiphany
Partition Magic	GParted
Picasa	DigiKam
qBittorrent	Transmission
Qt Creator	KDevelop



QuarkXPress	Scribus
ReGet	Steadyflow JDownloader
Remote Desktop Connection	Remmina
Rufus	Etcher
Scite	Medit Screem Scribes Bluefish Editor SciTE Komodo Edit Geany
SketchUp	Blender
Skype	Skype
Slack	Discord
SmartFTP	Filezilla
SolidWorks	FreeCAD
Steam	Steam



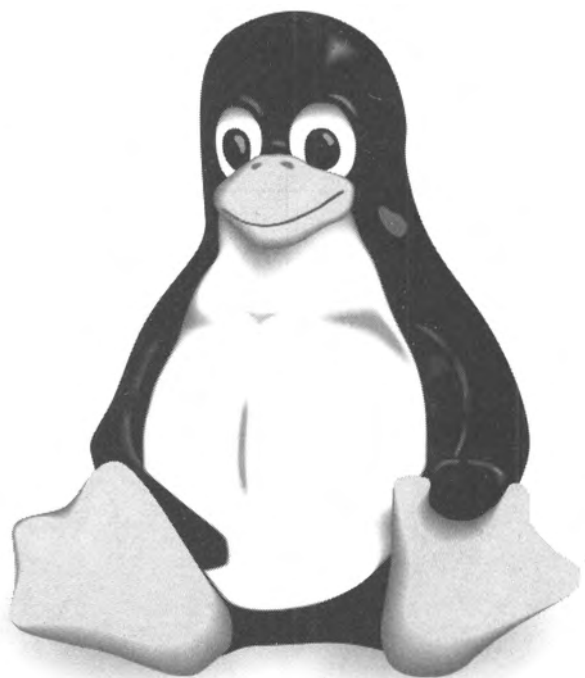
TeamViewer	Remmina
The Bat	Thunderbird Claws Mail
Total Commander	4Pane GNOME Commander Midnight Commander Krusader emelFM2
UltraISO	Etcher
Universal USB Installer	Etcher
uTorrent	qBittorrent Deluge Tixati Transmission
Viber	Viber
VirtualDub	PiTiVi Avidemux Open Movie Editor



Visual Studio Code	Visual Studio Code
VLC	VLC
Win32 Disk Imager	Etcher
WinAMP	Decibel Audio Player QMMP Clementine Totem Audacious AlsaPlayer Sayonara MPlayer
Wireshark for Windows	Wireshark
Xara	Inkscape Xara Xtreme



XnView	DigiKam Mirage GPicView PhotoQt gThumb
Zoom	Discord
3D Studio Max	Blender



ГЛАВА 5.

ФАЙЛОВАЯ СИСТЕМА





5.1. Какие файловые системы поддерживает Linux

Операционная система Linux поддерживает очень много файловых систем. Но самое главное — это модульный принцип организации ядра Linux. Даже если кто-то сегодня создаст файловую систему, о которой еще вчера никто ничего не знал, то ему достаточно создать модуль ядра и Linux будет поддерживать его операционную систему.

Родным для Linux является семейство файловых систем ext*.

Самая древняя файловая система Linux называлась ext (сегодня вы вряд ли с ней столкнетесь), затем появились ext2, ext3 и ext4. Еще в 2010 году ходили слухи о ext5, но ее так и не создали.

Файловые системы ext3 и ext4 являются журналируемыми, то есть они ведут «журналы» своей работы, что позволяет произвести восстановление информации в случае сбоя. Журналы работают так:



перед осуществлением операции файловая система записывает в журнал эту операцию, а после выполнения операции — удаляет запись из журнала. Если после занесения информации в журнал произошел сбой (например, отключение электричества), то после его устранения (подача электричества) файловая система выполнит все действия, которые она не успела выполнить. Конечно, это не панацея и резервные копии никто не отменял. Но все же лучше, чем ничего.

Однако не во всех дистрибутивах ext4 используется по умолчанию. Linux также поддерживает и другие файловые системы: XFS, ReiserFS, BtrFS, ZFS, JFS. Вы можете встретить дистрибутивы, в которых по умолчанию используется одна из этих файловых систем. У каждой из этих файловых систем есть свои отличия:

- **JFS (Journaled File System)** — 64-битная журналируемая файловая система созданная IBM, распространяется по лицензии GPL и благодаря этому факту она оказалась в Linux. Обладает высокой производительностью, но у нее маленький размер блока (от 512 байт до 4 Кб), поэтому на сервере данных ее можно использовать с большим успехом, но не на рабочих станциях, на которых производится обработка видео в реальном времени, так как размер блока для этих задач будет маловат. В отличие от ext3, в которую поддержка журнала была добавлена (по сути, ext3 — это то же самое, что и ext2, но с журналом), JFS была изначально журналируемой. Максимальный размер тома — 32 Пб, максимальный размер файла — 4 Пб.
- **ReiserFS** — самая экономная файловая система, поскольку позволяет хранить в одном блоке несколько файлов. В других файловых системах файл должен занимать как минимум 1 блок и получается, что если размер файла меньше размера блока, то «остаток» просто не используется. Когда в системе много небольших файлов, дисковое пространство используется очень нерационально. В ReiserFS все иначе. Если размер блока, скажем, 4 Кб, то в него могут поместиться несколько файлов общим размером 4 Кб, а не только один, например, два файла по



2 Кб. Максимальный размер тома и файла зависят от версии ReiserFS и разрядности системы.

- **XFS** — высокопроизводительная (до 7 Гбайт/с) файловая система, разработанная Silicon Graphics. Изначально была рассчитана на большие размеры накопителей (более 2 Тб) и большие размеры файлов. Очень хорошо проявила себя при работе с файлами большого размера. Размер блока у этой файловой системы — от 512 байт до 64 Кбайта. Выделяет место экстен-тами (Extent — указатель на начало и число последовательных блоков). В экстентах выделяется место для хранения файлов, а также экстентами хранятся свободные блоки. Именно эта фай-ловая система используется по умолчанию в современных вер-сиях Fedora Server.
- **Btrfs (B-tree FS, «Better FS» или «Butter FS»)** — файловая система, разработанная специально для Linux и основанная на структурах Б-деревьев. Работает по принципу «копирование при записи» (copy-on-write). Создана компанией Oracle Corporation в 2007 году, распространяется по лицензии GPL. Изначально планировалась как конкурент популярной файло-вой системы ZFS.
- **ZFS (Zettabyte File System)** — файловая система, созданная в Sun Microsystems для операционной системы Solaris. Позже она появилась и в Linux. Ее особенность — полный контроль над физическими и логическими носителями.

Кроме перечисленных выше файловых систем, Linux поддержива-ет еще и файловые системы Windows — FAT, FAT32, NTFS. Также поддерживаются всевозможные сменные носители вроде оптических дисков, флешек, внешних жестких дисков и, соответственно, фай-ловые системы на этих сменных носителях. На внешних жестких дисках используются файловые системы FAT32 или NTFS, если вы специально не переформатировали их в файловую систему Linux. На оптических дисках могут использоваться UDF, ISO 9660, Joliet и по-добные.



5.2. Какую файловую систему выбрать?

Файловых систем довольно много, так какую из них выбрать для вашей системы? На рабочих станциях и серверах общего назначения я бы использовал ext4 или ReiserFS. Последняя особенно хороша, если у вас много мелких файлов — тогда дисковое пространство будет использоваться более рационально.

На сервере баз данных лучше использовать JFS — тогда прирост производительности вам гарантирован. А вот XFS для некоторых видов серверов подходит так себе, однако это не помешало разработчикам Fedora Server использовать эту файловую систему по умолчанию в своем дистрибутиве (начиная с версии 22). Видимо, повлияла высокая производительность и ориентация на большие объемы накопителей и файлов.

Очень неплохой является файловая система ZFS, особенно она хороша при управлении различными дисковыми устройствами. Вы можете создать пул и добавить в него несколько дисковых устройств. Этим ZFS чем-то похожа на LVM — менеджер логических томов, который мы рассмотрим чуть позднее.

Не спешите с выбором файловой системы именно сейчас. В конце этой главы вы найдете подробную информацию об ext4. Может, в ней вы найдете ответы на все ваши вопросы и остановите свой выбор на ext4.

Выбор файловой системы нужно производить даже не на основе характеристик самой файловой системы, а исходя из назначения компьютера. Например, для рабочей станции с головой хватит ext4. С сервером сложнее — нужно учитывать, какой это будет сервер. Как



уже отмечалось, для сервера БД — JFS, для хостинга (где хранятся файлы других пользователей) — ReiserFS, если нужно работать с большими объемами данных — XFS.

5.3. Что нужно знать о файловой системе Linux

5.3.1. Имена файлов и каталогов

Нужно помнить следующие правила именования файлов и каталогов в Linux:

- Linux чувствительна к регистру символов, то есть файлы Document.txt и document.TXT — это разные документы.
- В Linux нет понятия «расширение» файла. В Windows мы привыкли, что последние символы после последней точки (обычно от 1 до 4 символов) называются расширением. В Linux такого понятия нет. Если кто-то и употребляет термин «расширение» в Linux, то это только для того, чтобы бывшим Windows-пользователям (которыми являемся, по сути, все мы) было понятнее, что имеется в виду.
- Максимальная длина имени файла — 254 символа.
- Имя может содержать любые символы (в том числе и кириллицу), кроме `/ \ ? < > * «|`.
- Разделение элементов пути осуществляется с помощью символа `/`, а не `\`, как в Windows. В Windows мы привыкли к путям вида `C:\Users\John`, в Linux используется прямой слеш: `/home/john`.



- Если имя файла начинается с точки, он считается скрытым. Пример: `.htaccess`.

5.3.2. Файлы устройств

Уникальность файловой системы Linux в том, что для каждого устройства в Linux создается собственный файл в каталоге `/dev`. Загляните в каталог `/dev` — в нем вы найдете множество файлов для всех устройств вашей системы. Вот примеры некоторых файлов устройств:

- `/dev/sda` — первый жесткий диск, как правило, подключенный к первому SATA-контроллеру;
- `/dev/sda1` — первый раздел на первом жестком диске. Нумерация разделов жестких дисков в Linux начинается с 1;
- `/dev/mouse` — файл устройства мыши;
- `/dev/cpu` — файл устройства процессора;
- `/dev/cdrom` — ваш CD/DVD-привод;
- `/dev/random` — файл устройства-генератора случайных чисел;
- `/dev/tty1` — первая консоль (терминал).

Файлы устройств бывают двух типов: символьные, обмен информацией с которыми осуществляется посимвольно, и блочные, обмен информацией с которыми осуществляется блоками данных. Пример символьного устройства — `/dev/ttyS0` — последовательный (COM) порт, пример блочного устройства — `/dev/sda1` — раздел жесткого диска.



5.3.3. Корневая файловая система и основные подкаталоги первого уровня

Самое большое отличие, к которому придется вам привыкнуть, — это наличие корневой файловой системы. Вспомните, как Windows управляет жесткими дисками. Представим, что у нас есть жесткий диск с двумя логическими дисками (разделами). Первый будет в Windows называться C:, а второй — D:. У каждого из этих логических дисков будет свой корневой каталог — C:\ и D:\.

В Linux все иначе. Представьте, что мы разбили жесткий диск /dev/sda на два раздела (как и в случае с Windows). Первый будет называться /dev/sda1 (Windows бы его назвала C:), а второй — /dev/sda2 (в Windows он был бы D:).

Мы установили Linux на первый раздел /dev/sda1. Точка монтирования этого раздела будет /, что соответствует корневой файловой системе. Второй раздел вообще никак не будет отображаться, пока вы его не *подмонтируете*. Подмонтировать можно к любому каталогу. Например, вы можете подмонтировать раздел /dev/sda2 к каталогу /home, и тогда домашние каталоги пользователей будут храниться физически на другом разделе. Точка монтирования — это каталог, через который осуществляется доступ к другому разделу. Правильнее сказать, даже к другой файловой системе, которая физически может находиться на другом разделе, на другом жестком диске, на внешнем жестком диске, флешке и т.д.

Корневая файловая система содержит стандартные каталоги. У каждого каталога есть свое предназначение, например, в каталоге **/bin** хранятся стандартные программы, в каталоге **/home** — домашние каталоги пользователей, в каталоге **/tmp** — временные файлы и т.д. Назначение стандартных каталогов приведено в таблице 5.1.



Таблица 5.1. Назначение стандартных каталогов корневой файловой системы Linux

Каталог	Описание
/	Каталог корневой файловой системы
/bin	Содержит стандартные утилиты (cat, ls, cp и т.д.)
/boot	Содержит конфигурационный файл загрузчика и некоторые модули загрузчика
/dev	Содержит файлы устройств
/etc	В этом каталоге находятся конфигурационные файлы системы и программ
/home	Здесь хранятся домашние каталоги пользователей
/lib	Содержит библиотеки и модули
/lost+found	В этом каталоге хранятся восстановленные после некорректного размонтирования файловой системы файлы
/misc, /opt	Опциональные каталоги, могут содержать все что угодно. Некоторые программы могут устанавливаться в каталог /opt
/media	Некоторые дистрибутивы монтируют сменные устройства (оптические диски, флешки) к подкаталогам этого каталога.



/mnt	Содержит точки монтирования. Как правило, здесь хранятся стационарные точки монтирования, которые обычно описываются в файле <i>/etc/fstab</i>
/proc	Каталог псевдофайловой системы <i>procfs</i>
/root	Каталог пользователя <i>root</i>
/sbin	Содержит системные утилиты. Запускать эти утилиты имеет право только пользователь <i>root</i>
/tmp	Содержит временные файлы
/usr	Может содержать много чего — пользовательские программы (несистемные программы), документацию, исходные коды ядра и т.д.
/var	Содержит постоянно изменяющиеся данные системы — почтовые ящики, очереди печати, блокировки (<i>locks</i>) и т.д.

5.4. Ссылки

Ссылки позволяют одному и тому же файлу существовать в системе под разными именами. Ссылки бывают жесткими и символическими. Сейчас разберемся, в чем разница. Если на файл указывает хотя бы одна жесткая ссылка, вы не сможете его удалить. Количество ссылок на файл можно узнать командой *ls -l*. Что касается символических ссылок, то вы можете удалить файл, если на него указывает хоть 100 символических ссылок. После этого они будут «оборваны» —



ссылки, как файлы, останутся на жестком диске, но они будут указывать на несуществующий файл.

У жестких ссылок есть одно ограничение. Они не могут указывать на файл, находящийся за пределами файловой системы. Представим, что каталог `/tmp` находится физически на одном и том же разделе, что и `/`. Тогда вы сможете создать ссылки на файлы, которые находятся в каталоге `/tmp`. Но если `/tmp` — это точка монтирования, к которой подмонтирован другой раздел, вы не сможете создать жесткие ссылки.

Для создания ссылок используется команда `ln`:

```
ln [-s] файл ссылка
```

Если параметр `-s` не указан, то будет создана **жесткая ссылка** на *файл*. Если параметр `-s` указан, то будет создана **символическая ссылка**.

5.5. Права доступа

5.5.1. Общие положения

В Linux, как и в любой многопользовательской системе, есть понятия владельца файла и прав доступа. Владелец — это пользователь, которому принадлежит файл. В большинстве случаев — это пользователь, создавший файл.

Права доступа определяют, кто и что может сделать с файлом. Права доступа файла может изменять владелец файла или пользователь *root*. Владелец может назначить, например, кто имеет право читать и изменять файл. Владелец также может «подарить» файл другому пользователю. После этого владельцем станет уже другой пользователь.



Права доступа у пользователя *root* максимальные, а это означает, что он может изменить владельца любого файла (вы можете создать файл, а *root* может сделать владельцем любого другого пользователя) и изменить права доступа любого файла. Пользователь *root* может удалить и изменить любой файл, может создать файл в любой папке и т.д. С одной стороны, это хорошо, но если злоумышленник завладеет паролем *root*, то хорошего в этой ситуации мало.

Права доступа в Linux по умолчанию настроены так, что пользователь владеет только своим домашним каталогом `/home/<имя_пользователя>`. Поэтому создавать файлы и выполнять другие операции по работе с файлами (удаление, редактирование, копирование и т.д.) пользователь может только в этом каталоге и то при условии, что файлы принадлежат ему.

Если в домашнем каталоге пользователя *root* создал файл, пользователь не сможет удалить или изменить его, поскольку он не является его владельцем. Сможет ли он прочесть этот файл, зависит от прав доступа к файлу (о них мы поговорим позже).

Остальные файлы, которые находятся за пределами домашнего каталога, пользователь может только просмотреть, и то если это не запрещено правами доступа. Например, файл `/etc/passwd` пользователь может просмотреть, а `/etc/shadow` — нет. Также пользователь не может создать файлы в корневой файловой системе или в любом другом каталоге, который ему не принадлежит, если иное не установлено правами доступа к этому каталогу.

5.5.2. Смена владельца файла

Команда *chown* используется для изменения владельца файла/каталога.



Формат такой:

```
chown <пользователь> <файл/каталог>
```

Здесь пользователь — это новый владелец файла. Чтобы подарить другому пользователю файл, вы должны быть или его владельцем, или пользователем *root*.

5.5.3. Определение прав доступа

Для изменения прав доступа используется команда *chmod*. Для изменения прав доступа вы должны быть владельцем файла/каталога или же пользователем *root*. Формат команды следующий:

```
chmod <права> <файл/каталог>
```

Права доступа состоят из *трех наборов*: для *владельца*, для *группы владельца* и для *прочих пользователей*.

Первый набор задает возможности владельца файла, второй — для группы пользователей, в которую входит владелец, и третий — для всех остальных пользователей.

В наборе может быть *три права* — *чтение* (r), *запись* (w) и *выполнение* (x). Для файла право на выполнение означает возможность запустить этот файл на выполнение (обычно используется для программ и сценариев). Право выполнение для каталога — возможность просматривать этот каталог.

Права доступа в наборе определяются четко в определенном порядке и могут быть представлены как в символьном, так и в числовом виде



(в двоичной или восьмеричной системе). Рассмотрим несколько наборов прав доступа:

- 100 — только чтение,
- 110 — чтение и запись,
- 101 — чтение и выполнение,
- 111 — чтение, запись, выполнение.

Учитывая, что права доступа задаются для владельца, группы и остальных пользователей, полный набор прав доступа может выглядеть так:

```
111 100 000
```

В этом наборе мы предоставляем полный доступ (в том числе и выполнение) владельцу, группе владельца разрешено только чтение, остальным пользователям доступ к файлу/каталогу вообще запрещен.

В двоичной системе права доступа мало кто записывает. В основном их преобразуют в восьмеричную систему. Если вы забыли, как это делается, то вам поможет следующая таблица (табл. 5.2).

Таблица 5.2. Преобразование из двоичной в восьмеричную систему

Двоичная система	Восьмеричная система
000	0
001	1
010	2
011	3

100	4
101	5
110	6
111	7

Если вы видите право доступа 666, то никакой дьявольщины в нем нет, это всего лишь полный доступ к обычному файлу (не к программе и не к сценарию). Для каталога полные права доступа выглядят как 777 — чтение, изменение и просмотр каталога для владельца, группы и прочих пользователей.

Просмотреть текущие права доступа можно командой `ls -l <файл/каталог>`, например:

```
# ls -l config
```

```
-rw-r--r--. 1 root root 110375 янв 2 08:28 config
```

Как мы видим, задано три набора *rw-*, *r--*, *r--*. Выходит, владельцу разрешена запись и чтение файла, остальным пользователям (группа и прочие) — только чтение. В восьмеричной системе этот набор прав доступа выглядит как 644.

Первый символ (в нашем случае это «-») является признаком каталога. Если бы мы выводили права доступа каталога, то вместо «-» здесь был бы символ *d*. Для файла выводится просто «-».

Символьный способ задания прав доступа немного проще, но лично я предпочитаю числовой. Рассмотрим, как использовать символьный:

```
# chmod +x config
```



Посмотрим опять права доступа:

```
# ls -l config
```

```
-rwxr-xr-x. 1 root root 110375 sep 2 08:28 config
```

Как видите, право выполнение было добавлено во все три набора прав доступа.

5.5.4. Специальные права доступа

В Linux есть еще специальные права доступа SUID (Set User ID root) и SGID (Set Group ID root), позволяющие обычным пользователям запускать программы, которые требуют для своей работы прав *root*.

В современных дистрибутивах Linux вам придется изменять эти права доступа чрезвычайно редко (может быть даже вообще никогда), но вам нужно знать, как их изменить. Например, если программе `/usr/sbin/program` вы хотите разрешить запускать с правами *root* обычным пользователям, установите права доступа так:

```
# chmod u+s /usr/sbin/program
```

Использование SUID — плохое решение с точки зрения безопасности. Правильнее использовать команду *sudo*, если какому-то пользователю будут нужны права *root*.

5.6. Атрибуты файла

В Linux кроме прав доступа есть еще и атрибуты файла, подобно атрибутам файла в других операционных системах. Изменить атрибуты файла можно командой *chattr*:



```
chattr +/-<атрибуты> <файл>
```

Просмотреть установленные атрибуты можно командой *lsattr*:

```
lsattr <файл>
```

Некоторые полезные атрибуты файлов приведены в таблице 5.3.

Таблица 5.3. Полезные атрибуты файлов

Атрибут	Описание
<i>i</i>	Запрещает изменение, переименование и удаление файла. Этот атрибут можно установить для критических конфигурационных файлов или для каких-либо других критических данных. Установить (как и сбросить) этот атрибут может только пользователь <i>root</i> или процесс с возможностью <i>CAP_LINUX_IMMUTABLE</i> . Другими словами, сбросить этот атрибут просто так нельзя — нужны только права <i>root</i>
<i>u</i>	При удалении файла с установленным атрибутом <i>u</i> его содержимое хранится на жестком диске, что позволяет легко восстановить файл
<i>c</i>	Файл будет сжиматься. Можно установить этот атрибут для больших файлов, содержащих несжатые данные. Доступ к сжатым файлам будет медленнее, чем к обычным, поэтому плохое решение — устанавливать этот атрибут для файлов базы данных. Этот атрибут нельзя устанавливать для файлов, уже содержащих сжатые данные — архивы, JPEG-фото, MP3/MP4-файлы и т.д. Этим вы не только не уменьшите его размер, но и замедлите производительность



<i>S</i>	Данные, записываемые в файл, сразу будут сброшены на диск. Аналогично выполнению команды <i>sync</i> сразу после каждой операции записи в файл
<i>s</i>	Прямо противоположен атрибуту <i>u</i> . После удаления файла принадлежащие ему блоки будут обнулены и восстановить их уже не получится

Пример установки атрибута:

```
# chattr +i config
```

Пример сброса атрибута:

```
# chattr -i config
```

5.7. Поиск файлов

Для поиска файлов вы можете использовать команды *which*, *locate* и *find*.

Первая используется только для поиска программ. Она позволяет определить, в каком каталоге находится исполнимый файл той или иной программы, например:

```
# which pppd  
/sbin/pppd
```

Данную программу очень удобно использовать администратору, когда нужно вычислить месторасположение программы, например, чтобы указать точный путь к программе в каком-то сценарии или конфигурационном файле.



Команда *locate* позволяет произвести быстрый поиск файла. Однако команда *locate* будет работать не во всех дистрибутивах, а только там, где доступен *updatedb*, который и формирует базу данных, по которой производит поиск команда *locate*. Если файл будет на диске, но его не будет в базе данных, то *locate* его не найдет — вот в чем основной недостаток этой команды. Для обновления базы данных нужно ввести команду *updatedb* (или дождаться, пока планировщик обновит базу данных).

Преимущество команды *locate* в том, что поиск файла производится практически мгновенно, особенно по сравнению с командой *find*. Однако если файла не будет в базе данных (файл был создан после обновления базы данных *locate*), команда *locate* его не найдет.

Конфигурация *updatedb* хранится в файле */etc/updatedb.conf* (листинг 5.1).

Листинг 5.1. Файл */etc/updatedb.conf*

```
PRUNE_BIND_MOUNTS = "yes"
```

```
PRUNEFS = "9p afs anon_inodefs auto autofs bdev binfmt_misc  
cgroup cifs coda configfs cpuset debugfs devpts ecryptfs  
exofs fuse fuse.sshfs fusectl gfs gfs2 hugetlbfs inotifyfs  
iso9660 jffs2 lustre mqueue ncpfs nfs nfs4 nfsd pipefs proc  
ramfs rootfs rpc_pipefs securityfs selinuxfs sfs sockfs  
sysfs tmpfs ubifs udf usbfs"
```

```
PRUNENAMES = ".git .hg .svn"
```

```
PRUNEPATHS = "/afs /media /mnt /net /sfs /tmp /udev /var/  
cache/ccache /var/lib/yum/yumdb /var/spool/cups /var/spool/  
squid /var/tmp"
```

Если параметр *PRUNE_BIND_MOUNTS* равен *yes*, файловые системы, смонтированные в режиме *bind*, не исследуются



при помощи *updatedb*. Параметр *PRUNEFs* задает типы файловых систем, которые не будут исследоваться *updatedb*. Аналогично, параметры *PRUNENAMES* и *PRUNEPATHS* задают имена файлов (у нас заданы «расширения») и пути (каталоги).

В листинге 5.1 приведен пример файла *update.conf* по умолчанию из Fedora Server. Вы можете отредактировать его под свои нужды, например, закомментировать параметр *PRUNENAMES*, отредактировать параметр *PRUNEPATHS* и т.д.

Теперь перейдем к третьей и самой универсальной команде поиска — *find*. Формат вызова следующий:

```
$ find список_поиска выражение
```

Полное описание команды *find* вы найдете в справочной системе (команда *man mount*), а мы рассмотрим несколько примеров.

```
$ find / -name test.txt
```

Мы ищем все файлы с именем *test.txt*, начиная с корневого каталога */*. Если нужно найти все текстовые файлы (**.txt*), начиная с корневого каталога, тогда команда будет такой:

```
$ find / -name '*.txt'
```



Следующая команда ищет только пустые файлы (параметр *-empty*):

```
$ find . -empty
```

Если нужно задать размер файла, тогда можем указать размер явно. В следующем примере мы задаем размер файла — от 500 до 700 Мб:



```
$ find ~ -size +500M -size -700M
```

Команда *find* может не только находить файлы, но и выполнять действие для каждого найденного файла. В следующем примере мы находим все старые резервные копии («расширение» .bak) и удаляем их:

```
# find / -name *.bak -ok rm {} \;
```

Поиск с помощью *find* занимает немало времени — ведь нет никакой базы данных. Команде *find* нужно «пройтись» по всем каталогам и проверить в них наличие искомых файлов.

5.8. Монтирование файловых систем

5.8.1. Монтируем файловые системы вручную

Ранее было сказано, что такое точка монтирования — это каталог, через который происходит доступ к файловой системе, физически размещенной на другом носителе (другом разделе жесткого диска, флешке, оптическом диске или даже на другом компьютере).

Для монтирования файловой системы используется команда *mount*, для размонтирования — *umount*:

```
# mount [опции] <имя устройства> <точка монтирования>  
# umount <имя устройства или точка монтирования>
```



Для монтирования файловой системы нужны права *root*, поэтому команды *mount* и *umount* нужно вводить с правами *root*.

Представим, что мы подключили флешку. Если у вас один жесткий диск (*/dev/sda*), то флешке будет назначено имя */dev/sdb*, если жестких дисков два, то флешке будет назначено следующее имя — */dev/sdc* и т.д.

На одном носителе (это касается жестких дисков, флешек и подобных носителей) может быть несколько разделов, которым назначаются номера, нумерация начинается с единицы. Поэтому вы не можете подмонтировать все устройство */dev/sdc*. Вы должны указать номер раздела.

Подмонтируем нашу флешку (пусть это будет устройство */dev/sdc* и на нем будет всего один раздел с номером 1):

```
# mount /dev/sdc1 /mnt/usb
```

Каталог */mnt/usb* — это и есть точка монтирования. Точка монтирования должна существовать до вызова команды *mount*, то есть вы не можете подмонтировать файловую систему к несуществующему каталогу.

После этого вы можете обращаться к файлам и каталогам на флешке через каталог */mnt/usb*:

```
# ls /mnt/usb
```

Итак, последовательность действий такая: создание точки монтирования (один раз), монтирование файловой системы, работа с файловой системой и размонтирование. Размонтирование осуществляется командой *umount*. В качестве параметра команды *umount* нужно передать или название точки монтирования, или имя устройства:



```
# mkdir /mnt/usb
# mount /dev/sdc1 /mnt/usb
# cp test.txt /mnt/usb
# umount /mnt/usb
```

Очень важно размонтировать файловую систему, особенно это касается внешних файловых систем. При завершении работы система автоматически размонтирует все смонтированные файловые системы.

Думаю, в общих чертах операция монтирования должна быть понятной. Теперь поговорим о параметрах команды *mount*. Самый часто используемый параметр — это параметр *-t*, позволяющий задать тип монтируемой файловой системы. Обычно команда *mount* сама в состоянии распознать тип файловой системы, но в некоторых случаях ей нужно указать его вручную. Вот наиболее распространенные типы файловых систем:

- **vfat** — файловая система Windows (FAT/FAT32);
- **ntfs** — файловая система Windows NT;
- **ntfs-3g** — драйвер ntfs-3g для чтения и записи NTFS (рекомендуется);
- **ext2/ext3/ext4** — различные версии файловой системы Linux;
- **iso9660** — файловая система оптического диска CD/DVD;
- **udf** — иногда Windows форматирует оптический диск как UDF;
- **reiserfs** — файловая система ReiserFS;
- **smbfs** — файловая система Samba ;
- **nfs** — сетевая файловая система.



Например, в случае с NTFS рекомендуется использовать драйвер **ntfs-3g**:



```
# mount -t ntfs-3g /dev/sdcl /mnt/usb
```

Параметр **-r** позволяет смонтировать файловую систему в режиме «только чтение», параметр **-w** монтирует файловую систему в режиме «чтение/запись», но обычно в этом режиме файловая система монтируется по умолчанию, поэтому в нем нет необходимости.

Параметр **-a** монтирует все файловые системы, перечисленные в файле `/etc/fstab`, за исключением тех, для которых указана опция *noauto*.

5.8.2. Имена устройств

Интерфейсов жестких дисков довольно много — IDE (ATA/PATA), SATA (Serial ATA), SCSI, USB. Раньше жесткие диски с интерфейсом IDE назывались в Linux `/dev/hd?` (? — буква, которая зависит от того, как подключен жесткий диск). Жесткие диски с интерфейсом SATA и SCSI назывались `/dev/sd?` (? — буква диска, соответствующая его порядковому номеру при подключении к интерфейсу).

Сейчас даже IDE-диски называются `/dev/sd?` (как и SATA/SCSI), что сначала вносило некую путаницу. Но жесткие диски с интерфейсом IDE вышли из моды и практически не используются. Мода на SCSI-диски также практически закончилась, поскольку SATA-диски такие же быстрые, как и SCSI. Некоторые обеспечивают такую же производительность, как и SCSI — иногда чуть меньше, иногда — даже больше. Так что SCSI уже можно списывать со счетов — если вам достался сервер со SCSI-диском, отказываться от него не стоит, а вот новый сервер будет поставляться или с интерфейсом SATA, или с интерфейсом SAS.



Интерфейс SAS (Serial Attached SCSI) обратно совместим с интерфейсом SATA и позволяет последовательно подключать SATA-диски, обеспечивая пропускную способность в 6 Гбит/с. Если вы купите сервер с интерфейсом SAS, то в большинстве случаев он будет оснащен высокопроизводительными SATA-дисками, а не SCSI-дисками. Поэтому никакой путаницы уже нет.

Что же касается USB-дисков (флешки и внешние жесткие диски), то они также получают обозначение `/dev/sd?`.

Оптические диски (приводы CD/DVD) в большинстве случаев называются `/dev/sr?`, где `?` — номер привода, нумерация начинается с 0.

Чтобы узнать, какие жесткие диски и оптические приводы установлены в вашем компьютере, введите команды (рис. 5.1):

```
ls /dev/sd?  
ls /dev/sr0
```

```
root@localhost ~# ls /dev/sd?  
/dev/sda /dev/sdb  
root@localhost ~# ls /dev/sr?  
/dev/sr0  
root@localhost ~# ls /dev/cd*  
/dev/cdrom  
root@localhost ~#
```

Рис. 5.1. Жесткие и оптические диски

Если у вас один оптический диск, можно смело использовать ссылку `/dev/cdrom`. Из рис. 5.1 видно, что у нас установлено два жестких диска — `/dev/sda` и `/dev/sdb`, а также один оптический привод `/dev/sr0`.

Для монтирования недостаточно указать имя всего устройства, нужно уточнить номер раздела. Когда разделов много, вы можете забыть (или не знать), какой именно раздел вам нужен. Вы можете исполь-



зовать команду *fdisk*: запустите *fdisk* <имя устройства>, а затем введите команду *p* для вывода таблицы разделов и команду *q* для выхода из *fdisk*. Более подробно программу *fdisk* мы, возможно, рассмотрим позднее.

Посмотрите рис. 5.2. Из него становится понятно, что на нашем жестком диске есть два раздела — */dev/sda1* и */dev/sdb2*.

```

[root@localhost ~]# fdisk /dev/sda

Welcome to fdisk (util-linux 2.28).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Command (m for help): p
Disk /dev/sda: 28 GiB, 21474836480 bytes, 41943040 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x7e91b068

Device     Boot      Start         End      Sectors  Size Id Type
/dev/sda1  *           2048    37750783    37748736   18G 83 Linux
/dev/sda2             37750784    41943039    4192256    2G 82 Linux swap / Solaris

Command (m for help): _

```

Рис. 5.2. Программа *fdisk*

Кроме коротких имен вроде */dev/sd?* в современных дистрибутивах часто используются идентификаторы UUID. Загляните в файл */etc/fstab*, и в нем в большинстве случаев вместо привычных имен */dev/sd?* вы обнаружите вот такие «страшные» имена:

```

# B Fedora, Debian, Ubuntu
UUID=2f149af9-3bff-44bd-d16s-ff98s9a7116d / ext4 defaults 0 1

# openSUSE
/dev/disk/by-id/dm-name-suse-server-root / ext4 defaults 1 1

```



Преимущество идентификаторов UUID в том, что они не изменяются, если вы иначе подключите жесткий диск. Представим, что у вас есть два жестких диска — `/dev/sda` и `/dev/sdb`. На первый вы установили Linux (в раздел `/dev/sda1`), а второй используете для хранения данных (на нем всего один раздел `/dev/sdb1`, который монтируется, как `/home`). Вы отключили оба диска, а затем, подключая, перепутали их местами. В итоге второй диск стал диском `/dev/sda`, а первый — `/sdb`. При загрузке может случиться конфуз, точнее, система вообще не загрузится. Даже если вы выберете в BIOS SETUP загрузку со второго жесткого диска, тоже ничего хорошего не выйдет. С UUID достаточно выбрать загрузку со второго жесткого диска, и система будет загружена.

Вы можете использовать обычные стандартные имена, а можете использовать UUID-идентификаторы. Узнать, какие UUID-идентификаторы соответствуют каким обычным именам, можно с помощью команды:



```
ls -l /dev/disk/by-uuid/
```

Вывод этой команды представлен на рис. 5.3.

```
root@lonehost:~# ls -l /dev/disk/by-uuid/
total 0
lrwxrwxrwx. 1 root root 18 Sep 11 00:09 58ebdefc-5d70-4174-b44e-bbb85ec0f408 -> ../sdb1
lrwxrwxrwx. 1 root root 18 Sep 11 00:31 a3075470-3c3c-492d-96b3-442f83b4bacf -> ../sda2
lrwxrwxrwx. 1 root root 18 Sep 11 00:31 c7699670-7643-4b72-85e0-b1efc10b5b30 -> ../sda1
root@lonehost:~#
```

Рис. 5.3. Соответствие UUID-идентификаторов обычным именам

5.8.3. Монтируем файловые системы при загрузке

Вводить команду *mount* при каждой загрузке не очень хочется, поэтому проще «прописать» файловые системы в файле `/etc/fstab`, чтобы система смонтировала их при загрузке.



Формат файла `/etc/fstab` следующий:

устройство точка тип опции флаг_копирования флаг_проверки

Первое поле — это устройство, которое будет монтироваться.

Второе поле — точка монтирования. Вы можете использовать как обычные имена, так и UUID.

Третье поле — тип файловой системы.

Четвертое поле — параметры файловой системы (табл. 5.4), последние два поля — это флаг резервной копии и флаг проверки. Первый флаг определяет, будет ли файловая система заархивирована командой *dump* при создании резервной копии (1 — будет, 0 — нет). Второй флаг определяет, будет ли файловая система проверяться программой *fsck* на наличие ошибок (1, 2 — будет, 0 — нет). Проверка производится, если достигнуто максимальное число попыток монтирования для файловой системы или если файловая система была размонтирована некорректно. Для корневой файловой системы это поле должно содержать 1, для остальных файловых систем — 2.

Таблица 5.4. Параметры файловой системы

Опция	Описание
<i>defaults</i>	Параметры по умолчанию
<i>user</i>	Разрешает обычному пользователю монтировать/размонтировать данную файловую систему
<i>nouser</i>	Запрещает обычному пользователю монтировать/размонтировать данную файловую систему. Смонтировать эту ФС может только <i>root</i> . Используется по умолчанию



<i>auto</i>	ФС будет монтироваться при загрузке. Используется по умолчанию, поэтому указывать не обязательно
<i>noauto</i>	ФС не будет монтироваться при загрузке системы
<i>exec</i>	Разрешает запуск исполнимых файлов на данной ФС. Используется по умолчанию. Для Windows-файловых систем (vfat, ntfs) рекомендуется использовать опцию <i>noexec</i>
<i>noexec</i>	Запрещает запуск исполнимых файлов на данной ФС
<i>r--</i>	ФС будет монтироваться в режиме «только чтение»
<i>rw</i>	ФС будет монтироваться в режиме «чтение запуск». По умолчанию для файловых систем, которые поддерживают запись
<i>utf8</i>	Для преобразования имен файлов будет использоваться кодировка UTF8
<i>data</i>	Задаёт режим работы журнала (см. ниже)

5.8.4. Автоматическое монтирование файловых систем

В современных дистрибутивах Linux сменные носители вроде USB-дисков и оптических дисков монтируются автоматически:

- Debian, Ubuntu, Fedora, CentOS — монтирование производится к каталогу `/media/<метка_устройства>`. В качестве метки может использоваться или метка, установленная при форматирова-



нии, или серийный номер устройства, если метка не устанавливалась.

- openSUSE — монтирование будет производиться к каталогу /var/run/media/<имя пользователя>/<метка>.

За автоматическое монтирование отвечает демон automount, который вы можете отключить, если автоматическое монтирование вам не нужно.

5.9. Работа с журналом

Существует *три* режима работы журналируемой файловой системы ext3/ext4: **journal**, **ordered** и **writeback**.

По умолчанию используется режим **ordered** — оптимальный баланс между производительностью и надежностью. В этом режиме в журнал будет заноситься информация только об изменении метаданных.

Самый медленный режим **journal**. В этом режиме в журнал записывается максимум информации, которая понадобится при восстановлении в случае сбоя. Режим очень медленный и использовать его следует, только если безопасность для вас важнее, чем производительность.

Самый быстрый режим **writeback**, но в нем, по сути, журнал не будет использоваться, и у вас не будет никакой защиты, например, от той же перезагрузки.

Режим работы журнала задается параметром *data*, например:

```
/dev/sdb1 /home ext4 data=journal 1 2
```



5.10. Преимущества файловой системы *ext4*

Поговорим о преимуществах файловой системы *ext4*. Возможно, она вас полностью устроит и вам не придется искать другую файловую систему для своего компьютера.

Впервые файловая система *ext4* появилась в ядре версии 2.6.28. По сравнению с *ext3*, максимальный размер раздела был увеличен до 1 эксбибайта (1024 петабайтов), а максимальный размер файла составляет 2 Тб. По производительности новая файловая система *ext4* превосшла файловые системы *ext3*, *Reiserfs*, *XFS* и *Btrfs* (в некоторых операциях).

Так, *ext4* опередила знаменитую *XFS* в тесте на случайную запись. Файловая система *Btrfs* провалила этот тест с огромным «отрывом» от лидеров — *XFS* и *ext4*. Производительность *ext4* была примерно такой же, как у *XFS*, но все-таки немного выше, чем у *XFS*. В Интернете вы найдете множество тестов производительностей — просмотрите их, если вам интересно.

Основной недостаток *ext3* заключается в ее методе выделения места на диске. Ее способ выделения дискового пространства не отличается производительностью, а сама файловая система эффективна для небольших файлов, но никак не подходит для хранения огромных файлов. В *ext4* для более эффективной организации данных используются *экстенты*. *Экстенст* — это непрерывная область носителя информации. К тому же *ext4* откладывает выделение дискового пространства до последнего момента, что еще более увеличивает производительность.

Файловая система *ext3* может содержать максимум 32 000 каталогов, в *ext4* количество каталогов не ограничено.



В журнале ext4 тоже произошли изменения — в журнале ext4 используются контрольные суммы, что повышает надежность ext4 по сравнению с ext3.

Выходит, по сравнению с ext3, у ext4 есть следующие преимущества:

- **Улучшена производительность** — производительность почти достигла XFS, а в некоторых тестах даже превышает ее.
- **Улучшена надежность** — используются контрольные суммы журналов.
- **Улучшена масштабируемость** — увеличен размер раздела, размер файла и поддерживается неограниченное количество каталогов.

5.11. Специальные операции с файловой системой

5.11.1 Монтирование NTFS-разделов

Не думаю, что на сервере вам придется монтировать NTFS-разделы, но ситуации бывают разные. Для монтирования NTFS-раздела используется модуль **ntfs-3g**, который в большинстве случаев уже установлен по умолчанию. Если он не установлен, для его установки введите команду (замените *yum* на имя вашего менеджера пакетов):

```
# yum install ntfs-3g
```

Команда монтирования NTFS-раздела выглядит так:



```
# mount -t ntfs-3g раздел точка_монтирования
```

Например, вам кто-то принес флешку, отформатированную как NTFS. Для ее монтирования введите команду (измените только имя устройства и точку монтирования):

```
# mount -t ntfs-3g /dev/sdb1 /mnt/usb
```

Модуль **ntfs-3g** выполняет монтирование в режиме чтение/запись, поэтому вы при желании можете произвести запись на NTFS-раздел.

5.11.2. Создание файла подкачки

При нерациональном планировании дискового пространства может возникнуть ситуация, когда раздела подкачки стало мало или вы вообще его не создали. Что делать? Повторная разметка диска требует времени, а выключать сервер нельзя. Сервер тормозит, поскольку ему не хватает виртуальной памяти, а ждать от начальства подписи на дополнительные модули оперативной памяти придется еще неделю. А за это время пользователи вас окончательно достанут своими жалобами.

Выход есть. Он заключается в создании файла подкачки на жестком диске. Такой файл подкачки будет работать чуть медленнее, чем раздел подкачки, но это лучше, чем вообще ничего. Хотя, если у вас SSD-диск, никакой разницы в производительности практически не будет.

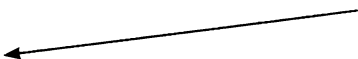
Первым делом нужно создать файл нужного размера. Следующая команда создает в корне файловой системы файл **swap01** размером 1 Гб:

```
# dd if=/dev/zero of=/swap01 bs=1k count=1048576
```



После этого нужно создать область подкачки в этом файле:

```
# mkswap /swap01 1048576
```



Наконец, чтобы система «увидела» файл подкачки, его нужно активировать:

```
# swapon /swap01
```

Чтобы не вводить эту команду после каждой перезагрузки сервера, нужно обеспечить ее автоматический запуск.

5.11.3. Файлы с файловой системой

Только что было показано, как создать файл произвольного размера, а потом использовать его в качестве файла подкачки. При желании этот файл можно отформатировать, как вам угодно. Даже можно создать в нем файловую систему.

Рассмотрим небольшой пример. Давайте опять создадим пустой файл размером 1 Гб:

```
# dd if=/dev/zero of=/root/fs01 bs=1k count=1048576
```

После этого нужно создать файловую систему в этом файле:

```
# mkfs.ext3 -F /root/fs01
```

Чтобы не заморачиваться, я создал самую обычную файловую систему ext3. После этого файл с файловой системой можно подмонтировать и использовать как обычный сменный носитель, то есть записывать на него файлы:



```
# mkdir /mnt/fs01
# mount -t ext3 -o loop /root/fs01 /mnt/fs01
```

После того как закончите работу с файлом, его нужно размонтировать:

```
# umount /mnt/fs01
```

Зачем это вам нужно — знаете только вы. В конце концов, можно использовать или зашифрованную файловую систему, или просто архив. Но для общего развития это очень важно, особенно если вы намереваетесь создавать собственный дистрибутив Linux.

5.11.4. Создание и монтирование ISO-образов

Все мы знаем утилиты, позволяющие в Windows подмонтировать ISO-образ диска. В Linux все подобные операции делаются с помощью штатных средств и никакие дополнительные программы не нужны.

Представим, что нам нужно создать образ диска. Если диск вставлен в привод, для создания его ISO-образа выполните команду:

```
$ dd if=/dev/cdrom of=/dvd.iso
```

Здесь, `/dev/cdrom` — имя устройства (в Linux это имя соответствует любому оптическому приводу — CD или DVD), а `dvd.iso` — файл образа.

Иногда ставится другая задача: есть папка, по которой нужно создать ISO-образ. То есть у нас нет диска, но есть файлы, которые нужно записать на диск, но прежде вы хотите создать его ISO-образ.



Пусть у нас есть папка `~/dvd` и нужно создать ISO-образ, содержащий все файлы из этой папки. Файл образа будет опять называться `~/dvd.iso`. Для этого используйте команду *mkisofs*:

```
$ mkisofs -r -jcharset utf8 -o ~/dvd.iso ~/dvd
```

Чтобы проверить, что образ был создан корректно, его нужно подмонтировать к нашей файловой системе:

```
# mkdir /mnt/iso-image  
# mount -o loop -t iso9660 dvd.iso /mnt/iso-image
```

Здесь все просто: опция *-o loop* означает, что будет монтироваться обычный файл, а не файл устройства, опция *-t* задает тип файловой системы, далее следуют название файла и название папки, к которой будет выполнено монтирование.

5.12. Файлы конфигурации Linux

5.12.1. Содержимое каталога `/etc`

Думаю, среди читателей этого справочника нет таких, которые бы не знали о реестре Windows. В реестре Windows хранятся настройки самой системы и практически всех программ (исключения составляют лишь устаревшие программы, хранящие свои параметры в `ini`-файлах).

Так вот, в Linux есть свое подобие реестра — это каталог `/etc`. И вы можете быть уверены, что в `/etc` находятся все настройки системы и всех программ. Да, некоторые программы также хранят персональные настройки в домашних каталогах пользователя, но глобальные настройки обычно хранятся в каталоге `/etc`.



Преимущество каталога `/etc` перед реестром — в том, что для его редактирования вам не нужен какой-то особый редактор. В каталоге `/etc` просто хранится набор текстовых конфигурационных файлов. Вы можете просмотреть или отредактировать файл с помощью любого текстового редактора. Также можно легко скопировать файлы/каталоги, например, перед изменением. Единственный недостаток — вам нужно знать формат каждого файла, но обычно с этим проблем никаких нет, поскольку файлы снабжены подробными комментариями, путь и на английском языке.

5.12.2. Конфигурационные файлы

Чтобы эффективно настраивать систему Linux, нужно ориентироваться в содержимом конфигурационного каталога. Если заглянете в `/etc`, то кроме подкаталогов в нем обнаружите и файлы, которые находятся непосредственно в `/etc`. Описание этих файлов приведено в таблице 5.5. Содержимое каталога может отличаться в зависимости от используемого дистрибутива и установленного программного обеспечения. Но общая картина будет примерно одинаковой во всех дистрибутивах.

Таблица 5.5. Файлы из каталога `/etc`

Файл	Описание
<i>DIR_COLORS</i>	Конфигурация цвета для утилиты ls . Здесь вы можете указать, каким цветом будут выводиться каталоги, файлы, ссылки и т.д.
<i>GREP_COLORS</i>	Конфигурация цвета для утилиты grep
<i>adjtime</i>	Содержит параметры для корректировки аппаратных часов



<i>aliases</i>	База данных псевдонимов для почтовых агентов (MTA)
<i>at.deny</i>	Содержит пользователей, которым запрещено использовать планировщик at . Дополнительная информация доступна в <i>man at</i>
<i>anacrontab</i>	Конфигурация (таблица расписания) планировщика anacron
<i>bash.bashrc</i>	Глобальный файл конфигурации оболочки bash . Локальные файлы конфигурации находятся в домашнем каталоге каждого пользователя
<i>bind.keys</i>	Содержит ключи для DNS-сервера bind9. Настройку DNS-сервера пока рассматривать не будем
<i>bindresvport.blacklist</i>	Содержит список номеров портов в диапазоне от 600 до 1024, которые не могут быть использованы <i>bindresvport</i> , который обычно вызывается IRC-службами. По умолчанию запрещены порты 623, 631, 636, 664, 774, 921, 993 и 995, которые используются различными сетевыми службами
<i>cron.deny</i>	Список пользователей, которым запрещено использовать cron
<i>crontab</i>	Таблица расписания демона crond
<i>crony*</i>	Конфигурация NTP (сервер времени)



<i>ssh.cshrc</i>	Файл конфигурации для оболочки C Shell
<i>ssh.login</i>	Глобальный файл конфигурации для C Shell. Определяет поведения во время регистрации пользователя (login) в системе. На самом деле — это сценарий, который запрещено редактировать вручную. Он должен изменяться только во время обновления системы. Вместо этого лучше редактировать <i>/etc/ssh.login.local</i> , если вам нужно внести изменения в настройки вашего локального окружения
<i>crypttab</i>	Содержит информацию о зашифрованных томах
<i>defaultdomain</i>	Содержит доменное имя для сервисов NIS и NIS+
<i>dhclient.conf</i>	Файл конфигурации для DHCP-клиента
<i>dhclient6.conf</i>	Файл конфигурации для DHCP-клиента, версия IPv6
<i>dhcpd.conf</i>	Файл конфигурации для DHCP-сервера
<i>dhcpd6.conf</i>	Файл конфигурации для DHCP-сервера, версия IPv6
<i>dialogrc</i>	Конфигурационный файл для пакета dialog , определяет цветовую схему диалоговых интерфейсов, построенных с помощью пакета dialog



<i>dnsmasq.conf</i>	Конфигурационный файл для dnsmasq (DNS-маскарадинга)
<i>dracut.conf</i>	Содержит параметры <i>dracut</i> — средства, которое формирует <i>initramfs</i>
<i>drirc</i>	Конфигурационный файл для репозитория DRI CVS
<i>environment</i>	Файл используется PAM-модулем <i>ram_env</i> . Содержит переменные окружения, описанные в виде пар КЛЮЧ=ЗНАЧЕНИЕ, по одной паре в одной строке
<i>esd.conf</i>	Параметры Esound (Enlightened Sound Daemon), который используется для смешивания вместе некоторых цифровых аудиопотоков для проигрывания на одином устройстве
<i>ethers</i>	Содержит 48-битные Ethernet-адреса и соответствующие им IP-адреса или имена узлов. Может использоваться некоторыми сетевыми службами для разрешения MAC-адресов в IP-адреса
<i>exports</i>	Содержит список экспортируемых файловых систем
<i>fedora-release</i>	Информация о релизе Fedora



<i>filesystems</i>	Информационный файл, содержит некоторые характеристики и атрибуты файловых систем. В нем нет ничего интересного
<i>fstab</i>	Содержит список файловых систем, которые будут монтироваться автоматически при загрузке системы
<i>ftpusers</i>	Список пользователей, которые НЕ могут войти в систему по FTP. Среди них вы найдете пользователя <i>root</i> и многие другие системные учетные записи, которые используются для сетевых сервисов и обычно обладают повышенными привилегиями
<i>group</i>	Содержит группы пользователей
<i>host.conf</i>	Задает порядок разрешения доменных имен
<i>hostname</i>	Содержит доменное имя узла
<i>hosts</i>	Ранее использовался для разрешения IP-адресов в доменные имена. Сейчас для этого используется система DNS, но вы все равно можете определить в нем некоторые IP-адреса, если ваша сеть не использует DNS или же вам нужно переопределить разрешение для определенного IP-адреса. Обе ситуации в наше время настолько редки, что похожи на что-то из области технической фантастики



<i>hosts.*</i>	Файл конфигурации локальной сети
<i>hushlogins</i>	Если существует файл <code>file ~/.hushlogin</code> или <code>/etc/hushlogins</code> , осуществляется «тихий» вход (это отключает проверку e-mail и вывод последнего времени входа и сообщения дня (Message of Day)). Если существует файл <code>/var/log/lastlog</code> , то выводится время последнего входа в систему
<i>idmapd.conf</i>	Конфигурация демона <code>idmapd</code>
<i>idn.conf</i>	Файл конфигурации для <code>idnkit</code>
<i>idnalias.conf</i>	Псевдонимы кодировок для <code>idnkit</code>
<i>inputrc</i>	Позволяет задавать обработку отображения символов в специальных ситуациях. Используется редко
<i>issue</i>	Приглашение, которое выводится при входе в систему (ссылка на <code>/usr/lib/issue</code>)
<i>issue.net</i>	Приглашение, которое выводится при сетевом входе в систему (ссылка на <code>/usr/lib/issue.net</code>)
<i>krb5.conf</i>	Параметры Kerberos
<i>ld.so.cache</i>	Хеш-версия файла <code>ld.so.conf</code> . Создается утилитой <code>ldconfig</code>



<i>ld.so.conf</i>	Настройка динамического связывания во время выполнения
<i>lesskey</i>	Задаёт параметры преобразования некоторых символов/клавиш, вы не будете редактировать этот файл
<i>libao.conf</i>	Конфигурация библиотеки libao (обычно здесь задается аудиодрайвер по умолчанию)
<i>libaudit.conf</i>	Конфигурация библиотеки libaudit
<i>login.defs</i>	Контрольные определения для пакета shadow . Например, здесь можно задать количество неудачных попыток входа в систему и многие другие параметры
<i>logrotate.conf</i>	Задаёт параметры ротации журналов
<i>machine-id</i>	Содержит идентификатор машины
<i>mail.rc</i>	Файл конфигурации программы mail
<i>manpath.config</i>	Этот файл используется пакетом man-db для настройки путей man и cat
<i>mime.types</i>	Содержит список MIME-типов и соответствующих им расширений файлов
<i>mke2fs.conf</i>	Файл конфигурации программы mke2fs



<i>motd</i>	Содержит сообщение дня (Message of the Day). В зависимости от настроек системы может выводиться при входе в систему
<i>mtools.conf</i>	Данный файл является частью пакета mtools.conf
<i>netconfig</i>	Файл конфигурации сети. Теперь используется только с кодом TI-RPC в библиотеке libtirpc
<i>netgroup</i>	Содержит описание конфигурации сетевых групп
<i>networks</i>	Статическая информация о сетевых именах
<i>nfsmount.conf</i>	Файл конфигурации монтирования NFS
<i>nscd.conf</i>	Конфигурационный файл для nscd (Name Service Cache)
<i>nsswitch.conf</i>	Параметры NSS (Network Service Switch)
<i>ntp.conf</i>	Файл конфигурации сервера времени ntpd
<i>os-release</i>	Содержит информацию о релизе: номер версии, кодовое имя и т.д. (ссылка на <code>/usr/lib/os-release</code>)
<i>passwd</i>	База данных паролей. Подробнее, возможно, будем рассматривать позднее



<i>permissions</i>	Этот файл используется программой chkstat и косвенно некоторыми RPM-скриптами для проверки или установки прав и режимов файлов и каталогов при установке
<i>printcap</i>	Этот файл автоматически генерируется cupsd , его не нужно редактировать. Все изменения, внесенные в этот файл, будут потеряны
<i>profile</i>	Не изменяйте этот файл во избежание потери изменений во время очередного обновления системы. Если вам нужно изменить этот файл, отредактируйте <code>/etc/profile.local</code> , чтобы установить ваши локальные настройки, например, глобальные псевдонимы, переменные <code>VISUAL</code> и <code>EDITOR</code> и т.д.
<i>protocols</i>	Список IP-протоколов
<i>python3start</i>	Startup-сценарий Python 3 для сохранения истории интерпретатора и автодополнения имен
<i>pythonstart</i>	То же, что и <code>python3start</code> , но для старых версий Python
<i>raw</i>	Определяет параметры привязки raw-устройств к блочным устройствам
<i>rc.splash</i>	Сценарий, определяющий внешний вид индикатора начальной загрузки



<i>resolv.conf</i>	Конфигурационный файл для системы разрешения имен
<i>rpc</i>	Список протоколов удаленного вызова процедур (RPC)
<i>rsyncd.conf</i>	Файл конфигурации для rsyncd
<i>rsyncd.secrets</i>	Пароли rsyncd
<i>screenrc</i>	Параметры программы screen (менеджер экрана с эмуляцией терминала VT100/ANSI)
<i>securetty</i>	Содержит имена устройств терминалов (tty), на которых разрешает вход в систему пользователю <i>root</i>
<i>services</i>	Список служб (сервисов)
<i>shadow</i>	Пароли из файла <i>/etc/passwd</i> физически хранятся в <i>/etc/shadow</i> . Поэтому фактически в <i>/etc/passwd</i> хранится список пользователей, а пароли этих пользователей находятся в <i>/etc/shadow</i> , доступ к которому ограничен
<i>shells</i>	Список установленных в системе интерпретаторов
<i>slp.conf</i>	Файл конфигурации OpenSLP SPI
<i>smartd.conf</i>	Файл конфигурации демона smartd



<i>sudoers</i>	Позволяет определить, кому можно использовать команду <i>sudo</i>
<i>suspend.conf</i>	Некоторые параметры питания
<i>sysctl.conf</i>	Файл конфигурации sysctl . Кроме этого файла, sysctl также читает параметры из файлов <code>/etc/sysctl.d/*.conf</code> , <code>/run/sysctl.d/*.conf</code> и некоторых других
<i>ttymtype</i>	Содержит список терминалов и определяет их тип
<i>usb_modeswitch</i>	Параметры для пакета usb_modeswitch
<i>vconsole.conf</i>	Конфигурационный файл для виртуальной консоли
<i>vi</i>	Файл конфигурации текстового редактора vi
<i>wgetrc</i>	Параметры программы wget
<i>xattr.conf</i>	Задаёт, как обработать расширенные атрибуты при копировании между файлами

5.12.3. Подкаталоги с конфигурационными файлами

Далее мы «пройдемся» по подкаталогам каталога `/etc` с целью выяснить, что находится в каждом из них (табл. 5.6). Содержимое вашего каталога `/etc` может отличаться в зависимости от дистрибутива и уже



установленных программ. Вполне вероятно, что у вас не будет некоторых каталогов, представленных в таблице 5.6, но будут некоторые другие каталоги, назначение которых можно найти или в справочной системе Linux, или в Интернете.

Таблица 5.6. Подкаталоги каталога /etc

Каталог	Описание
<i>NetworkManager</i>	Содержит файл конфигурации Network Manager и файлы конфигураций сетевых соединений в некоторых дистрибутивах
<i>PackageKit</i>	PackageKit — это открытый и свободный набор приложений для обеспечения высокоуровневого интерфейса для различных менеджеров пакетов. Этот каталог содержит файлы конфигурации PackageKit
<i>X11</i>	Параметры графического интерфейса X11 (X Window)
<i>abrt</i>	Конфигурация abrt — демона автоматических отчетов о сбоях
<i>alternatives</i>	Каталог альтернатив по умолчанию. Файл является частью подсистемы update-alternatives, которая обслуживает символические ссылки, определяющие команды, файлы и каталоги, используемые по умолчанию



<i>audit u audisp</i>	В этих каталогах находятся конфигурационные файлы демона аудита — auditd и его диспетчера событий (audit event dispatcher). Основной конфигурационный файл — /etc/audit/auditd.conf. В нем задается поведение демона и некоторые настройки, например, расположение журнала по умолчанию /var/log/audit/audit.log.
<i>apt</i>	Конфигурация менеджера пакетов apt
<i>bash_comletion.d</i>	Параметры автодополнения командной строки для оболочки bash
<i>binfmt.d</i>	Демон binfmt.d настраивает дополнительные двоичные файлы для выполнения во время загрузки. В этом каталоге находятся его конфигурационные файлы
<i>cockpit</i>	Конфигурация панели управления Cockpit
<i>cifs-utils</i>	Пакет cifs-utils содержит средство мониторинга ресурсов общего доступа SMB/CIFS в Linux. В этом каталоге находятся конфигурационные файлы пакета cifs-utils
<i>crond.d</i>	Содержит файлы, которые загружаются в память одновременно с файлами из каталога /var/spool/cron. После этого демон cron загружает содержимое файла /etc/crontab и начинает его обработку



<i>cron.daily, cron.hourly, cron.mounthly, cron. weekly</i>	Содержат сценарии, которые будут выполнены демоном cron, соответственно ежедневно, ежечасно, ежемесячно и еженедельно
<i>cups</i>	Содержит параметры конфигурации системы печати CUPS (Common Unix Printing System)
<i>cupshelpers</i>	В пакете python-cupshelpers содержатся модули Python, которые помогают создавать приложения и утилиты с использованием Python-интерфейса к CUPS. В этом каталоге находятся параметры этого пакета
<i>crypto-policies</i>	Конфигурация криптополитик
<i>dbus-1</i>	Содержит файлы конфигурации демона dbus-daemon
<i>default</i>	Содержит некоторые параметры по умолчанию, например, параметры загрузчика GRUB
<i>depmod.d</i>	depmod — программа для создания файла modules.dep и map-файла. В этом каталоге находятся ее файлы конфигурации. Как администратор сервера, вы можете о них просто забыть, они вам не пригодятся
<i>dhcp</i>	Конфигурация DHCP-сервера



<i>dnf</i>	Конфигурация менеджера пакетов dnf
<i>dracut.conf.d</i>	dracut заменяет mkinitrd для создания загрузочной файловой системы в оперативной памяти (ramdisk). Здесь находятся конфигурационные файлы dracut
<i>firewalld</i>	Конфигурация брандмауэра
<i>fonts</i>	Содержит конфигурационные файлы подсистемы шрифтов. В частности, файл /etc/fonts/fonts.conf описывает каталоги со шрифтами, каталоги с кэшем шрифтов, а также описывает аналоги шрифтов
<i>gnupg</i>	Содержит конфигурационный файл GnuPg
<i>grub.d</i>	Содержит файлы, относящиеся к загрузчику GRUB. Вам не нужно редактировать эти файлы, они предназначены для служебных целей
<i>init.d</i>	Содержит сценарии системы инициализации
<i>iproute2</i>	Параметры подсистемы маршрутизации. Например, могут использоваться для IP-балансировки, то есть объединения нескольких интернет-каналов в один



<i>iscsi</i>	Параметры iSCSI
<i>java</i>	Параметры Java
<i>joe</i>	Содержит конфигурационные файлы текстового редактора joe
<i>jvm, jvm-common</i>	Параметры виртуальной машины Java
<i>ld.so.conf.d</i>	Содержит файлы с расширением <i>conf</i> , которые используются для поиска разделяемых библиотек
<i>libnl</i>	Конфигурационные файлы библиотеки libnl
<i>logrotate.d</i>	Конфигурация средства ротации журналов
<i>mc</i>	Файлы конфигурации файлового менеджера Midnight Commander
<i>mcelog</i>	Программа mcelog позволяет расшифровать аппаратные ошибки. Настраивается посредством конфигурационных файлов в каталоге <i>/etc/mcelog</i>
<i>modprobe.d</i>	<i>modprobe</i> — программа для добавления и удаления модулей из ядра Linux. Соответственно в одноименном подкаталоге каталога <i>/etc</i> находятся ее конфигурационные файлы



<i>modules-load.d</i>	Содержит параметры некоторых модулей ядра. По умолчанию в этом каталоге пусто
<i>openldap</i>	Содержит конфигурационные файлы сервера каталогов OpenLDAP
<i>opt</i>	Файлы конфигурации для /opt/
<i>pam.d</i>	Параметры конфигурации модулей аутентификации PAM (Pluggable Authentication Modules)
<i>pkcs11</i>	Параметры программы pkcs11, используемой для управления объектами данных, которые находятся на зашифрованных устройствах PKCS#11 (Cryptoki)
<i>pki</i>	Содержит список GPG-ключей
<i>plymouth</i>	Plymouth — свободный графический экран загрузки для Linux. Этот каталог содержит его конфигурационные файлы
<i>pm</i>	Содержит конфигурационные файлы пакета pm-utils. Пакет pm-utils — это инфраструктура управления питанием нового поколения
<i>postfix</i>	Конфигурационные файлы почтового агента Postfix



<i>ppp</i>	Файлы конфигурации протокола PPP
<i>pptp.d</i>	Файлы конфигурации демона pptpd (протокол PPTP)
<i>products.d</i>	Относится к системе миграции между версиями дистрибутива openSUSE. Дополнительная информация может быть найдена по желанию в сети Интернет
<i>pulse</i>	PulseAudio — это звуковой сервер для POSIX-систем. Его основное назначение — смешивать звуковые потоки от разных приложений, что позволяет нескольким потокам воспроизводиться одновременно. Здесь находятся конфигурационные файлы PulseAudio
<i>rc.d</i>	Ссылка на каталог init.d
<i>rsyslog.d</i>	Конфигурация демона протоколирования rsyslogd
<i>rpm</i>	Различные параметры системы управления пакетами RPM. Обычно файлы из этого каталога не требуют изменения. Дополнительную информацию можно получить по адресу http://wiki.opennet.ru/RPM



<i>samba</i>	Конфигурационные файлы Samba
<i>sasl2</i>	Параметры SASL (Simple Authentication and Security Layer)
<i>security</i>	Еще один параметр конфигурации, относящийся к модулям аутентификации PAM
<i>selinux</i>	Содержит файлы конфигурации системы безопасности SELinux
<i>skel</i>	При создании новой учетной записи пользователя создается его домашний каталог в каталоге /home, при этом в созданный домашний каталог пользователя копируются файлы из каталога /skel. Все помещенные в этот каталог файлы будут скопированы в созданный домашний каталог
<i>ssh</i>	Содержит файлы конфигурации SSH-клиента и SSH-сервера
<i>ssl</i>	Файлы конфигурации OpenSSL
<i>sudoers.d</i>	Кроме файла /etc/sudoers, настройки sudo могут определяться содержимым файлов из каталога /etc/sudoers.d



<i>sysconfig</i>	Содержит конфигурационные файлы всей системы. В этом каталоге очень много различных конфигурационных файлов. Например, в каталоге <code>/etc/sysconfig/network</code> вы найдете конфигурационные файлы сетевых интерфейсов. А в файле <code>clock</code> хранится выбранный при установке часовой пояс
<i>sysctl.d</i>	<code>sysctl.d</code> настраивает параметры ядра при загрузке, здесь находятся его конфигурационные параметры
<i>systemd</i>	Конфигурационные файлы демона <code>systemd</code>
<i>udev</i>	Файлы конфигурации и файлы правил менеджера устройств udev
<i>wpa_supplicant</i>	Конфигурационные файлы пакета <code>wpa_supplicant</code> (обеспечивает поддержку WEP, WPA и WPA2)
<i>xdg</i>	<code>xdg-open</code> — это независимый пользовательский инструмент для настройки приложений рабочего стола по умолчанию. В этом каталоге находятся его конфигурационные файлы. Например, в каталоге <code>/etc/xdg/autostart</code> находятся все программы, которые могут быть запущены автоматически. Однако запускаются лишь те, которым разрешен запуск в определенной сессии



<i>xinetd.d</i>	Содержит дополнительные файлы конфигурации суперсервера xinetd (в современных дистрибутивах не используется)
<i>xml</i>	Конфигурационные файлы библиотеки libxml

5.13. Псевдофайловые системы

Псевдофайловые системы **sysfs** (каталог `/sys`) и **proc** (каталог `/proc`) используются для настройки системы и получения различной информации о системе и процессах.

Свое название *псевдофайловые* системы получили из-за того, что они работают на уровне *виртуальной файловой системы*. В итоге оба эти средства (назовем их так) для конечных пользователей выглядят как обычная файловая система — вы можете зайти как в каталог `/sys`, так и в каталог `/proc`. В обоих этих каталогах будут файлы, вы можете просмотреть эти файлы и даже изменить их содержимое.

Содержимое многих файлов псевдофайловой системы `/proc` формируется «на лету». Обратите внимание на размер любого файла в каталоге `/proc` — он равен нулю, но если открыть файл, то информация в нем будет. Например, в файле `/proc/version` находится информация о версии Linux.

Монтирование файловых систем **sysfs** и **proc** осуществляется или в сценариях инициализации системы, или через `/etc/fstab`. В последнем случае записи монтирования псевдофайловых систем выглядят так:



sysfs	/sys	sysfs	defaults	0 0
proc	/proc	proc	defaults	0 0

5.13.1. Псевдофайловая система *sysfs*

Файловая система **sysfs** (каталог `/sys`) предоставляет пользователю информацию о ядре Linux, об имеющихся в системе устройствах и драйверах этих устройств. На рис. 5.4 представлено содержание каталога `/sys`. В нем вы найдете следующие подкаталоги:

- **block** — содержит каталоги для всех блочных устройств, которые есть в вашей системе в настоящее время. Здесь под устройством подразумевается наличие физического устройства и его драйвера. Если вы подключите внешний жесткий диск, то в каталоге `/sys/devices` появится новое устройство, но в каталоге `/sys/block` оно появится, только если в системе есть драйвер для работы с этим устройством или же драйвер (модуль) встроен в само ядро.
- **bus** — здесь находится список шин, которые поддерживает ваше ядро. Заглянув в этот каталог, вы обнаружите подкаталоги `pci`, `pci_express`, `scsi` и т.д. В каждом из этих каталогов будут подкаталоги `devices` и `drivers`. В первом находится информация об устройствах, подключенных к данной шине, во втором — информация о драйверах устройств.
- **class** — позволяет понять, как устройства формируются в классы. Для каждого класса есть отдельный подкаталог в каталоге `class`.
- **devices** — содержит дерево устройств ядра, точнее структуру файлов и каталогов, которая полностью соответствует внутреннему дереву устройств ядра.
- **firmware** — содержит интерфейсы, предназначенные для просмотра и манипулирования `firmware`-специфичными объектами и их параметрами.



- **fs** — информация о файловых системах, которые поддерживает ваше ядро.
- **kernel** — общая информация о ядре.
- **module** — здесь вы найдете подкаталоги для каждого загруженного модуля ядра. Имя подкаталога соответствует имени модуля. В каждом из подкаталогов модулей вы найдете подкаталог `parameters`, содержащий специфичные для модуля параметры.
- **power** — позволяет управлять параметрами питания, а также переводить систему из одного состояния питания в другое. Далее будет показано несколько примеров.

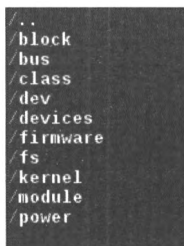


Рис. 5.4. Содержание каталога /sys

Довольно интересен с практической точки зрения каталог `/sys/power`. В файле `state` находится состояние питания. Изменив должным образом содержимое этого файла, можно изменить состояние питания. Например, вот как можно перевести систему в состояние «Suspend to RAM», когда питание процессора отключается, но питание на память подается, благодаря чему ее содержимое не уничтожается:

```
$ sudo echo -n mem > /sys/power/state
```



При желании можно отправить систему в состояние «Suspend to Disk», когда содержимое памяти будет записано на жесткий диск, после чего питание будет отключено:



```
$ sudo echo -n disk > /sys/power/state
```

5.13.2. Псевдофайловая система *proc*

Файловая система *proc* позволяет отправлять информацию ядру, модулям и процессам.

Вы можете не только получать информацию о процессах, но изменять параметры ядра и системы «на лету». Эта файловая система интересна тем, что позволяет изменять такие параметры ядра, которые невозможно изменить другим способом, к тому же вносимые изменения вступают в силу сразу же.

Некоторые файлы в */proc* доступны только для чтения — вы можете только просмотреть их. А некоторые вы можете изменять, и эти изменения сразу же отразятся на работе системы. Просмотреть файлы из */proc* можно любой программой для просмотра файлов, проще всего в консоли использовать команду *cat*:

```
cat /proc/<название файла>
```

Записать информацию в файл можно с помощью команды *echo*, как уже было показано выше:

```
sudo echo "информация" > /proc/<название файла>
```

В каталоге */proc* очень много файлов, и рассмотреть все мы не сможем. Прежде чем мы приступим к самым интересным с моей точки зрения файлам, нужно понять, что означают каталоги с числами. Эти каталоги содержат информацию о запущенных процессах.



Итак, самые полезные информационные файлы:

- */proc/cmdline* — содержит параметры, переданные ядру при загрузке;
- */proc/cpuinfo* — содержит информацию о процессоре, откройте этот файл, думаю, вам будет интересно. Кроме общей информации о процессоре вроде модели и частоты, здесь выводится точная частота, размер кэша и псевдорейтинг производительности, выраженный в *BogoMIPS*. Значение *BogoMIPS* показывает «сколько миллионов раз в секунду компьютер может абсолютно ничего не делать». Способ измерения производительности пусть и не самый удачный, но от него до сих пор не отказались, а «на дворе» уже 3-я версия ядра.
- */proc/devices* — список устройств.
- */proc/filesystems* — полный список поддерживаемых вашим ядром файловых систем.
- */proc/interrupts* — информация по прерываниям.
- */proc/ioports* — информация о портах ввода/вывода.
- */proc/meminfo* — полная информация об использовании оперативной памяти. Как по мне, вывод этого файла более понятен и удобен, чем вывод команды *free*.
- */proc/mounts* — содержит список подмонтированных файловых систем.
- */proc/modules* — список загруженных модулей и их параметры.
- */proc/swaps* — содержит список активных разделов и файлов подкачки.
- */proc/version* — здесь находится версия ядра.



Используя `/proc`, можно не только получить информацию о системе, но и изменить ее. Например, в файлах `/proc/sys/kernel/hostname` и `/proc/sys/kernel/domainname` содержится информация об имени компьютера и домена. Вы можете не только просмотреть, но и изменить содержимое этих файлов, изменив, соответственно, имя узла и имя домена. Хотя практика изменения доменных имен через `/proc/sys` практикуется не часто, никто не мешает вам это сделать:

```
sudo echo "server" > /proc/sys/kernel/hostname
sudo echo "example.com" > /proc/sys/kernel/domainname
```

Файл `/proc/sys/kernel/ctrl-alt-del` позволяет регулировать тип перезагрузки системы при нажатии комбинации клавиш `Ctrl+Alt+Del`.

По умолчанию в этом файле содержится значение 0, что означает так называемую «мягкую перезагрузку» (soft reboot). Если же вы внесете в этот файл значение 1, то при нажатии `Ctrl+Alt+Del` эффект будет такой же, как при нажатии кнопки **Reset** на корпусе компьютера:

```
sudo echo "1" > /proc/sys/kernel/ctrl-alt-del
```

Файл `/proc/sys/kernel/printk` позволяет задать, какие сообщения ядра будут выведены на консоль, а какие — попадут в журнал демона `syslog`.

По умолчанию в этом файле содержатся значения 4 4 1 7. Сообщения с приоритетом 4 и ниже (первая четверка) будут выводиться на консоль. Вторая четверка — это уровень приоритета по умолчанию. Если для сообщения не задан уровень приоритета, то считается, что его приоритет будет равен 4.



Третье значение определяет номер самого максимального приоритета. Последнее значение — это уровень приоритета по умолчанию для первого значения.

В большинстве случаев изменяют только первое значение, позволяющее определить, будут ли сообщения с указанным уровнем приоритета выводиться на консоль или нет. Остальные параметры оставляют без изменения.

В файле `/proc/sys/net/core/netdev_max_backlog` содержится максимальное число пакетов в очереди. Значение по умолчанию — 1000.

Файл `/proc/sys/fs/file-max` позволяет изменить максимальное количество заголовков файлов, которое может быть одновременно открыто.

Другими словами, этот файл задает, сколько одновременно может быть открыто файлов. Значение по умолчанию для ядра 3.16 и файловой системы `btrfs` — 73054.

Чтобы сохранить внесенные «на лету» изменения и не вводить их снова при следующей перезагрузке сервера, нужно отредактировать файл `/etc/sysctl.conf`. Представим, что вы изменили значение из файла `/proc/sys/fs/file-max`. Тогда в файл `/etc/sysctl.conf` нужно добавить строку:



```
fs.file-max = 16 384
```

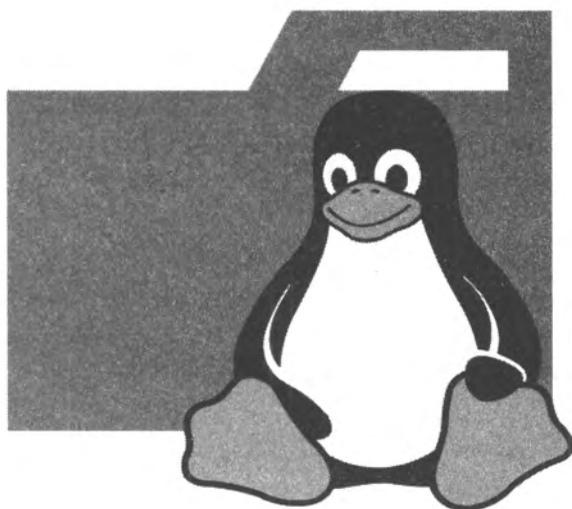
Принцип прост: `/proc/sys/` отбрасывается совсем, а в оставшейся строке все слэши заменяются точками. Само же значение указывается через знак равенства. Если нужно указать несколько значений, то они указываются через пробел.



О файловой системе в Linux можно написать отдельную книгу, которая будет не меньше, чем справочник, который вы держите в руках. Поэтому в этой главе мы рассмотрели только самое необходимое.

ГЛАВА 6.

УПРАВЛЕНИЕ ХРАНИЛИЩЕМ





В этой справочной главе мы рассмотрим довольно важные вещи — подключение нового жесткого диска и его разметку в классическом варианте — без всяких менеджеров томов, затем мы рассмотрим LVM и то, как можно расширить пространство группы томов, например, когда вы увеличили дисковое пространство виртуального сервера.

6.1. Подключение нового жесткого диска и его разметка

Классической программой для разметки жесткого диска в Linux и других операционных системах является программа **fdisk**. Конечно, в той же Windows программа **fdisk** совсем другая, но названия программ совпадают.

Рассмотрим пример использования этой программы. Представим, что мы подключили новый жесткий диск и нам нужно ввести его в эксплуатацию. Тренироваться лучше всего в виртуальной машине, особенно если вы в первый раз осуществляете разметку диска.



Формат вызова *fdisk* такой:

```
# fdisk <устройство>
```

Да, команду *fdisk* нужно запускать с правами *root*. Далее мы будем считать, что новым является устройство */dev/sdb*:

```
# fdisk /dev/sdb
```

Посмотрите на рис. 6.1. Я запустил программу *fdisk* для нового и не-размеченного жесткого диска. Программа сообщила мне, что:

1. Все изменения хранятся только в памяти и не переносятся на жесткий диск до тех пор, пока вы их не запишете.
2. Устройство не содержало таблицы разделов, и была создана таблица разделов DOS (по умолчанию).

```
root@localhost:~# fdisk /dev/sdb
Welcome to fdisk (util-linux 2.20).
Changes will remain in memory only, until you decide to write them.
Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS disklabel with disk identifier 0x4fc726db.

Command (m for help):
```

Рис. 6.1. Запуск *fdisk* для нового жесткого диска

Первым делом ознакомимся со списком команд *fdisk*. Введите команду *m* для получения справки. Список команд в последних версиях *fdisk* разбит на группы (рис. 6.2). В таблице 6.1 приведен список команд *fdisk*.



```

Welcome to fdisk (util-linux 2.20).
Changes will remain in memory only, until you decide to write them
to disk. Be careful before using the write command.

Device does not contain a recognized partition table.
Created a new DOS disklabel with disk identifier 0x00000000.

Command (for help): m

DOS (MBR)
a toggle a bootable flag
b edit second BSD disklabel
c toggle the dos compatibility flag

Generic:
d delete a partition
l list known partition types
n add a new partition
p print the partition table
t change a partition type
w write the partition table
z print information about a partition

Misc:
u print this menu
u change display/entry units
e extra functionality (expert only)

Script:
f load disk layout from fdisk script file
F dump disk layout to fdisk script file

Save & Exit:
w write table to disk and exit
q quit without saving changes

Create a new label:
g create a new empty GPT partition table
G create a new empty GPT (BIOS) partition table
o create a new empty BSD partition table
n create a new empty Sun partition table

Command (for help):

```

Рис. 6.2. Список команд fdisk

Таблица 6.1. Команды программы fdisk

Команда	Описание
<i>a</i>	Сделать раздел активным. Данный флаг был нужен для старых версий Windows, которые не могли загружаться с неактивных разделов. Сейчас эта команда попросту не нужна, а в мире Linux — тем более
<i>b</i>	Редактировать вложенную BSD-метку
<i>c</i>	Применить флаг совместимости с DOS
<i>d</i>	Удалить раздел
<i>l</i>	Вывести известные типы разделов
<i>n</i>	Добавить новый раздел



<i>p</i>	Вывод таблицы разделов
<i>t</i>	Изменить тип раздела
<i>v</i>	Проверить таблицу разделов
<i>m</i>	Вывод справки
<i>u</i>	Изменить единицы измерения
<i>x</i>	Дополнительная функциональность (только для экспертов)
<i>w</i>	Записать таблицу разделов на диск и выйти
<i>q</i>	Выход без сохранения изменений
<i>g</i>	Создать новую пустую таблицу разделов GPT
<i>G</i>	Создать новую пустую таблицу разделов SGI (для ОС IRIX)
<i>o</i>	Создать новую пустую таблицу разделов DOS
<i>s</i>	Создать новую пустую таблицу разделов Sun

Наша задача — создать раздел (или несколько разделов, здесь решать вам) и подмонтировать его (их) к корневой файловой системе.

Первым делом выведем таблицу разделов командой *p* (рис. 6.3). Как видно из рис. 6.3, таблица разделов пуста, а размер нашего жесткого диска всего 60 Гб.



```
Command (m for help): p
Disk /dev/sdb: 60 GiB, 64424509440 bytes, 125829120 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: dos
Disk identifier: 0x4fc726d8

Command (m for help):
```

Рис. 6.3. Пустая таблица разделов

Наш диск довольно скромного размера, поэтому программа создала таблицу разделов DOS. Для больших жестких дисков лучше создать таблицу разделов GPT. Если программа неправильно выбрала тип таблицы разделов или вы хотите изменить его принудительно, введите команду `g`. Посмотрите на рис. 6.4 — я изменил тип таблицы разделов, а затем опять отобразил таблицу разделов. Она по-прежнему пуста, но обратите внимание на ее тип — теперь у нас таблица разделов GPT.

ПРИМЕЧАНИЕ. Таблица разделов GPT (GUID Partition Table) является частью стандарта EFI (Extensible Firmware Interface) — стандарта, который был предложен компанией Intel на смену стандарта BIOS. Таблица GPT использует современную систему адресации логических блоков (LBA), а не старую систему CHS (цилиндр-головка-сектор). Но самое главное — это размер раздела. В GPT можно создать раздел размером до 9.4 Збайт (9.4×10^{21} байт), а в MBR — максимальный размер раздела всего 2.2 Тб (2.2×10^{21} байт).

```
Command (m for help): g
Created a new GPT disklabel (GUID: Z99E41E1-4A89-42E6-911F-C39BC06E7DE3).

Command (m for help): p
Disk /dev/sdb: 60 GiB, 64424509440 bytes, 125829120 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: Z99E41E1-4A89-42E6-911F-C39BC06E7DE3

Command (m for help):
```

Рис. 6.4. Изменение таблицы разделов



Настало время создать раздел. Введите команду *n*. Программа попросит вас ввести (рис. 6.5):

1. Номер раздела — это первый раздел, поэтому введите 1. В принципе, когда вы будете создавать второй раздел, программа автоматически предложит вам ввести номер 2.
2. Первый сектор раздела. Просто нажмите **Enter** — программа автоматически предложит правильный вариант.
3. Последний сектор раздела. Если вы хотите создать раздел на весь жесткий диск (то есть использовать все доступное пространство), тогда просто нажмите **Enter**. В противном случае укажите размер раздела. Проще всего это сделать, используя модификаторы +M и +G, например, для создания раздела размером 30 Гб укажите +30G. Если у вас очень большой жесткий диск, где пространство измеряется терабайтами, используйте модификатор T, например +1T.

```
Command (m for help): n
Partition number (1-128, default 1): 1
First sector (2048-125829036, default 2048):
Last sector, +sectors or +size (K,M,G,T,P) (2048-125829036, default 125829036):
Created a new partition 1 of type 'Linux filesystem' and of size 60 GiB.
Command (m for help):
```

Рис. 6.5. Создание нового раздела

Программа сообщит вам, что один раздел создан. Также будет сообщен размер раздела. По умолчанию тип раздела — файловая система Linux (Linux filesystem). Если вы хотите изменить тип раздела, введите команду:

t <номер раздела>

Например:

t 1

Далее нужно или ввести код типа раздела, или ввести команду L для вывода подсказки (рис. 6.6). Проблема вся в том, что нет способа постраничного просмотра типов разделов, а все они не помещаются на одном экране. Поэтому все равно придется обращаться к документации. Однако в большинстве случаев изменять тип раздела не нужно, поскольку при создании раздела он создается уже нужного типа. Исключение может возникнуть разве что для раздела подкачки (Linux swap). Его код — 82.

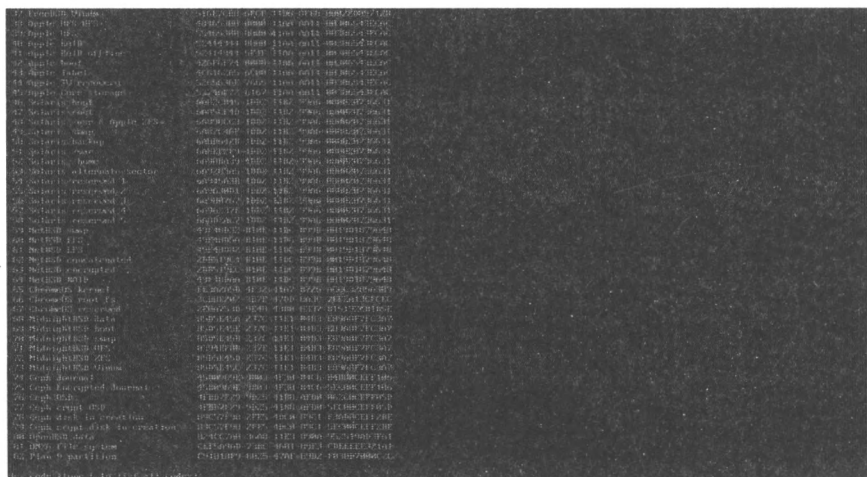


Рис. 6.6. Подсказка по типу раздела

Если вы передумали менять тип раздела, просто нажмите **Enter**.

Теперь введите `p` для просмотра нашей таблицы разделов. Всегда просматривайте таблицу перед ее записью. После этого введите команду `w` для сохранения изменений и выхода из программы (рис. 6.7).

Создать раздел мало. Нужно еще создать файловую систему. Не будем ничего выдумывать и создадим стандартную файловую систему ext4 командой `mkfs.ext4`:



После этого нужно создать точку монтирования для нового раздела и подмонтировать раздел (название точки монтирования можете изменить по своему усмотрению):

```
# mkdir /mnt/sdb1
# mount /dev/sdb1 /mnt/sdb1
# ls /mnt/sdb1
```

Почти все готово. Осталось только добавить запись в `/etc/fstab` для автоматического монтирования созданного раздела:

```
/dev/sdb1 /mnt/sdb1 ext4 defaults      1 1
```

Вот теперь можно приступать к использованию нового жесткого диска.

6.2. Менеджер логических томов

6.2.1. Введение в LVM

Впервые менеджер логических томов (LVM, Logical Volume Manager) появился в ядре 2.4 (его первая версия LVM 1), но более активно он стал применяться только в дистрибутивах с ядром 2.6 (уже вторая версия LVM 2).

Некоторые современные дистрибутивы используют LVM по умолчанию, создают пулы LVM, в некоторых же LVM даже не установлен по умолчанию. Чтобы понять, нужен ли вам LVM, нужно разобраться, что это такое. В этой главе все будет изложено максимально доступно, так что бояться использовать LVM не стоит. В то же время, если вам необходимы точные академические определения и до-



полнительная информация, обратитесь к Linux LVM Howto (<http://tldp.org/HOWTO/LVM-HOWTO/>), в котором много технических подробностей (если они вам нужны) и не все сразу понятно.

LVM — это дополнительный уровень абстракции над аппаратными средствами, позволяющий собрать вместе несколько дисков в один логический диск, а затем разбить его так, как вам хочется.

Примеров использования LVM множество. Самый простой из них — объединение нескольких небольших дисков в один диск большего размера. Например, вам досталось даром (или почти даром) несколько SSD-дисков небольшого размера, скажем, по 128 Гб, и вы хотите объединить эти 2-3 диска, чтобы получить один большой диск 256–384 Гб.

Второй пример тоже часто распространен. Представим, что система у вас установлена на небольшом диске, пусть даже на том же SSD-диске размером 80 Гб. Для Linux такой объем вполне достаточен, но рано или поздно свободное место закончится (все зависит от файлов, с которыми вы работаете). Вы покупаете диск большего размера, скажем, на 500 Гб или даже на 1 Тб. Но система уже установлена, и переустанавливать ее не хочется. Что делать? Здесь вам поможет LVM.

Первый приведенный мною пример (объединение трех дисков во время установки) слишком тривиален, и с ним справится любой, даже самый начинающий пользователь — просто во время установки нужно выбрать LVM (если, конечно, дистрибутив его поддерживает) и выбрать диски, которые вы объединяете в группу (пул).

Второй пример более сложный только за счет того, что система уже установлена, и мы договорились, что переустанавливать ее не будем. Поэтому он и заслуживает рассмотрения в этой главе, а дополнительную информацию вы сможете найти в Linux LVM Howto, ссылка на который была приведена ранее.



6.2.2. Уровни абстракции LVM

Прежде чем перейти к рассмотрению практической стороны вопроса, нужно разобраться с тремя уровнями абстракции, с которыми вам придется столкнуться в LVM. Вот эти уровни:

1. **PV** (Physical Volume) — физические тома. Это могут быть разделы или целые, еще не размеченные диски.
2. **VG** (Volume Group) — группа томов. Физические тома объединяются в группу, и создается единый диск, который вы можете разбить так, как вам хочется.
3. **LV** (Logical Volume) — логический раздел. Это раздел нашего единого диска (VG), который вы можете отформатировать в любую файловую систему и использовать так, как вам хочется, как обычный раздел обычного жесткого диска.

Прежде чем мы продолжим, вы должны знать об одном недостатке LVM. Тома LVM не поддерживаются загрузчиком GRUB. Поэтому если вы используете этот устаревший загрузчик, вам нужно создать отдельный раздел /boot за пределами LVM. Грубо говоря, если у вас есть диск /dev/sda, раздел /dev/sda1 должен монтироваться к /boot. В него будет установлен загрузчик. Размер этого раздела должен быть небольшим, примерно 100 Мб (вполне будет достаточно).

Что же касается GRUB2, то он нормально загружается с LVM и никакой дополнительный раздел создавать не нужно.

6.2.3. Немного практики

Первым делом вам нужно установить пакет `lvmd`, если он еще не установлен.

```
# apt-get install lvmd
# yum install lvmd
```



Команда установки этого пакета для Fedora/CentOS приведена на всякий случай, так как в большинстве случаев этот пакет в этих дистрибутивах установлен по умолчанию.

Будем считать, что система сейчас установлена на `/dev/sda1`, который подмонтирован как `/`. Больше никаких разделов на этом диске не создано — для упрощения примера. Мы подключили второй жесткий диск, который пока еще не разбит. Имя этого диска — `/dev/sdb`.

Если на втором диске не созданы разделы, то создавать их и не нужно. Вы можете сделать все устройство сразу физическим томом (PV). Если на нем есть разделы — не беда, вы можете добавить в группу томов все разделы поочередно.

Тратить время на создание разделов не хочется, тем более что это и не нужно, поэтому создаем PV на все устройство `/dev/sdb`. Для этого используется команда *pvc*reate:

```
# pvcreate /dev/sdb
Physical volume "/dev/sdb" successfully created
```

Теперь нужно создать группу томов с помощью команды *vg*create. Данной команде нужно передать имя группы (пусть это будет *my_vg*) и указать физическое устройство:

```
# vgcreate my_vg /dev/sdb
Volume group "vg0" successfully created
```

Создаем отдельные логические тома (команда *lv*create) для раздела подкачки (*swap*) и разделов `/home`, `/tmp` и `/var`. Параметр *-L* задает размер раздела, например, *-L30G* задает размер 30 Гб. Параметр *-n* задает имя логического тома:



```
# lvcreate -n swap -L8G my_vg
# lvcreate -n home -L500G my_vg
# lvcreate -n var -L30G my_vg
# lvcreate -n tmp -L5G my_vg
```

Последний параметр — это имя нашей группы. При желании можно создать отдельный раздел и для `/usr`, но, как правило, программное обеспечение (а оно в основном устанавливается в `/usr`) в Linux много места не занимает. В общем, смотрите сами — никто не мешает ввести еще одну команду *lvcreate*. Тем более что у нас еще осталось место (если учитывать, что у нас жесткий диск на 1 ТБ).

Теперь разберемся, что и где мы создали.

Просмотреть информацию по физическим томам, группам томов и логическим разделам можно с помощью команд *pvdisplay*, *vgdisplay* и *lvdisplay* соответственно.

Созданные нами разделы будут храниться в папке `/dev/my_vg/`. В этом каталоге вы найдете файлы `home`, `tmp`, `var` (правда, это будут ссылки, а не файлы, но суть от этого не меняется).

Когда созданы логические тома, можно создать на них файловые системы (отформатировать их). Вы можете использовать любую файловую систему, я предпочитаю `ext4`:

```
# mkfs.ext4 -L var /dev/my_vg/var
# mkfs.ext4 -L home /dev/my_vg/home
# mkfs.ext4 -L tmp /dev/my_vg/tmp
# mkswap -L swap /dev/my_vg/swap
# swapon /dev/my_vg/swap
```

Первые три команды создают файловую систему `ext4` на устройствах `/dev/my_vg/var`, `/dev/my_vg/home` и `/dev/my_vg/tmp`. Последние две создают раздел подкачки и активируют его.



Настало время заняться перемещением данных. Суть в следующем — нужно подмонтировать поочередно новые тома и скопировать в них содержимое `/home` и `/var`:

```
# mkdir /mnt/home
# mkdir /mnt/var

# mount /dev/my_vg/home /mnt/home
# mount /dev/my_vg/var /mnt/var

# cp -a /home/* /mnt/home
# cp -a /var/* /mnt/var

# umount /mnt/home
# umount /mnt/var
```

В папку `/tmp` копировать ничего не нужно. Нужно только изменить права доступа:

```
# mkdir /mnt/tmp
# mount /dev/my_vg/tmp /mnt/tmp
# chmod -R a+rwX /mnt/tmp
# umount /tmp
```

Почти все. Теперь нужно добавить в `/etc/fstab` записи, монтирующие файловые системы `/home`, `/var`, `/tmp`, и указать в нем раздел подкачки:

<code>/dev/mapper/my_vg-home</code>	<code>/home</code>	<code>ext4 relatime 1 1</code>
<code>/dev/mapper/my_vg-var</code>	<code>/var</code>	<code>ext4 relatime 1 1</code>
<code>/dev/mapper/my_vg-tmp</code>	<code>/tmp</code>	<code>ext4 noatime 0 2</code>
<code>/dev/mapper/my_vg-swap</code>	<code>none</code>	<code>swap sw 0 0</code>

Все готово. Осталось только ввести команду *reboot*, чтобы система перезагрузилась. Корневая файловая система осталась на старом жестком диске (как и `/usr`), а каталоги, которые занимают больше всего места, были перемещены на логические разделы LVM.

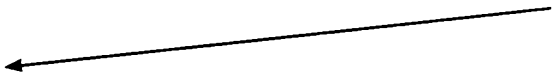


6.3. Расширение LVM-пространства

Предположим, что вы расширили дисковое пространство виртуального сервера. Однако одного расширения в панели управления недостаточно — чтобы система увидела изменения, нужно произвести определенные действия. В принципе, задачу расширения пространства сервера можно было решить иначе, например, добавить еще один виртуальный жесткий диск, а дальше или использовать классический способ (раздел 6.1), или же добавить диск в группу томов (раздел 6.2) — все зависит от того, как настроена операционная система виртуального сервера. Но случилось то, что случилось — вы уже расширили жесткий диск, а вернуть ресурсы в пул, как правило, нельзя. Поэтому рассмотрим процедуру расширения виртуального диска.

Первым делом посмотрим, сколько сейчас дискового пространства доступно. Для этого используется уже известная команда `df` с параметром `-h`, чтобы вывод был в удобочитаемом формате:

`df -h`



```
root@ubuntu1804:~# df -h
Filesystem            1.5G  used Avail Use% Mounted on
udev                  1.5G   0    1.5G   0% /dev
tmpfs                 301M  4.5M  296M   2% /run
/dev/mapper/vgroup1-root 19G  2.1G   17G  12% /
tmpfs                 1.5G   0    1.5G   0% /dev/shm
tmpfs                 5.0M   0   5.0M   0% /run/lock
tmpfs                 1.5G   0    1.5G   0% /sys/fs/cgroup
/dev/sda1             92.0G 14.0G  77.9G  17% /boot
tmpfs                 301M   0   301M   0% /run/user/0
root@ubuntu1804:~#
```

Рис. 6.9. Команда `df -h`

На данный момент общий размер группы томов `/dev/mapper/vgroup1-root` составляет 19 Гб. Однако мы расширили диск, и теперь нам нужно расширить размер этой группы томов до полного размера диска.

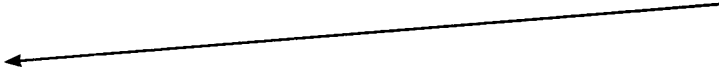


Чтобы система увидела новый объем жесткого диска, нужно пересканировать аппаратную конфигурацию. Для этого мы будем использовать следующую команду:

```
echo 1 > /sys/block/sda/device/rescan
```

Запустите утилиту **parted** (используется для работы с разделами диска):

parted



Введите команду *p* для просмотра имеющихся разделов (рис. 6.10). Запомните номер раздела, который мы будем расширять (2), и новый размер диска (42.9GB).

```
root@ubuntu1804:~# echo 1 > /sys/block/sda/device/rescan
root@ubuntu1804:~# parted
Using /dev/sda
Welcome to GNU Parted! Type 'help' to view a list of commands.
(parted) p
Model: VMware Virtual disk (scsi)
Disk /dev/sda: 42.9GB
Sector size (logical/physical): 512B/512B
Partition Table: msdos
Disk Flags:

Number   Start   End     Size    Type     File system  Flags
 1        1049KB  1000MB  999MB   primary  ext4         boot
 2        1000MB  21.5GB  20.5GB  primary                lvm
```

Рис. 6.10. Текущая таблица разделов

Запустим команду изменения раздела:

```
resizepart
```

Укажем номер раздела:



```
Partition number? 2
```



А затем — новый размер раздела. Нужно указать как раз то самое значение 42.9GB — именно так, без пробелов.

Введите команду *quit* для выхода из **parted**.

Parted сделал свою работу. Осталось сообщить ядру об изменениях размера:

```
pvresize /dev/sda2
Physical volume "/dev/sda2" changed
1 physical volume(s) resized / 0 physical volume(s) not resized
```

Расширим логический том:

```
lvextend -r -l +100%FREE /dev/mapper/vgroup1-root
```

По окончании работ введем *df -h*, чтобы убедиться, что дисковое пространство расширилось.

Посмотрите на рис. 6.12. На нем результат выполнения команд *pvresize*, *lvextend* и *df*. Последний вывод сообщает нам, что размер группы томов *vgroup1-root* теперь составляет 41 Гб.

Вот таким образом мы успешно расширили том до нового размера.

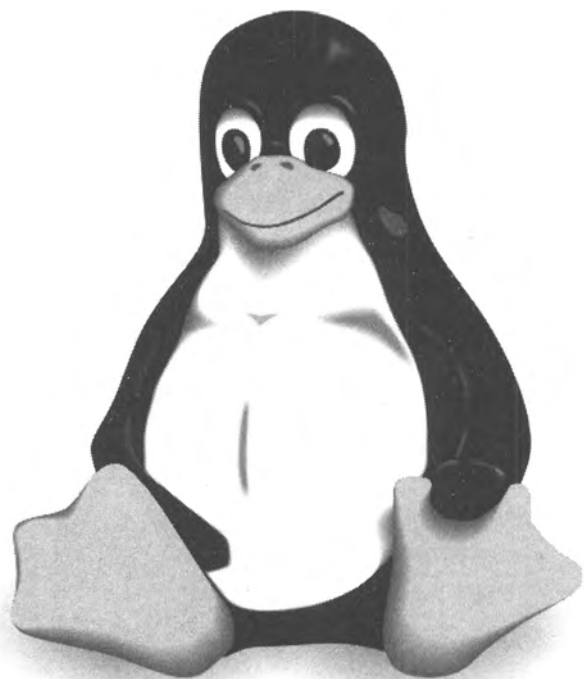


```

root@ubuntu1804:~# pvresize /dev/sda2
Physical volume "/dev/sda2" changed
1 physical volume(s) resized / 0 physical volume(s) not resized
root@ubuntu1804:~# lvextend -r -l +100%FREE /dev/mapper/vgroup1-root
Size of logical volume vgroup1/root changed from 18.11 GiB (4637 extents) to 38.07 GiB (9745 extents).
Logical volume vgroup1/root successfully resized.
meta-data=/dev/mapper/vgroup1-root isize=512    agcount=12, agsize=400640 blks
       =                               sectsz=512   attr=2, projid32bit=1
       =                               crcs=1      finobt=1 spinodes=0 rmapbt=0
       =                               reflink=0
data     =                               bsize=4096   blocks=4748288, imaxpct=25
       =                               sunit=0      swidth=0 blks
naming   =version 2                   bsize=4096   ascii-ci=0 ftype=1
log      =internal                   bsize=4096   blocks=2560, version=2
       =                               sectsz=512   sunit=0 blks, lazy-count=1
realtime =none                      extsz=4096   blocks=0, rtextents=0
data blocks changed from 4748288 to 9978880
root@ubuntu1804:~# df -H
Filesystem      Size  Used Avail Use% Mounted on
udev            1.6G   0  1.6G   0% /dev
tmpfs           315M  4.8M  311M   2% /run
/dev/mapper/vgroup1-root 41G  2.3G   39G   6% /
tmpfs           1.6G   0  1.6G   0% /dev/shm
tmpfs           5.3M   0  5.3M   0% /run/lock
tmpfs           1.6G   0  1.6G   0% /sys/fs/cgroup
/dev/sdal       967M  147M  754M  17% /boot
tmpfs           315M   0  315M   0% /run/user/0

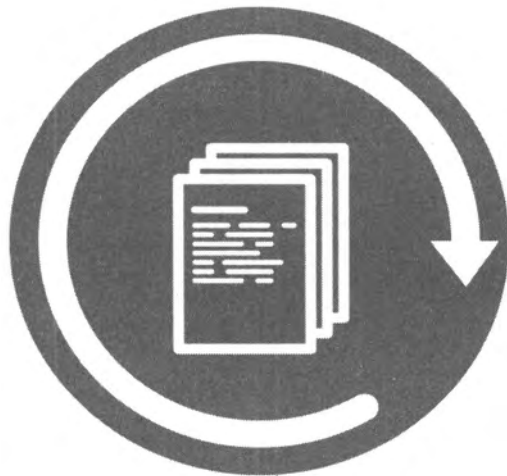
```

Рис. 6.12. Том расширен



ГЛАВА 7.

УПРАВЛЕНИЕ ЗАГРУЗКОЙ ОС





7.1. Загрузчики Linux

Существует несколько загрузчиков Linux. На сегодняшний день основным загрузчиком является GRUB2, который устанавливается по умолчанию во всех современных дистрибутивах Linux.

Одним из самых «древних» загрузчиков является LILO (Linux LOader). Этот загрузчик давно уже не используется, и ему на смену пришел загрузчик GRUB (GRand Unified Bootloader). GRUB является более гибким загрузчиком и «понимает» много разных файловых систем, в том числе FAT/FAT32, ext2, ext3, ReiserFS, XFS, BSDFS.

На смену GRUB пришел загрузчик GRUB2. Отличие GRUB — очень запутанный и неудобный файл конфигурации, но время не стоит на месте, и этот загрузчик уже не используется ни в одном современном дистрибутиве — только GRUB2. К преимуществам GRUB2 можно отнести — поддержку ext4 (а эта файловая система сейчас стандарт де-факто), загрузку с LVM, поддержку UEFI. Всех этих вещей в GRUB нет, поэтому его использование сегодня невозможно.



Собственно, GRUB2 сейчас устанавливается во всех современных дистрибутивах, и нет смысла возвращаться на GRUB. Если возникнет необходимость, вы можете вернуться на обычный GRUB, установив пакет **grub-legacy**, но такая возможность есть только для платформы x86.

7.2. Загрузчик *GRUB2*

7.2.1. Конфигурационные файлы

В каталоге `/etc/grub.d` хранятся шаблоны, определяющие настройки GRUB2. Также некоторые его параметры хранятся в файле `/etc/default/grub`. По шаблонам из `/etc/grub.d` и файлу `/etc/default/grub` программой `/usr/sbin/grub-mkconfig` создается рабочий конфигурационный файл `/boot/grub/grub.cfg`, который по задумке разработчиков GRUB2 вы не должны редактировать вручную.

Поэтому есть две стратегии настройки GRUB2. Первая заключается в непосредственном редактировании файла `/boot/grub/grub.cfg`. Загрузчику GRUB2 все равно, кто или что отредактирует этот файл — вы или программа **grub-mkconfig**. Вторая заключается в редактировании файлов из каталога `/etc/grub.d` и файла `/etc/default/grub`. После чего вы будете должны ввести команду *grub-mkconfig* для создания файла `/boot/grub/grub.cfg` по заданным вами настройкам.

Чтобы решить, какая из стратегий для вас лучше, нужно знать формат и содержимое всех этих файлов.

Начнем с основного файла конфигурации, который сложнее и длиннее файла конфигурации обычного GRUB (см. лист. 7.1).

**Листинг 7.1. Файл конфигурации /boot/grub/grub.cfg**

```
#
# Не редактируйте этот файл вручную!
#
# Он автоматически генерируется программой grub-mkconfig по шаблонам
# из /etc/grub.d и настройкам из /etc/default/grub
#

### НАЧАЛО файла /etc/grub.d/00_header ###
if [ -s $prefix/grubenv ]; then
    load_env
fi
# Загрузочная метка по умолчанию
set default="0"
if [ "${prev_saved_entry}" ]; then
    set saved_entry="${prev_saved_entry}"
    save_env saved_entry
    set prev_saved_entry=
    save_env prev_saved_entry
    set boot_once=true
fi

function savedefault {
    if [ -z "${boot_once}" ]; then
        saved_entry="${chosen}"
        save_env saved_entry
    fi
}

function load_video {
    insmod vbe
    insmod vga
    insmod video_bochs
    insmod video_cirrus
```




```
}

insmod part_msdos
insmod ext2
# Корневое устройство
set root='(hd0,msdos1)'
search --no-floppy --fs-uuid --set=root b7300e54-fff5-4f31-8002-bea43c64f344
if loadfont /usr/share/grub/unicode.pf2 ; then
    set gfxmode=640x480
    load_video
    insmod gfxterm
    insmod part_msdos
    insmod ext2
    set root='(hd0,msdos1)'
    search --no-floppy --fs-uuid --set=root b7300e54-fff5-4f31-8002-bea43c64f344
    set locale_dir=($root)/boot/grub/locale
    set lang=ru_RU
    insmod gettext
fi
terminal_output gfxterm
set timeout=5
### КОНЕЦ файла /etc/grub.d/00_header ###

### НАЧАЛО файла /etc/grub.d/05_debian_theme ###
insmod part_msdos
insmod ext2
# Корневое устройство
set root='(hd0,msdos1)'
search --no-floppy --fs-uuid --set=root b7300e54-fff5-4f31-8002-bea43c64f344
insmod png

if background_image /usr/share/images/desktop-base/joy-grub.png; then
    set color_normal=white/black
    set color_highlight=black/white
else
    set menu_color_normal=cyan/blue
```



```
set menu_color_highlight=white/blue
fi
### КОНЕЦ файла /etc/grub.d/05_debian_theme ###

### НАЧАЛО файла /etc/grub.d/10_linux ###
# Содержит главную загрузочную метку. Далее мы ее рассмотрим подробнее
menuentry 'AstraLinuxCE GNU/Linux with Linux 5.10.0-1038.40-generic'
--class debian --class gnu-linux --class gnu --class os {
    load_video
    insmod gzio
    insmod part_msdos
    insmod ext2
    set root='(hd0,msdos1)'
    search --no-floppy --fs-uuid --set=root b7300e54-fff5-4f31-8002-bea43c64f344
    echo 'Loading Linux 5.10.0-1038.40-generic...'
    linux /boot/vmlinuz-5.10.0-1038.40-generic root=UUID=b7300e54-fff5-4f31-8002-bea43c64f344 ro initrd=/install/gtk/initrd.gz quiet
    echo 'Loading ramdisk ...'
    initrd /boot/initrd.img-5.10.0-1038.40-generic
}
menuentry 'AstraLinuxCE GNU/Linux with Linux 5.10.0-1038.40-generic
(recovery mode)' --class debian --class gnu-linux --class gnu --class os {
    load_video
    insmod gzio
    insmod part_msdos
    insmod ext2
    set root='(hd0,msdos1)'
    search --no-floppy --fs-uuid --set=root b7300e54-fff5-4f31-8002-bea43c64f344
    echo 'Loading Linux 5.10.0-1038.40-generic...'
    linux /boot/vmlinuz-5.10.0-1038.40-generic root=UUID=b7300e54-fff5-4f31-8002-bea43c64f344 ro single initrd=/install/gtk/initrd.gz
    echo 'Loading ramdisk ...'
    initrd /boot/initrd.img-5.10.0-1038.40-generic
}
### КОНЕЦ /etc/grub.d/10_linux ###
```



```
### НАЧАЛО /etc/grub.d/20_linux_xen ###
### КОНЕЦ /etc/grub.d/20_linux_xen ###

### НАЧАЛО /etc/grub.d/20_memtest86+ ###
# Метка для memtest86 - программы для проверки памяти
menuentry "Memory test (memtest86+)" {
    insmod part_msdos
    insmod ext2
    set root='(hd0,msdos1)'
    search --no-floppy --fs-uuid --set=root b7300e54-fff5-4f31-8002-bea43c64f344
    linux16 /boot/memtest86+.bin
}

menuentry "Memory test (memtest86+, serial console 115200)" {
    insmod part_msdos
    insmod ext2
    set root='(hd0,msdos1)'
    search --no-floppy --fs-uuid --set=root b7300e54-fff5-4f31-8002-bea43c64f344
    linux16 /boot/memtest86+.bin console=ttyS0,115200n8
}

menuentry "Memory test (memtest86+, experimental multiboot)" {
    insmod part_msdos
    insmod ext2
    set root='(hd0,msdos1)'
    search --no-floppy --fs-uuid --set=root b7300e54-fff5-4f31-8002-bea43c64f344
    multiboot /boot/memtest86+_multiboot.bin
}

menuentry "Memory test (memtest86+, serial console 115200, experimental multiboot)" {
    insmod part_msdos
    insmod ext2
    set root='(hd0,msdos1)'
    search --no-floppy --fs-uuid --set=root b7300e54-fff5-4f31-8002-bea43c64f344
    multiboot /boot/memtest86+_multiboot.bin console=ttyS0,115200n8
}

### КОНЕЦ /etc/grub.d/20_memtest86+ ###
```



Далее этот файл я немного сократил, поскольку дальше в нем нет ничего интересного

Файл огромный, и его синтаксис напоминает синтаксис `bash`-сценариев. Если вы просмотрели этот конфигурационный файл, то вы уже догадались, что делает программа **grub-mkconfig**: она собирает воедино все файлы из каталога `/etc/grub.d` (кстати, в листинге 7.1 перечислена большая часть из этих файлов) и вносит в общий конфигурационный файл из `/etc/default/grub`.

Основная запись из всего листинга 7.1 — это запись *menuentry*. Именно в таких записях описываются элементы меню загрузчика GRUB.

```
menuentry 'AstraLinuxCE GNU/Linux with Linux 5.10.0-1038.40-generic'
--class debian --class gnu-linux --class gnu --class os {
    load_video
    insmod gzio
    insmod part_msdos
    insmod ext2
    set root='(hd0,msdos1)'
    search --no-floppy --fs-uuid --set=root b7300e54-fff5-4f31-8002-
bea43c64f344
    echo 'Loading Linux 5.10.0-1038.40-generic...'
    linux /boot/vmlinuz-5.10.0-1038.40-generic root=UUID=b7300e54-fff5-
4f31-8002-bea43c64f344 ro initrd=/install/gtk/initrd.gz quiet
    echo 'Loading ramdisk ...'
    initrd /boot/initrd.img-5.10.0-1038.40-generic
}
```

В одинарных кавычках после *menuentry* указывается название загрузочной метки. Далее идут параметры, которые вообще можно не указывать, и от этого Debian загружаться не перестанет. В фигурных скобках — основная конфигурация. Директива **load_video** — это не что иное, как вызов функции *load_video*, которая также описана в этом файле конфигурации. Функция вставляет некоторые моду-



ли (команда *insmod*), необходимые для работы графического режима. Обратите внимание, что команды *insmod* загружают не модули ядра Linux, а модули GRUB2, которые находятся в каталоге `/boot/grub`.

Внутри `{ }` можно использовать команду *echo* для обозначения различных этапов загрузки, что и сделано в нашем примере. Можете отказаться от *echo*, на загрузку это никак не повлияет.

Основные команды — это *linux* и *initrd*. Первая указывает путь к ядру Linux и задает параметры ядра. В нашем случае параметр ядра *root* указывает устройство, на котором находится корневая файловая система. Устройство указано в виде UUID. UUID-имена очень удобны. Представим, что у вас есть один жесткий диск SATA и два контроллера. Если вы подключите его ко второму контроллеру, обычное имя изменится (например, было `/dev/sda`, а стало `/dev/sdb`), а UUID-имя — нет. При желании вы можете указать имя в старом формате, например, *root=/dev/sda1*. Параметр ядра *ro* задает монтирование корневой файловой системы в режиме «только чтение» (это нормально, позже она будет перемонтирована), *initrd* — задает файл RamDisk, а последний параметр ядра *quiet* задает «тихую» загрузку ядра, при которой будут выводиться только самые важные сообщения.

Команда *initrd* задает путь к файлу *initrd*.

Теперь рассмотрим файл `/etc/default/grub` (листинг 7.2).

Листинг 7.2. Файл `/etc/default/grub`

```
# После редактирования этого файла запустите команду 'update-grub' для
# обновления файла /boot/grub/grub.cfg.
# Для получения полной информации об этом файле введите команду
#   info -f grub -n 'Simple configuration'

# Загрузочный элемент (menuentry по умолчанию)
```



```
GRUB_DEFAULT=0
# Таймаут
GRUB_TIMEOUT=5
# Задаёт название дистрибутива, не изменяйте эту строку
GRUB_DISTRIBUTOR='lsb_release -i -s 2> /dev/null || echo Debian'
# Параметры ядра Linux по умолчанию
GRUB_CMDLINE_LINUX_DEFAULT="quiet"
# Ещё одна строка для задания параметров ядра
GRUB_CMDLINE_LINUX="initrd=/install/gtk/initrd.gz"

# Раскомментируйте эту строку, если вы хотите отключить графический режим
#GRUB_TERMINAL=console

# Разрешение в графическом режиме
# Вы можете использовать только те режимы, которые ваша видеокарта
# поддерживает через VBE. Просмотреть список
# таких режимов можно с помощью команды 'vbeinfo'
#GRUB_GFXMODE=640x480

# Раскомментируйте эту строку, если вы не хотите
# использовать UUID-имена устройств
#GRUB_DISABLE_LINUX_UUID=true

# Раскомментируйте, если хотите запретить генерирование меток восстановления
#GRUB_DISABLE_RECOVERY="true"

# Раскомментируйте, если хотите получить гудок при загрузке GRUB
#GRUB_INIT_TUNE="480 440 1"
```

Как видите, параметры из файла `/etc/default/grub` понятны и не нуждаются в особых комментариях. Но в нем мы узнали о еще одной команде — *update-grub*. Так какую из них использовать — *update-grub* или *grub-mkconfig*?



На самом деле это почти одна и та же команда. Дело в том, что команда *grub-mkconfig* по умолчанию выводит конфигурацию GRUB2 на экран, поэтому, чтобы она записалась в файл `/boot/grub/grub.cfg`, запускать ее нужно так:

```
sudo grub-mkconfig > /boot/grub/grub.cfg
```

Или же вы можете ввести команду *update-grub*, которая сделает то же самое. Другими словами, команда *update-grub* — это сценарий, который вызывает только что приведенную команду. Как по мне, то использовать команду *update-grub* удобнее.

Так какую стратегию GRUB2 использовать? Редактировать шаблоны и параметры или сразу конфигурационный файл? Если вы работаете за компьютером в гордом одиночестве и нет и не предвидится других администраторов, тогда можете выбрать ту стратегию, которая вам больше нравится.

Если же есть или планируются другие администраторы, то нужно редактировать шаблоны и параметры вместо редактирования конфигурационного файла вручную. Дело в том, что если вы внесете изменения непосредственно в конфигурационный файл, а потом другой администратор захочет изменить какой-то незначительный параметр, например, добавить гудок при загрузке GRUB2, то команда *update-grub* перезапишет все сделанные вами изменения.

7.2.2. Выбор метки по умолчанию

Как правило, даже если у вас установлена одна только Linux, у вас будет несколько загрузочных меток (несколько записей *menuentry*). Выбрать метку по умолчанию можно с помощью параметра `GRUB_`



DEFAULT. Нумерация меток начинается с 0, то есть первой метке соответствует значение 0.

После того, как вы установите другой номер метки по умолчанию, нужно ввести команду *update-grub* и перезагрузить систему.

Другими словами, последовательность такая: редактируем файл `/etc/default/grub`, изменяем значение параметра `GRUB_DEFAULT` и вводим команду *update-grub*.

7.2.3. Загрузка Windows

Упор в этой книге делается на справочное описание различного применения Linux в разных сферах жизнедеятельности, однако вдруг вы захотите установить Linux на домашний компьютер, скорее всего, ей придется «сожительствовать» с Windows.

Чтобы добавить возможность загрузки Windows из GRUB2, нужно отредактировать файл `/etc/grub.d/40_custom` и добавить в него следующие строки:

```
menuentry "Windows (on /dev/sda1)" {
    insmod part_msdos
    insmod ntfs
    set root='(hd0,msdos1)'
    search --no-floppy --fs-uuid --set UUID
    drivemap -s (hd0) ${root}
    chainloader +1
}
```

Здесь Windows находится на `/dev/sda1`, и адрес этого раздела указывается в *set root*. Значение UUID нужно заменить на UUID вашего Windows-раздела. О том, как «вычислить» UUID раздела мы уже по-



говорили, надеюсь, с этим не возникнет трудностей. Остальные параметры можете оставить без изменения (еще не забудьте исправить `hd0` в `drivemap`, если Windows установлена не на первом жестком диске).

7.2.4. Пароль загрузчика GRUB2

Загрузчик GRUB позволял только установить пароль — или общий, или на загрузку определенной метки. Загрузчик GRUB2 более гибкий в этом плане, поскольку вы можете настроить не только пароли, но и логины. Также есть минимальная система разграничения прав доступа.

Итак, в GRUB2 есть суперпользователь, который может редактировать загрузочные метки. Существует возможность восстановить пароль *root* путем передачи ядру параметра *init*. Но для этого нужно отредактировать конфигурацию GRUB2. Если вы установите пароль суперпользователя, то изменить конфигурацию загрузчика вы сможете только после ввода этого пароля.

Также в GRUB2 есть обычные пользователи, которые имеют право только выбирать загрузочную метку. Они не имеют права редактировать конфигурацию загрузчика. В принципе, можно обойтись одним паролем суперпользователя, но при желании GRUB2 может довольно гибко разграничить права пользователей.

Давайте сначала добавим пароль суперпользователя. Для этого в файл `/etc/grub.d/00_header` добавьте строки:

```
set superusers="main_admin"  
password main_admin 123456789
```

Первая команда задает суперпользователя *main_admin*, а вторая — задает для него пароль. Старайтесь избегать общепринятых имен



вроде *admin*, *root* и т.д. Так у злоумышленника, который хочет изменить конфигурацию GRUB2, будет две неизвестных.

Пароль пока в незашифрованном виде, и это не очень хорошо. Поскольку если загрузиться с LiveCD или LiveUSB, то его можно будет увидеть. Позже я покажу, как зашифровать пароль.

Обычные пользователи задаются инструкцией *password*, например:

```
password me 12345
```



По сути *main_admin* тоже был бы обычным пользователем, если бы не инструкция *set superusers*, которая делает его суперпользователем.

Представим, что у нас есть следующие строки:

```
set superusers="main_admin"  
password main_admin 123456789  
password me 12345
```

Пользователь *main_admin* может загружать операционные системы и редактировать конфигурацию GRUB2. Пользователь *me* может только загружать операционные системы.

Если вы хотите, чтобы определенные метки могли загружать только определенные пользователи, добавьте к *menuentry* параметр *--users*:

```
menuentry "Windows" --users me {  
    insmod part_msdos  
    insmod ntfs  
    set root='(hd0,msdos1)'  
    search --no-floppy --fs-uuid --set UUID  
    drivemap -s (hd0) ${root}  
    chainloader +1  
}
```



Теперь зашифруем пароль. Введите команду:

```
grub-mkpasswd-pbkdf2
```

Программа запросит пароль, зашифрует его и выведет на экран его кэш. Вы увидите что-то подобное:

```
grub.pbkdf2.sha512.10000.9290F727ED06C38BA4549EF7DE25CF564265
9211B7FC076F2D27080136.887CFF169EA83D5235D8004742AA7D6187A41E3
187DF0CE14E256D85ED97A979080136.887CFF169EA8335235D8004242AA-
7D6187A41E3187DF0CE14E256D85ED97A97357AAA8FF0A3871AB9EEF-
F458392F462F495487387F685B7472FC6C29E293F0A0
```

Данный хеш нужно указать вместо пароля пользователя. Однако вместо инструкции *passwd* нужно использовать *password_pbkdf2*. Например:

```
password_pbkdf2 me
grub.pbkdf2.sha512.10000.9290F727ED06C38BA4549EF7DE25CF564265
9211B7FC076F2D27080136.887CFF169EA83D5235D8004742AA7D6187A41E3
187DF0CE14E256D85ED97A979080136.887CFF169EA8335235D8004242AA-
7D6187A41E3187DF0CE14E256D85ED97A97357AAA8FF0A3871AB9EEF-
F458392F462F495487387F685B7472FC6C29E293F0A0
```

После изменения файлов из каталога */etc/grub.d* не забудьте ввести команду *upgrade-grub* для обновления основного файла конфигурации.

7.2.5. Установка загрузчика

Команда установки загрузчика такая же, как и в случае с GRUB:



```
# /sbin/grub-install <устройство>
```

Например:

```
# /sbin/grub-install /dev/sda
```

Поскольку GRUB2 у вас уже установлен, вряд ли вам придется вводить эту команду. Исключение может составить разве что переустановка Windows, которая перезапишет загрузочный сектор своим загрузчиком. Поэтому вам придется загрузиться с LiveCD, выполнить *chroot* для вашей старой корневой системы и ввести команду *grub-install* для установки загрузчика GRUB2.

7.3. Система инициализации

После своей загрузки ядро передает управление системе инициализации. Цель этой системы — выполнить дальнейшую инициализацию системы. Самая главная задача системы инициализации — запуск и управление системными службами.

Служба (сервис, демон) — специальная программа, выполняющаяся в фоновом режиме и предоставляющая определенные услуги (или, как говорят, сервис — отсюда и второе название).

Что превращает обычный компьютер, скажем, в FTP-сервер? Правильно, запущенная служба FTP — тот же ProFTPD или подобная. Вы можете установить программу ProFTPD и настроить ее на автоматический запуск системой инициализации. Тогда при каждой загрузке наш компьютер будет превращаться в FTP-сервер.



Аналогично и с другими сервисами — достаточно установить определенную программу, чтобы превратить компьютер в веб-сервер или почтовый сервер. Но стоит вам отключить ее, и компьютер уже прекращает предоставлять обеспечиваемые программой услуги, следовательно, превращается в самый обычный компьютер.

В мире Linux существовало очень много разных систем инициализации — *init*, *upstart*, *init-ng*. Все их рассматривать уже нет смысла, поскольку в современных дистрибутивах используется современная система инициализации *systemd*.

systemd — подсистема инициализации и управления службами в Linux, фактически вытеснившая в 2010-е годы традиционную подсистему *init*. Основная особенность — интенсивное распараллеливание запуска служб в процессе загрузки системы, что позволяет существенно ускорить запуск операционной системы. Основная единица управления — модуль, одним из типов модулей являются «службы» — аналог демонов — наборы процессов, запускаемые и управляемые средствами подсистемы и изолируемые контрольными группами.

7.3.1. Принцип работы

Система инициализации ***systemd*** используется во многих современных дистрибутивах, в частности в Fedora, Ubuntu, CentOS и openSUSE. На данный момент это самая быстрая система инициализации. Для примера, в Astra Linux используется немного модифицированный ее вариант — ***systemdgenie***.

Давайте подумаем, как можно ускорить запуск Linux? Можно пойти по пути *upstart* — параллельно запускать службы. Но параллельный запуск — не всегда хорошо. Нужно учитывать зависимости служб. Например, сервис *d-bus* нужен многим другим сервисам. Пока сервис



d-bus не будет запущен, нельзя запускать сервисы, которые от него зависят.

Если сначала запускать основные сервисы и ждать, пока они будут запущены, а потом уже запускать службы, которые от них зависят, особого выигрыша в производительности по сравнению с *init* вы не увидите. Но если сервис *d-bus* (или любой другой, от которого зависят какие-то другие сервисы) запускается долго, то все остальные службы будут ждать его.

Как обойти это ограничение? При своем запуске службы проверяют, запущена ли необходимая им служба, по наличию файла сокета. Например, в случае с *d-bus* — это файл `/var/run/dbus/system_bus_socket`. Если мы создадим сокеты для всех служб, то мы можем запускать их параллельно, особо не беспокоясь, что произойдет сбой какой-то службы при запуске из-за отсутствия службы, от которой они зависят. Даже если несколько служб, которым нужен сервис *d-bus*, запустятся раньше, чем сам сервис *d-bus*, — ничего страшного. Каждая из этих служб отправит в сокет (главное, что он уже открыт!) сообщение, которое обработает сервис *d-bus* после того, как он запустится. Вот и все.

Но это не единственное «ухищрение», посредством которого осуществляется ускорение запуска компьютера, инициализацию которого производит **systemd**. Эта система инициализации запускает только необходимые сервисы. Остальные же будут запущены по мере необходимости. Концепция отложенного запуска используется и в других операционных системах — например, в MacOS X (там система инициализации называется **launchd**) и в Windows (концепция отложенного запуска служб). Так что решение не очень новое, но зато проверенное.

Основными функциями **systemd** являются:



- **Активация на основании сокетов** — система инициализации **systemd** прослушивает сокетов всех системных служб. Сокеты передаются системным службам сразу после запуска сервисов. Благодаря этому осуществляется параллельный запуск сервисов. Также это позволяет перезапускать сервисы без потери любых отправленных им сообщений, то есть пока сервис перезапускается, отправленные ему сообщения накапливаются, и он сможет их обработать после того, как будет запущен.
- **Активация на основании устройств** — **systemd** может запустить определенные службы, когда станет доступным определенный тип оборудования. Например, вы подключили Bluetooth-адаптер, может быть запущен сервис **bluetooth**.
- **Активация на основании d-bus** — служба инициализации может запустить сервисы, которые используют *d-bus* для межпроцессного взаимодействия, например, когда клиентское приложение попытается связаться с системной службой.
- **Активация на основании путей** — **systemd** может запустить службу, если изменится содержание каталога.
- **Управление точками монтирования и автоматическим монтированием** — система инициализации отслеживает и управляет точками монтирования и автоматического монтирования.
- **Снимки системных состояний** — благодаря этой возможности **systemd** может сохранить состояние всех модулей и восстановить предыдущее состояние системы.
- **Параллелизация** — **systemd** запускает системные службы параллельно благодаря активации на основании сокетов. Параллельная активация существенно сокращает время загрузки системы.
- **Обратная совместимость с SysV** — поддерживаются сценарии инициализации SysV, что упрощает переход на **systemd**. Однако все устанавливаемые в современных дистрибутивах пакеты служб уже адаптированы под **systemd**, поэтому не нуж-



но надеяться, что во время установки пакета какого-то сервиса будут установлены SysV-сценарии. Будут созданы файлы, необходимые для запуска сервиса посредством **systemd**.

7.3.2. Конфигурационные файлы *systemd*

Обилие различных конфигурационных файлов **systemd** может ввести в ступор даже бывалого линуксоида, не говоря уже о пользователе, который впервые видит **systemd**. Когда я впервые познакомился с **systemd**, у меня было только одно желание — снести ее и установить вместо нее **init**. Но мы это не будем делать. Чтобы разобраться со всеми файлами, нужно понимать, как работает эта система.

В **systemd** используется концепция модулей (юнитов). Существующие типы модулей описаны в таблице 7.1.

Таблица 7.1. Типы модулей системы инициализации *systemd*

Тип	Описание
<i>service</i>	Служба (сервис, демон), которую нужно запустить. Пример имени модуля: <i>network.service</i> . Изначально systemd поддерживала сценарии SysV (чтобы управлять сервисами можно было, как при использовании init), но в последнее время в каталоге <i>/etc/init.d</i> систем, которые используют systemd , практически нет (или вообще нет), а управление сервисами осуществляется только посредством systemd
<i>target</i>	Цель. Используется для группировки модулей других типов. В systemd нет уровней запуска, вместо них используются цели. Например, цель <i>multi-user.target</i> описывает, какие модули должны быть запущены в многопользовательском режиме



<i>snapshot</i>	Снимок. Используется для сохранения состояния systemd . Снимки могут использоваться для перевода системы из одного состояния в другое, например, в состояние сна и пробуждение
<i>mount</i>	Точка монтирования. Представляет точку монтирования. Система инициализации systemd контролирует все точки монтирования. При использовании systemd файл <code>/etc/fstab</code> уже не главный, хотя все еще может использоваться для определения точек монтирования
<i>automount</i>	Автоматическая точка монтирования. Используется для монтирования сменных носителей — флешек, внешних жестких дисков, оптических дисков и т.д.
<i>socket</i>	Сокет. Представляет сокет, находящийся в файловой системе или в Интернете. Поддерживаются сокеты <code>AF_INET</code> , <code>AF_INET6</code> , <code>AF_UNIX</code> . Реализация довольно интересная. Например, если сервису <code>service1.service</code> соответствует сокет <code>service1.socket</code> , то при попытке установки соединения с <code>service1</code> . <code>socket</code> будет запущен <code>service1.service</code>
<i>device</i>	Устройство. Представляет устройство в дереве устройств. Работает вместе с <i>udev</i> : если устройство описано в виде правила <i>udev</i> , то его можно представить в systemd в виде модуля device
<i>path</i>	Файл или каталог, созданный где-то в файловой системе



<i>scope</i>	Процесс, который создан извне
<i>slice</i>	Управляет системными процессами. Представляет собой группу иерархически организованных модулей
<i>swap</i>	Представляет область подкачки (раздел подкачки) или файл подкачки (свопа)
<i>timer</i>	Представляет собой таймер системы инициализации systemd

Модули хранятся в следующих каталогах:

- **/etc/systemd/system/** — обладает самым высоким приоритетом. Здесь содержатся модули, которые созданы и управляются системным администратором.
- **/run/systemd/system/** — модули, созданные во время выполнения. Приоритет этого каталога ниже, чем каталога **/etc/systemd/system/**, но выше, чем у **/usr/lib/systemd/system**.
- **/usr/lib/systemd/system/** — модули, которые установлены из пакетов.

Типичный файл модуля типа **service** приведен в листинге 7.3.

Листинг 7.3. Типичный файл модуля типа *service*

```
[Unit]
Description=Daemon to detect crashing apps
After=syslog.target
```



```
[Service]
ExecStart=/usr/sbin/abrttd
Type=forking

[Install]
WantedBy=multi-user.target
```

В секции Unit содержится общая информация о сервисе. Эта секция есть и в других модулях, а не только в сервисах.

Секция `Service` содержит информацию о сервисе. Параметр `ExecStart` описывает команду, которую нужно запустить. Параметр `Type` указывает, как сервис будет уведомлять **systemd** об окончании запуска.

Секция `Install` содержит информацию о цели, в которой должен запускаться сервис. В нашем видно, что сервис будет запущен при активации цели *multi-user.target*.

Вы можете использовать эту «болванку» для написания собственного сервиса, который потом нужно поместить в файл `/etc/systemd/system/имя_сервиса.service`. После этого нужно перезапустить саму **systemd**, чтобы она узнала о новом сервисе:

```
# systemctl daemon-reload
```

7.3.3. Цели

Теперь поговорим о целях. Файлы целей **.target* предназначены для группировки вместе других юнитов **systemd** через цепочку зависимостей. Так, модуль цели *graphical.target*, который используется для запуска графического сеанса, запускает системные службы GDM (файл *gdm.service*) и Accounts Service (*accounts-daemon.service*), а также



активирует цель *multi-user.target*. В свою очередь, цель *multi-user.target* запускает другие системные службы, например, D-Bus (*dbus.service*) и активирует другие цели вроде *basic.target*.

В **systemd** имеются предопределенные цели, которые напоминают стандартный набор уровней запуска.

Некоторые цели называются *runlevelN.target*, чтобы упростить переход бывших пользователей **init** на **systemd**, а именно:

- *poweroff.target* (*runlevel0.target*) — завершение работы и отключение системы;
- *rescue.target* (*runlevel1.target*) — однопользовательский режим, среда восстановления;
- *multi-user.target* (*runlevel2.target*, *runlevel3.target*, *runlevel4.target*) — многопользовательский режим, без графического интерфейса;
- *graphical.target* (*runlevel5.target*) — многопользовательский режим с графическим интерфейсом;
- *reboot.target* (*runlevel6.target*) — завершение работы и перезагрузка системы.

Управление службами осуществляется с помощью программы **systemctl**. Подробнее о службах мы поговорим чуть позже, а пока разберемся, как использовать **systemctl** для завершения работы системы:

- *systemctl halt* — останавливает систему;
- *systemctl poweroff* — выключает систему;
- *systemctl reboot* — перезагружает систему.



Многим пользователям будет удобнее использовать старые команды *halt*, *poweroff* и *reboot*. Но все же теперь вы знаете, что есть альтернативные способы завершения работы.

7.4. Управление сервисами при использовании *systemd*

При использовании системы инициализации **systemd** управление службами осуществляется посредством программы **systemctl**. Команда **systemctl** используется для разных целей, поэтому в таблице 7.2 представлены не все ее параметры, а только те, которые имеют отношение к сервисам.

Таблица 7.2. Параметры программы *systemctl*

Параметр	Описание
<code>start <имя.service></code>	Запускает сервис
<code>stop <имя.service></code>	Останавливает сервис
<code>restart <имя.service></code>	Перезапускает сервис
<code>try-restart <имя.service></code>	Перезапуск сервиса только, если он запущен
<code>reload <имя.service></code>	Перезагружает конфигурацию сервиса
<code>status <имя.service></code>	Отображает подробное состояние сервиса



<code>is-active <имя.service></code>	Отображает только строку <i>active</i> (сервис запущен) или <i>inactive</i> (остановлен)
<code>list-units --type service --all</code>	Выводит состояние всех сервисов
<code>enable <имя.service></code>	Включает сервис (обеспечивает его автоматический запуск)
<code>disable <имя.service></code>	Отключает сервис (сервис не будет автоматически запускаться при запуске системы)
<code>reenable <имя.service></code>	Деактивирует сервис и сразу его использует
<code>list-unit-files --type service</code>	Выводит список всех сервисов и сообщает, какие из них активированы, а какие — нет

Примеры:

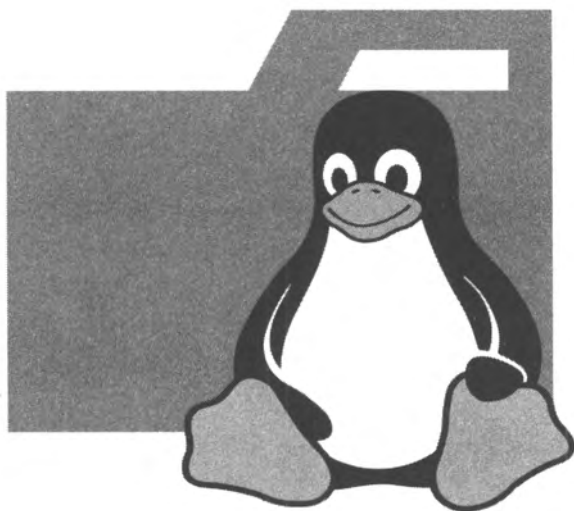
```
# systemctl start httpd.service  
# systemctl stop httpd
```

Первая команда запускает сервис `httpd` (веб-сервер), вторая — останавливает. Обратите внимание, что «`.service`» можно не указывать.

Бывалые пользователи Linux сразу заметят удобства. Ранее, чтобы отключить службу на определенном уровне запуска, нужно было удалить ее символическую ссылку из определенного каталога. Аналогично, чтобы служба запускалась на определенном уровне запуска (например, в графическом режиме), нужно было создать символическую ссылку. Сейчас всего этого нет, а есть только команды *enable* и *disable*, что гораздо удобнее.

ГЛАВА 8.

УПРАВЛЕНИЕ ПРОЦЕССАМИ





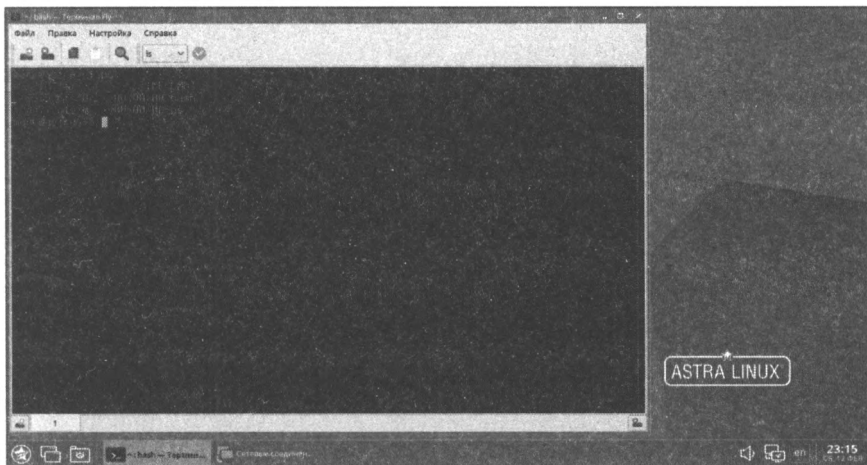
8.1. Команды *ps*, *nice* и *kill*

8.1.1. Получение информации о процессе

Современные операционные системы устроены так, что каждому процессу присваивается уникальный номер — PID (Process ID, ИД процесса), используя который можно управлять процессом, например, можно завершить процесс или изменить его приоритет.

Узнать PID можно с помощью команды `ps`.

Команда `ps`, введенная без параметров, просто показывает список процессов, запущенных на текущем терминале. Видно, что сейчас запущен `bash` и сама команда `ps` (правда, на момент завершения вывода процесс с ID 3535 уже не будет существовать, но на момент самого вывода такой процесс существовал), см. рис. 8.1.



*Рис. 8.1. Команда **ps***

Параметр *-a* позволяет вывести список всех процессов пользователя. Посмотрите на рис. 8.2 — на консоли *tty3* запущена программа *nano*, на консоли *tty1* — программа *ps*.

```
root@localhost ~# ps -a
  PID TTY          TIME CMD
 1835 tty3        00:00:00 nano
 1841 tty1        00:00:00 ps
root@localhost ~#
```

*Рис. 8.2. Команда **ps -a***

Если нужно вывести процессы какого-то определенного пользователя, тогда используйте параметр *-u*:

```
$ ps -u root
```



Вывести абсолютно все процессы можно с помощью опции `-A`. Обратите внимание на регистр опции! Поскольку в системе процессов будет очень много, лучше перенаправить вывод программы на команду `less` для более удобного просмотра (рис. 8.3):

```
$ ps -A | less
```

```

PID TTY          TIME CMD
1 ?        00:00:00 systemd
2 ?        00:00:00 kthreadd
3 ?        00:00:00 ksoftirqd/0
4 ?        00:00:12 kworker/0:0
5 ?        00:00:00 kworker/0:0
6 ?        00:00:27 rcu_sched
7 ?        00:00:00 rcu_bh
8 ?        00:00:12 rcuop/0
10 ?       00:00:00 rcuob/0
11 ?       00:00:00 watchdog/0
12 ?       00:00:00 ksoftirqd/0
13 ?       00:00:00 syrtask/1
14 ?       00:00:00 watchdog/0
15 ?       00:00:00 kworker/0:0
16 ?       00:00:00 kworker/0:0
17 ?       00:00:00 kworker/0:0
18 ?       00:00:00 kworker/0:0
19 ?       00:00:00 kworker/0:0
20 ?       00:00:00 kworker/0:0
21 ?       00:00:00 kworker/0:0
22 ?       00:00:00 kworker/0:0
23 ?       00:00:00 kworker/0:0
24 ?       00:00:00 kworker/0:0
25 ?       00:00:00 kworker/0:0
26 ?       00:00:00 kworker/0:0
27 ?       00:00:00 kworker/0:0
28 ?       00:00:00 kworker/0:0
29 ?       00:00:00 kworker/0:0
30 ?       00:00:00 kworker/0:0
31 ?       00:00:00 kworker/0:0
32 ?       00:00:00 kworker/0:0
33 ?       00:00:00 kworker/0:0
34 ?       00:00:00 kworker/0:0
35 ?       00:00:00 kworker/0:0
36 ?       00:00:00 kworker/0:0
37 ?       00:00:00 kworker/0:0
38 ?       00:00:00 kworker/0:0
39 ?       00:00:00 kworker/0:0
40 ?       00:00:00 kworker/0:0
41 ?       00:00:00 kworker/0:0
42 ?       00:00:00 kworker/0:0
43 ?       00:00:00 kworker/0:0
44 ?       00:00:00 kworker/0:0
45 ?       00:00:00 kworker/0:0
46 ?       00:00:00 kworker/0:0
47 ?       00:00:00 kworker/0:0
48 ?       00:00:00 kworker/0:0
49 ?       00:00:00 kworker/0:0
50 ?       00:00:00 kworker/0:0
51 ?       00:00:00 kworker/0:0
52 ?       00:00:00 kworker/0:0
53 ?       00:00:00 kworker/0:0
54 ?       00:00:00 kworker/0:0
55 ?       00:00:00 kworker/0:0
56 ?       00:00:00 kworker/0:0
57 ?       00:00:00 kworker/0:0
58 ?       00:00:00 kworker/0:0
59 ?       00:00:00 kworker/0:0
60 ?       00:00:00 kworker/0:0
61 ?       00:00:00 kworker/0:0
62 ?       00:00:00 kworker/0:0
63 ?       00:00:00 kworker/0:0
64 ?       00:00:00 kworker/0:0
65 ?       00:00:00 kworker/0:0
66 ?       00:00:00 kworker/0:0
67 ?       00:00:00 kworker/0:0
68 ?       00:00:00 kworker/0:0
69 ?       00:00:00 kworker/0:0
70 ?       00:00:00 kworker/0:0
71 ?       00:00:00 kworker/0:0
72 ?       00:00:00 kworker/0:0
73 ?       00:00:00 kworker/0:0
74 ?       00:00:00 kworker/0:0
75 ?       00:00:00 kworker/0:0
76 ?       00:00:00 kworker/0:0
77 ?       00:00:00 kworker/0:0
78 ?       00:00:00 kworker/0:0
79 ?       00:00:00 kworker/0:0
80 ?       00:00:00 kworker/0:0
81 ?       00:00:00 kworker/0:0
82 ?       00:00:00 kworker/0:0
83 ?       00:00:00 kworker/0:0
84 ?       00:00:00 kworker/0:0
85 ?       00:00:00 kworker/0:0
86 ?       00:00:00 kworker/0:0
87 ?       00:00:00 kworker/0:0
88 ?       00:00:00 kworker/0:0
89 ?       00:00:00 kworker/0:0
90 ?       00:00:00 kworker/0:0
91 ?       00:00:00 kworker/0:0
92 ?       00:00:00 kworker/0:0
93 ?       00:00:00 kworker/0:0
94 ?       00:00:00 kworker/0:0
95 ?       00:00:00 kworker/0:0
96 ?       00:00:00 kworker/0:0
97 ?       00:00:00 kworker/0:0
98 ?       00:00:00 kworker/0:0
99 ?       00:00:00 kworker/0:0
100 ?      00:00:00 kworker/0:0

```

Рис. 8.3а. Команда `ps -A | less` (начало вывода)

```

315 ?      00:00:00 kworker/0:0
316 ?      00:00:00 kworker/0:0
317 ?      00:00:00 kworker/0:0
318 ?      00:00:00 kworker/0:0
319 ?      00:00:00 kworker/0:0
320 ?      00:00:00 kworker/0:0
321 ?      00:00:00 kworker/0:0
322 ?      00:00:00 kworker/0:0
323 ?      00:00:00 kworker/0:0
324 ?      00:00:00 kworker/0:0
325 ?      00:00:00 kworker/0:0
326 ?      00:00:00 kworker/0:0
327 ?      00:00:00 kworker/0:0
328 ?      00:00:00 kworker/0:0
329 ?      00:00:00 kworker/0:0
330 ?      00:00:00 kworker/0:0
331 ?      00:00:00 kworker/0:0
332 ?      00:00:00 kworker/0:0
333 ?      00:00:00 kworker/0:0
334 ?      00:00:00 kworker/0:0
335 ?      00:00:00 kworker/0:0
336 ?      00:00:00 kworker/0:0
337 ?      00:00:00 kworker/0:0
338 ?      00:00:00 kworker/0:0
339 ?      00:00:00 kworker/0:0
340 ?      00:00:00 kworker/0:0
341 ?      00:00:00 kworker/0:0
342 ?      00:00:00 kworker/0:0
343 ?      00:00:00 kworker/0:0
344 ?      00:00:00 kworker/0:0
345 ?      00:00:00 kworker/0:0
346 ?      00:00:00 kworker/0:0
347 ?      00:00:00 kworker/0:0
348 ?      00:00:00 kworker/0:0
349 ?      00:00:00 kworker/0:0
350 ?      00:00:00 kworker/0:0
351 ?      00:00:00 kworker/0:0
352 ?      00:00:00 kworker/0:0
353 ?      00:00:00 kworker/0:0
354 ?      00:00:00 kworker/0:0
355 ?      00:00:00 kworker/0:0
356 ?      00:00:00 kworker/0:0
357 ?      00:00:00 kworker/0:0
358 ?      00:00:00 kworker/0:0
359 ?      00:00:00 kworker/0:0
360 ?      00:00:00 kworker/0:0
361 ?      00:00:00 kworker/0:0
362 ?      00:00:00 kworker/0:0
363 ?      00:00:00 kworker/0:0
364 ?      00:00:00 kworker/0:0
365 ?      00:00:00 kworker/0:0
366 ?      00:00:00 kworker/0:0
367 ?      00:00:00 kworker/0:0
368 ?      00:00:00 kworker/0:0
369 ?      00:00:00 kworker/0:0
370 ?      00:00:00 kworker/0:0
371 ?      00:00:00 kworker/0:0
372 ?      00:00:00 kworker/0:0
373 ?      00:00:00 kworker/0:0
374 ?      00:00:00 kworker/0:0
375 ?      00:00:00 kworker/0:0
376 ?      00:00:00 kworker/0:0
377 ?      00:00:00 kworker/0:0
378 ?      00:00:00 kworker/0:0
379 ?      00:00:00 kworker/0:0
380 ?      00:00:00 kworker/0:0
381 ?      00:00:00 kworker/0:0
382 ?      00:00:00 kworker/0:0
383 ?      00:00:00 kworker/0:0
384 ?      00:00:00 kworker/0:0
385 ?      00:00:00 kworker/0:0
386 ?      00:00:00 kworker/0:0
387 ?      00:00:00 kworker/0:0
388 ?      00:00:00 kworker/0:0
389 ?      00:00:00 kworker/0:0
390 ?      00:00:00 kworker/0:0
391 ?      00:00:00 kworker/0:0
392 ?      00:00:00 kworker/0:0
393 ?      00:00:00 kworker/0:0
394 ?      00:00:00 kworker/0:0
395 ?      00:00:00 kworker/0:0
396 ?      00:00:00 kworker/0:0
397 ?      00:00:00 kworker/0:0
398 ?      00:00:00 kworker/0:0
399 ?      00:00:00 kworker/0:0
400 ?      00:00:00 kworker/0:0

```

Рис. 8.3б. Команда `ps -A | less` (завершение вывода)

ПРИМЕЧАНИЕ. Для выхода из программы `less` нажмите `q` на клавиатуре. Листать вывод можно стрелками вверх и вниз.

Команда `ps` сортирует процессы по `PID`. Колонка `TTY` — это терминал, к которому привязан процесс. Если в этой колонке вы видите знак `?`, значит, процесс не привязан ни к одному из терминалов. Как правило, это системные процессы-службы. Они запускаются без привязки к терминалу. Чтобы отобразить только процессы без привязки к терминалу, используется опция `-x` (рис. 8.4).

Line	Time	Address
179	0.00	usr/lib/systemd/systemd --switched-root --system --deserialize=25
1	0.00	(kshenv)
2	0.00	(kshenv)
3	0.00	(kshenv)
4	0.23	(bashrc:0.00)
5	0.00	(bashrc:0.00)
7	0.50	(rc:0.00)
6	0.00	(rc:0.00)
5	0.51	(rc:0.00)
10	0.00	(rc:0.00)
11	0.00	(initramfs)
12	0.00	(initramfs)
13	0.00	(initramfs)
14	0.00	(initramfs)
15	0.00	(initramfs)
16	0.01	(initramfs)
19	0.00	(initramfs)
20	0.00	(initramfs)
21	0.00	(initramfs)
22	0.00	(initramfs)
23	0.00	(initramfs)
24	0.00	(initramfs)
25	0.00	(initramfs)
26	0.00	(initramfs)
27	0.00	(initramfs)
28	0.00	(initramfs)
29	0.00	(initramfs)
30	0.00	(initramfs)
31	0.00	(initramfs)
32	0.00	(initramfs)
33	0.00	(initramfs)
34	0.00	(initramfs)
35	0.00	(initramfs)
36	0.00	(initramfs)
37	0.00	(initramfs)
38	0.00	(initramfs)
39	0.00	(initramfs)
40	0.00	(initramfs)
41	0.00	(initramfs)
42	0.00	(initramfs)
43	0.00	(initramfs)
44	0.00	(initramfs)
45	0.00	(initramfs)
46	0.00	(initramfs)
47	0.00	(initramfs)
48	0.00	(initramfs)
49	0.00	(initramfs)
50	0.00	(initramfs)
51	0.00	(initramfs)
52	0.00	(initramfs)
53	0.00	(initramfs)
54	0.00	(initramfs)
55	0.00	(initramfs)
56	0.00	(initramfs)
57	0.00	(initramfs)
58	0.00	(initramfs)
59	0.00	(initramfs)
60	0.00	(initramfs)
61	0.00	(initramfs)
62	0.00	(initramfs)
63	0.00	(initramfs)
64	0.00	(initramfs)
65	0.00	(initramfs)
66	0.00	(initramfs)
67	0.00	(initramfs)
68	0.00	(initramfs)
69	0.00	(initramfs)
70	0.00	(initramfs)
71	0.00	(initramfs)
72	0.00	(initramfs)
73	0.00	(initramfs)
74	0.00	(initramfs)
75	0.00	(initramfs)
76	0.00	(initramfs)
77	0.00	(initramfs)
78	0.00	(initramfs)
79	0.00	(initramfs)
80	0.00	(initramfs)
81	0.00	(initramfs)
82	0.00	(initramfs)
83	0.00	(initramfs)
84	0.00	(initramfs)
85	0.00	(initramfs)
86	0.00	(initramfs)
87	0.00	(initramfs)
88	0.00	(initramfs)
89	0.00	(initramfs)
90	0.00	(initramfs)
91	0.00	(initramfs)
92	0.00	(initramfs)
93	0.00	(initramfs)
94	0.00	(initramfs)
95	0.00	(initramfs)
96	0.00	(initramfs)
97	0.00	(initramfs)
98	0.00	(initramfs)
99	0.00	(initramfs)
100	0.00	(initramfs)

Рис. 8.4. Команда `ps -x | less`

Колонка STAT — это состояние, в котором находится процесс. Возможные значения для этой колонки приведены в таблице 8.1. Обратите внимание: колонка STAT есть только тогда, когда программа запущена с параметром -x. Если программа запущена с другими параметрами, например -A, вместо нее будет колонка TIME, которая сообщает занимаемое процессом процессорное время.



Таблица 8.1. Возможные состояния процесса

Состояние	Описание
D	Процесс в непрерывном сне (как правило, ожидает ввода/вывода)
R	Выполняется в данный момент
S	Ожидание (то есть процесс «спит» менее 20 секунд, после чего он переходит или в состояние R или в состояние D)
T	Процесс остановлен
t	То же, что и T, но причина остановки — останов отладчиком
W	Процесс в свопинге (подкачке)
X	Процесс мертв, вы его никогда не увидите
Z	Процесс-зомби — он уже завершен, но не «похоронен» его родителем, то есть процесс-родитель еще не считал код завершения

У команды `ps` есть несколько синтаксисов установки параметров. Мы использовали BSD-синтаксис. Например, для вывода всех процессов в стандартном синтаксисе используется команда `-e`, а в нашем случае — `-A`. Вы вольны использовать любой синтаксис, но в случае с BSD-синтаксисом программа `ps` выводит дополнительное состояние процесса (работает подобно программе `ps` в системе BSD). Дополнительное состояние процесса описано в таблице 8.2.

**Таблица 8.2. Дополнительное состояние процесса (BSD-синтаксис)**

Состояние	Описание
<	Высокий приоритет
N	Низкий приоритет
L	У процесса есть страницы, заблокированные в памяти
s	Это лидер сессии
l	Процесс является многопоточковым
+	Находится на первом плане в группе процессов

Последняя колонка вывода `ps` — это `CMD`. Она содержит команду, которой был запущен процесс. Не просто название исполнимого файла, но путь (если он был указан в команде) и переданные программе параметры.

Если программа была запущена без указания полного пути к исполняемому файлу и вы хотите знать, где он находится, введите команду `which`, например:

```
$ which nano
/bin/nano
```

Если вам нужно узнать PID определенного процесса, но вам не хочется просматривать длинный список системных процессов, используйте команду `grep` как фильтр.

Например, следующая команда позволит нам узнать PID процесса `sshd` (это SSH-сервер):



```
# ps -A | grep sshd
```

Если такой процесс не запущен, вывод будет пуст. Или же вы получите вывод вроде этого:

```
929 ?    00:00:00 sshd
```

8.1.2. Изменение приоритета процесса

Когда мы знаем PID процесса, мы можем изменить его приоритет. В некоторых случаях полезно изменить приоритет процесса. Например, можно повысить приоритет процесса, выполняющего резервное копирование, чтобы программа успела за ночь создать все необходимые резервные копии и чтобы этот процесс утром не мешал нормальной работе сервера.

Запустить программу с определенным приоритетом можно командой `nice`:



```
# nice -n <приоритет> команда аргументы
```

Здесь приоритет задается от -20 (максимальный приоритет) до 19 (минимальный). Если процесс уже был запущен и вы не можете его прерывать, но повысить приоритет нужно, используйте команду *renice*:

```
# renice -n <приоритет> -p PID
```

8.1.3. Аварийное завершение процесса

Если процесс завис и его нельзя завершить, как обычно, тогда для его аварийного завершения используется команда `kill`



Формат вызова этой команды следующий:

```
$ kill [опции] PID
```

Конечно, перед этим нужно узнать PID процесса. На рис. 8.5 изображена команда `kill` в действии: сначала я вывел список процессов, чтобы узнать PID процесса `nano` (1035), затем я ввел команду `kill 1035`, чтобы «убить» этот процесс. Наконец, я вывел список процессов еще раз, чтобы убедиться, что процесс `nano` завершен.

```
[root@localhost ~]# ps -a
  PID TTY          TIME CMD
 1035 tty3        00:00:00 nano
 1125 tty1        00:00:00 ps
[root@localhost ~]# kill 1035
[root@localhost ~]# _
```

Рис. 8.5. Использование команды `kill`

Используя параметры программы, можно по-разному завершить процесс. Самый эффективный сигнал 9 (KILL) — означает аварийное завершение процесса. Программа не может игнорировать или как-либо обработать этот процесс.

Если нужно попытаться корректно завершить работу программы, ей отправляют сигнал 15 (TERM), означающий, что программа должна освободить все занятые ресурсы, сохранить все данные. Вот только если программа зависла и не отвечает на запросы пользователя, этот сигнал мало чем поможет, но попытаться стоит.



Сигнал 19 (STOP) позволяет временно приостановить работу программы, а сигнал 18 (CONT) — возобновить приостановленный ранее процесс.

Для сетевых служб полезен сигнал 1 (HUP), означающий, что процесс должен перезапуститься и перечитать файл конфигурации. Полезно, когда вы изменили файл конфигурации и хотите, чтобы демон был перезапущен (хотя для этого правильнее использовать команду *service*). Обычная программа при получении сигнала *1* завершает работу.

Пример отправки сигнала:

```
$ kill -9 1035
```

Если вам лень получать PID процесса, можно завершить его и по имени, используя команду *killall*, например:

```
$ killall nano
```

Вот только если в вашей системе есть два процесса с именем *nano*, например, один на консоли *tty2*, а другой — на *tty4*, то будут завершены оба процесса. Если это то, что вам нужно, используйте *killall*, в противном случае лучше использовать команду *kill* для завершения именно того процесса, который можно завершить.

Еще есть команда *xkill*, позволяющая «убить» программу, имеющую графический интерфейс. Такие программы можно завершить и командой *kill*, но программа *xkill* предоставляет графический метод завершения. После ввода этой команды указатель мыши примет вид черепа. Для завершения программы нужно щелкнуть по ее окну.



8.2. Команда *top*

Как было отмечено ранее, программа *ps* по умолчанию сортирует процессы по колонке PID, а не по колонке TIME. Конечно, можно использовать различные параметры программы, чтобы добиться нужного нам вывода, но все равно программа не будет показывать ситуацию в реальном времени.

Если же вам нужно знать, что происходит с вашими процессами в реальном времени, вам нужно использовать программу *top*

Пример можно увидеть на рис. 8.6.

PID	USER	PR	NI	VIRT	RES	SHR	S	CPU	MEM	COMMAND
1	root	0	0	0	0	0	S	0.0	0.0	init
2	root	0	0	0	0	0	S	0.0	0.0	smm
3	root	0	0	0	0	0	S	0.0	0.0	smm
4	root	0	0	0	0	0	S	0.0	0.0	smm
5	root	0	0	0	0	0	S	0.0	0.0	smm
6	root	0	0	0	0	0	S	0.0	0.0	smm
7	root	0	0	0	0	0	S	0.0	0.0	smm
8	root	0	0	0	0	0	S	0.0	0.0	smm
9	root	0	0	0	0	0	S	0.0	0.0	smm
10	root	0	0	0	0	0	S	0.0	0.0	smm
11	root	0	0	0	0	0	S	0.0	0.0	smm
12	root	0	0	0	0	0	S	0.0	0.0	smm
13	root	0	0	0	0	0	S	0.0	0.0	smm
14	root	0	0	0	0	0	S	0.0	0.0	smm
15	root	0	0	0	0	0	S	0.0	0.0	smm
16	root	0	0	0	0	0	S	0.0	0.0	smm
17	root	0	0	0	0	0	S	0.0	0.0	smm
18	root	0	0	0	0	0	S	0.0	0.0	smm
19	root	0	0	0	0	0	S	0.0	0.0	smm
20	root	0	0	0	0	0	S	0.0	0.0	smm
21	root	0	0	0	0	0	S	0.0	0.0	smm
22	root	0	0	0	0	0	S	0.0	0.0	smm
23	root	0	0	0	0	0	S	0.0	0.0	smm
24	root	0	0	0	0	0	S	0.0	0.0	smm
25	root	0	0	0	0	0	S	0.0	0.0	smm
26	root	0	0	0	0	0	S	0.0	0.0	smm
27	root	0	0	0	0	0	S	0.0	0.0	smm
28	root	0	0	0	0	0	S	0.0	0.0	smm
29	root	0	0	0	0	0	S	0.0	0.0	smm
30	root	0	0	0	0	0	S	0.0	0.0	smm
31	root	0	0	0	0	0	S	0.0	0.0	smm
32	root	0	0	0	0	0	S	0.0	0.0	smm
33	root	0	0	0	0	0	S	0.0	0.0	smm
34	root	0	0	0	0	0	S	0.0	0.0	smm
35	root	0	0	0	0	0	S	0.0	0.0	smm
36	root	0	0	0	0	0	S	0.0	0.0	smm
37	root	0	0	0	0	0	S	0.0	0.0	smm
38	root	0	0	0	0	0	S	0.0	0.0	smm
39	root	0	0	0	0	0	S	0.0	0.0	smm
40	root	0	0	0	0	0	S	0.0	0.0	smm
41	root	0	0	0	0	0	S	0.0	0.0	smm
42	root	0	0	0	0	0	S	0.0	0.0	smm
43	root	0	0	0	0	0	S	0.0	0.0	smm
44	root	0	0	0	0	0	S	0.0	0.0	smm
45	root	0	0	0	0	0	S	0.0	0.0	smm
46	root	0	0	0	0	0	S	0.0	0.0	smm
47	root	0	0	0	0	0	S	0.0	0.0	smm
48	root	0	0	0	0	0	S	0.0	0.0	smm
49	root	0	0	0	0	0	S	0.0	0.0	smm
50	root	0	0	0	0	0	S	0.0	0.0	smm
51	root	0	0	0	0	0	S	0.0	0.0	smm
52	root	0	0	0	0	0	S	0.0	0.0	smm
53	root	0	0	0	0	0	S	0.0	0.0	smm
54	root	0	0	0	0	0	S	0.0	0.0	smm
55	root	0	0	0	0	0	S	0.0	0.0	smm
56	root	0	0	0	0	0	S	0.0	0.0	smm
57	root	0	0	0	0	0	S	0.0	0.0	smm
58	root	0	0	0	0	0	S	0.0	0.0	smm
59	root	0	0	0	0	0	S	0.0	0.0	smm
60	root	0	0	0	0	0	S	0.0	0.0	smm
61	root	0	0	0	0	0	S	0.0	0.0	smm
62	root	0	0	0	0	0	S	0.0	0.0	smm
63	root	0	0	0	0	0	S	0.0	0.0	smm
64	root	0	0	0	0	0	S	0.0	0.0	smm
65	root	0	0	0	0	0	S	0.0	0.0	smm
66	root	0	0	0	0	0	S	0.0	0.0	smm
67	root	0	0	0	0	0	S	0.0	0.0	smm
68	root	0	0	0	0	0	S	0.0	0.0	smm
69	root	0	0	0	0	0	S	0.0	0.0	smm
70	root	0	0	0	0	0	S	0.0	0.0	smm
71	root	0	0	0	0	0	S	0.0	0.0	smm
72	root	0	0	0	0	0	S	0.0	0.0	smm
73	root	0	0	0	0	0	S	0.0	0.0	smm
74	root	0	0	0	0	0	S	0.0	0.0	smm
75	root	0	0	0	0	0	S	0.0	0.0	smm
76	root	0	0	0	0	0	S	0.0	0.0	smm
77	root	0	0	0	0	0	S	0.0	0.0	smm
78	root	0	0	0	0	0	S	0.0	0.0	smm
79	root	0	0	0	0	0	S	0.0	0.0	smm
80	root	0	0	0	0	0	S	0.0	0.0	smm
81	root	0	0	0	0	0	S	0.0	0.0	smm
82	root	0	0	0	0	0	S	0.0	0.0	smm
83	root	0	0	0	0	0	S	0.0	0.0	smm
84	root	0	0	0	0	0	S	0.0	0.0	smm
85	root	0	0	0	0	0	S	0.0	0.0	smm
86	root	0	0	0	0	0	S	0.0	0.0	smm
87	root	0	0	0	0	0	S	0.0	0.0	smm
88	root	0	0	0	0	0	S	0.0	0.0	smm
89	root	0	0	0	0	0	S	0.0	0.0	smm
90	root	0	0	0	0	0	S	0.0	0.0	smm
91	root	0	0	0	0	0	S	0.0	0.0	smm
92	root	0	0	0	0	0	S	0.0	0.0	smm
93	root	0	0	0	0	0	S	0.0	0.0	smm
94	root	0	0	0	0	0	S	0.0	0.0	smm
95	root	0	0	0	0	0	S	0.0	0.0	smm
96	root	0	0	0	0	0	S	0.0	0.0	smm
97	root	0	0	0	0	0	S	0.0	0.0	smm
98	root	0	0	0	0	0	S	0.0	0.0	smm
99	root	0	0	0	0	0	S	0.0	0.0	smm
100	root	0	0	0	0	0	S	0.0	0.0	smm

Рис. 8.6. Команда *top*

Назначение колонок программы описано в таблице 8.3.

Таблица 8.3. Колонки программы *top*

Колонка	Описание
PID	PID процесса
USER	Владелец процесса (пользователь, запустивший программу)
PR	Приоритет процесса
NI	Значение <i>nice</i> (см. ранее)
VIRT	Виртуальная память, которая используется процессом
RES	Размер процесса, который не перемещается в область подкачки
SHR	Разделяемая память, используемая процессом
S	Состояние процесса (см. табл. 8.1)
%CPU	Процессорное время, занимаемое процессом в данный момент
%MEM	Память, используемая процессом
TIME+	Процессорное время, которое было потрачено с момента запуска процесса
COMMAND	Команда запуска процесса

При просмотре списка программы *top* вы можете управлять сортировкой процессов с помощью нажатия клавиши **F**, которая изменя-





гда полезно знать, как просмотреть информацию об использовании памяти (команда `free`) и дискового пространства (команда `df`), см. рис. 8.8.

```

root@localhost ~# free
              total        used        free      shared  buff/cache   available
Mem:          2039768      132908      1582496         816       324372      1720744
Swap:          2096124           0       2096124

root@localhost ~# df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        987M   0 987M   0% /dev
tmpfs            996M   0 996M   0% /dev/shm
tmpfs            996M  888K 996M   1% /run
tmpfs            996M   0 996M   0% /sys/fs/cgroup
/dev/sda1        18G  1.3G  17G   7% /
tmpfs            996M  8.8K 996M   1% /tmp
tmpfs            200M   0 200M   0% /run/user/0
tmpfs            200M   0 200M   0% /run/user/1000
root@localhost ~#

```

Рис. 8.8. Команды `free` и `df -h`

Программа `free` выводит информацию об использовании памяти (`Mem`) и подкачки (`Swap`). Колонка `total` — это общее количество памяти в килобайтах, `used` — использованное количество памяти (тоже в килобайтах), `free` — свободно памяти, `shared` — разделяемая память, `buff/cache` — размер кэша, `available` — общий объем доступной памяти.

Параметр `-h` команды `df` означает вывод информации об объеме в удобных для восприятия человеком единицах, то есть в мегабайтах и гигабайтах.

Обратите внимание на значение `buff/cache` в выводе команды `free`. Оно показывает, сколько памяти задействовано под буфер ввода/вывода и кэш. В нашем случае (рис. 8.8) — примерно 323 Мб. На реальном сервере это значение будет гораздо выше. Немного освободить память можно, очистив кэш. Для этого введите команду:



```
sync; echo 3 > /proc/sys/vm/drop_caches
```

Сначала мы командой `sync` сбрасываем содержимое буферов на диск, а затем уничтожаем кэш. Если посмотреть затем информацию об использовании памяти, то вы увидите, что размер кэша был уменьшен почти в три раза (рис. 8.9). Однако помните, что эта команда может негативно отразиться на стабильности системы и на скорости ее работы. Не всегда очистка кэша таким вот варварским образом — это хорошо.

```
root@localhost ~# free
              total        used        free      shared  buff/cache   available
Mem:          2839768      132988      1582496         816       324372      1728744
Swap:          2896124           0       2896124

root@localhost ~# df -h
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs         987M   0  987M   0% /dev
tmpfs            996M   0  996M   0% /dev/shm
tmpfs            996M 800K  996M   1% /run
tmpfs            996M   0  996M   0% /sys/fs/cgroup
/dev/sda1        18G  1.3G   17G   7% /
tmpfs            996M 8.0K  996M   1% /tmp
tmpfs            200M   0  200M   0% /run/user/0
tmpfs            200M   0  200M   0% /run/user/1000
root@localhost ~# sync; echo 3 > /proc/sys/vm/drop_caches
root@localhost ~# free
              total        used        free      shared  buff/cache   available
Mem:          2839768      117852      1888844         816       113872      1771468
Swap:          2896124           0       2896124

root@localhost ~#
```

Рис. 8.9. До и после ввода команды `sync; echo 3 > /proc/sys/vm/drop_caches`

8.4. Команда *fuser*

Команда `fuser` позволяет узнать, какой процесс открыл тот или иной ресурс, например, файл или сетевой порт.



Примеры использования программы:

```
fuser -va 23/tcp  
fuser -va /chroot/etc/resolv.conf
```

В первом случае мы получим идентификатор процесса, открывшего TCP-порт 23, во втором — идентификатор процесса, открывшего файл /chroot/etc/resolv.conf. Что делать далее — решать вам, например, можно «убить» этот процесс командой `kill`.

8.5. Планировщики заданий

8.5.1. Планировщик *cron*

Планировщик заданий нужен для периодического выполнения каких-либо заданий. Задания могут быть самыми разнообразными — очистка временного каталога для экономии места, очистка кэша, обновление баз антивируса, запущенного на почтовом сервере и т.д. Никаких ограничений нет — вы можете написать на **bash** небольшой сценарий с необходимыми вам действиями, а затем настроить планировщик для его периодического выполнения.

Самый древний планировщик заданий — **cron**. Он появился еще во времена первых версий UNIX. Но примечательно то, что в современных дистрибутивах используется его модифицированная версия, которая настраивается практически так же. Так, в современных версиях дистрибутивов openSUSE и Fedora используется демон **cronie**, который является потомком того самого демона **cron**.

Не будем вникать в тонкости новой версии, а просто разберемся, как ее настроить. Как и в том самом **cron**, в **cronie** есть файл /etc/crontab —



это таблица расписания планировщика задач. Формат записей в этом файле следующий:

минуты (0-59) часы (0-23) день (1-31) месяц (1-12) день_недели (0-6, 0 — Вс) команда

В листинге 8.1 приведен пример этого файла по умолчанию.

Листинг 8.1. Пример файла /etc/crontab

```
SHELL=/bin/bash
PATH=/usr/bin:/usr/sbin:/sbin:/bin:/usr/lib/news/bin
MAILTO=root

*.15 * * * * root test -x /usr/lib/cron/run-crons && /usr/
lib/cron/run-crons >/dev/null 2>&1
```

ПРИМЕЧАНИЕ. Просмотреть (даже просмотреть, не говоря уже о модификации!) файл /etc/crontab может только пользователь *root*. Поэтому сначала нужно получить права *root*, а затем уже что-либо делать с файлом /etc/crontab.

Переменная *SHELL* задает путь к оболочке, *PATH* — путь поиска программ, а *MAILTO* определяет имя пользователя, которому будет отправлен отчет о выполнении расписания. В таблице расписания всего одна запись — она проверяет наличие сценариев в каталогах *cron.hourly*, *cron.daily*, *cron.weekly* и *cron.monthly* и их выполнение. Об этом мы поговорим чуть позже.

Представим, что вам нужно выполнять какой-то сценарий каждый день в 8:30. Для этого в /etc/crontab нужно добавить строку:



Однако планировщик предлагает более удобный способ изменения таблицы расписания. Представим, что у вас есть команда, которую нужно выполнять периодически. Создайте файл **myscript** со следующим содержанием:

```
#/bin/bash
/путь/ myscript аргументы
```

Сохраните файл и установите право выполнения для этого файла:

```
chmod +x myscript
```

Только что вы создали простейший сценарий, который просто выполняет вашу команду с заданными аргументами. При желании, необходимости и знании **bash** (информацию по этой оболочке вы без особых проблем найдете в Интернете) вы можете усовершенствовать этот сценарий.

После того, как сценарий создан, его нужно поместить в один из каталогов:

- **/etc/cron.hourly** — содержит сценарии, которые будут выполняться каждый час;
- **/etc/cron.daily** — содержит сценарии, который будут выполняться ежедневно;
- **/etc/cron.weekly** — сюда нужно поместить сценарии, которые будут выполняться еженедельно;
- **/etc/cron.monthly** — содержит сценарии, которые будут выполняться раз в месяц.

Просто поместите сценарий в один из этих каталогов и положитесь на **cron** — далее вашего вмешательства не требуется.



Также существует возможность создать отдельное расписание для каждого пользователя. Для этого используется команда `crontab`. Однако такая возможность на современных серверах (когда пользователи не работают с терминалом сервера непосредственно) используется довольно редко. При этом в файл `/etc/cron.deny` заносятся пользователи, которым запрещено использовать планировщик **cron**. Если вам нужно отредактировать таблицу расписания для каждого пользователя, используйте команду `crontab`. В большинстве случаев команда будет такой:

```
crontab -e -u <имя_пользователя>
```

После этого откроется текстовый редактор (определенный в переменной окружения `EDITOR`) для редактирования таблицы расписания указанного пользователя. Синтаксис такой же, как для общесистемной таблицы расписания.

8.5.2. Планировщик *anacron*

Планировщик **anacron** — еще один форк старого доброго планировщика **cron**. Ситуация с **anacron** такая: когда стало понятно, что **cron** устарел, начали появляться альтернативные планировщики, подобные ему. У каждого из них были свои преимущества и недостатки, но некоторые разработчики дистрибутивов остановили свой выбор на планировщике **anacron**. Однако прошло время, и сейчас этот планировщик практически не используется. В основном используется планировщик **cronie**, который настраивается практически так же, как и **cron**, что и было показано ранее.

Конечно, вам могут встретиться дистрибутивы, в которых по каким-то причинам используется **anacron**. Как правило, это устаревшие версии дистрибутивов. Если же вы обнаружили **anacron** в современной



версии дистрибутива, то, скорее всего, это личное предпочтение разработчиков дистрибутивов.

Основное «визуальное» отличие этого планировщика — наличие файла `/etc/anacrontab` вместо просто `/etc/crontab`. Формат записей также другой:

Период	Задержка	ID	Команда
--------	----------	----	---------

Пример таблицы расписания **anacrontab**:

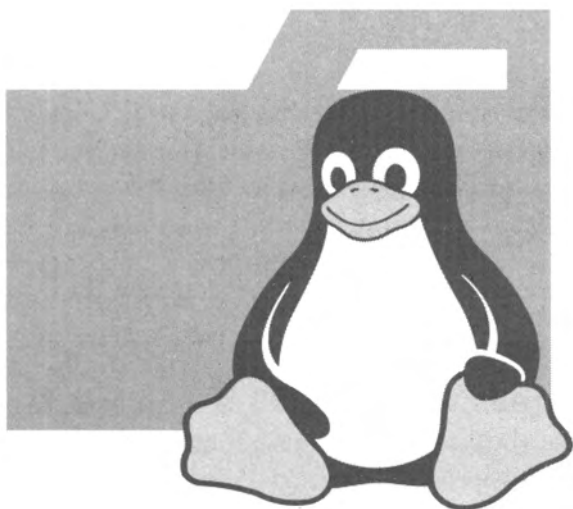
1	5	cron.daily	run-parts /etc/cron.daily
7	10	cron.weekly	run-parts /etc/cron.weekly
30	75	cron.monthly	run-parts /etc/cron.monthly

Принцип прост: как и в предыдущем случае, вам нужно поместить ваш сценарий в один из каталогов `/etc/cron.daily`, `/etc/cron.weekly` или `/etc/cron.monthly` (каталога `cron.hourly` для этого планировщика не было предусмотрено).

По сути, как только вы увидели каталоги `cron.daily`, `cron.weekly` или `cron.monthly`, можете сразу помещать в них свои сценарии, особо не разбираясь, какой планировщик установлен. Самое главное, чтобы служба **cron** была включена.

ГЛАВА 9.

ПОЛЬЗОВАТЕЛИ И ГРУППЫ





9.1. Введение в учетные записи Linux

Операционная система Linux поддерживает регистрацию и одновременную работу множества пользователей. Обратите внимание: именно одновременную работу. Раньше, еще во времена UNIX, были компьютеры, к которым подключалось несколько мониторов и клавиатур. Каждый комплект *монитор+клавиатура* назывался *терминалом* и представлял собой отдельное рабочее место пользователя. Пользователь входил в систему, а его рабочее место в системе отображалось как `ttyN`, где `N` — номер рабочего места.

Сегодня такие компьютеры уже более не востребованы, их вытеснили персональные компьютеры, которые и стали называться персональными, поскольку предполагают подключение только одного рабочего места. Мониторов можно подключить несколько, а устройство ввода — клавиатура — будет одна. Но даже на таких компьютерах возможна одновременная работа нескольких пользователей. Например,



вы можете войти в систему как обычно — посредством графического интерфейса. Другие пользователи смогут войти через **ssh** или FTP. И все вы будете работать с системой одновременно. SSH-пользователи смогут выполнять команды и получать результат их выполнения, FTP-пользователи — обмениваться с вашим компьютером файлами.

Все учетные записи можно разделить на три вида:

Учетные записи обычных пользователей;

Учетные записи системных служб;

Учетная запись root.

С *учетными записями обычных пользователей* все ясно — они имеют право входить в систему разными способами (если тот или иной способ не запрещен настройками системы), для них определен домашний каталог (обычно /home/<имя_пользователя>), пароль и командная оболочка (как правило, в последнее время используется /bin/bash).

Права обычных учетных записей:

- **Право на вход в систему** — по умолчанию обычный пользователь может войти в систему самыми разными способами, если это не ограничено настройками системы (например, модулями PAM). Пользователь может войти локально — через консоль или в графическом режиме через дисплей-менеджер вроде **gdm**. Также никто не запрещает (опять-таки по умолчанию) удаленный вход, например, по SSH или FTP, если на компьютере, в который осуществляется вход, установлены соответствующие службы.
- **Право на запуск программ, не требующих для своего выполнения прав root** — как правило, такие программы находятся в каталогах /bin и /usr/bin. А вот из каталога /sbin запустить программу может только суперпользователь. Программы, действие



которых распространяется на всю систему, например, программы изменения сетевых интерфейсов, программы разметки диска, находятся в каталоге `/sbin` (*super-bin*). Чтобы запустить эти программы, пользователю нужно получить полномочия *root*. О том, как это сделать, будет сказано в следующем разделе.

- **Обычный пользователь может создавать, удалять, читать, изменять, запускать, устанавливать права** и выполнять другие операции над файлами, которые находятся в его домашнем каталоге. Как правило, это каталог `/home/<имя_пользователя>`. Хотя администратор может назначить пользователю любой другой каталог, хоть `/users/bagira`, как правило, этого никто не делает. Каталог `/home` может находиться физически на одном разделе, что и корневая файловая система, а может находиться на другом разделе и даже на другом диске. На крупных серверах, как правило, под `/home` отводят целый диск или даже создают RAID-массивы дисков.
- **Право на чтение файлов** — обычный пользователь может читать большую часть файлов за пределами домашнего каталога. Исключения разве что составляют домашние каталоги других пользователей (если эти другие пользователи явно не разрешили этому пользователю читать их файлы) и некоторые файлы/каталоги в `/etc`. Например, файл `/etc/passwd` могут читать все пользователи, а вот файл `/etc/shadow` — только *root*.
- **Пользователь не имеет право вносить изменения в конфигурацию всей системы**, то есть устанавливать программы, изменять глобальные настройки устройств, параметры ядра, параметры загрузчика и т.д.
- **Пользователь имеет право изменить свои пользовательские параметры**, например, обои рабочего стола, некоторые переменные окружения, которые будут влиять только на его работу и т.д.
- **Право на изменение своего пароля**, но обычный пользователь не имеет право изменять пароли других пользователей.



Учетные записи системных служб не имеют право входить в систему.

Для них не задан ни пароль, ни домашний каталог, а в качестве оболочки используется `/bin/true` или `/bin/false` — чтобы пользователь, используя учетную запись службы, не мог войти в систему через консоль. От имени таких учетных записей выполняются различные службы, например, от имени пользователя `www-data` выполняется веб-сервер, **gdm** — учетная запись для GNOME Display Manager и т.д.

Пользователь `root` — пользователь с максимальными правами, он может делать все.

Например:

- **Право на изменение любого файла** — *root* может читать, записывать, удалять любые файлы, в том числе и файлы в домашних каталогах других пользователей.
- **Право на изменение конфигурации системы** — пользователь *root* может изменять конфигурацию систему посредством редактирования файлов в каталоге `/etc`, `/proc`, запуска конфигураторов системы.
- **Право на запуск любых программ** — *root* может запустить любую программу, в каком бы каталоге она ни находилась.
- **Право на создание, удаление, изменение** (в том числе изменение пароля) других учетных записей.
- **Право на установку и удаление программ.**

Власть пользователя *root* неограниченна. Так было до определенного момента, пока не появились системы принудительного контроля доступа вроде SELinux, LIDS, Tomoyo, которые могут ограничить даже самого *root*. Вот только беда подобных систем в том, что по



умолчанию они неактивны или даже не установлены, поэтому пока их не активировать, пользователь *root* будет все равно самым главным.

ПРИМЕЧАНИЕ. Напомним, что когда у вас привилегии пользователя *root*, приглашение командной строки заканчивается символом **#**, а когда вы работаете как обычный пользователь — **\$**.

9.2. Получение полномочий *root*

Самый простой способ получить права *root* — это войти как *root*. То есть при входе в систему вы указываете имя пользователя *root* и пароль, указанный при установке. Проблема в том, что не во всех дистрибутивах этот трюк сработает. Учитывая всю опасность, которую несет использование учетной записи *root*, во многих дистрибутивах учетная запись *root* отключена, а вход как *root* ограничен самыми разными способами. Например, в том же дистрибутиве Ubuntu учетная запись *root* попросту отключена. Включать учетную запись *root* не рекомендуется — ведь злоумышленник знает, что учетка *root* есть везде и ему не придется угадывать имя пользователя, останется только подобрать пароль *root*. А так вы исключаете даже самую возможность входа как *root*. Тем более права *root* можно получить другим образом, о котором мы поговорим далее. А пока рассмотрим способы блокировки входа (кроме отключения учетной записи) *root*.

Часто вход как *root* ограничен на уровне менеджера дисплея. В дистрибутивах, где используется графическая среда KDE и менеджер экрана KDM (K Display Manager), нужно отредактировать файл `/etc/alternatives/kdm4-config`. В нем нужно найти директиву `AllowRootLogin` и присвоить ей значение `true`, чем вы разрешите вход как *root* в графическом режиме.

При использовании GDM (Gnome Display Manager) вход как *root* ограничен не столько конфигурацией самого GDM, сколько PAM-модулями. Нужно открыть `/etc/pam.d/gdm-password` и найти строку, отвечающую за запрет входа как *root* (рис. 9.1). Она может выглядеть, например, так:

```
auth required pam_succeed_if.so user != root quiet
```



Рис. 9.1. Файл `/etc/pam.d/gdm-password`

Некоторые дистрибутивы разрешают заходить как *root* даже в графическом режиме. Просто они отображают предупреждение о том, что работать как *root* небезопасно. Пример такого дистрибутива — CentOS.

Настоятельно рекомендуется работать в системе как обычный пользователь, а максимальные права получать только тогда, когда они вам действительно нужны. Например, когда понадобится запустить какую-то программу, требующую права *root*. При этом вам особо ничего не придется делать, кроме как ввести пароль.



Рассмотрим пример. Вы установили Ubuntu, при установке создали учетную запись *ubuntu* и задали пароль. По умолчанию инсталлятор создает первую учетную запись так, что она вносится в файл *sudoers*. В этом файле указываются все учетные записи, имеющие право выполнять административные задачи. Когда вы попытаетесь выполнить одну из таких задач, например, добавить нового пользователя, графический интерфейс автоматически запросит у вас ваш пароль. Это будет не пароль *root*, а ваш пароль, то есть пароль пользователя *ubuntu* (рис. 9.2). Ему разрешено выполнять административные задачи, просто система пытается убедиться, что вы – это вы, а не некто, кто оказался за вашим компьютером во время вашего отсутствия.

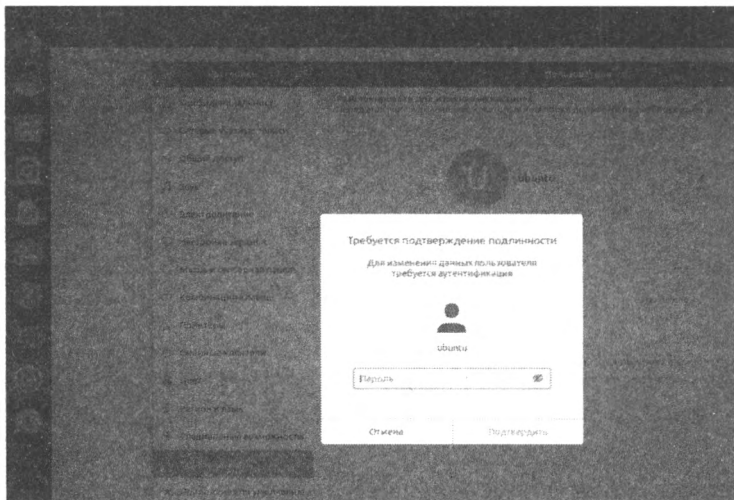


Рис. 9.2. Ввод пользователя

Когда же вам нужна командная строка с правами *root*, не обязательно даже переключаться в консоль. Достаточно открыть терминал и ввести команду *su*. Она попросит вас ввести пароль *root*. После ввода пароля вы получаете терминал с правами *root*. Это означает, что все команды, которые вы будете вводить после ввода команды *su* и успешной аутентификации, будут выполняться с правами *root*.



ПРИМЕЧАНИЕ. Если учетная запись *root* отключена, то вы не сможете использовать команду *sudo*, так как она предполагает ввод пароля *root*, а вы его не знаете.

Когда вы единственный администратор, команда *su* — идеальный вариант. Но когда администраторов несколько, команда *su* — не выход, поскольку пароль *root* нужно будет сообщить всем остальным администраторам. Если потом возникнет нестандартная ситуация, выяснить, кто виноват, будет сложнее.

На этот случай у пользователя *root* могут быть доверенные лица. Это может быть помощник администратора, его заместитель — называйте, как хотите. Есть лица, которым разрешено получать права *root*. Такие лица вносятся в файл */etc/sudoers*.

Редактировать файл */etc/sudoers* можно только через команду *visudo* (рис. 9.3):

```
export EDITOR=nano
sudo visudo
```

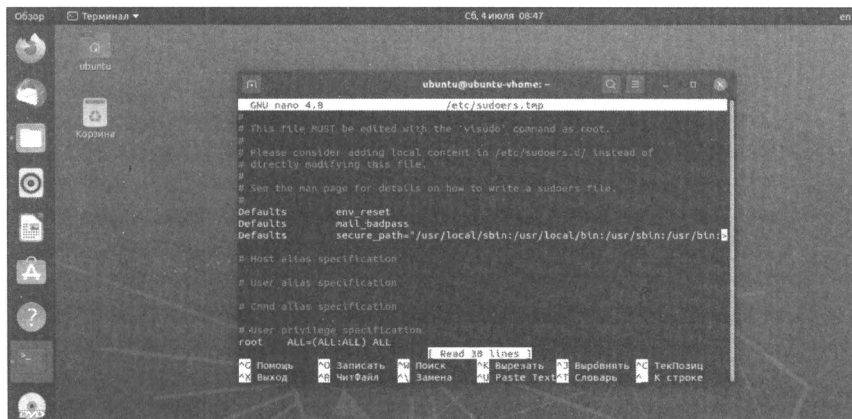


Рис. 9.3. Редактирование файла */etc/sudoers*



Первая команда устанавливает переменную окружения *EDITOR*, задающую удобный текстовый редактор, который будет использован для */etc/sudoers*. Вторая команда вызывает утилиту для редактирования файла */etc/sudoers*.

Представим, что у нас есть пользователь *bagira*, которому нужно разрешить делать все, что можно пользователю *root*. Для этого нужно добавить в */etc/sudoers* запись вида:

```
bagira ALL=(ALL:ALL) ALL
```

Можно также добавить запись:

```
%sudo ALL=(ALL:ALL) ALL
```

Она означает, что членам группы *sudo* можно делать все, что можно делать пользователю *root*. Тогда всех администраторов-помощников нужно добавить в группу *sudo* (далее будет показано, как это сделать).

Сохраните файл и выйдите из редактора. Войдите как пользователь, которому вы предоставили право *sudo*. В нашем случае это пользователь *bagira*. Далее введите команду, которая требует прав *root*, через команду *sudo*:

```
sudo <команда>
```

Например:

```
sudo aptitude
```

Обратите внимание: система запрашивает пароль пользователя, а не пароль *root*.



Пользователь указывает свой пароль, а система знает, что ему разрешено получать права *root*. В итоге наши помощники не знают пароль *root* и смогут выполнять определенные действия с правами *root* под своим именем.

Посмотрите на рис. 9.4. Хотя пользователь внесен в *sudoers*, ему отказано в доступе при попытке запуска программы, требующей максимальные права. Системе нужно указать, что вы явно хотите запустить такую программу и предоставить максимальные права. Для этого нужно использовать команду *sudo*, а в качестве аргумента – передать ей команду, которую вы хотите запустить с максимальными правами:

```
sudo visudo
```



Рис. 9.4. Нюанс запуска программы с максимальными правами

Контролировать получение прав *sudo* можно командой¹:

```
# tail /var/log/auth.log | grep sudo
```

1

В некоторых дистрибутивах журнал аутентификации называется *secure*, а не *auth.log*.



Посмотрите на рис. 9.5. Например, 4 июля 2020 года в 8:56 пользователь *ubuntu* пытался выполнить команду *sudo* для выполнения команды *tail /var/log/secure*. То есть в журнале отображаются не только попытки использования *sudo*, но и журналируются даже вводимые пользователями команды.

```
ubuntu@ubuntu-vhorne: ~$ sudo tail /var/log/auth.log | grep sudo
Jul  4 08:56:28 ubuntu-vhorne sudo:  ubuntu : TTY=pts/0 ; PWD=/home/ubuntu ; USER=root ; COMMAND=/usr/bin/tail /var/log/secure
Jul  4 08:56:28 ubuntu-vhorne sudo: pam_unix(sudo:session): session opened for user root by (uid=0)
Jul  4 08:56:28 ubuntu-vhorne sudo: pam_unix(sudo:session): session closed for user root
Jul  4 08:56:42 ubuntu-vhorne sudo:  ubuntu : TTY=pts/0 ; PWD=/home/ubuntu ; USER=root ; COMMAND=/usr/bin/tail /var/log/auth.log
Jul  4 08:56:42 ubuntu-vhorne sudo: pam_unix(sudo:session): session opened for user root by (uid=0)
Jul  4 08:56:42 ubuntu-vhorne sudo: pam_unix(sudo:session): session closed for user root
Jul  4 08:56:54 ubuntu-vhorne sudo:  ubuntu : TTY=pts/0 ; PWD=/home/ubuntu ; USER=root ; COMMAND=/usr/bin/tail /var/log/auth.log
Jul  4 08:56:54 ubuntu-vhorne sudo: pam_unix(sudo:session): session opened for user root by (uid=0)
ubuntu@ubuntu-vhorne: ~$
```

Рис. 9.5. Журнал аутентификации *secure*

Если вам нужно получить некоторый аналог команды *su*, чтобы вы могли вводить сразу неограниченное количество команд с максимальными правами без приставки *sudo*, используйте следующий трюк — запустите с максимальными правами оболочку **bash**. Все команды, вводимые в этой оболочке, будут выполнены с максимальными правами:

```
sudo bash
```

Закрыть такой сеанс можно командой *exit*.

Прежде, чем перейти к следующему разделу, разберемся, как включить учетную запись *root* в Ubuntu. Для этого нужно просто задать пароль:



```
sudo passwd root
```

Сначала нужно ввести ваш пароль, затем новый пароль для *root*, после этого — подтвердить пароль. После этого вы сможете войти в систему как *root* в консоли. Для входа в графическом режиме нужно редактировать файл, относящийся к RAM, как было сказано ранее. Отметим, что активация пользователя *root* — занятие небезопасное, гораздо правильнее использовать команду *sudo*. На личном компьютере еще такое мероприятие допускается, но на сервере — такое делать воспрещено.

9.3. Управление учетными записями пользователей

9.3.1. Создание учетной записи пользователя

Создать новую учетную запись пользователя можно командой *adduser* или *useradd*.

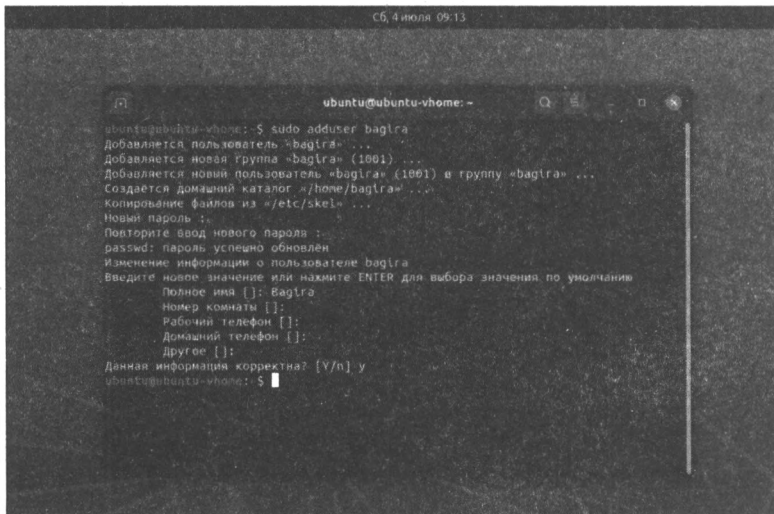
Чаще всего используется именно первая команда, вторая используется гораздо реже.

В большинстве случаев *adduser* просто добавляет в файл */etc/passwd* учетную запись пользователя. В дистрибутивах Debian и Ubuntu команда *adduser* запрашивает контактную информацию (полное имя пользователя, номера телефонов и т.д.), а также сразу устанавливает пароль пользователя (рис. 9.6).

Если в вашем дистрибутиве команда *adduser* не запросила пароль пользователя, а просто добавила его учетную запись, тогда вам нужно



еще ввести команду *passwd* <имя пользователя> для установки его пароля, иначе пользователь не сможет войти в систему.



```
ubuntu@ubuntu-vhorne: ~  
$ sudo adduser bagira  
Добавляется пользователь «bagira» ...  
Добавляется новая группа «bagira» (1001) ...  
Добавляется новый пользователь «bagira» (1001) в группу «bagira» ...  
Создается домашний каталог «/home/bagira» ...  
Копирование файлов из «/etc/skel» ...  
Новый пароль :  
Повторите ввод нового пароля :  
passwd: пароль успешно обновлен  
Изменение информации о пользователе bagira  
Введите новое значение или нажмите ENTER для выбора значения по умолчанию  
Полное имя []: Bagira  
Номер комнаты []:  
Рабочий телефон []:  
Домашний телефон []:  
Другое []:  
Данная информация корректна? [Y/n] y  
ubuntu@ubuntu-vhorne: $
```

Рис. 9.6. Создание нового пользователя в *Ubuntu 20.04*

Итак, в одних дистрибутивах достаточно команды:

```
# adduser <имя>
```

В других же нужно ввести две команды:

```
# adduser <имя>  
# passwd <пароль>
```

Напомню, что для добавления учетной записи пользователя нужны права *root*, которые пользователь может получить через *su* или *sudo*, если он внесен в */etc/sudoers* и ему разрешена операция добавления пользователя.



9.3.2. Файлы `/etc/passwd` и `/etc/shadow`

При добавлении учетной записи происходят следующие действия (вкратце):

- Добавляется запись в файл `/etc/passwd` — это небольшая база данных о пользователях в текстовом формате. Этот файл могут просмотреть все пользователи.
- Если при создании учетной записи утилита запрашивает пароль, то он будет внесен в файл `/etc/shadow`. Пароли в этом файле хранятся в зашифрованном виде, а доступ имеет только *root*. Команда `passwd <имя>`, изменяющая пароль пользователя, вносит изменения как раз в этот файл.
- Создается домашний каталог `/home/<имя>`, и в него копируется содержимое каталога `/etc/skel`.
- Создается почтовый ящик пользователя в каталоге `/var/spool/mail`.
- Владелец каталога `/home/<имя>` и всех файлов и каталогов в нем назначается создаваемый пользователь.

Рассмотрим формат файла `/etc/passwd`:

`имя_пользователя:пароль:UID:GID:полное_имя:домашний_каталог:оболочка`

Вот фрагмент этого файла:

```
ubuntu:x:1000:1000:ubuntu:/home/ubuntu:/bin/bash
bagira:x:1001:1001::/home/bagira:/bin/bash
```

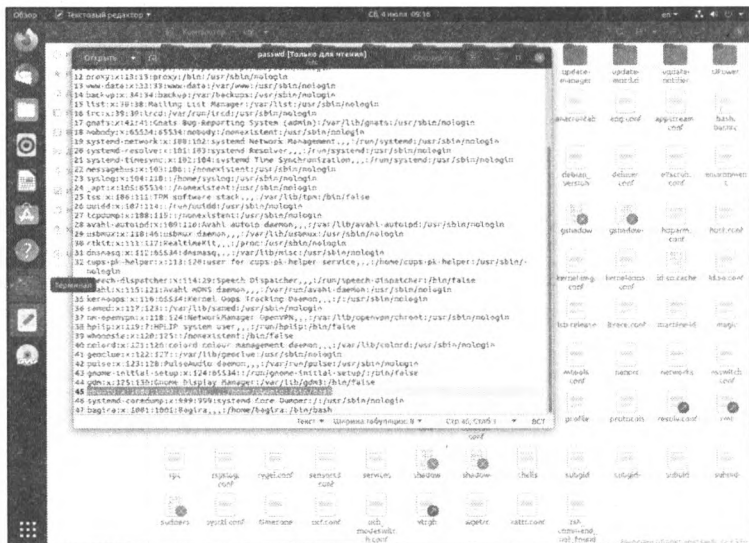


Рис. 9.7. Файл /etc/passwd

Формат файла /etc/passwd приведен в таблице 9.1.

Таблица 9.1. Формат файла /etc/passwd

Номер поля	Название	Описание
1	Имя пользователя	Имя, использующееся при входе в систему
2	Пароль	Поскольку пароль пользователя хранится в файле /etc/shadow, то в файле /etc/passwd вместо пароля просто указывается символ 'x'
3	UID	Идентификатор пользователя
4	GID	Идентификатор группы пользователя



5	Полное имя пользователя	Устанавливается администратором и ни на что не влияет. В крупных организациях имя пользователя помогает установить контакт с пользователем. Это поле может также содержать номер телефона, номер комнаты и прочую информацию, которую запрашивает <i>adduser</i> при создании учетной записи пользователя
6	Домашний каталог пользователя	Обычно это <code>/home/<имя_пользователя></code>
7	Оболочка	Программа, которая будет запущена при входе пользователя в систему из консоли (для графического режима это поле не имеет значения). Список доступных оболочек хранится в файле <code>/etc/shells</code>

В файле `/etc/shadow` полей больше, чем в `/etc/passwd`. Как и в случае с `/etc/passwd`, поля разделяются двоеточиями:

1. Имя пользователя. Совпадает с именем пользователя в файле `/etc/passwd`.
2. Зашифрованный пароль. Позже мы поговорим о том, как распознать алгоритм шифрования, которым был зашифрован пароль.
3. Количество дней (с 1 января 1970 года), когда пароль был сменен в последний раз.



4. Число дней до смены пароля. Если в этом поле 0, то пароль может быть сменен в любой момент.
5. Количество дней, после которых пароль должен быть сменен. Обычно здесь значение 999999, которое показывает, что пользователь может никогда не менять свой пароль.
6. Число дней, в течение которых пользователь получает предупреждение о необходимости изменить пароль. Обычно такие предупреждения пользователь получает за неделю (7 дней) до часа «X».
7. Число дней после окончания действия пароля, когда пользователь еще может работать со старым паролем. Если после этого срока пользователь не сменит пароль, учетная запись будет заблокирована.
8. Число дней, начиная с 1 января 1970, после которых пароль будет заблокирован.
9. Не используется.

Обычно последние три поля не используются. По зашифрованному паролю можно понять, какой алгоритм шифрования использует система. Посмотрите на начало зашифрованного пароля:

- **\$1\$** — MD5. Ранее часто использовался, сейчас чаще используется SHA-512, поскольку в MD5 обнаружались математические уязвимости;
- **\$2\$, \$2a\$** — Blowfish. Чаще используется в FreeBSD/OpenBSD, чем в Linux;
- **\$5\$** — SHA-256;
- **\$6\$** — SHA-512. Используется в современных дистрибутивах.



Форматы файлов `/etc/passwd` и `/etc/shadow` были приведены «для общего развития», чтобы вы понимали, что происходит. Модифицировать учетную запись пользователя правильнее с помощью команды *usermod*, а не с помощью редактирования файла `/etc/passwd`. Конечно, можно внести небольшие изменения, например, изменить полное имя пользователя. А вот для изменения остальных параметров, например, домашнего каталога, правильнее использовать *usermod*, чтобы потом не делать много ручной работы.

9.3.3. Изменение и удаление учетных записей

Как было отмечено ранее, для модификации учетной записи пользователя нужно использовать команду *usermod*, но прежде поговорим об изменении пароля, так как изменение пароля — это тоже, по сути, изменение учетной записи.

Для установки и изменения пароля пользователя используется команда *passwd*:

```
# passwd <имя>
```

Если пользователь хочет изменить собственный пароль, то указывать имя не нужно:

```
$ passwd
```



А вот теперь можно приступить к рассмотрению команды *usermod*. Формат вызова этой команды следующий:

```
# usermod [параметры] учетная_запись
```

Параметры команды *usermod* описаны в таблице 9.2.

Таблица 9.2. Параметры команды *usermod*

Параметр	Описание
<i>-a, -append</i>	Добавляет пользователя в дополнительную группу. Используется только с параметром <i>-G</i>
<i>-c, --comment комментарий</i>	Добавляет комментарий для учетной записи пользователя
<i>-d, --home каталог</i>	Задаёт новый домашний каталог пользователя. Если указать параметр <i>-m</i> , то текущий домашний каталог пользователя будет перенесен в новый домашний каталог, который будет создан, если не существует
<i>-e, --expiredate дата</i>	Указывает дату устаревания учетной записи пользователя. По достижении этой даты учетная запись пользователя будет заблокирована. Дата указывается в формате ГГГГ-ММ-ДД. Если дату не указывать, то устаревание учетной записи будет отключено
<i>-f, --inactive дни</i>	После указанного числа дней, которые пройдут после устаревания пароля, учетная запись будет заблокирована. Значение <i>-1</i> означает, что эта возможность не используется, а 0 — запись будет заблокирована сразу же после устаревания пароля



<i>-g, --gid группа</i>	Указывает имя или GID первичной группы пользователя. Группа с таким именем/GID должна существовать. Все файлы в домашнем каталоге пользователя, которые принадлежали бывшей первичной группе, теперь будут принадлежать новой группе
<i>-G, --groups группа1[, группа2, ..., группаN]</i>	Список дополнительных групп, в которых находится пользователь. Перечисление групп осуществляется через запятую без дополнительных пробелов. Например, <i>-G group1,group2</i>
<i>-l, --login новое_имя</i>	Изменяет имя пользователя на новое_имя
<i>-L, --lock</i>	Блокирует учетную запись пользователя. Нельзя использовать этот параметр с <i>-p</i> или <i>-I</i>
<i>-m, --move-home</i>	Перемещает домашний каталог. Используется вместе с параметром <i>-d</i>
<i>-o, --non-unique</i>	При использовании с <i>-u</i> позволяет указать не уникальный UID (идентификатор пользователя)
<i>-p, --password пароль</i>	Шифрованное значение пароля, которое возвращает функция <i>crypt</i> . Использовать этот параметр не рекомендуется, поскольку другие пользователи увидят незашифрованный пароль в списке процессов



<i>-R, -root chroot</i>	Выполняет изменения в каталоге chroot и использует файлы конфигурации из этого каталога
<i>-s, --shell оболочка</i>	Задаёт оболочку для пользователя. Если оболочка не указана, то будет использована оболочка по умолчанию
<i>-u, --uid UID</i>	Задаёт новый UID пользователя, который должен быть уникальным
<i>-U, --unlock</i>	Разблокирует учётную запись пользователя
<i>-Z, --selinux-user SEUSER</i>	Новый пользователь SELinux для пользовательского входа

Рассмотрим несколько примеров:

```
# usermod -d /home/new_home -m ubuntu
# usermod -L bagira
# usermod -G admins,sudo mark
```

Первая команда задаёт новый каталог для пользователя *ubuntu*. Теперь он будет называться */home/new_home*. Старые файлы (из каталога */home/ubuntu*) будут перемещены в новый домашний каталог.

Вторая команда блокирует учётную запись пользователя *bagira*. Третья команда вносит пользователя *mark* в группы *admins* и *sudo*.

Теперь рассмотрим команду *userdel* (см. табл. 9.3):

```
# userdel [параметры] пользователь
```


Таблица 9.3. Параметры команды *userdel*

Параметр	Описание
<i>-f, --force</i>	Удаляет учетную запись, даже если пользователь работает в системе. Также будет удален домашний каталог и почтовый ящик, даже если другой пользователь использует тот же домашний каталог. Если в файле <code>/etc/login.defs</code> параметр <code>USERGROUPS_ENAB</code> равен <code>yes</code> , то будет удалена и первичная группа пользователя, даже если она является первичной и для другого пользователя. Довольно опасный параметр, который может привести систему в нерабочее состояние
<i>-r, --remove</i>	Удаляет домашний каталог пользователя и почтовый ящик. Файлы этого пользователя, созданные на других файловых системах, нужно искать и удалять вручную
<i>-R, --root chroot</i>	Выполняет изменения в каталоге chroot и использует файлы конфигурации из этого каталога
<i>-Z, --selinux-user</i>	Удаляет все пользовательские сопоставления SELinux для учетной записи пользователя

Пример удаления учетной записи *ubuntu*, домашний каталог и почтовый ящик также будут удалены:

```
# userdel -r ubuntu
```

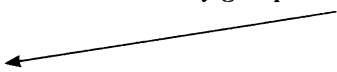


9.3.4. Группы пользователей

Для более простого управления пользователями их можно объединять в группы. Например, можно задать ограничения ресурсов для группы пользователей. Тогда они будут распространяться на всех пользователей, входящих в группу, и вам не придется их устанавливать для каждого пользователя отдельно.

Но прежде чем устанавливать права для группы, нужно эту группу создать. Добавить группу можно командой *groupadd*, однако ничего плохого не случится, если вы просто отредактируете файл */etc/group* (не *groups*, а именно *group*!) и добавите группу вручную. При добавлении группы следите, чтобы ID группы был уникальным. Если же вы не хотите ни за чем следить, тогда просто введите команду *groupadd*:

```
# groupadd [параметры] имя_группы
```



С параметрами команды *groupadd* можно ознакомиться в справочной системе — **man groupadd**.

9.4. Графические конфигураторы

Управлять учетными записями пользователя можно и с помощью графических конфигураторов, что будет привычнее для бывших Windows-пользователей. В Ubuntu для управления учетными записями пользователей нужно перейти в окно **Настройки**, далее – в раздел **Пользователи** (рис. 9.8). Для управления другими учетными записями нажмите кнопку **Разблокировать**, введите свой пароль, после чего вы сможете управлять другими учетными записями — изменять их пароли, создавать и удалять учетные записи и т.д.

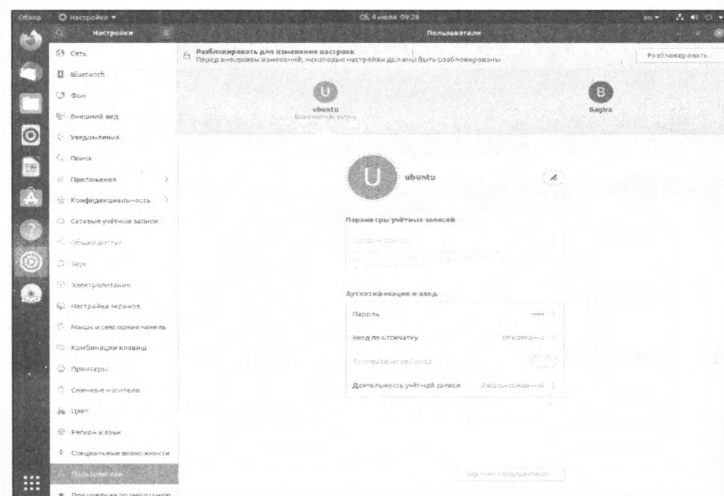


Рис. 9.8. Графический конфигуратор в Ubuntu

Например, в Astra Linux нужно открыть панель управления, а далее перейти в раздел **Безопасность**, выбрать **Политика безопасности** и в открывшемся окне перейти в раздел **Пользователи** (рис. 9.9). Немного запутано, но как есть.

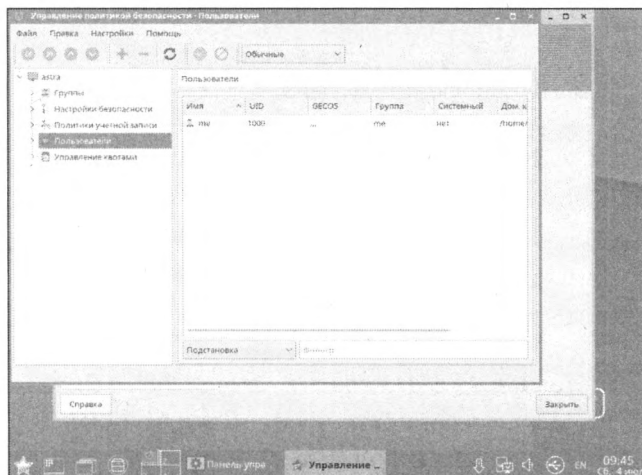


Рис. 9.9. Управление пользователями в Astra Linux



9.5. Модули PAM

Подключаемые модули аутентификации PAM (Pluggable Authentication Modules) предоставляют администраторам дополнительные методы подтверждения подлинности пользователя.

Модули PAM — это не новинка в мире Linux. Они были разработаны очень давно, но до сих пор есть даже в самых современных дистрибутивах Linux, поскольку заменить их, по сути, нечем.

Модули PAM позволяют использовать несколько схем аутентификации. Большинство приложений, которые нуждаются в проверке подлинности пользователя, используют PAM. Модули PAM позволяют реализовать альтернативную аутентификацию, например, по отпечаткам пальцев или по сетчатке глаз, но для этого необходимо дополнительное оборудование, например, сканер отпечатков. В этом справочнике мы рассмотрим традиционный вариант использования PAM — когда аутентификация происходит посредством ввода пароля с клавиатуры.

Основной файл конфигурации называется `/etc/pam.conf`. В каталоге `/etc/pam.d/` находится конфигурация для разных сервисов, которые поддерживают PAM, например, в `/etc/pam.d/sshd` находится конфигурация PAM-модулей для SSH, в `/etc/pam.d/gdm-passwd` — конфигурация пароля для менеджера дисплея GDM и т.д. В каталоге `/etc/security` также есть файлы конфигурации, относящиеся к PAM, например, файл `access.conf` управляет доступом в систему.

Безопасность вашей системы зависит от используемых вами модулей. Модули хранятся в каталоге `/lib/security` или `/lib64/security` (для 64-битных систем), однако некоторые дистрибутивы не следуют этому стандарту. К примеру, в некоторых системах модули можно найти

в каталоге `/usr/lib/security`. При желании можно написать и собственные модули, но для начала следует разобраться с уже имеющимися.

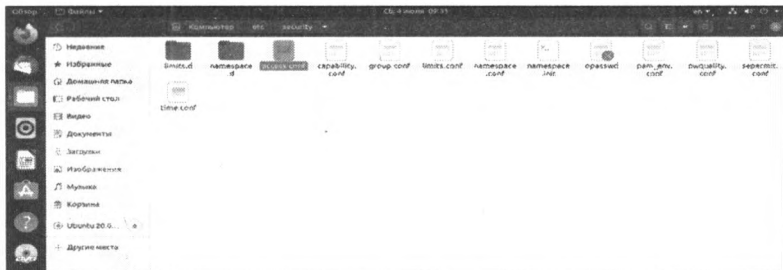


Рис. 9.10. Содержимое каталога `/etc/security`

Ниже приведен список наиболее часто используемых модулей. Больше информации по каждому из них можно получить, набрав `man` модуль, к примеру, `man pam_pwcheck`. Обратите внимание, что нет «стандартного списка» модулей. Их состав варьируется от дистрибутива к дистрибутиву.

- **pam_access** — разрешает или запрещает доступ, в зависимости от IP-адреса, имени пользователя, имени хоста или доменного имени и т.п. По умолчанию, правила доступа определены в файле `/etc/security/access.conf`. Когда пользователь входит, проверяются правила доступа до первого совпадения, и принимается решение, разрешить или запретить доступ. Также смотрите модуль `pam_time` — там другие ограничения.
- **pam_cracklib** и **pam_pwcheck** — предоставляют функции проверки прочности пароля (проверки на легкость угадывания или повторяемость). У пользователя спрашивают пароль, и если тот проходит предустановленные правила и считается прочным, тогда нужно ввести его еще раз для проверки правильности ввода.
- **pam_deny** — безусловно запрещает доступ. Этот модуль можно использовать для блокирования пользователей, как политику по умолчанию. См. также `pam_permit`.



- **pam_echo** — выводит предустановленное текстовое сообщение. См. также **pam_motd**.
- **pam_env** — позволяет присвоение значений переменным окружения. Правила по умолчанию берутся из файла `/etc/security/pam_env.conf`.
- **pam_exec** — вызывает внешнюю программу.
- **pam_lastlog** — выводит дату и время последнего входа в систему.
- **pam_limits** — устанавливает ограничения на системные ресурсы, используемые пользователем. Ограничения по умолчанию берутся из файла `/etc/security/limits.conf`.
- **pam_listfile** — разрешает или запрещает сервис в зависимости от значений в файле. К примеру, если вы хотите открыть FTP-доступ лишь для некоторых пользователей, перечень которых указан в файле `/etc/ftpusers_ok`, нужно добавить строку `auth required pam_listfile.so item=user sense=allow file=/etc/ftpusers_ok onerr=fail` в файл `/etc/pam.d/ftpd`. См. также модуль **pam_nologin**.
- **pam_mail** — сообщает пользователю о наличии свежей электронной почты.
- **pam_mkhomedir** — создает домашний каталог пользователя, если он не существует на локальной машине. Таким образом, можно использовать централизованную авторизацию (к примеру, в NIS или LDAP) и создавать домашние каталоги лишь при необходимости.
- **pam_motd** — выводит «сообщение дня». См. также модуль **pam_echo**.
- **pam_nologin** — запрещает доступ, когда существует файл `/etc/nologin`.
- **pam_permit** — безусловно разрешает доступ — очень небезопасно! См. также модуль **pam_deny**.



- **pam_rootok** — разрешает доступ для пользователя *root* без дополнительных проверок. Обычно этот модуль используется в */etc/pam.d/su*, чтобы пользователь *root* мог войти под любым другим пользователем даже без ввода пароля. Файл должен содержать следующие строки (обратите внимание на вторую строку, см. модуль *pam_wheel*):
 - » `auth sufficient pam_rootok.so`
 - » `auth required pam_wheel.so`
 - » `auth required pam_unix.so`
- **pam_succeed_if** — проверяет некоторые характеристики учетной записи, к примеру, принадлежность к определенной группе, значение UID и т.п.
- **pam_time** — запрещает доступ к службе в зависимости от дня недели и времени дня. По умолчанию правила берутся из файла */etc/security/time.conf*. Однако запрет накладывается лишь на момент входа в систему. Способа принудительно заставить пользователя выйти из системы нет.
- **pam_umask** — устанавливает маску создания файлов.
- **pam_unix** или **pam_unix2** — классическая аутентификация в UNIX-стиле, основана на файлах */etc/passwd* и */etc/shadow*. См. также модуль *pam_userdb*.
- **pam_userdb** — аутентифицирует пользователя с помощью базы данных. См. также модуль *pam_unix*.
- **pam_warn** — заносит название службы, номер терминала, пользователя и другие данные в системный журнал. Модуль можно использовать везде, он не влияет на процесс аутентификации.
- **pam_wheel** — позволяет root-доступ лишь для членов группы *wheel*. Часто этот модуль используется для *su*, чтобы лишь избранные пользователи могли пользоваться этой программой. Пример использования можно найти в описании модуля *pam_rootok*.



В рамках этой книги не получится рассмотреть РАМ очень подробно, поэтому лучше всего рассматривать РАМ на отдельных примерах.

9.5.1. Ограничиваем доступ к системе по IP-адресу

Файл `/etc/security/access.conf` используется модулем `ram_access.so`, чтобы определить, каким пользователям позволено входить в систему и с каких IP-адресов.

Если открыть файл `access.conf`, то в нем будет достаточно много различных примеров, которые хорошо прокомментированы. Если вы знаете английский язык, то не составит особого труда во всем разобраться самостоятельно.

Формат этого файла следующий:

разрешения : пользователи : источники

Разрешение может начинаться с символа "+" (доступ разрешен) или "-" (доступ запрещен). Если нужно указать несколько пользователей, то их имена разделяют пробелом. Если нужно сделать исключение для некоторых пользователей, то перед их именами указывают служебное слово EXCEPT.

Третье поле может содержать список из одного или более имен консолей (tty) — для несетевого доступа к системе, имен узлов (для сетевого доступа), доменных имен (начинаются с "."), IP-адресов узлов, IP-адресов сетей (заканчиваются "."). Также можно указать все источники (ALL), ни один из источников (NONE) или только локальные источники (LOCAL).

Теперь несколько примеров:

```
-:ALL EXCEPT root:tty1
```




Первая консоль — это только консоль *root*. Другим пользователям запрещено ее занимать. Мы запрещаем доступ (-) всем пользователям (ALL) кроме (EXCEPT) пользователя *root* на консоли *tty1*.

Следующий пример — разрешение регистрации как *root* с определенных IP-адресов:

```
+ : root : 192.168.1.1 192.168.1.4 192.168.1.9  
+ : root : 127.0.0.1
```

Если нужно разрешить регистрацию *root* со всей подсети 192.168.1.0, тогда укажите адрес этой подсети, указав точку вместо 0:

```
+ : root : 192.168.1.
```

Самый жесткий пример — запрещаем *root* вообще входить в систему:

```
- : root : ALL
```

ПРИМЕЧАНИЕ. Обратите внимание, что комментарии в этом файле начинаются с #, если вы хотите использовать один из примеров, приведенных в файле, убедитесь, что вы раскомментировали нужную вам строку.

Чуть выше мы разрешили вход пользователя *root* с определенных IP-адресов. К сожалению, одного только редактирования *access.conf* будет недостаточно. Нужно еще отредактировать соответствующие файлы в */etc/pam.d*. Нас интересует регистрация по SSH (*telnet* уже не используется, поэтому вы будете регистрироваться по SSH) и обычная регистрация в системе. Поэтому нам нужно отредактировать файлы */etc/pam.d/ssh* и */etc/pam.d/system-auth*. В этих файлах вам нужно добавить строчку:



```
account    required  /lib64/security/pam_access.so
```

Если у вас 32-разрядная система, тогда нужно добавить немного другую строку:

```
account    required  /lib/security/pam_access.so
```

9.5.2. Ограничиваем время входа в систему

Безопасностью системы лучше управлять, когда вы бодрствуете. Поэтому имеет смысл разрешить регистрацию только в это время, например, с 8:00 до 19:00 (вдруг кто-то немного задержится на работе).

Откройте файл `/etc/security/time.conf` и добавьте в него строку:

```
login;tty* & !ttyp*; !root & admin & ; !A10800-1900
```

Здесь мы разрешаем пользователям регистрироваться только с 8:00 по 19:00. На пользователей *root* и *admin* это правило не распространяется. Также в файле `time.conf` вы найдете еще несколько примеров.

Как и в случае с предыдущим файлом, вам нужно изменить файлы `/etc/pam.d/sshd` и `/etc/pam.d/system-auth`, в которые нужно добавить строку:

```
account    required  /lib64/security/pam_time.so
```

или строку (для 32-разрядной системы):

```
account    required  /lib/security/pam_time.so
```



9.5.3. Ограничение системных ресурсов с помощью PAM

С помощью PAM-модулей можно ограничить системные ресурсы, что полезно для защиты системы от DoS-атаки. Принцип DoS-атаки заключается в том, что злоумышленник узурпирует все ресурсы системы, в результате обычным пользователям ничего не остается. Ограничив системные ресурсы, вы можете смягчить последствия DoS-атаки на ваш сервер. Конечно, полной защиты этот способ не даст, но все равно ваш сервер будет продолжать работать, хоть и медленно. Все же это лучше, чем ничего.

Ограничить системные ресурсы можно с помощью `/etc/security/limits.conf`. Формат записей в этом файле такой:

домен	тип	ресурс	значение
-------	-----	--------	----------

В качестве домена указывается или **имя пользователя**, или **имя группы пользователей** (@имя). Также можно указать звездочку (*), если ограничение должно распространяться на всех пользователей.

Ограничения бывают мягкими (soft) и жесткими (hard).

Мягкое ограничение можно незначительно превысить, жесткое превысить нельзя.

Возможные значения третьего поля задают тип ограничиваемого ресурса и представлены в таблице 9.4.



Таблица 9.4. Ресурсы, которые можно ограничить с помощью *limits.conf*

Элемент	Описание
core	Позволяет ограничить размер файла ядра (в килобайтах)
cpu	Задаёт максимальное процессорное время (в минутах)
data	Определяет максимальный размер сегмента данных (в килобайтах)
fsize	Позволяет указать максимальный размер файла (в килобайтах)
maxlogins	Определяет максимальное количество параллельных регистраций пользователя. По умолчанию пользователю разрешается войти неограниченное количество раз разными способами — по SSH, FTP, с разных консолей и т.д.
nofile	Задаёт максимальное число одновременно открытых файлов
nproc	Определяет число процессов, которые может запустить пользователь
priority	Задаёт приоритет, с которым будут выполняться процессы пользователя или группы
stack	Максимальный размер стека (в килобайтах)



Последнее поле определяет значение лимита. Теперь несколько примеров:

*	hard	maxlogins	3
@ssh_users	hard	nproc	5
@ssh_users	hard	fsize	24576

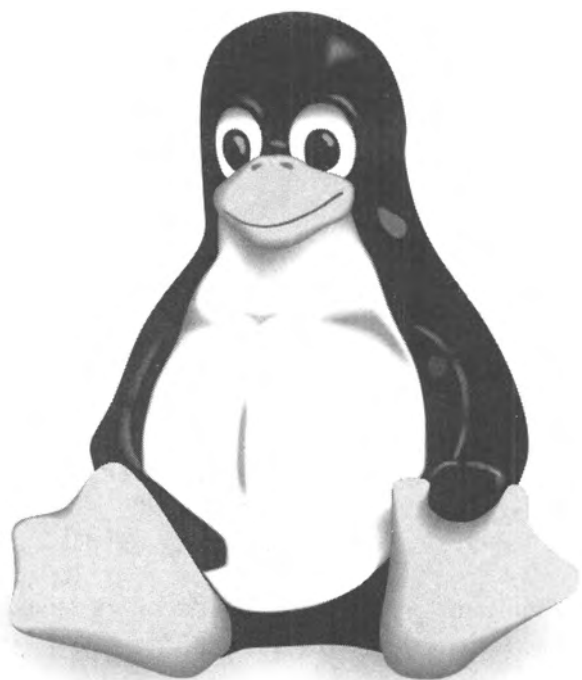
В первом случае мы ограничиваем число одновременных регистраций пользователей до 3 (консоль, X11, если есть, и SSH — этого более чем достаточно). Во втором случае пользователям из группы **ssh_users** мы разрешаем запускать не более 5 процессов одновременно. Также SSH-пользователям не разрешается создавать файлы размером более 24 Мб.

Обратите внимание: здесь мы просто задаем лимит на максимальный размер файла. В принципе, 24 Мб вполне достаточно даже для хранения фотографий с зеркальной камеры и больших документов Word, содержащих изображения и другие объемные объекты. А видео и файлы большего размера пусть пользователи хранят или на своих компьютерах, или входят иным способом, например, по FTP, где можно более качественно ограничить операции с файлами.

После редактирования `/etc/security/limits.conf` никакие другие файлы редактировать не нужно. Но описанные вами изменения будут действовать для новых сеансов пользователей, поэтому желательно перезагрузить систему, чтобы изменения действовали сразу для всех пользователей.

Если вам нужна дополнительная информация о PAM, предлагаем ознакомиться с официальной документацией, доступной по адресу:

<https://mirrors.edge.kernel.org/pub/linux/libs/pam/>



ГЛАВА 10.

ЯДРО LINUX





10.1. Что такое ядро

В этой части нашего справочника мы поговорим о ядре Linux.

***Ядро Linux** — это и есть основная программа, которую запускает загрузчик при выборе загрузочной метки.*

У вас (даже если установлен всего один дистрибутив) может быть установлено несколько ядер. Например, вы можете установить дистрибутив, в нем будет одна версия ядра, затем установить более новую версию ядра вручную. При загрузке вы сможете выбрать нужную вам версию ядра. На рис. 10.1 показано меню загрузчика Astra Linux, где пользователю предлагается выбрать одну из версий ядра — *generic* или *hardened*. Вторая версия повышает общую защищенность системы от взлома. Так, *hardened*-ядро умеет блокировать массу потенциально опасных операций, а компилятор *hardened-gcc* позволяет защитить компилируемые им программы от взлома типовыми методами вроде переполнения буфера. Грубо говоря, если у вас стоит «дырявая» версия программы X, и ее пытается взломать хакер, то в



обычной системе у него это получится, а в *hardened* — не получится, да еще и в лог запись пойдет.

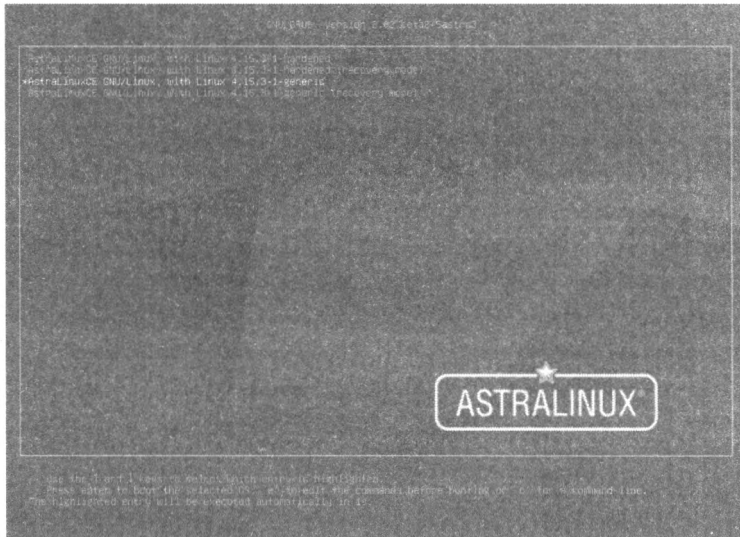


Рис. 10.1. Выбор ядра при загрузке системы

На данный момент одной из последних стабильных версий ядра является версия 6.2.2. Однако не нужно думать, что в вашем дистрибутиве будет самая последняя версия. Как правило, в вашем дистрибутиве будет именно та версия ядра, которая была стабильной на момент выпуска дистрибутива.

Номер версии ядра выводится при входе в систему (если вы входите в консоли) или его можно узнать командой *uname -a* (рис. 10.2). На рис. 10.2 показано, что, например, в Ubuntu 20.04 используется версия ядра 5.4.0 — даже не 5.7.0, и уж тем более не 6.2.2.

При загрузке ядро выводит сообщения, просмотреть которые можно командой:

```
dmesg | less
```



```

C6, 4 июля 10:43
ubuntu@ubuntu-vhorne: ~
ubuntu@ubuntu-vhorne:~$ uname -a
Linux ubuntu-vhorne 5.4.0-39-generic #43-Ubuntu SMP Fri Jun 19 10:28:31 UTC 2020
x86_64 x86_64 x86_64 GNU/Linux
ubuntu@ubuntu-vhorne:~$

```

Рис. 10.2. Команда `uname -a`

Обычно сообщения ядра понятны сами по себе без особых комментариев. Например, в самом начале выводится версия ядра и переданные ядру параметры:

```
[ 0.000000] Linux version 5.4.0-39-generic (builddd@lcy01-amd64-016)
(gcc version 9.3.0 (Ubuntu 9.3.0-10ubuntu2)) #43-Ubuntu SMP Fri Jun 19
10:28:31 UTC 2020 (Ubuntu 5.4.0-39.43-generic 5.4.41)
```

```
[ 0.000000] Command line: BOOT_IMAGE=/boot/vmlinuz-5.4.0-39-generic
root=UUID=05fd3e4e-1605-478c-8db9-b6df62a01ad3 ro quiet splash
```

Карта физической памяти:

```
[ 0.000000] BIOS-e820: [mem 0x0000000000000000-0x0000000000009eff] usable
[ 0.000000] BIOS-e820: [mem 0x0000000000009f00-0x0000000000009fff] reserved
[ 0.000000] BIOS-e820: [mem 0x000000000000ca000-0x000000000000cbfff] reserved
[ 0.000000] BIOS-e820: [mem 0x000000000000dc000-0x000000000000ffff] reserved
[ 0.000000] BIOS-e820: [mem 0x0000000000100000-0x00000000001feffff] usable
[ 0.000000] BIOS-e820: [mem 0x0000000001fee0000-0x0000000001feffff] ACPI data
[ 0.000000] BIOS-e820: [mem 0x0000000001fef000-0x0000000001feffff] ACPI NVS
[ 0.000000] BIOS-e820: [mem 0x0000000001ff00000-0x0000000001fffffff] usable
```



```
[ 0.000000] BIOS-e820: [mem 0x00000000e0000000-0x00000000effffff] reserved
[ 0.000000] BIOS-e820: [mem 0x00000000fec00000-0x00000000fec0fff] reserved
[ 0.000000] BIOS-e820: [mem 0x00000000fee00000-0x00000000fee00fff] reserved
[ 0.000000] BIOS-e820: [mem 0x00000000fffe0000-0x00000000ffffffff] reserved
```

Сообщит, что найдена SMP-таблица (значит, наша машина является многопроцессорной или хотя бы содержит процессор с несколькими ядрами):

```
[ 0.000000] found SMP MP-table at [mem 0x000f6bf0-0x000f6bff]
mapped at [ffff8800000f6bf0]
```

Выведет информацию о процессоре:

```
[ 0.694470] smpboot: CPU0: Intel(R) Core(TM) i5-7200U CPU @ 2.50GHz
(family: 0x6, model: 0x8e, stepping: 0x9)
...
[ 0.715491] smp: Brought up 1 node, 2 CPUs
```

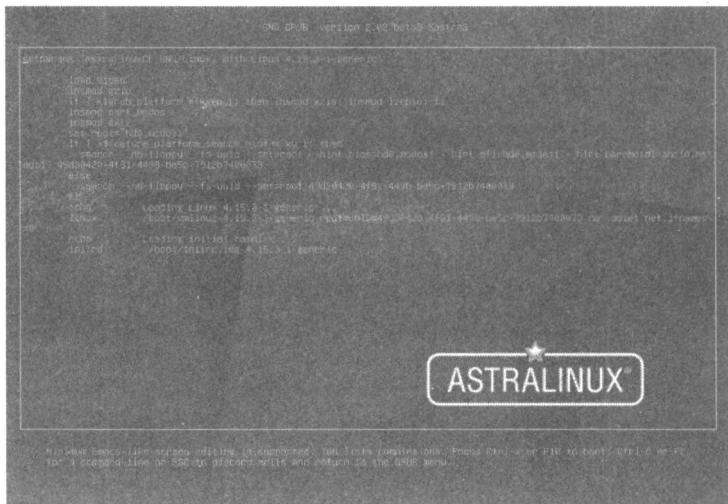
Частоту процессора и рейтинг в «попугаях»:

```
[ 0.000001] tsc: Detected 2711.997 MHz processor
[ 0.715491] smpboot: Total of 2 processors activated (10847.98 BogoMIPS)
```

BogoMIPS — это псевдорейтинг, который показывает, сколько миллионов пустых операций процессор может выполнить за секунду. Судить о производительности по этому рейтингу можно только косвенно.

Исследуйте вывод ядра самостоятельно. Уверен, в нем вы найдете много интересного, в том числе и о своей системе.

Поскольку ядро — это программа, то ей можно передать параметры, влияющие на поведение ядра. О параметрах ядра мы поговорим в





Ранее было показано, как сохранить конфигурацию GRUB2 (после перезагрузки внесенные вами изменения будут потеряны, поэтому их нужно внести в конфигурационные файлы GRUB2), а пока нажмите Ctrl+X для загрузки с отредактированной вами конфигурацией.

Самые часто используемые параметры ядра описаны в таблице 10.1.

Таблица 10.1. Часто используемые параметры ядра Linux

Параметр	Описание
<i>Параметры корневой файловой системы</i>	
<i>root=ysmpoysmo</i>	Указывает устройство, содержащее корневую файловую систему. Вы можете указать как короткое имя устройства (/dev/sda1), так и UUID устройства
<i>rootfstype=mun</i>	Тип корневой файловой системы. Обычно в нем нет необходимости, поскольку ядро само определяет тип ФС
<i>rootwait</i>	Ядро будет ждать появления устройства с корневой файловой системой. Вы можете держать ядро на жестком диске, а корневую файловую систему на USB-диске. С точки зрения производительности, может, и не очень хорошо, но зато неплохо с точки зрения безопасности, когда весь жесткий диск с данными можно быстро извлечь и удалить



<i>rootdelay=N</i>	Подождать N секунд перед монтированием файловой системы
<i>ro</i>	Монтирует корневую ФС в режиме «только чтение». После проверки утилитой fsck корневая ФС будет перемонтирована в режим rw
<i>rw</i>	Монтирует корневую ФС в режим «чтение/запись». Но тогда вам нужно отредактировать сценарии инициализации системы и удалить из них вызов утилиты fsck , поскольку проверять ФС в режиме rw этой утилитой нельзя
<i>Параметры, связанные с аппаратными средствами</i>	
<i>noscsi</i>	Отключает поддержку SCSI
<i>nousb</i>	Отключает поддержку USB
<i>norpmcia</i>	Отключает поддержку PCMCIA-карт
<i>noapic</i>	Отключает поддержку APIC. Полезен, если вы увидите при загрузке сообщение об ошибке, связанной с APIC
<i>nodmraid</i>	Отключает программные RAID-массивы, созданные на уровне BIOS
<i>mem=xxxxM</i>	Задаёт точный размер оперативной памяти, например, mem=4096M



<i>vga=режим</i>	Задаёт VGA-режим. Можно попросить ядро позволить вам выбрать один из режимов, для этого передайте ядру такой параметр — vga=ask
<i>Параметры управления питанием</i>	
<i>noapm</i>	Отключает расширенное управление питанием APM
<i>acpi=off</i>	Отключает ACPI (Advanced Configuration and Power Interface)
<i>pci=noacpi</i>	Отключить ACPI для PCI
<i>Общесистемные параметры</i>	
<i>init=программа</i>	Позволяет указать систему инициализации
<i>reboot=mun</i>	Указывает тип перезагрузки: <i>warm</i> (теплая) или <i>cold</i> (холодная)
<i>single</i>	Однопользовательский режим, который может использоваться для восстановления системы в случае сбоя
<i>quiet</i>	Отключает большинство сообщений ядра при загрузке системы



<i>boot_delay=N</i>	Задаёт задержку в N секунд перед выводом следующего сообщения ядра. Система будет загружаться очень медленно, лучше дождаться загрузки, выполнить команду <i>dmesg</i> и в спокойной обстановке прочитать все сообщения
---------------------	---

10.3. Обновление ядра до версии 5.7

Любой Linux-пользователь может скачать исходный код ядра и откомпилировать его. Должен отметить, что в последнее время компиляция ядра используется очень и очень редко. Хотя бы потому, что в репозиториях современных дистрибутивов уже есть образы ядра, в которых активированы те или иные функции. Чаще всего переходят на более новую версию ядра для обеспечения поддержки тех или иных устройств, повышения производительности системы или же для устранения каких-либо ошибок, например, в версии 5.4 драйвер звуковой карты может работать как-то не так, зато, например, в версии 5.7 все будет хорошо. Дабы не дожидаться, пока выйдет новая версия дистрибутива (а ждать придется как минимум полгода), то проще установить новую версию ядра самостоятельно.

Это можно сделать, как уже было отмечено, **двумя способами**.

1. Первый — **компиляция ядра из исходного кода**. Метод так себе:

- Вам нужно установить тонны программного обеспечения, необходимого для разработчиков. Это ПО понадобится вам только один раз, потом его придется или удалять, или оно будет просто занимать много места на жестком диске. А если у вас небольшой SSD-диск, то проблема свободного места еще более актуальна для вас.



- Процесс компиляции требует наличия большого количества свободного места на диске, которого у вас может и не быть.
- Процесс компиляции требовательный к ресурсам и занимает много времени. В принципе, можно оставить компьютер на ночь и утром получить готовое ядро. Здесь уже на ваше усмотрение.

2. Второй способ заключается в **установке уже откомпилированного ядра**. Все, что вам нужно сделать, — это скачать и установить несколько deb-пакетов. Способ, который подойдет большинству пользователей — он быстрый, не требует много места на диске, а результат будет тем же.

Рассмотрим, как установить ядро 5.7.0 в Ubuntu и других Debian-ориентированных системах (Mint, Astra Linux и др).

Существует две основных ветки ядра — *generic* (для всех систем) и *lowlatency* (для систем, где нужна низкая задержка, например, для записи аудио, для работы с видео).

Вам нужно загрузить следующие пакеты:

linux-headers-5.7.0-xxxxxx_all.deb

linux-headers-5.7.0-xxx-generic(/lowlatency)_xxx_amd64.deb

linux-modules-5.7.0-xxx-generic(/lowlatency)_xxx_amd64.deb

linux-image-xxx-5.7.0-xxx-generic(/lowlatency)_xxx_amd64.deb

В именах пакетов исправьте *generic* на *lowlatency*, если вам это нужно. Команды для загрузки будут такими:



```
cd /tmp/
```

```
wget -c https://kernel.ubuntu.com/~kernel-ppa/mainline/v5.7/linux-headers-5.7.0-050700_5.7.0-050700.202005312130_all.deb
```

```
wget -c https://kernel.ubuntu.com/~kernel-ppa/mainline/v5.7/linux-headers-5.7.0-050700-generic_5.7.0-050700.202005312130_amd64.deb
```

```
wget -c https://kernel.ubuntu.com/~kernel-ppa/mainline/v5.7/linux-image-unsigned-5.7.0-050700-generic_5.7.0-050700.202005312130_amd64.deb
```

```
wget -c https://kernel.ubuntu.com/~kernel-ppa/mainline/v5.7/linux-modules-5.7.0-050700-generic_5.7.0-050700.202005312130_amd64.deb
```

Мы загружаем все эти пакеты во временный каталог. После чего нужно их установить:

```
sudo dpkg -i *.deb
```

После установки загруженные deb-файлы можно удалить для экономии места:

```
rm *.deb
```

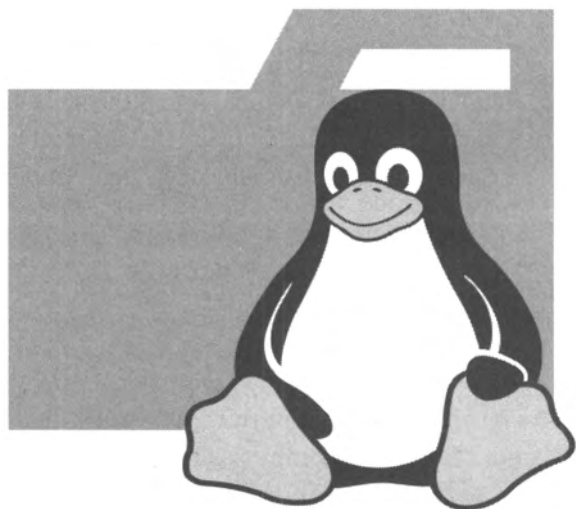
Как только пакеты будут установлены, перезагрузите систему. При перезагрузке выберите новую версию ядра в меню загрузчика GRUB2.

Если что-то пошло не так, удалить версию 5.7.0 можно командой:

```
sudo dpkg --purge linux-image-unsigned-5.7.0-050700-generic
```

ГЛАВА 11.

МАРШРУТИЗАЦИЯ И НАСТРОЙКА БРАНДМАУЭРА





11.1. Просмотр таблицы маршрутизации

Заканчивая наш небольшой справочник, хотелось бы коснуться темы маршрутизации и брандмауэра. Отправляемые данные, как вы знаете, не отправляются целиком, они разбиваются на меньшие части — *пакеты*.

Маршрутизация — это процесс перенаправления пакета по различным сетям вплоть до места назначения.

В сетях TCP/IP информация маршрутизации хранится в виде таблицы маршрутизации. Таблица маршрутизации содержит ряд простых правил. Например, если пакет отправляется в **сеть 1**, он должен быть отправлен на **маршрутизатор M1**, если пакет отправляется в **сеть 2**, то его нужно отправить на **маршрутизатор M2**. Пакет, отправляемый



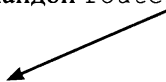
на любую другую сеть (не 1 и не 1), нужно отправить на **маршрутизатор М** (шлюз по умолчанию). Получив пакеты, маршрутизаторы М1, М1 и М сами знают, что с ними делать. Возможно, передать дальше другому маршрутизатору, а может, отправить узлу-получателю пакета. Все зависит от такой же таблицы маршрутизации, но уже на тех маршрутизаторах. Что будут делать маршрутизаторы с полученными пакетами — это их дело, наше дело — доставить пакеты до этих маршрутизаторов.

Таблица маршрутизации ядра Linux хранит данные о маршрутизации. Каждая строка в этой таблице содержит несколько параметров — адрес сети назначения, маску сети, флаги, интерфейс и т.д.

Ядро при отправке пакета исследует таблицу маршрутизации: оно определяет, в какую сеть направляется пакет — если она есть в таблице маршрутизации, то пакет отправляется через заданный в таблице интерфейс. Если нужной сети в таблице нет, пакет отправляется или на шлюз по умолчанию, или же (если он не задан) отправителю пакета передается ICMP-сообщение «Network Unreachable» (сеть недоступна).

Просмотреть таблицу маршрутизации ядра можно или командой `netstat -rn`, или командой `route`:

```
# netstat -rn
# route
```



Вывод команд немного отличается, как видно из рис. 11.1. Здесь видно, что шлюз по умолчанию — 192.168.52.2. По сути, это самая простая таблица маршрутизации, какая только может быть.



```

root@localhost ~# netstat -rn
Kernel IP routing table
Destination        Gateway           Genmask          Flags   MSS Window  irtt Iface
0.0.0.0            192.168.52.2     0.0.0.0          UG        0  0           0 ens33
192.168.52.0       0.0.0.0          255.255.255.0    U           0  0           0 ens33
root@localhost ~# route
Kernel IP routing table
Destination        Gateway           Genmask          Flags Metric Ref    Use Iface
default            gateway          0.0.0.0          UG        100    0     0 ens33
192.168.52.0       0.0.0.0          255.255.255.0    U        100    0     0 ens33
root@localhost ~#

```

Рис. 11.1. Команды `netstat -rn` и `route`

Разберемся, что содержится в столбцах таблицы маршрутизации:

- **Столбец Destination** хранит адрес сети назначения, Gateway — шлюз (маршрутизатор), которому нужно отправить пакеты, чтобы они достигли сети из колонки Destination.
- **Столбец Genmask** содержит маску сети, а Flags — флаги маршрута. Флаги могут быть следующими:
 - » U — маршрут активен;
 - » H — маршрут для хоста (H – host), а не для сети;
 - » G — флаг шлюза (G – gateway);
 - » D — динамический маршрут, который был установлен демоном маршрутизации;
 - » M — маршрут, который был модифицирован демоном маршрутизации;
 - » C — запись кэширована;
 - » ! — запрещенный маршрут.
- **Колонка MSS** (Maximum Segment Size) содержит значение MSS — максимальный размер сегмента для TCP-соединений по этому маршруту.



- **Столбец Window** показывает размер окна по умолчанию для TCP-соединений по этому маршруту.
- **Столбец irtt** — это начальное время RTT. В большинстве сетей время RTT не нужно изменять, но в некоторых медленных сетях время RTT можно увеличить, чтобы избежать лишних повторений пакетов. Система отправляет пакет и ждет некоторое время (RTT) от получателя подтверждения получения. Если подтверждения получения не было, тогда система отправляет пакет еще раз. В медленных сетях подтверждение получения может не успеть дойти до отправителя пакета, поэтому RTT увеличивают (это можно сделать с помощью команды *route*).
- **Столбец Iface** задает интерфейс, используемый для отправки пакета. В выводе команды *route* вместо столбцов MSS и Windows есть колонки Metric и Ref. Первая содержит метрику, то есть расстояние до маршрутизатора в хопах (переходах): один хоп — это один маршрутизатор.
- **Столбец Ref** — это количество ссылок на маршрут. Ядром Linux этот параметр не учитывается.

11.2. Изменение и сохранение таблицы маршрутизации

Для редактирования таблицы маршрутизации используется команда *route*. Записи в таблице маршрутизации бывают *статическими* (добавляются командой *route*) или *динамическими* (добавляются по мере работы системы демоном маршрутизации).



Вот как можно добавить маршрут по умолчанию командой *route*:

```
# route add default gw 192.168.1.1 eth0
```

Думаю, эта команда понятна: 192.168.1.1 — это новый шлюз по умолчанию, а пакеты к нему будут отправляться через интерфейс eth0.

При перезагрузке таблица маршрутизации очищается, поэтому она формируется вручную, то есть командами *route*, а не каким-либо демоном маршрутизации, то есть эти команды нужно добавить в конфигурационные файлы сети или сценарии инициализации системы, чтобы внесенные вами изменения не были потеряны.

Шлюз по умолчанию можно сохранить в файле `/etc/sysconfig/networking-scripts/ifcfg-имя_интерфейса`. В листинге 11.1 приводится пример этого файла.

Листинг 11.1. Файл `/etc/sysconfig/networking-scripts/ifcfg-имя_интерфейса`

```
DEVICE=eth0
HWADDR=00:0C:29:E8:F0:C4
TYPE=Ethernet
ONBOOT=yes
NETMASK=255.255.255.0
IPADDR=192.168.1.101
GATEWAY=192.168.1.1
NETWORK=192.168.1.0
BROADCAST=192.168.1.255
BOOTPROTO=none
NM_CONTROLLED=no
```




Параметр `GATEWAY` позволяет указать IP-адрес шлюза. Но, как правило, нет необходимости редактировать файлы настройки сетевых интерфейсов обычных компьютеров, поскольку такая общая информация, как IP-адрес шлюза, задается DHCP-сервером примерно так:

```
subnet 192.168.1.0 netmask 255.255.255.0 {  
    # Список маршрутизаторов (через пробел)  
    option routers                192.168.1.1;  
    ...
```

Теперь рассмотрим общий формат вызова команды *route*:

```
# route [операция] [тип] адресат gw шлюз [метрика] [dev интерфейс]
```

Параметр *операция* может принимать значения *add* и *del*.

- **Первый** добавляет маршрут, а второй — удаляет.
- **Второй** параметр (*тип*) необязательный — он позволяет задать тип маршрута:
 - » *default* (маршрут по умолчанию);
 - » *-net* (маршрут к сети);
 - » *-host* (маршрут к узлу).
- **Третий** параметр, *адресат*, содержит адрес сети, если вы зададите маршрут к сети, или адрес узла, если добавляется маршрут к узлу. Если вы зададите маршрут по умолчанию, то этот параметр вообще не нужно указывать.

Параметр *шлюз* задает IP-адрес шлюза. Можно также указать и доменное имя, но обычно указывается IP-адрес. Параметр *метрика* задает число переходов (маршрутизаторов) на пути к адресату. Параметр *dev* нужно указывать, если в системе установлено несколько сетевых интерфейсов и нужно указать, через какой именно сетевой



интерфейс нужно отправить пакеты. Оба последних параметра не являются обязательными.

Рассмотрим несколько примеров использования команды *route*:

```
# route add -net 192.168.16.0 netmask 255.255.255.0 dev ens33
# route add -net 192.168.16.0 netmask 255.255.255.0 gw 192.168.16.1
# route add -net 10.100.0.0 netmask 255.0.0.0 reject
# route del 10.100.0.0
```

Первая команда добавляет маршрут к сети 192.168.16.0 через устройство ens33. Как видите, мы не указываем IP-адрес шлюза, а просто указываем имя интерфейса — все пакеты, адресованные сети 192.168.16.0, будут отправлены через интерфейс ens33.

Вторая команда добавляет маршрут к сети 192.168.16.0 через шлюз 192.168.16.1. Все пакеты, адресованные этой сети, будут отправлены маршрутизатору с IP-адресом 192.168.16.1. В этом случае сетевой интерфейс указывать не обязательно.

Третья команда задает запрещающий маршрут. Отправка пакетов в сеть 10.100.0.0 запрещена (параметр **reject**). Последняя команда удаляет ранее заданный запрещающий маршрут.

Удаление маршрутов в Linux заслуживает отдельного разговора. Команда удаления маршрута выглядит так:

```
# route del IP-адрес
```



В Linux нет параметра *-f* (как в FreeBSD), который позволяет удалить сразу все маршруты (то есть очистить таблицу маршрутизации), поэтому вам придется ввести ряд команд *route*.



Как уже было отмечено, таблица маршрутизации очищается при перезагрузке. Чтобы этого не произошло, маршруты нужно определить в файлах конфигурации сети.

В Debian / Astra Linux / старых версиях Ubuntu настройка статических маршрутов производится в файле `/etc/network/interfaces`. Просто добавьте в секцию настройки интерфейса команду *ip* и укажите команду *route*, которую нужно выполнить. Допустим, для настройки маршрутов вы ввели три команды:

```
route add -net 192.168.1.0 netmask 255.255.255.0 gw 192.168.17.254 eth0
route add -net 192.168.14.0 netmask 255.255.255.0 gw 192.168.17.254 eth0
route add -net 192.168.22.0 netmask 255.255.255.0 gw 192.168.17.254 eth0
```

Тогда в файл `/etc/network/interfaces` нужно добавить следующие строки:

```
up route add -net 192.168.1.0 netmask 255.255.255.0 gw 192.168.17.254 eth0
up route add -net 192.168.12.0 netmask 255.255.255.0 gw 192.168.17.254 eth0
up route add -net 192.168.21.0 netmask 255.255.255.0 gw 192.168.17.254 eth0
```

В этом примере пакеты ко всем трем сетям направляются на шлюз 192.168.17.254, а он уже сам решает, что с ними делать. Прежде чем мы рассмотрим настройку брандмауэра, нужно разобраться, как превратить компьютер в шлюз. Для этого нужно разрешить пересылку пакетов протокола IPv4 (IPv4 forwarding). Для этого добавьте значение 1 в файл `/proc/sys/net/ipv4/ip_forward`:

```
# echo 1 > /proc/sys/net/ipv4/ip_forward
```

Конечно, после перезапуска это значение будет потеряно. Чтобы его сохранить, добавьте в файл `/etc/sysctl.conf` следующую строку:

```
net.ipv4.ip_forward=0
```



Итак, ваш компьютер теперь превращен в шлюз. Но сам по себе перенаправлять пакеты он не будет. Нужно задать ряд правил перенаправления пакетов. Для этого мы будем использовать брандмауэр **iptables**, который имеется во всех современных дистрибутивах. Именно о нем и пойдет речь в следующем разделе. Забегая наперед, скажу, что **iptables** можно использовать не только для настройки шлюза, но и просто для защиты сервера или рабочей станции от нежелательных подключений.

В новых версиях Ubuntu (начиная с версии 18.04) настройка маршрутов осуществляется через файл `/etc/netplan/01-netcfg.yaml`. Откройте этот файл:



```
sudo nano /etc/netplan/01-netcfg.yaml
```

В нем вы найдете конфигурацию по умолчанию, которая может быть примерно такой:

```
# This file describes the network interfaces available on your system
# For more information, see netplan(5).
network:
  version: 2
  renderer: networkd
  ethernets:
    enol:
      dhcp4: yes
```

Добавьте следующие строки:

```
routes:
- to: 192.168.44.0/24
  via: 192.168.0.1
```



Данная конфигурация означает, что маршрут к сети 192.168.44.0/24 (маска 255.255.255.0) будет проходить через маршрутизатор 192.168.0.1. Полная конфигурация будет выглядеть так:

```
# This file describes the network interfaces available on your system
# For more information, see netplan(5).
network:
  version: 2
  renderer: networkd
  ethernets:
    eno1:
      dhcp4: true
      routes:
        - to: 192.168.44.0/24
          via: 192.168.0.1
```

Обратите внимание, что YAML-файл очень требователен к отступам и разметке. Убедитесь, что оператор «routes» находится на расстоянии двух пробелов от имени интерфейса (в нашем случае eno1), к которому вы применяете маршрут.

Сохраните изменения в файле и примените их посредством команды:

```
sudo netplan apply
```

Проверим, все ли хорошо:

```
ip route show
```

Убедитесь, что вы видите эту строку в выводе предыдущей команды:

```
192.168.44.0/24 via 192.168.0.1 dev eno1 proto static
```



Если вы не видите данный статический маршрут в выводе, значит, что-то не так с вашей конфигурацией. Вернитесь к YAML-файлу и проверьте разметку/отступы. Вы также можете проверить конфигурацию командой:

```
sudo netplan try
```

11.3. Настройка брандмауэра *iptables*

Брандмауэр **iptables** на данный момент считается устаревшим, но до сих пор используется даже в самых новых версиях дистрибутивах. От него пытаются отказаться последние лет 5, но до сих пор этого не произошло и, на наш взгляд, в ближайшие лет 5 он будет все еще актуален. В следующем разделе мы рассмотрим современный брандмауэр **ufw**, который, судя по всему, вытеснит **iptables** в будущем, хотя на данный момент он даже не устанавливается по умолчанию.

11.3.1. Преобразование сетевого адреса

Что такое брандмауэр, я надеюсь, объяснять не нужно. А вот о преобразовании сетевого адреса (NAT, Network Address Translation) поговорить стоит, чтобы не было лишних вопросов. Пространство IP-адресов не безразмерное. Рано или поздно IPv4-адреса закончатся. Именно поэтому и был разработан протокол IPv6. Учитывая тенденции роста Интернета, думаю, что переход на IPv6 состоится в ближайшие годы, но пока будем говорить исключительно о IPv4, поскольку новый протокол IPv6 пока практически не используется.



Если бы реальные IP-адреса раздавались бы всем желающим, они бы уже давно закончились. Поэтому обычно при подключении к Интернету клиенту выдается всего один IP-адрес, даже если у вас целая организация. Этот один реальный IP-адрес, как правило, используется на шлюзе. А во внутренней сети используются локальные IP-адреса: 10.*.* (сеть класса А), 172.16.*.*–172.31.*.* (класс В) и 192.168.*.* (сеть класса С). Эти IP-адреса не могут пройти через маршрутизатор Интернета — как только маршрутизатор увидит локальный IP-адрес, пакет сразу же будет отброшен. Но зато такие IP-адреса вполне подойдут для использования в локальных сетях. Данные IP-адреса уникальны только в пределах вашей организации. В соседней организации могут использоваться такие IP-адреса, как и у вас, но поскольку между компьютерами организаций нет взаимодействия, в этом нет ничего страшного.

Когда компьютер с локальным IP-адресом 192.168.1.102 отправляет пакет компьютеру, обладающему реальным IP-адресом, например, веб-серверу `www.example.com`, наш шлюз должен выполнить NAT. То есть он должен перезаписать локальный IP-адрес так, чтобы он выглядел как реальный. Поскольку в нашем распоряжении только один реальный IP-адрес (IP-адрес шлюза), то веб-сервер `www.example.com` будет «думать», что запрос пришел от нашего шлюза, и он ничего не будет подозревать о компьютерах, которые находятся за шлюзом. Веб-сервер обработает запрос и отправит пакет. Наш шлюз получит этот пакет (с ответом) и отправит его компьютеру 192.168.1.102, перезаписав поле отправителя так, что компьютер 192.168.1.102 будет «думать», что ответ пришел непосредственно от `www.example.com`.

11.3.2. Цепочки и правила

При настройке брандмауэра **iptables** очень важно разобраться с цепочками и правилами. Ведь основная задача брандмауэра — это



фильтрация пакетов, проходящих через сетевой интерфейс. Когда брандмауэр получает пакет, он анализирует его и затем принимает решение — принять его или уничтожить. Брандмауэр может выполнять и более сложные действия, но обычно достаточно этих двух.

Обработка пакета заключается в его прохождении по цепочке правил. Каждое правило состоит из условия и действия. Если пакет соответствует условию, то выполняется указанное действие. Действие также называется целью. Если пакет не соответствует условию правила, то он передается следующему правилу. Если же пакет не соответствует ни одному из правил цепочки, выполняется действие по умолчанию.

Цепочки правил существует не сами по себе, а собираются в таблицы.

- Таблица **filter** является основной таблицей, отвечает за фильтрацию пакетов, в ней содержатся правила фильтрации пакетов.
- Таблица **nat** используется при преобразовании сетевого адреса.
- Есть еще одна таблица — **mangle**, которая используется, если нужно произвести специальные действия над пакетом, отличные от фильтрации и NAT.

В состав каждой таблицы входят три цепочки: INPUT, OUTPUT и FORWARD. Первая используется для входящих пакетов, вторая — для исходящих, а третья для пересылаемых пакетов. При желании вы можете создать собственную таблицу, но в ней все равно будут цепочки INPUT, OUTPUT и FORWARD.

Теперь поговорим о действиях над пакетом. Действие ACCEPT принимает пакет, DROP — уничтожает пакет. Действие MASQUERADE позволяет скрыть IP-адрес пакета. Если же в качестве действия указано имя цепочки, то пакет будет отправлен для обработки в указанную цепочку.



Рассмотрим процесс обработки входящего пакета. Сначала пакет поступает в цепочку PREROUTING таблицы **mangle**. После чего, если он не был уничтожен правилами таблицы **mangle**, он попадает в цепочку PREROUTING таблицы **nat**. Здесь правила проверяют, нужно ли модифицировать назначение пакета или нет. После этого пакет направляется или в цепочку FORWARD (если его нужно передать дальше — другому компьютеру), или в цепочку INPUT (если пакет адресован этому компьютеру).

Если пакет был адресован этому компьютеру, то он передается правилам цепочки INPUT таблицы **mangle** и **filter**. Если эти правила не уничтожили пакет, то он отправляется приложению, например, веб-серверу. Приложение получает данные, обрабатывает их и отправляет ответ. Ответ приложения преобразуется в пакет, который сначала обрабатывается цепочкой OUTPUT таблиц **mangle**, **nat** и **filter**, а затем отправляется в цепочку POSTROUTING и обрабатывается правилами таблиц **mangle** и **nat**. Если после всего этого пакет еще жив, он становится исходящим пакетом и отправляется в сеть.

11.3.3. Команда *iptables*

Для управления правилами брандмауэра используется команда *iptables*. У этой команды очень и очень много различных параметров. В этой книге не будет полного руководства по **iptables**, потому что есть документация (man iptables), доступная каждому пользователю. Вместо переписывания руководства другими словами мы остановимся на практическом применении *iptables*. Сначала рассмотрим часто используемые параметры (чтобы вы понимали, что происходит), а затем настроим шлюз для небольшой организации.

Чтобы добавить правило в цепочку, нужно использовать параметр **-A**:

```
# iptables -A <цепочка> <правило>
```



По умолчанию правило будет добавлено в таблицу **filter**. Если нужно указать другую таблицу, то для этого используется параметр *-t*:

```
# iptables -t <таблица> -A <цепочка> <правило>
```

Параметр *-P* позволяет задать действие по умолчанию:

```
# iptables -P INPUT DROP
# iptables -P FORWARD ACCEPT
# iptables -P OUTPUT DROP
```

В таблице 11.1 указываются возможные действия, которые вы можете использовать с **iptables**.

Таблица 11.1. Возможные действия над пакетами

Действие	Описание
<i>ACCEPT</i>	Принимает пакет и передает его дальше — в следующую цепочку
<i>DROP</i>	Уничтожает пакет, как будто бы его никогда не было
<i>REJECT</i>	Уничтожает пакет, а отправителю пакета сообщается об этом с помощью ICMP-сообщения. Параметр <i>--reject-with</i> позволяет уточнить тип ICMP-сообщения: icmp-host-unreachable — узел недоступен; icmp-net-unreachable — сеть недоступна; icmp-port-unreachable — порт недоступен; icmp-proto-unreachable — протокол недоступен



<i>LOG</i>	Протоколирует информацию о пакете в протокол. Полезно из соображений отладки, когда вы настраиваете шлюз
<i>RETURN</i>	Возвращает пакет в цепочку, откуда он прибыл. Использовать не рекомендуется, поскольку возможны зацикливания — вы можете легко попасть в бесконечный цикл, если будете использовать это действие
<i>SNAT</i>	Выполняет подмену IP-адреса источника (Source NAT, SNAT). Данное действие используется в цепочках POSTROUTING и OUTPUT таблицы nat
<i>DNAT</i>	Выполняет подмену IP-адреса получателя (Destination NAT, DNAT). Поддерживается только в цепочке POSTROUTING таблицы nat
<i>MASQUERADE</i>	Используется в таблице POSTROUTING таблицы nat . Чем-то похоже на SNAT, но используется при работе с динамическими IP-адресами, когда возможна «потеря» интерфейса при изменении IP-адреса

Прежде чем мы начнем создавать наш собственный шлюз, нужно рассмотреть параметры, касающиеся фильтрации пакетов (табл. 11.2).



Таблица 11.2. Параметры, касающиеся фильтрации пакетов

Параметр	Описание
<i>--source</i>	Указывает источник пакета. Вы можете указывать как доменное имя, так и IP-адрес или даже целую подсеть, например, 192.168.5.0/255.255.255.0
<i>--destination</i>	Указывает получателя пакета. Аналогично, можно указывать как доменное имя, так и IP-адрес
<i>--protocol (-p)</i>	Позволяет указать протокол. Вы можете использовать идентификаторы, определенные в файле /etc/protocols
<i>--source-port (--sport)</i>	Указывает порт отправителя. Опцию можно использовать вместе с параметром <i>-p</i>
<i>--destination-port (--dport)</i>	Указывает порт получателя. Можно использовать вместе с параметром <i>-p</i>
<i>--state</i>	Позволяет указать состояние пакета. Фильтр по состоянию возможен только при загрузке модуля state (<i>-m state</i>). Возможны следующие состояния пакета: NEW (новое соединение), ESTABLISHED (установленное соединение), RELATED (связанные с соединением пакеты), INVALID (неопознанные пакеты)
<i>--in-interface (-i)</i>	Указывает входящий интерфейс



<code>--out-interface (-o)</code>	Указывает исходящий интерфейс
-----------------------------------	-------------------------------

В таблице 11.2 указаны далеко не все параметры, но для организации собственного шлюза представленных параметров будет более чем достаточно.

11.3.4. Практический пример

Сейчас рассмотрим практический пример — настройку шлюза небольшой сети с нуля на базе дистрибутива Debian. У нас есть следующие вводные данные:

- Внешняя сеть — интерфейс `eth0`, реальный IP-адрес 88.99.88.99 (IP-адрес приведен для примера).
- Внутренняя сеть — интерфейс `eth1`, IP-адрес сети 192.168.1.0/24 (у вас, скорее всего, будет другой адрес).

Простым «перебросом» пакетов мы не ограничимся, а постараемся решить попутные задачи, а именно:

- Запретить доступ сотрудников к некоторым сайтам. Администрации доступ к этим сайтам будет разрешен;
- Запретить ICQ для сотрудников, но не для администрации;
- Запретить некоторые нежелательные приложения;
- Открыть доступ к порту 80 извне для доступа к будущему веб-сайту. Все, что вам останется сделать, — это установить `nginx` (или `Apache`), и ваш сайт будет виден из Интернета;



- Открыть доступ к шлюзу извне для его администрирования по SSH;
- Открыть порты 25 и 110, необходимые для работы будущего почтового сервера;
- Администратору (его IP-адрес 192.168.1.10) открываем все, что ему будет нужно.

Создайте каталог `/etc/firewall`, в котором мы будем хранить все необходимое для организации нашего шлюза. Далее создайте в нем два файла — `admins.txt` и `black.txt`. В первый нужно внести IP-адреса администрации (по одному IP-адресу в одной строке) — им будут доступны все сайты. Это могут быть сотрудники IT-отдела, директор, его зам и т.д. — в общем, все вышестоящее начальство. В файл `black.txt` нужно внести сайты, доступ к которым нужно ограничить (тоже по одному сайту в одной строке).

```
# mkdir /etc/firewall
# touch /etc/firewall/admins.txt
# touch /etc/firewall/black.txt
```

После этого можно приступить, собственно, к настройке шлюза. Первым делом нам нужно отключить Network Manager и настроить наши интерфейсы статически. На сервере (шлюзе) Network Manager не нужен, а сетевые интерфейсы можно легко настроить с помощью `/etc/network/interfaces`.

Сначала введите команду:

```
# runlevel
```

Вы узнаете текущий уровень запуска:



```
# runlevel
N 2
```

Далее переходим в каталог `/etc/rcX.d`, где `X` — это уровень запуска, и удаляем ссылку на `network-manager`. Можно также использовать команду:



```
# update-rc.d -f network-manager remove
```

Далее остановите Network Manager:

```
# /etc/init.d/network-manager stop
```

После этого отредактируйте файл `/etc/network/interfaces` и сконфигурируйте сетевые интерфейсы (лист. 11.2).

Листинг 11.2. Файл `/etc/network/interfaces`

```
# Локальный интерфейс lo
auto lo
iface lo inet loopback

# Внешняя сеть
auto eth0
iface eth0 inet static
address 88.99.88.99
netmask 255.255.255.0
network 88.99.88.99
dns-nameservers 8.8.8.8

# Внутренняя сеть
auto eth1
iface eth1 inet static
```



```
address 192.168.1.1
netmask 255.255.255.0
network 192.168.1.0

broadcast 192.168.1.255
```

Перезапустим сеть:

```
# service networking restart
```

Теперь создайте каталог `/etc/firewall` и поместите туда файл `firewall.sh`, который будет содержать команды, превращающие наш обычный компьютер в шлюз:

```
# touch /etc/firewall/firewall.sh
# chmod +x /etc/firewall/firewall.sh
# nano /etc/firewall/firewall.sh
```

Код файла `firewall.sh` приведен в листинге 11.3.

Листинг 11.3. Файл `firewall.sh`

```
#!/bin/bash

# Определяем некоторые переменные, чтобы облегчить редактирование
# конфигурации в будущем
# Внешний IP-адрес и внешний интерфейс
WAN_IP=88.99.88.99
WAN_IFACE=eth0

# Внутренний адрес сервера и внутренний интерфейс
LAN_IP=192.168.1.1
LAN_IFACE=eth1
```




```
# Внутренняя сеть
LOCAL_NET=192.168.1.0/24

# loopback
LO_IFACE=lo
LO_IP=127.0.0.1

# Путь к iptables
ip=/sbin/iptables

# Черный список
blacklist=( $(cat "/etc/firewall/black.txt") )
admins=( $(cat "/etc/firewall/admins.txt") )

# Очищаем все таблицы iptables
$ip -F -t filter
$ip -F -t nat
$ip -F -t mangle
$ip -F

# Правила по умолчанию: запрещаем все, что явно не разрешено
$ip -P INPUT DROP
$ip -P OUTPUT DROP
$ip -P FORWARD DROP

# Превращаем компьютер в шлюз, на всякий случай, если мы забыли сделать
# это раньше
echo 1 > /proc/sys/net/ipv4/ip_forward

# Включаем NAT, чтобы локальные узлы могли получать доступ к Интернету
$ip -t nat -A POSTROUTING -o $WAN_IFACE1 -s $LOCAL_NET ! -d $LOCAL_NET
-j SNAT --to-source $WAN_IP1

# Отбрасываем INVALID-пакеты
```



```
$ip -A INPUT -m state --state INVALID -j DROP
$ip -A FORWARD -m state --state INVALID -j DROP

# Разрешаем трафик через loopback
$ip -A INPUT -p all -i $LO_IFACE -j ACCEPT
$ip -A OUTPUT -p all -o $LO_IFACE -j ACCEPT

# Разрешаем трафик через внутренний адаптер
# Между сервером (шлюзом) и локальной сетью разрешаем все
$ip -A INPUT -p all -i $LAN_IFACE -s $LOCAL_NET --match state --state NEW,
ESTABLISHED -j ACCEPT

# Разрешаем исходящие новые и уже установленные соединения
# во внутреннюю сеть с адаптера локальной сети

$ip -A OUTPUT -p all -o $LAN_IFACE -d $LOCAL_NET --match state --state NEW,
ESTABLISHED -j ACCEPT

# Разрешаем новые и уже установленные соединения извне (с внешней сети)
# к портам 80 (веб-сервер) и 22 (ssh):

$ip -A INPUT -p tcp -i $WAN_IFACE1 -m multiport --dports
80,22,25,110 --match state --state NEW,ESTABLISHED -j ACCEPT
$ip -A OUTPUT -p tcp -o $WAN_IFACE1 -m multiport --sports
80,22,25,110 --match state --state ESTABLISHED,RELATED -j ACCEPT

# Разрешаем выход с сервера во внешнюю сеть, но только на определенные порты
# Разрешаем порты 80 (HTTP), 443 (SSL) и 53 (DNS)
$ip -A INPUT -i $WAN_IFACE1 -p tcp -m multiport --sports 80,53,443 -j ACCEPT
$ip -A OUTPUT -o $WAN_IFACE1 -p tcp -m multiport --dports 80,53,443 -j ACCEPT
$ip -A INPUT -i $WAN_IFACE1 -p udp -m multiport --sports 53 -j ACCEPT
$ip -A OUTPUT -o $WAN_IFACE1 -p udp -m multiport --dports 53 -j ACCEPT

# Открываем админу (192.168.1.10) доступ ко всему, что будет нужно
# Просмотрите список портов и откройте то, что вам будет нужно
# tcp
```



```
$ip -A FORWARD -p tcp -s 192.168.1.10 ! -d $LOCAL_NET -m multiport
--dports 80,53,443,22,25,110,5190 -j ACCEPT
$ip -A FORWARD -p tcp -d 192.168.1.10 ! -s $LOCAL_NET -m multiport
--sports 80,53,443,22,25,110,5190 -j ACCEPT

# udp
$ip -A FORWARD -p udp -s 192.168.1.10 ! -d $LOCAL_NET -m
multiport --dports 53 -j ACCEPT
$ip -A FORWARD -p udp -d 192.168.1.10 ! -s $LOCAL_NET -m
multiport --sports 53 -j ACCEPT

# Разрешаем ICQ только администраторам
i=0
for i in "${admins[@]}"
do
    $ip -A FORWARD -p tcp -d $i --sport 5190 -j ACCEPT
    $ip -A FORWARD -p tcp -s $i --dport 5190 -j ACCEPT
done

# Разрешаем избранным (список admins) доступ к сайтам из черного списка
j=0
for j in "${blacklist[@]}"
do
    i=0
    for i in "${admins[@]}"
    do
        $ip -A FORWARD -d $i -s $j -j ACCEPT
    done
done

# Всем остальным запрещаем доступ к сайтам из списка blacklist
i=0
for i in "${blacklist[@]}"
do
    $ip -A FORWARD -s $i -j DROP
```



```
done
```

```
# Разрешаем транзит некоторых пакетов (80, 443 и 53)
```

```
$ip -A FORWARD -p tcp -s $LOCAL_NET ! -d $LOCAL_NET -m  
multiport --dports 80,53,443 -j ACCEPT
```

```
$ip -A FORWARD -p tcp -d $LOCAL_NET ! -s $LOCAL_NET -m  
multiport --sports 80,53,443 -j ACCEPT
```

```
$ip -A FORWARD -p udp -s $LOCAL_NET ! -d $LOCAL_NET -m  
multiport --dports 53 -j ACCEPT
```

```
$ip -A FORWARD -p udp -d $LOCAL_NET ! -s $LOCAL_NET -m  
multiport --sports 53 -j ACCEPT
```

После этого нужно обеспечить автоматический запуск нашего сценария `/etc/firewall/firewall.sh`.

11.4. Настройка брандмауэра *ufw*

Традиционно в качестве брандмауэра (фильтра пакетов) в Ubuntu используется **iptables**, но поскольку Ubuntu позиционируется как простой дистрибутив, то и оболочка для **iptables** была разработана соответствующая — UTF (Uncomplicated Firewall) — несложный файрвол.

11.4.1. Проверяем состояние брандмауэра

Первым делом нужно убедиться, что пакет **ufw** вообще установлен, или установить его, если это не так:



```
sudo apt install ufw
```

Проблем с установкой у вас не должно возникнуть, все довольно стандартно. Итак, будем считать что уже установлена последняя версия пакета **ufw**.

Теперь посмотрим состояние брандмауэра:

```
# ufw status verbose
```

По умолчанию фильтр пакетов выключен, поэтому вы получите сообщение:



```
Состояние: неактивен
```

Не нужно спешить включать файрвол: сначала его нужно настроить. Ведь если порт 22 окажется по умолчанию недоступен, то вы потеряете доступ к своему серверу, если администрируете его удаленно. Об этом нужно помнить!

11.4.2. Базовая настройка

По умолчанию брандмауэр запрещает все входящие соединения и разрешает все исходящие. Такая политика идеальна с точки зрения безопасности (далее вы поймете почему) — ведь если кто-то (и вы в том числе) захочет к нему подключиться, то у него это не получится. В то же время приложения на сервере смогут создавать исходящие соединения.

Рассмотрим две команды:

```
ufw default deny incoming
ufw default allow outgoing
```



Данные два правила как раз и задают политику по умолчанию — запрещаются все входящие соединения и разрешаются все исходящие.

Итак, все входящие соединения запрещены. Чтобы к серверу можно было «достучаться» по определенному порту, его нужно сначала открыть. UFW хорош тем, что вам даже не нужно помнить номер порта — нужно знать только название сервиса. Например, вот как можно разрешить подключение по SSH:

```
ufw allow ssh
```

При этом UFW сам создаст правило для порта 22 — именно этот порт используется для SSH. Брандмауэр знает порты и имена всех распространенных служб (http, ssh, ftp, sftp и т.д.).

Однако если вы перенастроили **ssh** на нестандартный порт из соображений той же безопасности, нужно явно указать номер порта:

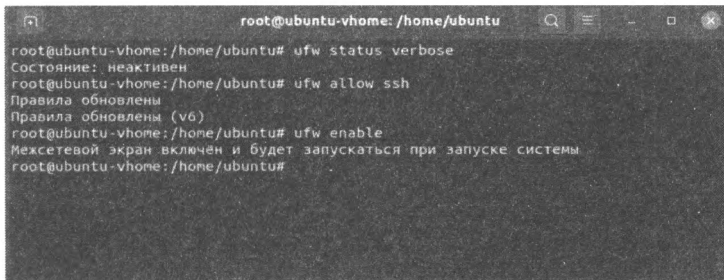
```
ufw allow 3333
```

После разрешения **ssh** (это главное, чтобы сейчас файрвол нам не разорвал соединение) можно включить **ufw** командой:

```
ufw enable
```

Вы увидите сообщение о том, что межсетевой экран включен и будет запускаться при загрузке системы.

Посмотрите на следующий скриншот (рис. 11.2).



```
root@ubuntu-vhome: /home/ubuntu
root@ubuntu-vhome:/home/ubuntu# ufw status verbose
Состояние: неактивен
root@ubuntu-vhome:/home/ubuntu# ufw allow ssh
Правила обновлены
Правила обновлены (v6)
root@ubuntu-vhome:/home/ubuntu# ufw enable
Мехсетевой экран включен и будет запускаться при запуске системы
root@ubuntu-vhome:/home/ubuntu#
```

Рис. 11.2. Процесс настройки ufw

Посмотрим, что произошло. Сначала мы разрешили **ssh**, на что получили ответ, что правила обновлены:

```
Правила обновлены
Правила обновлены (v6)
```

Затем мы включаем файрвол и получаем сообщение, что он активен и будет запускаться при загрузке системы.

На этом базовая настройка выполнена — **ssh** успешно работает, и мы можем приступить к дальнейшей настройке фильтра пакетов.

11.4.3. Создаем правила для других приложений

Теперь нужно разрешить работу других приложений. Как правило, нужно разрешить службу **http** (веб-сервер), **ftp** и постараться не забыть о **https** (что очень важно в последнее время):

```
ufw allow http
ufw allow https
ufw allow ftp
```



Сделать то же самое можно было бы и по номерам портов:

```
ufw allow 80
ufw allow 443
ufw allow 21
```

При желании можно разрешить целый диапазон портов, указав при этом транспортный протокол (UDP или TCP):

```
sudo ufw allow 2000:2200/tcp
sudo ufw allow 4000:4400/udp
```

11.4.4. Разрешаем IP-адреса

Ufw позволяет разрешить определенному IP-адресу доступ ко всем портам сервера, например:

```
ufw allow from 46.229.220.16
```

Если нужно разрешить доступ конкретному IP-адресу только к определенному порту, то делается это так:

```
ufw allow from 46.229.220.16 to any port 22
```

Здесь мы разрешаем не все подключения к **ssh**, а только подключения с IP-адреса 46.229.220.16.

Разрешить доступ целого диапазона IP-адресов (например, когда у администратора динамический IP), можно так:

```
ufw allow from 123.45.67.89/24 to any port 22
```




11.4.5. Запрещаем IP-адреса и службы

Запретить доступ с определенного IP-адреса можно так:

```
ufw deny from 123.45.67.89
```

При желании можно запретить все подключения к определенной службе:

```
ufw deny ftp
```

11.4.6. Удаление/сброс правил

Сбросить все правила можно командой:

```
ufw reset
```

Но убедитесь, что на момент ввода этой команды вы отключили фаервол, иначе вы потеряете доступ по **ssh**.

Удалить конкретное правило можно по номеру. Сначала введите следующую команду, чтобы узнать номер правила:

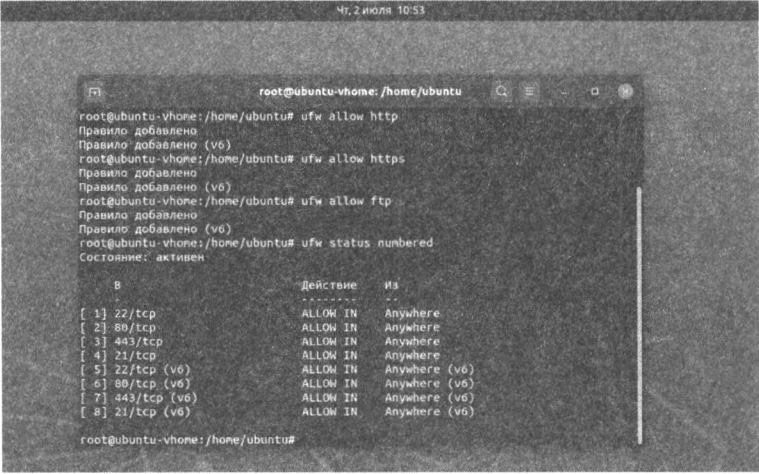
```
ufw status numbered
```

11.4.7. Отключение фаервола

Отключить **ufw** можно командой *ufw disable*. Отключение может понадобиться, если какие-то из правил работают неправильно и нужно



временно отключить фаервол, чтобы разрешить работу тех или иных сервисов.



```
Чт, 2 июля 10:53
root@ubuntu-vmh: /home/ubuntu
root@ubuntu-vmh: /home/ubuntu# ufw allow http
Правило добавлено
Правило добавлено (v6)
root@ubuntu-vmh: /home/ubuntu# ufw allow https
Правило добавлено
Правило добавлено (v6)
root@ubuntu-vmh: /home/ubuntu# ufw allow ftp
Правило добавлено
Правило добавлено (v6)
root@ubuntu-vmh: /home/ubuntu# ufw status numbered
Состояние: активен

      В      Действие  Из
      --      -
[ 1] 22/tcp      ALLOW IN    Anywhere
[ 2] 80/tcp      ALLOW IN    Anywhere
[ 3] 443/tcp      ALLOW IN    Anywhere
[ 4] 21/tcp      ALLOW IN    Anywhere
[ 5] 22/tcp (v6)  ALLOW IN    Anywhere (v6)
[ 6] 80/tcp (v6)  ALLOW IN    Anywhere (v6)
[ 7] 443/tcp (v6)  ALLOW IN    Anywhere (v6)
[ 8] 21/tcp (v6)  ALLOW IN    Anywhere (v6)

root@ubuntu-vmh: /home/ubuntu#
```

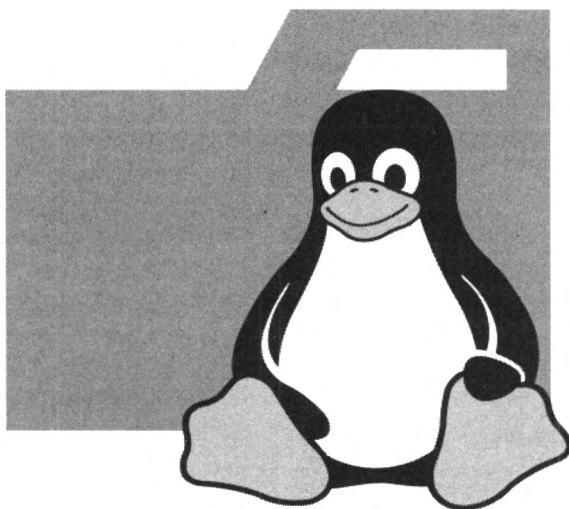
Рис. 11.3. Список правил

Если вы ранее использовали **iptables**, то наверняка заметили, что синтаксис **ufw** гораздо проще. Если же до этого вам не приходилось настраивать брандмауэр, то **ufw** — оптимальное решение, с которым не составит труда разобраться даже начинающему администратору.

На этом мы завершаем основную часть нашего справочного руководства по Linux. Надеемся, что вы нашли в этой книге полезную и нужную для себя информацию!

ПРИЛОЖЕНИЕ 1.

КОМАНДНЫЙ ИНТЕРПРЕТАТОР BASH





В первом приложении к нашему справочнику мы рассмотрим командный интерпретатор **bash**. Это больше, чем просто средство для ввода команд, у него есть свои параметры и свои команды. Да, да, именно команды. На **bash** можно написать сценарии (подобно тому, как в MS DOS создавались BAT-файлы). Возможности **bash** подобны языку программирования — есть управляющие структуры, циклы и т.д. Знание возможностей **bash** не менее важно для полноценного использования операционной системы Linux, чем использование функций ядра или рабочего окружения. Обо всем этом мы поговорим в данной главе.

П1.1. Настройка **bash**

bash — это самая популярная командная оболочка (командный интерпретатор) Linux.

Основное предназначение *bash* — выполнение команд, введенных пользователем.



Пользователь вводит команду и **bash** ищет программу, соответствующую команде, в каталогах, указанных в переменной окружения **PATH**. Если такая программа найдена, то **bash** запускает ее и передает ей введенные пользователем параметры. В противном случае выводится сообщение о невозможности выполнения команды.

Интерпретатор **bash** — не единственная оболочка, доступная в вашей системе. Список доступных оболочек доступен в файле `/etc/shells`. В большинстве случаев по умолчанию используется **bash**.

Файл `/etc/profile` содержит глобальные настройки **bash**, он влияет на всю систему — на каждую запущенную оболочку. Обычно `/etc/profile` не нуждается в изменении, а при необходимости изменить параметры **bash** редактируют один из файлов:

- `~/.bash_profile` — обрабатывается при каждом входе в систему;
- `~/.bashrc` — обрабатывается при каждом запуске дочерней оболочки;
- `~/.bash_logout` — обрабатывается при выходе из системы.

Файл `~/.bash_profile` часто не существует, а если и существует, то в нем есть всего одна строка:

```
source ~/.bashrc
```

Данная строка означает, что нужно прочесть файл `.bashrc`. Поэтому будем считать основным конфигурационным файлом файл `.bashrc`. Но помните, что он влияет на оболочку текущего пользователя (такой файл находится в домашнем каталоге каждого пользователя, не забываем: «`~`» означает домашний каталог). Если же вдруг понадо-



биться задать параметры, которые повлияют на всех пользователей, то нужно редактировать файл `/etc/profile`.

Файл `.bash_history` (тоже находится в домашнем каталоге) содержит историю команд, введенных пользователем. Здесь вы можете просмотреть свои же команды, которые вы накануне вводили.

Конфигурация `bash` хранится в файле `.bashrc`. Обычно в этом файле задаются псевдонимы команд, определяется внешний вид приглашения командной строки, задаются значения переменных окружения.

Псевдонимы команд задаются с помощью команды *alias*, вот несколько примеров:

```
alias h='fc -l'
alias ll='ls -laFo'
alias l='ls -l'
alias g='egrep -i'
```

Псевдонимы работают просто: при вводе команды *l* на самом деле будет выполнена команда `ls -l`.

Теперь рассмотрим пример изменения приглашения командной строки. Глобальная переменная `PS1` отвечает за внешний вид командной строки. По умолчанию командная строка имеет формат:

```
пользователь@компьютер:рабочий_каталог
```

Значение `PS1` при этом будет:

```
PS1='\u@\h:\w$'
```





В табл. П1.1 приведены допустимые модификаторы командной строки.

Таблица П1.1. Модификаторы командной строки

Модификатор	Описание
\a	ASCII-символ звонка (код 07). Не рекомендуется его использовать — очень скоро начнет раздражать
\d	Дата в формате «день недели, месяц, число»
\h	Имя компьютера до первой точки
\H	Полное имя компьютера
\j	Количество задач, запущенных в оболочке в данное время
\l	Название терминала
\n	Символ новой строки
\r	Возврат каретки
\s	Название оболочки
\t	Время в 24-часовом формате (ЧЧ: ММ: СС)
\T	Время в 12-часовом формате (ЧЧ: ММ: СС)



<code>\@</code>	Время в 12-часовом формате (АМ/РМ)
<code>\u</code>	Имя пользователя
<code>\v</code>	Версия bash (сокращенный вариант)
<code>\V</code>	Версия bash (полная версия: номер релиза, номер патча)
<code>\w</code>	Текущий каталог (полный путь)
<code>\W</code>	Текущий каталог (только название каталога, без пути)
<code>!\</code>	Номер команды в истории
<code>\#</code>	Системный номер команды
<code>\\$</code>	Если UID пользователя равен 0, будет выведен символ #, иначе — символ \$
<code>\ </code>	Обратный слэш
<code>\$ ()</code>	Подстановка внешней команды

Вот пример альтернативного приглашения командной строки:

```
PS1='[\u@\h] $(date +%m/%d/%y) \$'
```




Вид приглашения будет такой:

```
[denis@hosting] 08/04/19 $
```

В квадратных скобках выводится *имя пользователя* и *имя компьютера*, затем используется конструкция `$()` для подстановки даты в нужном нам формате и *символ приглашения*, который изменяется в зависимости от идентификатора пользователя.

Установить переменную окружения можно с помощью команды `export`, что будет показано позже.

П1.2. Зачем нужны сценарии *bash*

Представим, что нам нужно выполнить резервное копирование всех важных файлов, для чего создать архивы каталогов `/etc`, `/home` и `/usr`. Понятно, что понадобятся три команды вида:

```
tar -cvjf имя_архива.tar.bz2 каталог
```

Затем нам нужно записать все эти три файла на DVD с помощью любой программы для прожига DVD.

Если выполнять данную операцию раз в месяц (или хотя бы раз в неделю), то ничего страшного. Но представьте, что вам нужно делать это каждый день или даже несколько раз в день? Думаю, такая рутинная работа вам быстро надоест. А ведь можно написать сценарий, который сам будет создавать резервные копии, и записывать их на DVD! Все, что вам нужно, — это вставить чистый DVD перед запуском сценария.



Можно пойти и иным путем: написать сценарий, который будет делать резервные копии системных каталогов, и записывать их на другой раздел жесткого диска. Ведь не секрет, что резервные копии делаются не только на случай сбоя системы, но и для защиты от некорректного изменения данных пользователем. Помню, удалил важную тему форума и попросил своего хостинг-провайдера сделать откат. Я был приятно удивлен, когда мне предоставили на выбор три резервные копии — осталось лишь выбрать наиболее подходящую. Не думаете же вы, что администраторы провайдера только и занимались тем, что три раза в день копировали домашние каталоги пользователей? Поэтому, автоматизация — штука полезная, и любому администратору нужно знать, как автоматизировать свою рутинную работу.

П1.3. Сценарий "Привет, мир!"

По традиции напишем первый сценарий, выводящий всем известную фразу: Hello, world!. Для редактирования сценариев вы можете использовать любимый текстовый редактор, например, **nano** или **ee** (листинг П1.1).

Листинг ПР1.1. Первый сценарий

```
#!/bin/bash
echo "Привет, мир!"
```

Первая строка нашего сценария — это указание, что он должен быть обработан программой `/bin/bash`. Обратите внимание: если между `#` и `!` окажется *пробел*, то данная директива не сработает, поскольку будет воспринята как обычный комментарий. Комментарии начинаются, как вы уже наверняка знаете, с решетки:



Комментарий

Вторая строка — это оператор *echo*, выводящий нашу строку. Сохраните сценарий под именем *hello* и введите команду:

```
$ chmod +x hello
```

Для запуска сценария введите команду:

```
./hello
```

На экране вы увидите строку:

```
Привет, мир!
```

Чтобы вводить для запуска сценария просто `hello` (без `./`), сценарий нужно скопировать в каталог `/usr/bin` (точнее, в любой каталог из переменной окружения `PATH`):

```
# cp ./hello /usr/bin
```

П1.4. Переменные в сценариях

В любом серьезном сценарии вы не обойдетесь без использования переменных. Переменные можно объявлять в любом месте сценария, но до места их первого применения. Рекомендуется объявлять переменные в самом начале сценария, чтобы потом не искать, где вы объявили ту или иную переменную.



Для объявления переменной используется следующая конструкция:

```
переменная=значение
```

Пример объявления переменной:

```
ADDRESS=firma.ru  
echo $ADDRESS
```

При объявлении переменной *знак доллара* не ставится, но он обязателен при использовании переменной. Также при объявлении переменной не должно быть пробелов до и после знака =.

Значение для переменной указывать вручную не обязательно — его можно прочесть с клавиатуры:

```
read ADDRESS
```



или со стандартного вывода программы:

```
ADDRESS='hostname'
```



Чтение значения переменной с клавиатуры осуществляется с помощью инструкции *read*.

При этом указывать *символ доллара* не нужно. Вторая команда устанавливает в качестве значения переменной ADDRESS вывод команды *hostname*.

В Linux часто используются переменные окружения. Это специальные переменные, содержащие служебные данные. Вот примеры некоторых часто используемых переменных окружения:



- **BASH** — полный путь до исполняемого файла командной оболочки `bash`;
- **BASH_VERSION** — версия `bash`;
- **HOME** — домашний каталог пользователя, который запустил сценарий;
- **HOSTNAME** — имя компьютера;
- **RANDOM** — случайное число в диапазоне от 0 до 32 767;
- **OSTYPE** — тип операционной системы;
- **PWD** — текущий каталог;
- **PS1** — строка приглашения;
- **UID** — ID пользователя, который запустил сценарий;
- **USER** — имя пользователя.

Для установки собственной переменной окружения используется команда *export*:

```
# присваиваем переменной значение
$ADDRESS=firma.ru
# экспортируем переменную — делаем ее переменной окружения
# после этого переменная ADDRESS будет доступна в других сценариях
export $ADDRESS
```

П1.5. Передаем параметры сценарию

Очень часто сценариям нужно передавать различные параметры, например, режим работы или имя файла/каталога.



Для передачи параметров используются следующие специальные переменные:

- `$0` — содержит имя сценария;
- `$n` — содержит значение параметра (n — номер параметра);
- `$#` — позволяет узнать количество параметров, которые были переданы.

Рассмотрим небольшой пример обработки параметров сценария. Я понимаю, что конструкцию *case-esac* вы, возможно, не знаете, но общий принцип должен быть понятен (листинг П1.2).

Листинг П1.2. Получение параметров сценария

```
# Сценарий должен вызываться так:
# имя_сценария параметр

# Анализируем первый параметр
case "$1" in
    start)
        # Действия при получении параметра start
        echo "Запускаемся..."
        ;;
    stop)
        # Действия при получении параметра stop
        echo "Завершаем работу..."
        ;;
*)
    # Действия в остальных случаях
    # Выводим подсказку о том, как нужно использовать сценарий, и
    # завершаем работу сценария
    echo "Использовать: $0 {start|stop }"
```



```
exit 1
;;
esac
```

Думаю, приведенных комментариев достаточно, поэтому подробно рассматривать работу сценария из листинга П1.2 не будем.

П1.6. Обработка массивов

Интерпретатор *bash* позволяет использовать массивы. Массивы объявляются подобно переменным.

Вот пример объявления массива:

```
ARRAY[0]=1
ARRAY[1]=2
```

Выводим элемент массива:

```
echo $ARRAY[0]
```

П1.7. Циклы *for* и *while*

Как и в любом языке программирования, в **bash** можно использовать циклы. Мы рассмотрим циклы *for* и *while*, хотя вообще в **bash** доступны также циклы *until* и *select*, но они применяются довольно редко.

Синтаксис цикла *for* выглядит так:



```
for переменная in список
do
    команды
done
```

В цикле, при каждой итерации, переменной будет присвоен очередной элемент списка, над которым будут выполнены указанные команды. Чтобы было понятнее, рассмотрим небольшой пример:

```
for n in 1 2 3;
do
    echo $n;
done
```

Обратите внимание: список значений и список команд должны заканчиваться точкой с запятой.

Как и следовало ожидать, наш сценарий выведет на экран следующее:

```
1
2
3
```

Синтаксис цикла *while* выглядит немного иначе:

```
while условие
do
    команды
done
```

Цикл *while* выполняется до тех пор, пока истинно заданное условие.



Подробно об условиях мы поговорим, возможно, чуть позже, а сейчас напишем аналог предыдущего цикла, т. е. нам нужно вывести 1, 2 и 3, но с помощью *while*, а не *for*:

```
n=1
while [ $n -lt 4 ]
do
    echo "$n "
    n=$(( $n+1 ));
done
```

П1.8. Условные операторы

В *bash* доступны два условных оператора: *if* и *case*.

Синтаксис оператора *if* следующий:

```
if условие_1 then
    команды_1
elif условие_2 then
    команды_2
...
elif условие_N then
    команды_N
else
    команды_N+1
fi
```

Оператор *if* в *bash* работает аналогично оператору *if* в других языках программирования. Если истинно первое условие, то выполняется первый список команд, иначе — проверяется второе условие и т. д. Количество блоков *elif*, понятно, не ограничено.



Самая ответственная задача — это правильно составить условие. Условия записываются в квадратных скобках. Вот пример записи условий:

```
# Переменная N = 10
[ N==10 ]

# Переменная N не равна 10
[ N!=10 ]
```

Операции сравнения указываются не с помощью привычных знаков > или <, а с помощью следующих выражений:

- *-lt* — меньше;
- *-gt* — больше;
- *-le* — меньше или равно;
- *-ge* — больше или равно;
- *-eq* — равно (используется вместо ==).

Применять данные выражения нужно следующим образом:

```
[ переменная выражение значение|переменная ]
```

Например:

```
# N меньше 10
[ $N -lt 10 ]

# N меньше A
[ $N -lt $A ]
```



В квадратных скобках вы также можете задать выражения для проверки существования файла и каталога:

- *-e файл* — условие истинно, если файл существует;
- *-d каталог* — условие истинно, если каталог существует;
- *-x файл* — условие истинно, если файл является исполняемым.

С оператором *case* мы уже немного знакомы, но сейчас рассмотрим его синтаксис чуть подробнее:

```
case переменная in
значение_1) команды_1 ;;
...
значение_N) команды_N ;;
*) команды_по_умолчанию;;
esac
```

Значение указанной переменной по очереди сравнивается с приведенными значениями (значение_1, ..., значение_N). Если есть совпадение, то будут выполнены команды, соответствующие значению. Если совпадений нет, то будут выполнены команды по умолчанию. Пример использования *case* был приведен в листинге П1.2.

П1.9. Функции в *bash*

В *bash* можно использовать функции.

Синтаксис объявления функции следующий:

```
имя() { список; }
```



Вот пример объявления и использования функции:

```
list_txt()  
{  
  echo "Text files in current directory:"  
  ls *.txt  
}
```

П1.10. Практические примеры сценариев

Проверка прав пользователя

Для сценариев, требующих полномочий *root*, сначала нужно проверить, какой пользователь запустил сценарий. UID пользователя *root* всегда равен 0. Проверка, является ли пользователь, запустивший сценарий, пользователем *root*, может выглядеть так, как показано в листинге П1.3.

Листинг П1.3. Проверка полномочий пользователя

```
#!/bin/bash  
  
ROOT_UID=0  
  
if [ "$UID" -eq "$ROOT_UID" ]  
then  
  echo "Root"  
else  
  echo "Обычный пользователь"
```



```
fi

exit 0
```

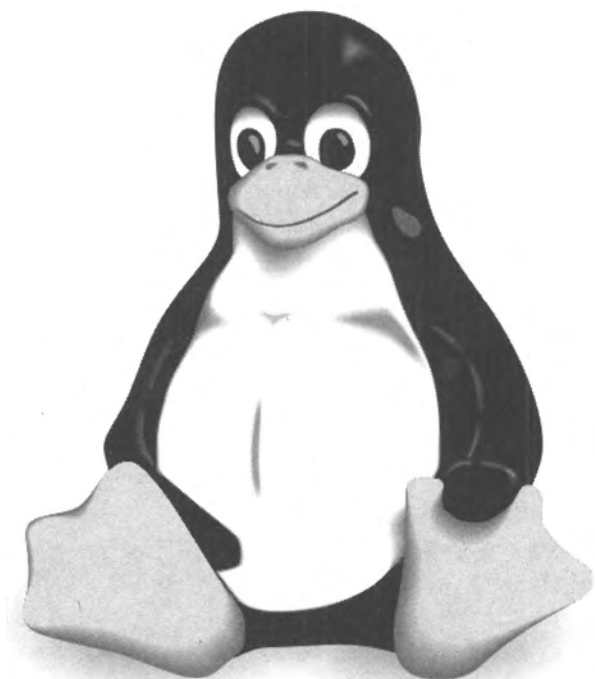
Проверка свободного дискового пространства с уведомлением по e-mail

Следующий сценарий проверяет свободное дисковое пространство сервера и отправляет уведомление по e-mail администратору, если осталось меньше 2000 Мбайт дискового пространства. Сценарий целесообразно запускать через планировщик заданий (например, через **cron**) с заданной периодичностью (например, каждый час). Код сценария приведен в листинге П1.4.

Листинг П1.4. Проверка свободного дискового пространства

```
#!/bin/bash
# В переменную freespace будет записано свободное пространство
# на контролируемом диске - /dev/sda1 (в Мбайт)
freespace=$(df -m | grep "/dev/sda1" | awk '{print $4}')

# Если места на диске < 2000 Мб, то отправляем письмо администратору
if [ $freespace -lt 2000 ];
then
echo "На сервера меньше < 2000 МБ" | mail -s "Свободное место заканчивается!" admin@firma.ru
fi
```



ПРИЛОЖЕНИЕ 2.

СЕТЕВАЯ ФАЙЛОВАЯ СИСТЕМА NFS





П2.1. Вкратце о NFS и установке необходимых пакетов

Сетевая файловая система (NFS, Network File System) – старый, но проверенный способ предоставления доступа к общим файлам. NFS позволяет монтировать файловые системы, которые физически находятся на других компьютерах в вашей локальной сети.

Принцип работы NFS прост: на сервере устанавливается демон **nfsd**, который экспортирует файловые системы. NFS-клиент монтирует экспортируемые файловые системы. После монтирования с удаленной файловой системой можно работать, как с локальной – для пользователя ничего не изменится, разве что скорость доступа к файлам будет чуть ниже, поскольку данные ведь передаются по сети.

В вашей сети может быть несколько NFS-серверов и несколько NFS-клиентов. По сути, каждый NFS-клиент может выступать и в роли



NFS-сервера, но обычно NFS используется иначе. Выделяется один NFS-серв

вер, который будет экспортировать общую файловую систему, которая может использоваться для хранения общих файлов – довольно частый сценарий.

Чтобы установить NFS-сервер и/или NFS-клиент произведите поиск пакета по слову ‘nfs’. Далее прочитайте описания пакетов и установите необходимые вам пакеты. Названия пакетов могут отличаться в зависимости от дистрибутива. Например, в openSUSE на сервере нужно установить следующие пакеты:

- **nfs-kernel-server** – содержит NFS-сервер;
- **nfswatch** – утилита мониторинга NFS-трафика (можно не устанавливать);
- **quota-nfs** – система дисковых квот для NFS.

На клиенте нужно установить только пакет `nfs-client`.

П2.2. Файл `/etc/exports`

В файле `/etc/exports` прописываются экспортируемые файловые системы.

Эти экспортируемые файловые системы могут монтировать клиенты. Общий формат записи в файле `exports` следующий:

файловая_система [компьютер] (опции)



Вот типичный пример:

```
/home/public (rw)
/home/templates (ro)
```

В данном случае каталог `/home/public` сервера будет доступен NFS-клиентам для **чтения и записи** (`rw`), а каталог `/home/templates` – только для **чтения** (`ro`). При желании можно еще и указать узлы, которым будет доступен экспортируемый каталог, например:

```
/home/share1 192.168.1.100 (rw)
/home/share2 host.example.com (rw)
/home/share3 *.example.com (rw)
```

Здесь первый каталог (`/home/share`) будет доступен только узлу с IP-адресом `192.168.1.100`, второй каталог – только узлу с доменным именем `host.example.com`, а третий – всем узлам из домена `example.com`. Также можно указать целую IP-сеть и ее маску, например:

```
/home/share4 192.168.1.0/28 (rw)
```

В данном случае получить доступ к `/home/share4` смогут лишь первые 16 IP-адресов – от `192.168.1.0` до `192.168.1.15`. Компьютеры с IP-адресами `192.168.1.16` и выше не смогут использовать эту файловую систему.

Обычно можно обойтись только опциями `ro`, `rw` и `noaccess`. Последняя используется для запрещения доступа к файловой системе. Например, вы хотите запретить узлу `192.168.1.5` доступ к файловой системе:

```
/home/share4 192.168.1.5 (noaccess)
```



Остальные опции NFS приведены в таблице П2.1.

Таблица П2.1. Опции экспорта файловой системы в NFS

Опция	Описание
<i>all_squash</i>	Преобразует идентификаторы групп и пользователей в анонимные
<i>insecure</i>	Разрешает запросы с любых портов, см. <i>secure</i>
<i>link_absolute</i>	Не изменяет символические ссылки (по умолчанию)
<i>link_relative</i>	Абсолютные ссылки будут преобразованы в относительные
<i>root_squash</i>	Преобразует все запросы от <i>root</i> в запросы от анонимного пользователя
<i>no_root_squash</i>	Разрешает доступ к файловой системе от имени <i>root</i> . Не рекомендуется
<i>secure</i>	Принимает запросы на монтирование файловой системы только с портов с номерами < 1024. Такие порты может создавать только <i>root</i> , поэтому такое соединение считается безопасным. Используется по умолчанию



После редактирования файла `/etc/exports` не забудьте перезапустить или запустить службу **nfs**:

```
# service nfs start
```

или

```
# service nfs restart
```

П2.3. Использование NFS-клиента

Для монтирования сетевой файловой системы используется та же команда *mount*.

Синтаксис вызова команды такой:

```
mount -t nfs сервер:ФС точка_монтирования
```

Вот пример:

```
mount -t nfs 192.168.1.157:/home/share /mnt/share
```

Сервер 192.168.1.157 экспортирует файловую систему `/home/share`. Мы подмонтируем эту файловую систему к каталогу `/mnt/share` (должен существовать на момент вызова команды *mount*). Также обратите внимание на параметр *-t*, который задает тип файловой системы.

Конечно, вы не будете вводить эту команду после каждой перезагрузки. Поэтому если удаленную файловую систему нужно использовать постоянно, лучше прописать ее в файле `/etc/fstab`:



```
192.168.1.157:/home/share /mnt/share nfs bg,hard,rw 0 0
```

Нужно отметить, что NFS – довольно старое решение, как говорят, классическое. Существуют различные варианты сетевых и распределенных файловых систем, например, ClusterNFS, OpenAFS, Parallel Virtual File System (PVFS) и др.

Основной недостаток NFS – отсутствие какого-либо шифрования передаваемых по сети данных. Если вы используете NFS в защищенной локальной сети, то это, как правило, не проблема (при отсутствии угроз внутри сети). Но когда есть необходимость передавать данные по Интернету, то нужно использовать шифрование. Один из способов добавления шифрования к NFS – использование виртуальных частных сетей (VPN), в которых все данные между удаленными системами передаются в зашифрованном виде. В этом случае NFS имеет право на существование в Интернете.

Список использованных источников информации:

- Колисниченко Д.Н. «LINUX. Полное руководство по работе и администрированию»
- https://ru.wikipedia.org/wiki/Softlanding_Linux_System
- https://docs.huihoo.com/doxygen/linux/kernel/3.7/structtask__struct.html
- <http://tldp.org/HOWTO/LVM-HOWTO/>
- <http://www.takovlev.org/index.html>
- <https://mirrors.edge.kernel.org/pub/linux/libs/pam/>
- <https://www.insight-it.ru/category/linux/>
- <http://wiki.opennet.ru/RPM>
- <https://icon-icons.com/ru/%D0%B7%D0%BD%D0%B0%D1%87%D0%BE%D0%BA/Linux-%D0%BA%D0%BE%D0%BD%D1%84%D0%B8%D0%B3%D1%83%D1%80%D0%B0%D1%86%D0%B8%D1%8F-%D0%BC%D0%B5%D1%82%D0%B8%D0%B7%D1%8B/5522>
- <https://icon-icons.com/ru/%D0%B7%D0%BD%D0%B0%D1%87%D0%BE%D0%BA/%D0%9B%D0%B8%D0%BD%D1%83%D0%BA%D1%81-%D0%BE%D1%80%D0%B8%D0%B3%D0%B8%D0%BD%D0%B0%D0%BB-%D0%BB%D0%BE%D0%B3-%D0%BE%D1%82%D0%B8%D0%BF/146433>
- <https://icon-icons.com/ru/%D0%B7%D0%BD%D0%B0%D1%87%D0%BE%D0%BA/%D0%BF%D0%B0%D0%BF%D0%BA%D0%B8-%D0%92%D0%B8%D0%B4-%D0%9B%D0%B8%D0%BD%D1%83%D0%BA%D1%81/129907>
- <https://icon-icons.com/ru/%D0%B7%D0%BD%D0%B0%D1%87%D0%BE%D0%BA/%D1%80-%D0%BB%D0%B8%D0%BD%D1%83%D0%BA%D1%81/102999>



Книги по компьютерным технологиям, медицине, радиоэлектронике

Уважаемые авторы!

Приглашаем к сотрудничеству по изданию книг по **IT-технологиям, электронике, медицине, педагогике.**

Издательство существует в книжном пространстве более 20 лет и имеет большой практический опыт.

Наши преимущества:

- Большие тиражи (в сравнении с аналогичными изданиями других издательств);
- Наши книги регулярно переиздаются, а автор автоматически получает гонорар с *каждого* издания;
- Индивидуальный подход в работе с каждым автором;
- Лучшее соотношение цена-качество, влияющее на объемы и сроки продаж, и, как следствие, на регулярные переиздания;
- Ваши книги будут представлены в крупнейших книжных магазинах РФ и ближнего зарубежья, библиотеках вузов, ссузов, а также на площадках ведущих маркетплейсов.

Ждем Ваши предложения:

тел. (812) 412-70-26 / эл. почта: nitmail@nit.com.ru

Будем рады сотрудничеству!

Для заказа книг:

➤ **интернет-магазин: www.nit.com.ru / БЕЗ ПРЕДОПЛАТЫ по ОПТОВЫМ ценам**

- более 3000 пунктов выдачи на территории РФ, доставка 3-5 дней
 - более 300 пунктов выдачи в Санкт-Петербурге и Москве, доставка 1-2 дня
 - тел. (812) 412-70-26
 - эл. почта: nitmail@nit.com.ru
-

➤ **магазин издательства: г. Санкт-Петербург, пр. Обуховской обороны, д.107**

- метро Елизаровская, 200 м за ДК им. Крупской
 - ежедневно с 10.00 до 18.30
 - справки и заказ: тел. (812) 412-70-26
-

➤ **крупнейшие книжные сети и магазины страны**

Сеть магазинов «Новый книжный» тел. (495) 937-85-81, (499) 177-22-11

➤ **маркетплейсы ОЗОН, Wildberries, Яндекс.Маркет, Myshop и др.**

Матвеев М. Д.

Справочник ЛИНУКСОИДА

Все, что нужно, под рукой

Группа подготовки издания:

Зав. редакцией компьютерной литературы: *М. В. Финков*

Редактор: *Е. В. Финков*

Корректор: *А. В. Громова*

12+

ООО "Издательство Наука и Техника"

ОГРН 1217800116247, ИНН 7811763020, КПП 781101001

192029, г. Санкт-Петербург, пр. Обуховской обороны, д. 107, лит. Б, пом. 1-Н

Подписано в печать 03.04.2023. Формат 70х100 1/16

Бумага офсетная. Печать офсетная. Объем 22 п. л.

Тираж 2000. Заказ 6450.

Отпечатано с готового оригинал-макета

ООО «Принт-М», 142300, М.О., г. Чехов, ул. Полиграфистов, д. 1